Programming and finance

Maths

- A real life case
- Ito's lemma
- Black and Scholes and more
- Expectation of the Max
- Efficient frontier
- Var
- Tranche pricing
- FFT
- Simplex
- LLMs

1- A real life time series

The financial industry is based upon financial markets activity nowadays. Up until 1990 that was rather a side activity that became more and more dominant as information technologies brought up more and more "data" and therefore "prices".
The S&P500 is a benchmark:

You can see at once that the path is chaotic and this is what market players, but also every financial decision maker, are focusing on every day despite repeating on and on "markets are wrong", "markets are crazy", "this will pass". This is true: markets have this reputation to have predicted 90% of the crisis that never happened. They also have this reputation of having the "memory of an elephant". And yet they also have this reputation of "missing every serious crisis". Of course…since we do have crisis…
So, beyond the self-conceived and self-serving pseudo interpretation of what the "markets think", they are just random in their behavior.
Looking at the longer term we might discern a more salient pattern: in the chart below we are tempted to infer "whoa! I have to buy it because it is just a one way train towards ever higher values!"

But the volumes histograms . the volumes of buyers sky-rocketed…When you have more "buyers" then, unsurprisingly, prices shoot up. Also, 90% of the rise since 2010 is due at 90% to the 7 companies at most: Microsoft, Apple, Amazon, Google, Facebook, Tesla, Nvidia. They have a common point: mobiles, data, entertainment, "services for free" on the surface, AI. And they all are heavily investing in cryptos since 2017…but the scandals and scams have pulled their ambitions back so far.

Just notice above the massive progression of the S&P since the trough of 2009: it rose 500%. Not bad.

What about the Bitcoin then?! In 2009 BTC was worth about 0.001$ and now it is worth around 80 000 $ ie a comfortable: 80 000 000 00%=16 000 000 times the S&P 500 'fabulous' performance. We should not expect as much in the future of course.

The really important thing is that here one has for free, the "volumes" of course and, better, the "book orders", ie "who" is buying "at what price" and "how much". It sounds like just a luxury of details. It is not…

First notice that here, when the buyers purchased BTC in 2022-2023 they stopped afterwards and the BTC rallied in relatively low volumes. Is this a “bubble” or is the S&P500 actually in a bubble?
We can answer with logic at least: the buyers of the S&P500 will feel at a loss and pressured to sell out as soon as the S&P500 goes down 10% or more. But for the BTC, the buyers purchased it at around 15000-20000 and they would not see a “loss” in a correction down, rather a “loss of opportunity to make magic gains”. Truth being told an 80% correction on the way down for BTC is as frequent as a correction of 15% on the way down for the S&P 500. So, finally, things are equivalent because an 80% correction on the way down of BTC would also put the BTC buyers of 2022 to sell out in haste. So, they are currently “waiting to gain even more” as if their trade paradoxically is done quite “at profit yet”.

Another key anecdote of our times is that on average companies in the S&P500 struggle to deliver a 15-20% return on capital: ironically the average return on capital of shareholders on the S&P500, that kept buying it all along for over 16 years now turns out to be 11.1% compounded every year how one is getting to that conclusion?
Let’s assume that one buys with 1$ of the S&P 500 in 2009 and in 2025 one has now 6$, ie the investment “made” 500% which means that the net gain if the original dollar is sold out at this market level, net, one gained 5$ ie 500% of 1$. Impressive even by century long historical standards: only the decade before the crisis of 1929 matched this kind of performance. Yet if we want to compute the yearly rate of return on capital we would write the following formula: $P_{today} = P_{2009}e^{(Today-2009)_{years}*R_{oe}}$

$$and\ then\ R_{oe} = \frac{Ln\left(\frac{P_{today}}{P_{2009}}\right)}{(Today-2009)_{in_{years}}} = Ln\left(\frac{5+1}{16}\right) \approx 11.1\%$$

The GAFAM+Tesla ie the 7 companies that contributed to 90% of these 500% return actually deliver on average 20% Roe, ie “Return on Equity” (Equity=shares).
Now, OpenAI, the designer of ChatGPT, delivered a “meager” 300% Roe to its shareholders, the biggest of them being Tesla, and Microsoft,..And Amazon next…and Google…What is the next project of OpenAI: “Orbs” that is a crypto that will store over the whole world your identity and therefore just everything that defines you as an individual in the eyes of the “other human beings”…worldwide…This will be your ID card, your debit card, you professional path, your “likes” and “dislikes” etc….everything ! Expected Roe? Guess…
What follows is unfortunately much less entertaining. It will focus on the the basic mathematical tools that prevail in the financial industry and support all the comments and undertakings that drive the financial markets. This will be a rather dry overview that is meant to introduce the main concepts given that people “maximize profits” based on “time series” using “models having many parameters” where the “normal distribution” and the “matrices” are ubiquitous all the same. All this is meant to be “efficient” at least in economic terms.

2- Ito's lemma

Let's start with some abstract vocabulary clarifications…..

We will often speak of "random variable x,y ,z": you should imagine that we speak of "prices", or "yields" or "returns" or "risks" or "indices" ie things that reflect values or value changes. In contrast we speak of "time" , "periods", "instant" in order to refer to deterministic measures that define a time series that is not random. These "instants" are separated by "time steps" and it is not always clear whether the "instant" features the "statistical expectation" over the surrounding time-step or the instants delimits the bounds of the time-steps that we then might feature through their "mid-point in time". Usually we would assume that the time-steps are small enough so that the ambiguity is not an issue. When we have set these 2 types of variables, one random and the other deterministic, we speak of some "stochastics process" to describe de resulting time series. In finance this kind of data is ubiquitous.

What is this "Itô's lemma" that actually was first established by Wolfgang Doeblin?

We need first to clarify a few definitions:

1- A Markov chain or Markov process: it is a process, ie an evolution over discrete time "t" increments, whereby a random variable "x" switches to a new state as per a distribution that is only determined by the value that "x" took at the former time "t". In mathematical terms we phrase it like that:

$$for\ a\ given\ series\ of\ \text{time steps}\ t_1, \mathrm{t}_2, \mathrm{t}_3, \dots, \mathrm{t}_\mathrm{n}\ \text{we can write that}$$
$$\mathrm{P}(\mathrm{x}(\mathrm{t}_\mathrm{n}) < \mathrm{x}_\mathrm{n} | \mathrm{x}(\mathrm{t}_1), \mathrm{x}(\mathrm{t}_2), \mathrm{x}(\mathrm{t}_3), \dots \mathrm{x}(\mathrm{t}_{\mathrm{n}-1})) = \mathrm{P}(\mathrm{x}(\mathrm{t}_\mathrm{n}) < \mathrm{x}_\mathrm{n} | \mathrm{x}(\mathrm{t}_{\mathrm{n}-1}))$$

So the "former events" would not matter so much : only the rule that defines the "next event" matter. But bear in mind that this "instant t_{n-1}" could very well be a "period of time" where the "rule" then may depend on a "vector of consecutive instants and associated events". The key here is that we assume that we can define these consecutive periods through which a known fixed rule applies uniformly.

2- Now let's consider seriously the mathematical description of the space into which one can handle a random variable over time….This variable "x", at any given "instant", can take as many values as per its distribution that might change from any time step to the next. A Markov process states that a random variable is independent in its current distribution from the path that it has just followed. But there is a subtlety here: the distribution at time step $\mathrm{t}_\mathrm{n}$does depend on the value taken by x at $(\mathrm{t}_{\mathrm{n}-1})$… Still, the "rule" that defines the distribution is known from the beginning of "time"… Thus if the "rule" was to change, we would have to define a "stopping time" and therefore start a brand new time axis for this new "rule".

This is all fine when we are at time "$\mathrm{t}_{\mathrm{n}-1}$" projecting what will happen for the next time step under the prevailing rule… But what about our projection for $\mathrm{t}_{\mathrm{n}+1}$ and $\mathrm{t}_{\mathrm{n}+2}$ and so on? Then the "Markov assumption" provides an answer: we know that we need to compound always the same set of distributions that will be conditioned upon the former value of x "only". The whole set of distributions (potentially one for every value of the couple (x, t_n) is defined from time t=0 as per fixed rules of composition. Yet we do not know "exactly" how these rules will combine iteratively in the future: in real life we can only make this premise that the rule is unchanged while unknown in its closed form expression. In other words, the path followed does not alter the future set of distributions but we cannot know in advance which the

distribution is the one that we will face at time "t" in the future. As in card games or any game, we will follow potentially an infinite number of "paths" but the rules remain the same all along.

We may well have a different distribution depending on the time step itself and the value that "x". will have followed. However "path dependent" as it may be, if we know the path and the value of "x" then we can know ahead of time what the distribution shall be: so we can run "simulations" and infer "mean values" from these simulations. People summarily say for that reason that Markov processes are not "path dependent". And projecting future losses or gains, these many distributions will all recombine as per a pre-defined set if we want to project in the future beyond the next time-step…And this opens the possibility of really infinite combinations: take a simple distribution admitting just 2 possible values for x at t_{n-1} and assume that x can also take only 2 values next at t_n. In fact for each value taken at t_{n-1}, there is a distinct distribution for the 2 values that x could take at t_n. After N time steps beyond t_{n-1}, one is therefore left, in this very simplistic example of a Markov chain with 2^N possible distributions for x at t_{n+N}.

There is thus a whole set of "events", ie combinations of the futurethat are "conditionally independent" distributions, that can be used in projecting the future. And one can see however that one initial state will impact the future ones within this steady set of distributions. One then speaks of "filtrations" ie a series of "Russian dolls" picked in this whole universe of future scenarios….A filtration can be expressed as a series of sets of scenarios that some random variable x would follow from one time step to the next where, as time steps add up, the former scenarios of t_{n-1} are all included into the set of scenarios that describes the next time-step t_n. So a filtration at time t_n will be "included" in a filtration that is referenced to time t_{n+1}, not only as such but also as a common denominator with a whole new set of other scenarios that are based on this filtration and have not incorporated a "stopping time" yet.

It is only when "time stops" because the Markov chain is broken that the filtration stops itself. Otherwise any scenario of time t_n, provides the basis for many subsequent new scenarios for time t_{n+1}. Again in mathematical terms, if we call S the set containing all the possible future scenarios, then we define a filtration as a series of subsets of S like F_{t_j} or F_{t_i} such that they actually define a "Russian dolls" series :

$$\forall (i,j) \epsilon \mathbb{N}^2 where\ i < j, \qquad F_{t_i} \subset S, F_{t_j} \subset S, \qquad t_i < t_j \Leftrightarrow F_{t_j} \subset F_{t_i}$$

3- A martingale now is defined with the help of a given filtration and a necessary "probability measure" in a given set of projective scenarios S that apply to "x", a random variable. The "probability measure" is an issue mostly when "x" can take possibly an infinite number of values or can at times take infinite values….This is unrealistic in real life but this becomes necessary when one wants to sketch "jump" scenarios or one hopes to use the CLT(Central Limit Theorem that states that the average of "n" independent random variables of equivalent mean and variance, irrespective of the underlying individual distributions, converges towards the normal distribution that has the average of the means and the average of the variances as respective mean and variance) for subsequent calculations. A "probability measure" is just one way to obtain a convergent series based upon the many values that a random variable "x" could take: we group the N values into "buckets", find the "number n " of such values and we record the density of probability as the ratio of "$\frac{n}{N}$" that is

associated to the mid point in this bucket or else the average value of the "n" values that stand in this bucket. The "probability measure" defines the bucketing itself since the buckets may have to be of different size. Then this density of probability that we noted "$\frac{n}{N}$" has to be weighted by the "size" of the bucket itself. Indeed a large bucket is more likely to store more values than a small one. Thus the probability measure makes the future integral sums consistent from one measure to another. It matters whenever we compute an "expectation" which amounts to say, virtually every time we actually compute something in finance.

We have now to properly define the "martingale".
Thus let's consider "x" a random variable measured in distribution as per this filtration F set within S. Then the mean variation of "x" since it is a "martingale" from one time-step to the next, is zero. In mathematical terms we write that for a filtration F set within the space of events S:

$$\forall (i,j) \epsilon \mathbb{N}^2 where\ i < j,$$
$$\left(F_{t_i}, F_{t_j}\right) \in F^2, \qquad t_i < t_j\ , E\left[x_{t_j} - x_{t_i} | given\ any\ event\ of\ F_{t_i}\right] = 0$$

Thus a martingale states that the "mean" (or statistical moment of first order of the distribution of "x") is constant over time in a Markov process where one assumes that one can find "one filtration" (ie a Russian dolls way of encompassing all the possible future scenarios) and one can "measure" the variation of x over any future time length! These are pretty strong assumptions…. That simply express that "on average" the value "x" is not "expected" to move away from zero, even if it does so temporarily. It is the case when on tosses a coin: we sort of "know" already that the "rule" is that either the coin falls on "heads" or it falls on "tails". However, if we repeat the game we may face very long series of consecutive "tails" (for example). We could even say that for any number of consecutive "tails" "N", the probability that we have this scenario is never "0". Also, the same applies for the "heads" face…So, if someone makes bets on every draw, one faces potentially infinite positive or negative returns while, "on average" they should be zero if the player plays long enough. Which is what defines intuitively a "martingale". As you may have noted, the "rule" is fixed all along, ie it is a Markov process. However, while we easily convince ourselves that this "rule" is fixed by considering the homogenous influence of the gravity and the symmetry of the coin itself, we can hardly set this "rule" in equation and even less so model easily the "motion" that the coin would follow once it has been thrown in the air. This is this approach that drives the financial calculations of all sorts.

4- A Wiener process is one certain type of Markov Process that involves also a martingale feature: it is a martingale of variance 1 over a unit time interval. On top of that a Wiener process is a martingale where the actual distribution for "x" obeys the normal distribution (see below for the definition) with a variance being equal to "T" over the time interval [0,T] that holds "T" time-steps through which the variance is "1". On top of that a Wiener process is deemed to be continuous over time "t" **'almost surely'**, ie there is no possible "infinite jump" within an infinitely small time interval. In pure mathematical terms, this means that there no actual smooth behavior in "x" from one instant to the next at all: if we used an electronic microscope to scrutinize the time series of "x", ie all the consecutive values that x took in some real life record, the "function" that "x" would seem to follow would be made of "jumps" from one time interval to the next. However, when we would compute this integral sum, weighed by this "probability measure" we would find a value that would tend to be a continuous function, not always stuck on "0" despite being a "martingale". The smoothness

would be the result still of the "martingale" property but of course the first elements might induce a non zero end value. This implies that on any finite time period, if we can indeed sum up this period into a number high enough of sub-intervals the "integral sum" would soon look as "smooth" as we want it to be. This is the reason why we say "almost surely": if you can really catch values between arbitrarily close instants in time relative to the your starting point, you very soon see that you integral, as a function of the "end time" becomes smooth. If you went as far was stripping your time period into an infinity of sub-intervals, then your integral would the nil function! What happens then when you increase the number of sub-intervals? Well you catch finer and finer observations that potentially all "see" a "jump" of finite dimension, but they all combine to converge towards 0 on every single sub-interval that is finite and can be infinitely stripped into sub-intervals again. All this implies that all the jumps are not "Dirac" spikes: they are of finite magnitude although, as in the case of coin tossing, this magnitude itself is NOT bounded. This is why we speak of "continuity almost surely".

The Wiener process is also known as a "Brownian motion": there is no stopping time risk but that does not preclude chaotic moves whereby there are sort of "mini-jumps" that have benign consequences over time… The rules remain unchanged and therefore the future distributions remain set by these original rules anyway.

Thus if one considers a Wiener process W_t, some fundamental relations appear

$$E[W_t] = 0, ie\ , the\ mean\ is\ 0 \ldots it\ is\ a\ martingale$$

$$Var[W_t] = E[(W_t - E[W_t])^2] = t$$

This property here means that (since $E[W_t] = 0$) $E[(W_t)^2 - t] = 0$, or equivalently that $(W_t)^2 - t$ is a martingale itself.

One last property of the Wiener process is that it follows a normal distribution continuously over time while being a martingale over any time length "t"

$$p_{W_t}(W_t = x) = \frac{1}{\sqrt{2\pi t}} e^{-\frac{x^2}{2t}}\ ie\ the\ normal\ law\ with\ mean\ at\ 0\ and\ variance\ t$$

One intuitive expression of the Wiener process comes as follows whereby the Markov process pattern shows up clearly for this martingale:

$$W_{t+\Delta t} = W_t + \sqrt{\Delta t} Z_{\aleph(0;1)}\ ie\ Z\ is\ a\ is\ random, normal, with\ mean\ 0\ and\ variance\ 1$$

This last formula shows that irrespective of what occurred until "t", what will happen over Δt in the future, follows a normal distribution with mean "0" (martingale) and variance Δt (Brownian motion whereby the noisy pattern is steady over time. That is the "in hindsight look" stating that "we had all the needed information from the beginning of times"….)

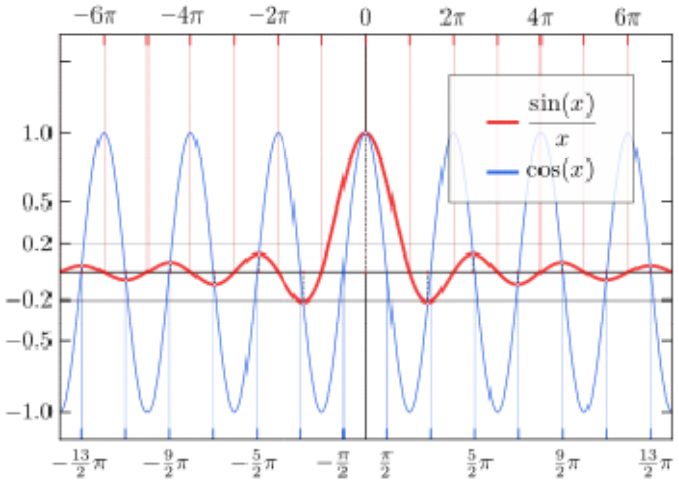
But this simplicity conveys already a complex behavior as Norbert Wiener put it in his own formula below. Here we just need to take for granted that this is a "continuous process over Δt", ie that we can depict the process into an infinite number of sub-intervals that is numerable with an index " i ", through which again we do not face a "stopping time":

$$W_{t+\Delta t} = \xi_t \Delta t + \sqrt{2} \sum_{i=1}^{+\infty} \xi_i \frac{\sin(i\pi\Delta t)}{i\pi\Delta t}\ where\ \xi_t \sim \aleph(0;1)\ and\ \forall i \epsilon \mathbb{N}, \xi_i \sim \aleph(0;1)$$

How did Norbert Wiener find this formula and for what purpose? Generally speaking, he wanted to find a generic way to approach numerically a Brownian motion ie a function as we described above that is "continuous almost surely" ie that potentially may "jump" at any moment in time without going "crazy". This happens to mimic quite well what we "see" in financial markets in general.

What is the “big change” between our modern formulation of a Wiener process and what Norbert Wiener initially constructed?
We have removed the term W_t and replaced it with a pseudo “integral of the past sub-intervals”. This is quite a complex series cumulating a infinite number of random variables. This one expression above is a “looking forward” view of a Wiener process. Not so simple is it?! This formula simply exhibits what will be shown soon: this risk—based risk management optimization of some invested capital induces a “Brownian” motion that is not so “intuitive” when we have to make numerical projections. It is “fine “ in “hindsight” only…

Thus, if one wants to project the possible paths in the future, this is not so simple as the “ ultimate simplistic” Wiener process that amounts in the end to a “simple normal law” “in hindsight, ie once we have survived the shocks”. This is an infinite series of other “simple normal laws” weighted by a peculiar function called the sine cardinal function: $\sin_{\text{card}}(n\pi\Delta t) = \frac{\sin(n\pi\Delta t)}{n\pi\Delta t}$. This function that is associated with everything in the series does not behave quite like the sine function as the chart below exhibits:

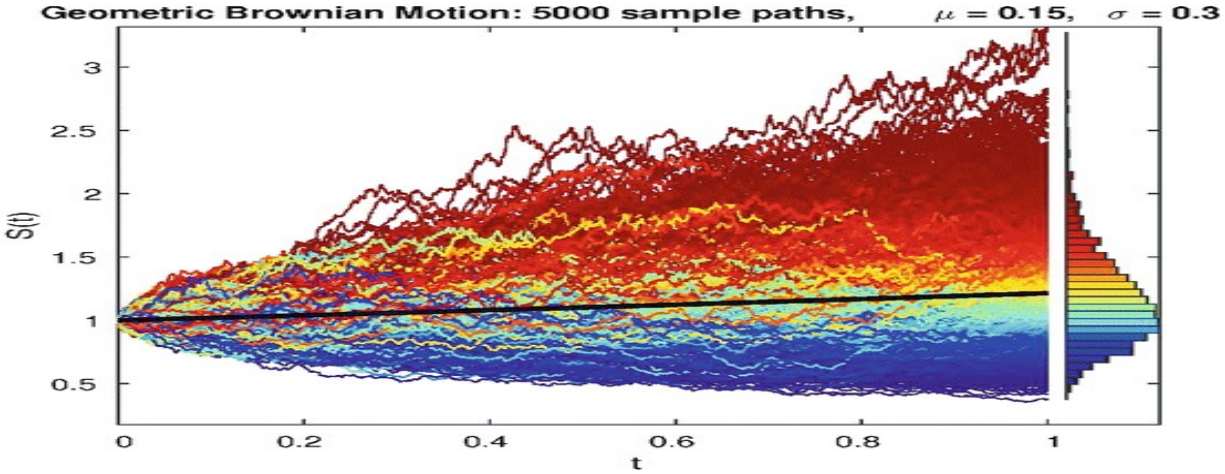


We can see that the “red line” vanishes when one goes away from the present time. This chart displays that such a process carries a rather “short term memory”, ie that shocks vanish rather quickly. This extinction provides an intuition as to why this complex series always converges back to a normal distribution. The sine cardinal functions can be understood as the “vector” of functions that secures the steadiness of the normal distribution over time, ie through some “continuous compounding”.
Thus when in the chart above “x” is actually $n\pi\Delta t$, one observes that the “weight” of the smallest “n”s given Δt, do have the bigger contribution in the aggregate series as written by Norbert Wiener. Yet, all the terms contribute and sometimes in a negative fashion. Thus, although the “ index 0” term matters first, other terms can nullify it which gives the ultimate maximum importance to higher orders… All this is random….with a rather fast extinction still….This is no match with the notorious “elephant memory” of financial markets. But this is what drives the maximum financial leverage in financial markets through banks and any other trading business! In hindsight again we always tend to look back at the past financial crisis at things that we escaped already. And they can easily be featured in our empirical database as “temporary disruptions”. The “elephant memory” is then very much felt as this extinguishing Sin Cardinal pattern….except that this memory does NOT vanish like a Sin Cardinal function anyway…
So, as a way of conclusion about our own psychological bias towards financial crisis, the “term 0” in financial crisis is really scary but does not last in practice. In our vocabulary, while usually the “term 0” in the function of Norbert Wiener is “expected to be small or 0”, indicating the value of some

"risk-free" interest rate, it implies that the terms of higher orders are never that small. They ultimately concur to induce a "crisis after a bubble" where we have some "stopping time" for the current observers. Then, quite locally in time this "term 0" would tend to grow like mad through the "bubble moment" when people start collectively think "yes! We have made a tremendous progress that opens a new era!". And all of a sudden the "term 0" turns into a very negative value, cancelling out the bubbly growth. Soon after the "term 0" gets back to its usual low value. It is quite "predictable"….
Yet our memory always works as per subjective patterns: people who went through the crisis itself remember it as an open wound, while others consider it as a thing of the past. This paradox is often referred to as "risk aversion" of "fear and greed". Although mind-boggling the formulation of Norbert Wiener relative to the "simplicity" of our initial definition exhibits the actual challenge that lies underneath the programing in finance through the use of stochastic processes albeit "simple" in apparence.

We can now look at the famously known "Ito's lemma" that is so generically used in the finance industry to price any kind of option…. It definitely exploits the properties of Wiener processes and therefore puts a frame on our risk aversion bias. We have seen right here the "bias" of such modeling: provided we survived the shocks : they would simply turn out to be extreme samples of a generic distribution that is continuously normal over time". Yes, we did have all the information ahead of time, "in hindsight", or say in a sort of Biblical mood: " we knew what we were doing right?"

Let's now assume that a measure of risk on some invested capital that comes along with some "expected return on capital" leads to a "minimum capital requirement". In our approach here, it is a measure over a single random variable (the instant based return on capital) that follows a Wiener process (or a Brownian motion). Let's assume simply that we purchase an asset at a price of "100" with some capital. The chart below provides an intuitive image of how an asset price behavior is modeled as per such a process (this is here a geometric Brownian motion ie one where the only the relative change is Brownian since a price cannot go negative for assets: so the underlying "x" is actually "Ln(P)" where "P" is the price.)



One can notice the outer bound that is delimited by "t", or more exactly by $\sqrt{t}$

The "Ito's lemma" simply states that if one considers a function f that is simply 'almost surely continuous' (ie f is continuous except for a numerable number of points and is bounded anyway), one has over any time interval [t; t+Δt]

$$\lim_{n\to+\infty} \sum_{i=0}^{i=n-1} f(W_{t_i})\left(W_{t_{i+1}} - W_{t_i}\right)^2 \overset{L^2(p)}{=} \int_t^{t+\Delta t} f(W_\tau)\, d\tau$$

This expression sounds abstract. However, looking at "$E[\left(W_{t_{i+1}} - W_{t_i}\right)^2] = (t_{i+1} - t_i)$" we can intuitively see the ground for this equality. We will see how this "continuity almost surely" property plays out. In the end this formula simply states that as a long as the function "f" is also 'continuous almost surely" then the integral sum does not depend upon the "path" that was followed: only the end points matter over the time interval that we consider. This has crucial consequences when one tries to integrate differential equations over time and wants to factor in the "optionality" that most financial instruments convey. For the intuition you can always consider a "price" as the source of the random process and consider the "ln(Price)" as the normally distributed "x". whether we then think of some function "f" that is "almost surely continuous" or "f°ln" we still have then an "almost surely continuous" function…and vice versa because a price is never "0" by assumption. We just need to be careful on the way we define our "measure of probability" since it comes along with "ln(P)". It means that if we consider a pay-off for an option we would have initially a simple linear by segment profile based on the "price" not "ln(Price)". But we only need next to be careful with our changes of variables.

"W" indicates a wiener process. Recall that W has a variance that is worth the value of the time interval. It is a martingale ie its expectation is 0 over time. W moves randomly. Now if one considers the integral of "f" over "W", whatever path W might take over a given time interval, the average value of "f" over time is "independent for sure" of the path that W will follow IF, AND ONLY IF, one can decompose this path in as many steps as one wants on a numerable set like the set of integers. This would not work if for example there was a "stopping time" ie a moment when actually there is a "jump", ie a break in the markov process that drives W.

So in overly simplistic terms, through a monte carlo simulation that would exhibit many paths for W, when trying to estimate an unknown function "f", we would converge towards the integral average value for "f" over time provided we can generate enough sub-intervals in time or else enough simulations covering this same time interval independently so.

The term $L^2(p)$ means that the integral and the limit of the series have a difference that is absolutely convergent towards zero under the weighted distribution of W that depends upon a law "p" that is such that $E[W_t] = \int W_t p(W_t) dW_t = 0$ while $\int p(W_t) dW_t = 1$ and $\int {W_t}^2 p(W_t) dW_t$=t.

We now see that 'dW_t"features the "measure of probability", that "$p(W_t)$" features the "density of probability" and that W_t features "x". Typically this means that if we do look at the expectation of $\lim_{n\to+\infty} \sum_{i=0}^{i=n-1} f(W_{t_i})\left(W_{t_{i+1}} - W_{t_i}\right)^2$ depending upon the value of W_τ, a weighted by "p" a probability of occurrence, we will not have necessarily this absolute convergence towards the integral $\int_t^{t+\Delta t} f(W_\tau)\, d\tau$ without the "p weighting". This is why the sign "$\overset{L^2(p)}{=}$" is here: the expectation does converge in "module" or in "absolute value" terms. As we saw with the CLT proof, this equality certainly holds for other distributions "p" than the sole normal distribution. Whether this distribution "p" must be a normal distribution everywhere at one point in time "t" or whether f is necessarily well defined "continuously enough", is beyond the scope of the course. This would push us into the theory of distributions for convergence proof matters that are way, way beyond easy understanding. The rigorous proof of Itô's lemma is thus beyond the scope of this course. We will review here a rather simple proof that applies to most cases in practice.

The main idea is to prove actually that

$$E\big[I_n[f]\big] = \lim_{n\to+\infty} E_{L^2(p)}\left[\sum_{i=0}^{i=n-1} f\big(W_{t_i}\big)\Big[\big(W_{t_{i+1}} - W_{t_i}\big)^2 - (t_{i+1} - t_i)\Big]\right]^{L^2(p)} \triangleq 0$$

Because surely we have $\int_t^{t+\Delta t} f(W_\tau)\, d\tau \overset{L^2(p)}{\triangleq} \lim_{n\to+\infty} E_{L^2(p)}\big[\sum_{i=0}^{i=n-1} f\big(W_{t_i}\big)[(t_{i+1} - t_i)]\big]$ given that f is "integrable" by assumption.

We would therefore simply establish a formal equality ignoring scenarios where the "sum" or the integral might diverge on a standalone basis without necessarily getting to infinite values.

In practice one therefore needs to prove the following (with benign adjustments this works also with complex numbers), which amounts to prove the convergence as per the norm that rules the infinite series:

$$E_p[(I_n[f])^2] = \lim_{n\to+\infty} E_p\left[\left[\sum_{i=0}^{i=n-1} f\big(W_{t_i}\big)\Big[\big(W_{t_{i+1}} - W_{t_i}\big)^2 - (t_{i+1} - t_i)\Big]\right]^2 |p(W_{t_0}, W_{t_1}, W_{t_2} \ldots . W_{t_n})\right]$$
$$= 0$$

Thanks to the fact that W^2-t is a martingale for every couple of instants, the cross terms are zeroed when one develops the squared sum $\Big[\sum_{i=0}^{i=n-1} f\big(W_{t_i}\big)\Big[\big(W_{t_{i+1}} - W_{t_i}\big)^2 - (t_{i+1} - t_i)\Big]\Big]^2$. We generically develop a sum having "n" terms exactly as we would develop a simple square of a sum of 2 variables "a" and "b": $(a+b)^2 = a^2 + b^2 + 2ab$. The development is made of squares like a^2 and b^2, and "cross terms", ie ab twice. In the bigger sum above the "cross terms" carry a zero expectation. The cross terms indeed mix two instants that are distinct ie independent (the value for one are set independently from the values taken by the other), one of the two occurring before the other one: whatever happened in the prior instant, the expectation for the future one is nil anyway. This is simply due to the Markovian nature of the process mixing with the martingale nature of the Wiener process that is used here: the instants are conditionally independent. Note that any martingale verifies this property that the cross products are nil in expectation as long as they follow a Markov process and a martingale.

Let's explain this property better since this is the only difficult thing to understand. The issue is that when developing the squared sum above all the cross terms involving two different instants carry a nil expectation. How is that occurring? There is always a "future instant" and a "past instant" in the cross terms. Given the "past instant", the "future instant" distribution is defined as per rules that are independent of whatever the "past state" from the "past instant" was. The expectation is independent of the past instant due to the MDP assumption in the way rules are determining it. This expectation is nil due to the Wiener process assumptions. As to the "future state" it certainly defines a certain number of specific conditions but by definition of a markovian process the martingale expectation of W into the next "dt" remains nil, whatever the "future value" of f can be in that "future instant" before one runs an expectation on the next "dt" then. Thus one is simply left to prove that :

$$E_p[(I_n[f])^2] = \lim_{n\to+\infty} E_p\left[\sum_{i=0}^{i=n-1} f^2\big(W_{t_i}\big)\Big[\big(W_{t_{i+1}} - W_{t_i}\big)^2 - (t_{i+1} - t_i)\Big]^2 |p(W_{t_0}, W_{t_1}, W_{t_2} \ldots . W_{t_n})\right]$$
$$= 0$$

And this works because as we wrote $[W_t]^2 - t$ is a martingale and therefore $\left[\left(W_{t_{i+1}} - W_{t_i}\right)^2 - (t_{i+1} - t_i)\right]$ is also a martingale.

Let's now set a convenient bound for f as, using the fact that it is bounded anyway in absolute terms and "almost" continuous. This assumption plays a crucial role here in simplifying the proof :

$$\|f\|_\infty^2 = \underbrace{Max}_{W_t}\left[f^2(W_t)\right]$$

Here we see the difference between the assumption that the Wiener process is "continuous almost surely" whereby it is not necessarily bounded over the time period that we consider, but he function "f" that is stated as being "continuous almost surely" is bounded over any time interval. The difference here lies with the fact that the Wiener process follows a normal distribution that will render the probability of extreme value close to 0 very fast. In contrast the function "f" takes one deterministic value for any value taken by "x" each time.

Then: $E_p[(I_n[f])^2] \leq \|f\|_\infty^2 \lim_{n\to+\infty} E_p\left[\sum_{i=0}^{i=n-1}\left[\left(W_{t_{i+1}} - W_{t_i}\right)^2 - (t_{i+1} - t_i)\right]^2\right]$

We then simplify the expression above, something which we will prove converges towards 0 even when "n" tends towards infinity writing in a simpler form that:

$$E_p[(I_n[f])^2] \leq \|f\|_\infty^2 \lim_{n\to+\infty} E_p\left[\sum_{i=0}^{i=n-1}\left[\left(\Delta W_{t_i}\right)^2 - (\Delta t_i)\right]^2\right]$$

Or

$$E_p[(I_n[f])^2] \leq \|f\|_\infty^2 \lim_{n\to+\infty} E_p\left[\sum_{i=0}^{i=n-1}(\Delta t_i)^2\left[\frac{\left(\Delta W_{t_i}\right)^2}{\Delta t_i} - 1\right]^2\right]$$

In the equation above, only the term "$\left(\Delta W_{t_i}\right)^2$" $is\ random.$ Hence our problem has become:

$$E_p[(I_n[f])^2] \leq \|f\|_\infty^2 \lim_{n\to+\infty}\left[\sum_{i=0}^{i=n-1}(\Delta t_i)^2 E_p\left[\left(\frac{\left(\Delta W_{t_i}\right)^2}{\Delta t_i} - 1\right)^2\right]\right]$$

Note here that the last move of the proof is made possible thanks to the fact that we assume that we can deterministically cut the time interval independently of the path followed by W. This amounts to say that we exclude the occurrence of an " infinite jump" or a stopping time. There is a subtlety here between "unbounded jumps" and "infinite jumps": on any sub-interval there is no bound to the possible jump while the density of probability of the Normal distribution makes them more and more likely very fast, but any jump is "finite" anyway. At no moment an "infinite jump" is possible: for sure the probability here is 0. The normal distribution thus makes the "Sigma" converge in "Norm" which secures an absolute convergence. Otherwise we could not move the "Expectation" within the "Sigma".

We can them simplify further the expression. The process $\frac{\left(\Delta W_{t_i}\right)^2}{\Delta t_i} - 1$ is a martingale. Thus we know that its expectation is 0. What about its variance, since this is what this is all about here? Whatever the interval $[t_i; t_{i+1}]$ that is considered we know that ΔW_{t_i} follows a normal law with mean "0" and

variance "Δt_i". Thus we should consider the random variable $x = \frac{\Delta W_{t_i}}{\sqrt{\Delta t_i}}$. This "x" in every case is obeying a normal law with a variance of 1. And we need to compute the variance of the variance of "x" as per the normal law. Whatever the result, it will be the same for all the intervals being considered….

$$E[(x^2-1)^2]_{\sim\aleph(0;1)} = \int_{-\infty}^{+\infty}(x^2-1)^2\frac{1}{\sqrt{2\pi}}e^{-\frac{x^2}{2}}dx$$

The computation is rather simple when developing the polynom in "x" since

$$\int_{-\infty}^{+\infty}x\frac{1}{\sqrt{2\pi}}e^{-\frac{x^2}{2}}dx = 0;\ \int_{-\infty}^{+\infty}(x)^2\frac{1}{\sqrt{2\pi}}e^{-\frac{x^2}{2}}dx = 1$$

So,

since $(x^2-1)^2 = x^4-2x^2+1 \Rightarrow E[(x^2-1)^2]_{x\sim\aleph(0;1)} = \int_{-\infty}^{+\infty}(x)^4\frac{1}{\sqrt{2\pi}}e^{-\frac{x^2}{2}}dx - 1 = 3-1 = 2$

(the result "3" is found through integration by parts of $(x)^4\frac{1}{\sqrt{2\pi}}e^{-\frac{x^2}{2}} = (x)^3\frac{x}{\sqrt{2\pi}}e^{-\frac{x^2}{2}}$)

So, back to our bounding problem: $E_p[(I_n[f])^2] \le \|f\|_\infty^2 \lim_{n\to+\infty}\left[\sum_{i=0}^{i=n-1}(\Delta t_i)^2."2"\right]$

Thus if we have divided "rather regularly" the interval length Δt into "n" sub time-steps rather homogeneously, then we can write that any time-step has a length that is below a finite multiple K of Δt, So we write that $\exists K \in \mathbb{N}, \forall i\epsilon(0,1,2\dots n)\ \Delta t_i < \frac{K\Delta t}{n}$ and therefore:

$$E_p[(I_n[f])^2] \le \|f\|_\infty^2 \lim_{n\to+\infty}\left[\sum_{i=0}^{i=n-1}(\Delta t_i)^2."2"\right] \le 2\|f\|_\infty^2 \lim_{n\to+\infty}\sum_{i=0}^{i=n-1}\left(\frac{K\Delta t}{n}\right)^2 \le 2\|f\|_\infty^2 \lim_{n\to+\infty} n\left(\frac{K\Delta t}{n}\right)^2$$

The squared term in n provides then a rather straight conclusion since we end up with:

$$E_p[(I_n[f])^2] \le 2\|f\|_\infty^2 (K\Delta t)^2 \lim_{n\to+\infty}(1/n\ )\ ie\ 0.$$

Please notice that the continuity assumption underneath the Wiener process matters a lot since this is what entitles us to state that as n goes towards infinity there always is this ability to strip any time interval into arbitrarily small and homogenous sub-intervals. Otherwise the conclusion here is not possible. **If there exists a finite number of time-steps that are not infinitely small when n becomes infinite then the convergence is not established. This is typically the case in a "jump" situation.**

> This approach applies on VaR (Value at Risk) or any other quantitative risk measure. It also applies to derivatives pricing like "Options" (Black& Scholes model).

3- Black&Scholes and more

In general an investment is a question of "reward", ie the expected return in the favorable conditions, against the "risk", ie the expected return in adverse scenarios. A prevailing assumption in financial markets consist in saying that, thanks to the "information" that flows through the prices as people trade and trade again the price fixing mechanism that leads every minute, every second, every millisecond in crypto markets, to a bid-offer price (one price to buy and another price to sell, that is supposed to be testimony of a "balanced market" ie a market between buyers and sellers that

is in equilibrium. Yet, this balance swings every milli-second in the most active markets. This is a way to picture a Wiener process with few exceptions: the mean is not zero but some interest rate that is labeled as a “risk-neutral” rate, the variance is not “1” and, crucially, this variance is not always stable. The instability of the variance is called “volatility” and it is often a precursor to a coming crisis when this “volatility” rises. Why is there “volatility”? answer: market players always act in anticipation and as such speculate against a current status quo: why wait for buying a stock if we all know that the company will make a fortune? So people “buy” ahead of time financial assets in expectation that they might get more than the “risk neutral” rate. But the “equilibrium” principle states that at every millisecond the “information” of this anticipation is factored in the prices already. The best evidence of that is when a big “news” comes out in the media that people could have expected, the market reaction is to “sell the news”: in practice it means that if a big positive news that you and I had not knowledge of about a company A, right after the moment the news is out in the media outlets and has therefore become accessible to all of us, the share price rather than go “up”, is actually heavily sold out. Why is that? Simple the market players had been hunting for validation of this future news weeks of months before and had speculated that it would become reality. Up until the day of publication that was just a more or less well founded “speculation” since it was not “official”. Over time more and more people would convince themselves that indeed this big positive news about company A would pop in the media one day and, projecting the impact on the share price, they would buy and buy again quite often as the clues confirmed it ahead of the D-day.

So, right before the big positive news, because the genesis of this outcome takes time and because markets are constantly “informed” through the “bid-offer” price changes after each prior trade, the market is net “long” the share of this company A ahead of time and offloads the share right after the news if nothing really new pops out then. People speculated and take profits on the news. Of course the opposite goes about a big concealed negative news on some other company B. This why one would observe a certain rather low variance of the share price ahead of the news, a short lived spike in the variance right after the news and likely a reversion to the historical mean of the variance. In the meantime the share price of company A will have outperformed the risk-neutral rate that prevails on average across the many financial assets, some going up, others going down more or less. All this is translated mathematically through the lens of the Wiener processes with the caveats mentioned above. As we can see the real life either induces “variance” or, and, “volatility”: it all depends upon your perspective when you are a market player.

Of course, if you have a perfect information you would still face temporary p&l losses, also called “draw-downs” that would mean that, you might have to consume some of your capital to keep the position waiting for better future outcomes. But, if you have made a appropriate prior assessment of the size of the “risk” here that you can take and still hold onto your position, you prepare for yourself a set of “options” to gain more than the “risk-neutral” rate. Market players usually provide soma “value added” services on the matter that consist in achieving better returns that the “expected risk-neutral rate”. The word “expected” is added in the last sentence because as you can see, in crisis times, most asset prices are down and therefore even the safest investors face under-par returns. While “reason” would provide a clear hint as to what the optimal course of action should be, right in the trough of a crisis, investors run usually with a deeply depleted capital cushion and typically will opt out for even safer assets…which amounts to lock the current loss for most pat.

One may argue that if investors factored in the fact that bull markets tend to be “trendy” ie they deliver a return that is clearly higher than the risk-neutral rate, which fuels speculation and therefore “bubbles” and mechanically “crisis” as corrective phases that would labeled as “bear markets”, investors should not be so surprised and thus not so compelled to liquidate their riskiest positions at

the worst moment in time. This is actually what the markets do and so investors as a group too: the "implied volatility" that is priced in the option markets is most of the time way higher than the actual observed variance: the implied volatility is usually close to the observed variance and is often mixed up. Yet, while they both refer to the fact that the prices and therefore the short term returns are random, subject to variance and expectations, the variance is measured on the past, while "volatility" refers to the fact that this variance in the future is itself random (ie we are NOT in a random distribution of the returns anyway). Thus is no such thing as a "risk-free" rate in practice for investors simply because they all take risks in order to try and beat the risk-neutral rate.

On safe way to take risks without exposing too much of the capital, ie securing that even in the trough of a crisis, investors have full control over their remaining capital, is to buy options which define from the very beginning how capital is set at risk. This imply though a "passive investment strategy" whereby an investor purchases an option on some underlying asset price either a "call" if the option exercise price is higher than the spot price, or a "put" price if the option exercise price is lower than the spot price. Thus investors may even be able to implement defensive strategies relative the eventual "next crisis" to come. However markets are trendy 90% of the time which means that defensive strategies are broadly neglected over the long run in favor of "bullish" strategies that leverage the upward trends in prices when markets are optimistic.

To be sure at this stage, here are some jargon:

"bull market": tendency of markets, more or less clear, to move asset prices higher and higher across asset classes

"asset classes": bonds, stocks, forex (foreign exchange ie currency cross-rates), ETFs, crypto-tokens, CDS, futures (expecially on raw materials, commodities, energy sources like oil or gas, precious metals , options, preferred shares, ABS tranches, MBS tranches, CMBS tranches, CDOs tranches, CLOs tranches, CDS, synthetic CDS tranches, CDS indices etc….

"bear markets": relatively short lived periods through which markets correct all the way down the past excesses of the last bubble to date or else plunge due to a major economic crisis, or, worse, a financial crisis.

"bid-offer price": typically in markets, people never trade at "mid price" which however is the one price that accountants and regulators look at for the "mark to market" that define the legally binding valuation of the positions that an investor like a bank or an insurance company have to produce every quarter. In practice, market makers provide a price to an investor who wants to "buy" an asset: this is the "offer" price. And they also provide a "sell" price to an investor that is the price at which the investor can sell the asset to the market maker: this is the "ask" price.

Let's now notice the relation that we have between "Ln(P)" and the relative return: if we consider relative variation of an asset price over a small time interval "dt" (mathematical and physical definition) we would say that the return is $\frac{dP}{Pdt}$ it happens to equal $\frac{d[Ln(P)]}{dt}$. In order to make it crystal clear relative to "real life", let's now consider a "short period" of time through which we have an "open price" and the "closing price" for this short period that we would write as $\delta t = t_{close} - t_{open}$, then we have a price P_{open} and a price P_{close}. Note that here we assume that both prices are "mid-prices" based on some hypothetical "bid-offer" pair of prices where we retain the mid value at the "open" of this period and at the "close" of this same period. Over the period we would consider the "middle price" that is

$$P_{middle} = \frac{1}{2}\left(P_{close} - P_{open}\right) = \frac{1}{2}\left(\frac{\left(p_{close_{offer}} + p_{close_{bid}}\right)}{2} - \frac{1}{2}\left(p_{open_{offer}} + p_{open_{bid}}\right)\right)$$

Usually data sources provide only the "mid values" not the "bid-offer", so this is purely explanative here.

One can see that if the period of time is sufficiently small so that the historical data compounds a high number of such consecutive time periods, we can use the assumptions relative to Wiener processes. This is the practice in the financial industry since the normal distribution is the only statistical distribution that offers tractable models. The word "tractable" means that we can develop rather easily measures of "correlation", but also we can easily deploy numerical resolution of differential equations that, for example, allow us to put a price on "options".

So, we can run the following approximation: $\frac{dP}{Pdt} \approx \frac{P_{close} - P_{open}}{P_{middle}\delta t} \approx \frac{\delta\left(Ln(P)\right)}{\delta t}$. This will be instrumental in all the following development of the differential equation that allow to "solve" the price of an option of an asset. All the future "dt" will actually refer to "δt" and the surrounding context.

"Options and strike price": the strike price is the price beyond which (higher for call options, lower for put options) the option has an "intrinsic" value ie at expiry the owner of the option receives some windfall money that represents a linear leveraged value of the difference between the market price and the strike price at the expiry of the option. Thus, for a 3 months call option that has a strike price at 110 when the current asset price is at 100, there is a non nil possibility that, within the next 3 months the asset price rise to say 125. It this is the price after 3 months then the investor will pocket "125-110=15". Also if the asset price at expiry ends at 135, then the investor pockets 135-110-25. The "pay-off"' function as you can see is "linear": this is the case for the "vanilla" options. More complex pay-offs exist but they derive from this simple model. If instead the price ends at 100 or below the investor pockets nothing but loses nothing. So this is the "option" where the investor would only win at expiry. Of course this "option to win or else not lose" is too good to be true: it has a price in itself: so if the investor at expiry pockets nothing because the asset price did not "cross" the strike price, the investor pockets 0 from the pay-off mechanism but he has lost the "premium" ie the price the investor paid in the first place to own this option. So, if say the option premium was "2", the investor paid "2" an therefore is immediate economic capital his is "-2". At expiry, if the asset price is at "102" then the investor pockets "2=102-100" from the pay-off but this only balance the initial disbursement of "2". Hence the investor has made no money while the investor was rather "right" on betting that the price would go up. If the expiry price is "101" then the investor pockets "1" but, paying "2" initially, the investor has lost"1"("1-2=-1"), while having the view right that the price might go up. So this security about securing always the right amount of capital ahead of crisis times that are rare is not a strategy that is profitable in general. Why would the last situation occur? Because the markets will factor in the option prices not the current price variance but "more" ie the "volatility" that was pictured anecdotally before with the case of company A stock prices.

Indeed it is logical that the more variable a price is the higher the chance that it happens to cross the strike, even before the expiry time: Then the investor might opportunistically sell his option out before expiry because then the option premium would have gained in price already: thus the investor may have purchased the option at 2, the day after the price moves up to "103": the distance to the strike "110" has shrunk by 10% which makes the option value increase to some 4 value. So, the investor then, one day after his purchase, that if he gets out now he sells at "3" the option" that he purchased the day before at "2". He pockets a gain of "1=3-2" but then he gives up on making

money betting that “P” would keep rising. On balance he knows that this price is volatile and might end up at “103” at the expiry of the option: then he anticipates that he will have made “1” while he has now the opportunity to lock “1” already instead of waiting for 3 months less 1 day. He then can focus on other opportunities and avoid the advent of a crash that would turn his current bet eventually to a loss of “2” as we saw since then P would likely end below 100 anyway.

Of course the more variable, or ”volatile”, the price P is, the higher the chance that the investor has this opportunity now and may be later. So the option premium factors that volatility in. We can see that this “volatility” feels like the “variance” of the price but it also factors in the surrounding speculation that is made in anticipation by the market players. Thus in general if a price is “trending up” the option prices will factor in an “implied volatility” that is way higher than the observed price variance: it is easy ti see “why” here. If you are this investor that is already at profit and knows that the price tends to “trend up”, he is going to keep the option even if the trade is already at profit because he expects the trend to go farther. Waiting for even better prices P over time, he will get closer and closer to expiry where anyway the “time value” of his option will converge to 0. An expired option has to time value left since it is expired. So, it is not possible for very long that such options are “free lunches” for investors and therefore “the implied volatility” tends to quote higher than the observed past variance through option prices, especially in bull markets.

The market makers and traders on options would consider the options as an asset class and trade actually on the evolution of this implied volatility. In other words they will “hedge” their direct exposure to the variations of the price P and only target profits on the evolution of the implied volatility alone. We can conceive now that, for a fixed price P and a fixed strike, the option “time value”, ie the “premium” that investors would pay for the option itself, is going to increase with the implied volatility. Thus option traders will compute, with models derived from the Black&Scholes formula that we will study here, what the “delta hedge is” so that buying a 3 months call option at 110 that would vary at a current price P of 100 would imply a delta of say 33%. We saw formerly that if the price “tomorrow” goes up to 103, the option price moves from “2” to “3” ie “2+33%*3”. We see here that, using an appropriate model a trader on options can predict how the option price will move as a consequence of the variations of P. So the option trader, might sell the option to the investor and buy 33% the asset in front. Thus the day after, the trader has not lost money since he lost on the option leg” given that he sold at “2” the day before what is now worth “3”. He lost “1” here. But ha also delta hedged his option sale by buying 33% at P=100 ie it was worth 100*33%=33 that now is worth 33%*103=33.99 ie 33+1. So net net the trader is flat in P&L (forget about the 0.01% difference). Now, since this move occurred, investors will tend to buy more of these options and the option traders will ask a higher time value, or premium for the future investors to buy more options. So the implied volatility will likely rise higher after this move even if the price P actually might always trend as it used to. Then the trader on option will face some uncertainty: if the price P keeps trending in more auto-correlated moves, the investors will make even more money and option traders would lose but they also would lose little thanks to their delta hedges that cover most of the rise. Likely the implied volatility will rise despite the fact that the variance would barely move….up until the moment when the price P when too high too fast and corrects down. Then the option traders would lose on their delta hedges but the implied volatility would tend to crash down putting the option traders at the net gain since the latest rise in prices cost them nothing. Imagine the price P rises, rises and then reverts to 100. Then the option traders will have lost the variance that P realized through their delta hedge but they will gain the “implied volatility” that they sold to investors that is structurally higher and higher in such situations.

In the end we could say that the option traders bet on the implied volatility while investors prefer to "pay for the security of their capital" by buying these options: The traders therefore earn the difference between the implied volatility that investors pay for good reason and the realized variance. Of course the traders notoriously suffer when a crash occurs. They will cover against this outcome by buying cheap "far out of the money" puts ie put options that have very low strikes which will cover the number adverse scenario for them: a surge in the implied volatility when prices fall heavily. But most of the time, option traders will favor "flat books" ie they would tend to promote highly "barbelled" option books where they sponsor call offer prices at strikes and tenors (3 months, 6 months, 1months etc…)that are close to calls that they originally sold to investors. In other words, they would play with their "bid-offer" quotes in order to have "long-short-long-short" exposures across the many tenors and strikes. They would only "tail hedge" with far out of the money puts the residual exposures. So at the end of the day they would bet on the direction of the implied volatility indirectly so, favoring the gains that they will make by simply having investors sell on the trader's bid price and buy at the trader's offer price. Then they lock the bid-offer.

This is the most profitable strategy by far in the markets: force players to "hit the bid" or "lift the offer" and this is why all banks have innumerable "market making desks" why the really profitable business in "crypto markets" also is being a " broker" that offer bid-offer quotations 7/7 24/24 across the planet. The buy and hold investor who would buy and wait until expiry usually will pay 5-10% of the ultimate return to be able to "trade". The active investor like a "value based" hedge fund that trades weekly or so would pay "20%" of the future return. The "daily" trader, in hindsight, would leave 40-to 60% of its future return depending on whether the investor has a privileged access to execution platforms and information in general. The most successful business models in trading to date consist in have computers trading on the nano second to actually "steal" the bid-offers from traditional market makers: but this implies investing billions in super fast machines and algorithm designs to pare against "slippage" and "deadlocks" and hunt for "crossed markets".

In order to hammer the point: due to this "volatility" that is inherent to the very functioning of financial markets and human psyche , the options are an astute way to mitigate the risks: indeed one would either just buy the option and wait until expiry: either the strike price of the option is crossed and the option then can have a great value, way higher than the premium that was paid to own this option itself. Or the strike price will not be crossed and the option value at expiry will be worth 0. Then the investor will have lost the money that serve to pay for the option premium at purchasing time. So, either the investor makes eventually a "fortune" if the strike price is crossed or the investor lost money for good, but paying the implied volatility the investor secures a predictable outcome in a crash scenario. This explains why the price for this insurance rises crazily during financial crisis and is crazily depleted when markets are optimistic (90% of the times).

The whole definition of the equation that we will study is based upon in crucial assumption: we can approach a closely as we want a "jump" in value that is always going to be of finite value. From there, we reduce the dynamics to first orders in "x" and "t" knowing that the variance of "x" is fully represented with "a dt" term…..How do we get to this conclusion?

We assume that we deal with a Wiener process (aka Brownian motion) and we look at the "variance" over some infinitesimal time period and the fact that the integral of a function over time will behave intuitively…which can only be warranted if our crucial assumption holds whatever the time interval δt that is all the same "arbitrarily small " and yes "finite" we can slice it virtually into infinitely small time sub-intervals and the integral reflects appropriately over this time interval "δt"the aggregate variation of the function, on "expectation", "almost surely" so. Below the "d" that points to the "derivative" in the coming differential equations relies on this conclusion that the lemma of Ito

secures. As a result this implies also that for any such function that is "continuous almost surely" we can define it as the primitive over "time" of another function that is also continuous almost surely". From this we will in.

Let's now consider a process X that is random with a mean μ and a variance σ relative to a Wiener process. This is how typically in finance we model a financial asset price P through its " almost instantaneous return" over δt , and here notation wise we set $\delta t = dt$…

Developing the $dX^2 = dXdX = (\mu dt + \sigma dW_t)(\mu dt + \sigma dW_t) = \mu^2 dt^2 + 2\mu\sigma dW_t dt + \sigma^2 dW_t{}^2$

We can see that the first term $\mu^2 dt^2$ is infinitesimally small versus any "dt" term. The second term is nil in essence since our random process is actually "random" ie unrelated to the arrow of time and centered on 0. Therefore: $dX^2 = \sigma^2 dt$ remains the predominant term in the end….

We can infer the differential of "f" (consider that "f" might be the price of some vanilla option, or else the stress VAR-related required capital, or else the expected loss of a tranche price as a function of normal latent random factor X) based on the underlying Wiener process. Here we develop the classical approach but on the follow we provide a more generic resolution method through the Fourier transforms and eventually the Hermites' functions.

We are going to develop now the core differential equation: a concise way to recall what the notation "dt" or the partial derivative signs like "$\frac{\partial \dots}{\partial t}$" point to is "on expectation on some arbitrarily small consecutive time intervals $\delta t = dt$" we have from the Taylor-Young formula:

$$df = \frac{\partial f}{\partial t}dt + \frac{\partial f}{\partial X}dX + \frac{1}{2}\frac{\partial^2 f}{\partial X^2}\underbrace{dX^2}_{\sim \sigma^2 dt} \underbrace{+ \frac{1}{3!}\frac{\partial^3 f}{\partial X^3}dX^3 + \cdots \frac{1}{n!}\frac{\partial^n f}{\partial X^n}dX^n + \cdots}_{negligible\ in\ \mathrm{dt}\ term}$$

The terms or higher order can be deployed and may be significant if we are not talking about the normal distribution the term "$\frac{1}{n!}$" is key here in comparison to the expectation (ie mean value over $\delta t = dt$) of dX^n. We saw that to prove the lemma of Ito p, for the order "2", we needed the moment of order "4" on our Wiener process and it was worth "3". What about the moment of order "n" then? We only needed to prove comfortably the convergence in the ultimate step when we look for some upper bound. At one point then, if we want to "prove" the relevance of higher moments, we need to consider the integral:

$$\int \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2}(z^n - 1)^2 dz = M_{2n} - 2M_n + 1\ with\ M_n = \int \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2} z^n dz$$

$$M_n = \int \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2} z^n dz = \left[-\frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}z^2} z^{n-1}\right]_{-\infty}^{+\infty} + (n-1)M_{n-2}$$

We have to consider separately the case of odd indices(n=2p+1) and the case of even indices 'n=2p) and write then $M_{n=2p} = (2p-1)(2p-3)\ldots.1 = \frac{n!}{2^p p!}$ and $M_{n=2p+1} = 0$

Thus in general the order of magnitude, as an expectation, for $\frac{1}{n!}(dW_t)^n$ is:

$$\frac{1}{n!}dX^n = \begin{cases} 0\ if\ n = 2p+1 \\ \frac{1}{2^p p!}\ if\ n = 2p \end{cases}$$

If “p=1” this is fine, we retrieve the classical expression that we will use. But already when p=2 we get a division by $2^p p! = 8$ and for p=3 the division factor becomes 48. Next, deploying the expression where every odd power law on dW_t for any symmetrical distribution including the normal distribution:

$$dX^n = dX \dots dX = (\mu dt + \sigma dW_t)^n = \sum_{k=0}^{p} \frac{n!}{(2k)!\,(n-2k)!} \mu^{2k} (dt)^{2k} \sigma^{n-2k} (dW_t)^{n-2k}$$

We can see that only the term with “k=0” can matter given that $\delta t = dt$ is arbitrarily small:

Therefore: $\frac{1}{n!}\frac{\partial^n f}{\partial X^n} dX^n \approx \frac{1}{2^p p!}\frac{\partial^n f}{\partial X^n} dt$. here we need to recall that , while the final “pay-off” function might not be a smooth function at all, we will assume that our “option price” function will be a smooth function up until expiry, ie its consecutive derivatives tend towards zero in norm when “n” grows to infinity. In other words, we will solve the differential equation into the set of smooth functions “f”. This is a relatively important point since “barrier options” require more care that “vanilla options”. But the ground work is the same.

So usually we keep only the first terms….

$$df = \frac{\partial f}{\partial t} dt + \frac{\partial f}{\partial X} dX + \frac{\sigma^2}{2}\frac{\partial^2 f}{\partial X^2} dt + 0 \dots$$

$$df = \frac{\partial f}{\partial t} dt + \frac{\sigma^2}{2}\frac{\partial^2 f}{\partial X^2} dt + \mu \frac{\partial f}{\partial X} dt + \sigma \frac{\partial f}{\partial X} dW_t + 0$$

$$\boxed{df = \left(\frac{\partial f}{\partial t} + \mu \frac{\partial f}{\partial X} + \frac{\sigma^2}{2}\frac{\partial^2 f}{\partial X^2}\right) dt + \sigma \frac{\partial f}{\partial X} dW_t + 0 \; equiv \dots}$$

We are quite “close” to the Black&Scholes model that focuses on “prices” which cannot take negative values in general. So usually we would write that in our case $X = Ln(P)$.We then say thet “P follows a log-normal process ie $dX = \frac{dP}{P}$ which implies that $\boxed{\mu_X P dX|_{dt} = dP|_{dt}}$ and $\boxed{P\sigma_X dW_t = \sigma|_{dW} dW_t}$…. If instead of writing the stochastic differential equation with a usual “normal” random process “X” we actually write the same equation with a “log-normal” random process, we indeed would simply move $\sigma \to \sigma X \; and \; \mu \to \mu X$ and assume thus that the “relative variance” and the “relative drift” are constants hopefully so. All this is NOT quite equivalent to assume that “Ln(P)” per se depends directly on some normal random variable: we just stick to the “mean” and the “variance” over some arbitrarily small time interval δt.

The next step will consist in ‘adding’ a layer of modeling by stating that we <u>actually</u> have log-normal” factors “, namely "σ" rather is σX with hopefully this new relative σ=Cst and $\mu X \; shows \; up \; instead \; of \; \mu$.

This yields the famed equation $\boldsymbol{df = \left(\frac{\partial f}{\partial t} + \mu X \frac{\partial f}{\partial X} + \frac{\sigma^2}{2} X^2 \frac{\partial^2 f}{\partial X^2}\right) dt + \sigma X \frac{\partial f}{\partial X} dW_t + 0 \; equiv}$:

From now on we will swap the “X” with a more intuitive “P”, that stands for “Price”.

<u>Typical resolution</u>

We thus start as follows, adopting the most popular notations in finance….

it is based on a “price V” of the option in question depending on the price P of the underlying asset. The delta neutralizes the derivative of V versus P. Thus the term in $\sigma P dW_t$ is neutralized. Doing so

the terms in μ disappear too. Remember that $dP = PdX = \mu Pdt + \sigma PdW_t$ and that the option V is a smooth convex function of P where we neutralize the dependence on $\frac{\partial V}{\partial P}$, rendering a "net exposure" that should reward us with a continuous "yield" that is "equivalent" across the markets: this is the "risk neutral" rate that we described formerly in order to set the stage for the use of an "option based investment strategy"…People would say "there is no free lunch…if you take no risk, then you only deserve to get the risk free rate, whatever the instrument you trade". Whether such a "risk-free" rate or even a "risk-neutral" rate ever existed is a philosophical question!

As we pictured in introduction to this part, some investors use options rather passively to take a "bet" while securing from the start the maximum loss they might endure. Option traders would sell the option to this investor, delta hedge their risk with regards to the evolution of the price P and actually bet on the outright level of the volatility. How they would eventually "hedge" their own bets is another topic. Enough is to know here that they customarily "barbell" their risks, accrue some opportunistic tail hedges and "sell the implied volatility" versus the realized variance of the price. Clearly we will assume the existence of some "equilibrium" in the markets where the spot volatility reflects that neither the "buy and hold" investor not the "option trader" have a "free lunch". In other words the delta hedged option strategy will not yield more than any other asset class on δt which is the "risk free rate" (assuming that there are risk-free assets like Treasury bonds for example), or the "risk neutral" rate. This is the core pricing principle that we will deploy now from the initial stochastic differential equation that we just established. We need to focus on one practical aspect of this "delta hedging" strategy: from "t" to "t+ δt" the market will change but the trader will only "know" the values and derivatives in "t", being kind of "blind" until t+ δt comes.

So we start with giving a name , "r", to this "risk free rate" that we will rather call a "risk neutral" rate

$$dV = \left(\frac{\partial V}{\partial t} + \mu P\frac{\partial V}{\partial P} + \frac{\sigma^2 P^2}{2}\frac{\partial^2 V}{\partial P^2}\right)dt + \sigma P\frac{\partial V}{\partial P}dW_t = rVdt$$

and on the follow we set the delta-hedge ratio as: $\mathrm{P}\left[\frac{\partial V}{\partial P}\right]_{fixed}$ and

$$d\left(\mathrm{P}\left[\frac{\partial V}{\partial P}\right]_{fixed}\right) = \mu P\left[\frac{\partial V}{\partial P}\right]_{fixed}dt + \sigma P\left[\frac{\partial V}{\partial P}\right]_{fixed}dW_t = r\left(\mathrm{P}\frac{\partial V}{\partial P}\right)dt$$ at first order

Note that the hedge as such is also perceived as an "asset" that can only yield on average the risk-neutral ratewe assume $\frac{\partial V}{\partial P}$ is actually "fixed" in "t" and would not change through to "t+dt", hence the notation $\left[\frac{\partial V}{\partial P}\right]_{fixed}$). We therefore indeed use the stochastic part and land on the seminal equation by considering the "portfolio" made of the option and its delta hedge , namely $V - \mathrm{P}\left[\frac{\partial V}{\partial P}\right]_{fixed}$.

The "portfolio" then follows the equation:

$$dV - d\left(\mathrm{P}\left[\frac{\partial V}{\partial P}\right]_{fixed}\right) = \left(\frac{\partial V}{\partial t} + \mu P\frac{\partial V}{\partial P} + \frac{\sigma^2 P^2}{2}\frac{\partial^2 V}{\partial P^2}\right)dt + \sigma P\frac{\partial V}{\partial P}dW_t - \mu P\left[\frac{\partial V}{\partial P}\right]_{fixed}dt - \sigma P\left[\frac{\partial V}{\partial P}\right]_{fixed}dW_t$$
$$= \left(rV - \mathrm{P}\left[\frac{\partial V}{\partial P}\right]_{fixed}\right)dt$$

Please notice that we will be able to relax the notation $\mathrm{P}\left[\frac{\partial V}{\partial P}\right]_{fixed}$ on the follow because now we will have a deterministic equation where the drift of our random variable disappears:

$$\frac{\partial V}{\partial t} + \frac{\sigma^2 P^2}{2}\frac{\partial^2 V}{\partial P^2} = r\left(\mathrm{V} - \mathrm{P}\frac{\partial V}{\partial P}\right)$$

$$\boxed{\frac{\partial V}{\partial t} + r\mathrm{P}\frac{\partial V}{\partial P} + \frac{\sigma^2 P^2}{2}\frac{\partial^2 V}{\partial P^2} - rV = 0}$$

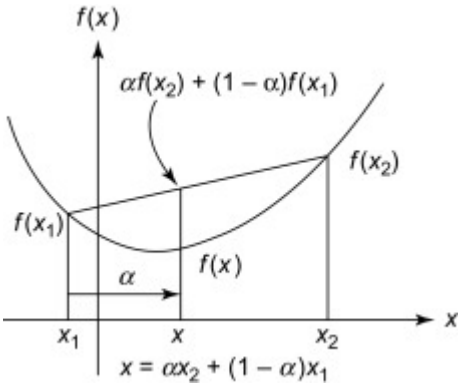
What could be the "point" of delta-hedging so that we actually take no risk or almost no risk?

We formerly approached the question from a pure risk taking standpoint: a cautious investor, anticipating a possible loss would prioritize the need to cap the maximum loss in capital he/she may face on this bet. Alternatively we saw that the ultimate liquidity provider to this investor would be a trader that had rather "bet" on the implied volatility price that is baked into the option price itself. Thus both parties trade on the option price but with independent strategies with regards to their respective capital.

Now the question can also be analyzed from pure risk management standpoint. How is the option market maker actually managing day to day the option exposure? How can we answer then?

The short answer is "because we actually take a risk on the level of implied volatility and risk-neutral rates altogether….because, truly, there is no free lunch and therefore there is not even a "rate that is risk free"….What is happening with this "portfolio" made of an option against its risk-neutral delta hedge?

Intuitively, the whole concept of "Time Value" lies in the chart that stands below and exhibits the "convexity" or the "gamma trading gains" that come along with "owning" such an option alone post any significant market move. The chart below exhibits the value of the option that moves as per some hyperbolic profile or close :



Comment on the chart above:

One can see that the delta hedge follows the tangent lines while the option value outperforms any time the market prices moves significantly. Then, whenever the price makes big ups and downs, one would adjust the delta hedge ratio at profit…which in theory shall be balanced by the value of the option, that in itself sinks down to 0 over time. On balance therefore, a careful risk manager adjusting the hedge ratios at profit all along the lifetime of the option, will record trading gains, aside from the associated execution costs, BUT will lose it all and more because the option time value will have shrunk to zero at expiry time. We formerly commented on the fact that the time value is computed not on the basis of the actual and predictable price variance but on some higher value (99% of the time) that is called the "implied volatility": you can see this value as the volatility of the price that is "implied" in the option price as time value. So in general the option traders sell the

options and therefore will make money due to the difference that is constantly in their favor when they sell the implied volatility and they have to pay losing delta hedging adjustments as per the actual price variance. Why would buy and hold investors pay this extra value? Answer: this is the insurance price they have to pay when choose an option rather than face the direct variance of the price itself. The convexity above of the option price as a function of the underlying asset price looks attractive since it feels like one always makes money when markets move whichever direction they take. Most of the time this is a losing strategy because the difference between the implied volatility that is then paid versus the gains extracted from the price variance is , on average, in favor of the option seller. It must be the case since otherwise the option sellers would always lose money and therefore go bankrupt. This strategy wins at times when, over time, the implied volatility has grown while one owns the option or when the rates decline…in general… To be successful this strategy when one buys an option and actively delte-hedges it, is equivalent to go "long" the implied volatility value. This strategy also implies that if one wants to lock gains for sure, one has to take profit before the expiry unless the option is deep in the money already. This means that one has to have the timing about right.

How do we "solve" for such equation as the one that we empathized before this comment?

We use a Laplace transform or a Fourier transform and obtain the formula after a couple of changes of variables and an inversion of the time scale (since the conditions are set in T through the pay-off of the option, not 0). We indeed form a differential equation on "t" for the integral transform. The underlying equation is the one of the notorious heat diffusion. So we inherit a natural solution that fits with the normal distribution. We will see later in more detail how we get the general resolution…

Then the fundamental issue we come across relates to facts:

- There is no such thing as a "risk less rate" and even a "risk neutral rate" is just a convenient arbitrary concept
- For sure though in practice, one has to rebalance the "hedge ratio" and this for sure costs money because the liquidity is never perfect and markets are NOT efficient

If we want to sum it up in light of the silky standard model of Black&Scholes we should write:

$$df - \Delta dP = r(f - \Delta)dt - d\Delta(cost_{of_{trading_x}})$$

Notice the "-" sign on the right hand side of the equation: the trading costs (<0) come to "reduce" the "time value" of the option right? Indeed the time accrues at the pace of the rate "r" over "dt". So, if trading in and out the delta hedge that is one part of this "portfolio", the time value of the option has to factor this cost in. Hence the presence of trading cost as a depletive factor for the time value of the option itself. The theoretical risk-neutral rate generates this "no arbitrage principle" that induces a discount of the "time value". This "time value" itself builds up over time due to the convexity element that is pictured in the chart above, ie there is an even higher time value for an option that is priced in low rates or over longer tenors (time to expiry of the option)

Thus a higher rate induces a higher discount for the present value of the future gains. The principle on this consideration is that you always have a choice between investing in "bonds" and earn this rate with little risk or put this money instead on this option here and take a risk. On average, market prices factor in the fact that no counterparty on an option trade is favored other than from the upcoming short term future that no one knows yet: they both would earn this "risk-neutral" rate. The option value must therefore be discounted at the risk-neutral rate.

Thus, just keeping the option and doing nothing other than adjusting the deltas, your money returns the same rate as a bond would do for you. Thus the higher the rate in question, the lower the present value of the option. Likewise the trading costs have the same effect: the higher they are the more the option present value has to be discounted. So, to be clear "-cost" must be a positive value and have a similar role to what the "risk neutral rate" does on the option present value in "t" (V in the equation). It is just as if the option price looks over discounted…while in fact it factors in additional costs. This explains "why" on some illiquid stocks or assets, the options look quite "cheap given the rates on bonds": they simply rely on volatile and illiquid assets where you will pay high execution costs when trying to rebalance your hedge-ratio. It may sound counter-intuitive if now we think of a typical option trader whose business is precisely to "sell" options to investors because he would also have to pay the execution costs to adjust its own delta hedge ratios over time. However, the market maker is usually quoting the underlying asset price too and would trade smaller changes in hedge ratio terms because its objective is to rebalance as often as possible. Thus if you are a market maker, earning both bid-offer differences on the asset price and the option prices, you may then "lock" rates that are quite higher than bond rates, provided you can get a better access to liquidity in the markets than what the bid-offer currently shows to standard investors. This is the bread and butter of all investment banks… So in liquid and confident markets, the execution costs are "given back" in part in the option prices depending upon the liquidity of the underlying asset.

So we actually work in real life with (showing "x" again now):

$$\partial_t f dt + \partial_x f dx + \frac{1}{2}\partial_x{}^2 f(dx)^2 + \frac{1}{3!}\partial_x{}^3 f(dx)^3 \ldots + \frac{1}{n!}\partial_x{}^n f(dx)^n \ldots - \Delta \boldsymbol{dx} = r(f-\Delta)dt - d\Delta\,(cost_{of_{trading_{x<0}}})$$

And re-introducing proportional behavior versus the price but with "z" now we get:

$$\partial_t f + \frac{1}{2}\sigma^2 z^2 \partial_z{}^2 f + d\Delta\, trading_{cost_{x<0}} - r(f-\Delta) = 0$$

So, due the cancellation of the drift term and the first order stochastic sensitivity based on $\mathrm{P}\frac{\partial V}{\partial P} = \Delta = \left[\mathrm{P}\frac{\partial V}{\partial P}\right]_{\substack{fixed \\ t\to t+dt}}$, we land finally on:

$$\boldsymbol{\partial_t f + \frac{1}{2}\sigma^2 z^2 \partial_z{}^2 f + rz\partial_z f + d\Delta\, trading_{cost_{x<0}} - rf = 0}$$

We are quite close to the classical Black & Scholes equation.

Let's get rid of the trading costs for the moment and assume the best of the world about the "risk neutral" rate that is "constant" and predictable and unbiased by the idiosyncratic running risk underneath the option…..

And here is the classical resolution…assuming no execution costs

<u>Current working assumption:</u> Let's assume, before we dive into more complex models, that we have NOT included yet the impact of trading costs and let's assume that the risk neutral rate issue is not addressed: "σ^2" or "r" $are\ constant\ wrt\ x\ in\ \ proportion.$

We thus revert to the classical Black & Scholes model….

We obtain a simplified equation once we use the change of variable $z = z_0 e^{+x}$:

$$\partial_t f = -\frac{1}{2}\sigma^2 \partial_x^2 f - (r - \frac{1}{2}\sigma^2)\partial_x f + rf$$

We can observe the apparition of the $(r - \frac{1}{2}\sigma^2)$ term here that results from the change of variable PLUS the fact we are not here governed by a "log-normal" process. We really consider the "log" of our price based on an assumption where volatility and drift as proportional to prices of the underlying asset that defines the option.

We now consider the Fourier Transform of f: $\mathcal{F}(f)(y) = f_F(y) = \frac{1}{\sqrt{2\pi}}\int f(x)e^{-ixy}dx$ and the property on the derivatives: $\mathcal{F}\left(f^{(n)}\right)(y) = \frac{1}{\sqrt{2\pi}}\int f^{(n)}e^{-i} \quad dx = [iy]^n\mathcal{F}(f)(y)$

$$\partial_t f_F = -\frac{1}{2}\sigma^2 (iy)^2 f_F - \left(r - \frac{1}{2}\sigma^2\right)(iy)f_F + rf_F$$

We derive an equation that is typical in heat diffusion processes:

$$\partial_t f_F = +\left(\frac{1}{2}\sigma^2 y^2 - iy\left[r - \frac{1}{2}\sigma^2\right] + r\right)f_F$$

The positive sign is an issue for the stability of the solution in general. In Black-and Scholes, we invert the time scale from "t" to "τ =T-t" which brings back a quite convenient "minus sign":

$$\partial_\tau f_F = -\left(\frac{1}{2}\sigma^2 y^2 - iy\left[r - \frac{1}{2}\sigma^2\right] + r\right)f_F$$

Then: $f_F(y,\tau) = f_F(y,0)e^{-\int_0^\tau\left(\frac{1}{2}\sigma^2 y^2 - iy\left[r-\frac{1}{2}\sigma^2\right]+r\right)d\tau'}$

We define a key function, namely the Fourier transform of the option pay-off: $f_F(y,0)$. This is a very convenient outcome with regards to the assumption that we made about the function "f" itself when "t>0" ie outside of the initial conditions: it has to be smooth ie all the derivatives have to be bounded and tend to "0" at infinite values for "t" or "x". This here does not imply that in "T=0", ie at expiry the ultimate function that we call the "pay-off" function is smooth. In particular, the pay-off is not smooth at the strike price in t=0 but this is just a "limit case". The subtle point here is that the function "f" will converge eventually randomly towards its "pay-off" function provided the Fourier transform of the pay-off function exists. Now, as we know from the theory on the Fourier transform, it the pay-off itself "jumps" then the Fourier transform, after inversion, would only retrieve the average value of the "jump" bounds. So this resolution does not work well if the pay-off function is not continuous all along.

We observe that solution of our core differential equation is a product of 2 functions: an exponential function and the Fourier transform of the Pay-off function. We can use the convolution product property to infer "f" from the product above since:

$$f(x,\tau) = \frac{1}{\sqrt{2\pi}}\int f_F(y,\tau)e^{2i\pi} \quad dy = \frac{1}{\sqrt{2\pi}}\int f_F(y,0)e^{-\int_0^\tau\left(\frac{1}{2}\sigma^2 y^2 - iy\left[r-\frac{1}{2}\sigma^2\right]+r\right)d\tau} e^{2i\pi} \quad dy$$

$$\tau = 0 \; is \; the \; time \; t = T \; and \; f_F(y,0) \; is \; the \; Fourier \; transform \; of \; the \; option \; pay - off$$

Actually we can do even better than that and prevent us from computing any Fourier transform actually. Indeed… **This formula above means ALSO that "f" is the convolution product of the primitives of the 2 Fourier transforms: one being $f_F(y,0)$, the other being a compounded normal distribution that has a Fourier transform $e^{-\int_0^\tau\left(\frac{1}{2}\sigma^2 y^2 - iy\left[r-\frac{1}{2}\sigma^2\right]+r\right)d\tau'}$.**

So we can write more directly the solution -if nothing changes over time- as:

$$\boldsymbol{f(x,\tau) = \int_{-\infty}^{+\infty} Payoff_{option}(x-u)e^{-\int_0^\tau r d\tau'} \aleph\left(u, mean = \int_0^\tau \left[\frac{1}{2}\sigma^2 - r\right] d\tau', var = \int_0^\tau (\sigma^2) d\tau'\right) du}$$

Where $x = Log\left(\frac{z}{z_0}\right)$ *and* z *is usually the price of the underlying asset*

This directly comes from the computation of the Fourier transform of a normal distribution like $d(u) = \aleph(u, mean = m, var = v) = \frac{1}{\sqrt{2\pi v^2}} e^{-\frac{\frac{1}{2}(u-m)^2}{v^2}}$

$$\mathcal{F}(d)(y) = \frac{1}{\sqrt{2\pi}} \int d(u) e^{-iuy} du = \frac{1}{\sqrt{2\pi}} \int \frac{1}{\sqrt{2\pi v^2}} e^{-\frac{\frac{1}{2}(u-m)^2}{v^2}} e^{-iuy} du$$

$$\mathcal{F}(d)(y) = \frac{1}{\sqrt{2\pi}} \int \frac{1}{\sqrt{2\pi v^2}} e^{-\frac{u^2 - 2um + 2iv^2uy + m^2}{2v^2}} du$$

$$u^2 - 2um + 2iv^2uy + m^2 = u^2 - 2u(m - iv^2y) + (m - iv^2y)^2 - (m - iv^2y)^2 + m^2$$

$$u^2 - 2um - 2iv^2uy + m^2 = \left(u - (m - iv^2y)\right)^2 + y^2(v^2)^2 + 2imv^2y$$

$$\mathcal{F}(d)(y) = \frac{1}{\sqrt{2\pi}} \int \frac{1}{\sqrt{2\pi v^2}} e^{-\frac{\left(u-(m+iv^2y)\right)^2 + y^2(v^2)^2 + 2imv^2y}{2v^2}} du$$

$$\mathcal{F}(d)(y) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}y^2v^2 - imy} \; because \int \frac{1}{\sqrt{2\pi v^2}} e^{-\frac{\left(u-(m+iv^2y)\right)^2}{2v^2}} du = 1$$

(Another way but one that does not fit well is about changing variables: $w = \frac{u-m}{v} \rightarrow dw = \frac{du}{v} \rightarrow u = m + vw$ …that can be solved the same way on the follow through a similar factorization from…

$\mathcal{F}(d)(y) = \frac{1}{\sqrt{2\pi}} \int \frac{1}{\sqrt{2\pi}} e^{-w^2} e^{-imy - iyv} \; du$)

This use of convolution products is a more generic formulation of an option that allows us to assume variability of the volatility over time…this is fine if this "volatility" itself is not random…hmmm…. But what happens if the factors are NOT constant when z varies…In other words, if the coefficients in the equation are not fixed independently of "z"?

Let's notice that their dependence on "time to maturity" is not a big issue in the resolution as long as they are not stochastic themselves (then we might really struggle to define them as "random functions"): the main concern is that the Fourier transform remains simple as long as the factors of the derivatives on "z" or "x" are independent of the random variable.

The most used "improved" models are named GARCH (aka Gaussian Auto-Regressive Conditional Hetero-skedasticity) that assume that the implied volatility is also random following a normal distribution around a "mean" value that mean reverts over the long term. This is fine and yet requires to set 2 hyper-parameters in general to account for the sensitivity of the mean moving "out" and the mean "reverting" next. The weakness of this model is that the volatility is not directly tied to the underlying random price. It works ok for risk measures ex-post but it does not make wonders for investment and trading purpose in price determination or tail-hedging. One strong point though is to start considering that the "path followed" matters: in other words the coefficients like σ or "r" will depend on the price and therefore on "z".

Extension: Hermite's polynomials and easy approximations

Now we should introduce the trading costs and some variability on the risk neutral rate that shall BOTH depend on the price and the path that this price has followed.

How can we calibrate properly this model with prices of traded options in the most simplistic fashion? Is there any "simple" solution in fact? Answer: no there is no magic wand…But there is a methodic approach…

The original problem with the model pictured above is that, beyond the mere pricing of vanilla options through the Balck&Scholes model we actually use mathematical technical that were invented by Louis Bachelier as early as 1900. He was the one who founded the modeling of the "speculative games" and had the idea to use stochastic processes. The genuine input of Black&Scholes was in the assumptions leading to the resolution that is as simple as one can wish. Nevertheless even these assumptions were first introduced by Harry Markowitz in the 1950ies, ie some 15-20 years before. The achievement of the Black&Scholes model was mostly to "find" the needed and yet practical assumptions that allowed to "price" vanilla options. However this simple method came across bit issues: first the "skew" of the volatility across the strikes and second the volatility of the volatility itself. Last but not least, there is no such thing as a "risk-neutral" rate. So, both the implied volatility and the "risk-neutral" rate are stochastic and the GARCH models are backward looking. What if we could, remaining as simple as possible, introduce the idea that the implied volatility and the "rate r" are stochastic but tied, in probability terms only, to the main underlying random process of the "price"?

We try to find a convenient distribution so that the adjusted "implied volatility" will then remain quite "flat" once we have anchored the coefficient to "z": no skew, no smile, maybe no term structure! But would it make sense fundamentally? Which would the most appropriate model be? We can see the advantage: if the volatility is flat, ie constant, then the projections and delta hedges are quite predictable risk management tools…therefore they also would become reliable pricing tools all the way. We may never be able to predict the actual level of volatility but if it always remains "flat" across the strikes, then we can use ancillary trades to hedge the volatility change. This is what the practitioners do but this is poisoned by the smile and the skew because the latter do move a lot in large market shifts, rendering the former hedge ratio useless if not deceptive.

Here we again will aim first at modeling with constant factors wrt "z" or "x"… and see whether these sole factors could reduce significantly this "smile+skew risk"…

We then can use the properties of the Fourier transforms to address these questions: the tool is called a convolution product….

If we want to consider now the hedging costs and the potential randomness of the risk-neutral rate that is therefore quite "risky" too…..we start with the following equation. You will notice that we switch to a more explicit model for the trading costs aka "T_c" $that\ is > 0$ now.

Indeed we established formerly that:

$$\partial_t f + \frac{1}{2}\sigma^2 z^2 \partial_z{}^2 f + rz\partial_z f + d\Delta\, trading_{cost_{x<0}} - rf = 0$$

We would set $-\frac{1}{2}\sigma^2 z^2 \partial_z{}^2 f T_c = d\Delta\, trading_{cost_x}\ with\ T_c \geq 0$ so that we keep some intuitive homogeneity: we say that the trading costs are ultimately a straight function of the "gamma" ie the

potential change in the delta that locally is featured fully by the term $\frac{1}{2}\sigma^2 z^2 \partial_z{}^2 f$. This is astute because this term has NO effect if the option is in the money.

It is a transposition of the classical equation except that here we state that the "factors" are in a special relation namely they contribute to the equation via a convolution product:

$$\partial_t g = -\frac{1}{2}\sigma^2[(1-T_c)(\partial_x^2 g)]_{conv} - \left[\left(r-\frac{1}{2}\sigma^2(1-T_c)\right)\partial_x g\right]_{conv} + [rg]_{conv}$$

<u>We introduce the idea that the equation itself is governed on "expectations under conditions set by x:"</u> for a given "x" many outcomes are possible and there is an equilibrium that only exists "on average" over "t->t+dt"

<u>Detailed comment:</u>

Here we would simply say that, while the original equation over a "dt" that is in reality some "δt" that, by working assumption, we assume to be arbitrarily small and we can "rebalance" at each "dt" as our convenience through some executing costs while earning the rate "r" on the cash balance :

$$df - \Delta dP = r(f-\Delta)dt - d\Delta(cost_{of_{trading_x}})$$

However, we then have to assume that, despite "constant rules" that secure a Markov process, as in so many games, the "path followed" would condition "r" and these trading costs. Thus, since we have to consider that for a given price at some time "t" there are many scenarios that would have led to this situation and each time we have to assume that "r" and the trading costs might well be different one from another. However, the rules being fixed and the paths being of arbitrary length, we fall down to some ability to "add up all the possible scenarios" as per a normal distribution (use of the Central Limit Theorem). This means that if we still consider that the only "possible" equation here results from some "equilibrium" we have to consider the "expectation" of prices crossed with either "r" or "trading costs". So it is just as if we introduced a convolution product that is based on some inner random variable. We thus start from :

$$\partial_t f = -\frac{1}{2}\sigma^2\partial_x^2 f - \left(r-\frac{1}{2}\sigma^2(1-T_c)\right)\partial_x f + rf$$

And introduce an inner random variable "u" that pictures the many "paths that could have been followed" : $\int_{-\infty}^{+\infty} d(u) = 1$ with d(u) exhibiting some normal distribution and we opt to represent this equation as a convolution product being very careful on the introduction of the "x-u" versus the "u" as indexation because this will have an importance when we will estimate these functions from empirical data given the inversion of the scales here ("u" for "f" but "-u" for the rate and the trading costs)

$$\int_{-\infty}^{+\infty} \partial_t f_x(x-u)d(u)$$
$$= -\int_{-\infty}^{+\infty}\frac{1}{2}\sigma^2\partial_x^2 f_x(x-u)d(u)$$
$$-\int_{-\infty}^{+\infty}\left(r(u)-\frac{1}{2}\sigma^2\big(1-T_c(u)\big)\right)\partial_x f_x(x-u)d(u) + \int_{-\infty}^{+\infty} r(u)f_x(x-u)\,d(u)$$

This equation above states in full words: "At the instant t we would observe one price but many others would have been observable on average given that many paths are possible for the same ultimate conclusion. However for any sub-scenario there would be a rate and a trading cost that

associate differently as per the inner random variable “u” that itself defines a distance from the observed price of the moment. We can see that the distance “u” determines the rate and trading cost and that some distribution is fixed for all values of x actually. So this means that the distance to the average price only determines the variation of the rate and of the trading costs, not the price level itself.” This sounds strange but it fits rather well with market practice that mostly speculate and therefore prices assets relative to the prevailing current price level anyway, not in absolute terms.

Next we would define a “special Fourier transform” and still keep the property of the convolution product… From the usual Fourier transform that is set as:

$$\mathcal{F}(f)(y) = f_F(y) = \frac{1}{\sqrt{2\pi}} \int f(x) e^{-ix} \; dx$$

We would define a Fourier transform that is tied to our distribution “d(u)” in a very simple way empirically: we would simply imagine that we have “N” scenarios of path followed and we would imagine that we “sum” all these scenarios before we “average” the sum by “N”.

Thus from every possible equation, knowing in advance the average price “x” in t between t and t+dt and the distance “u” from this average we can record :

$$\partial_t f_{x-u} = -\frac{1}{2}\sigma^2 \partial_x^2 f_{x-u} - \left(r_u - \frac{1}{2}\sigma^2 \left(1 - T_{c_u}\right) \right) \partial_x f_{x-u} + r_u f_{x-u}$$

Then we consider an indexing of the many possible scenarios, one being defined by a path followed leading to the very same ultimate outcome over [0,T]:

$$\frac{1}{N} \sum_{k=1}^{k=N} \partial_t f_{x-u_k} = \frac{1}{N} \sum_{k=1}^{k=N} -\frac{1}{2}\sigma^2 \partial_x^2 f_{x-u_k} - \left(r_{u_k} - \frac{1}{2}\sigma^2 \left(1 - T_{c_{u_k}}\right) \right) \partial_x f_{x-u_k} + r_{u_k} f_{x-u_k}$$

At each stage we would increase “N” in order to more and more finely approach the “real underlying model” for $N \to +\infty$ we can estimate in fact that we take $\frac{1}{N} \approx du$ while, the density of probability of the distance that is $d(u)$ would rather “count” the number of times the “distance” u takes the same value repeatedly. So for each “N” we would apply a standard Fourier transform to this discrete “sum” that would respect the well know property of Fourier transforms all along. And we finally would retain the proxy for our convolution product and we know this property is valid for discrete sums anyway.

So, provided we remain careful as to the “functions” that we would infer from empirical evidence that would be a “function” and a “distribution for u”, we certainly can define a “limit case” for the Fourier transform that will “work”. How to do that neatly so?

We just have to pay a little extra attention to “what” we include in our equation here:

$$\begin{aligned} \int_{-\infty}^{+\infty} \partial_t f_x(x-u) \frac{d(u)}{du} du & \\ &= -\int_{-\infty}^{+\infty} \frac{1}{2}\sigma^2 \partial_x^2 f_x(x-u) \frac{d(u)}{du} du \\ &- \int_{-\infty}^{+\infty} \partial_x f_x(x-u) \left(r(u) - \frac{1}{2}\sigma^2 \left(1 - T_c(u)\right) \right) \frac{d(u)}{du} du \\ &+ \int_{-\infty}^{+\infty} f_x(x-u)\, r(u) \frac{d(u)}{du} du \end{aligned}$$

Recall that in practice "x" is the log of the market price, hence "u" is the distance in logarithm terms. Also we can see that we will need, for each "distance" bucket to weight the empirical measure again as a sort of "sum" on this bucket: while "d(u)" is the density ie the "number" of instance of a given distance within a certain bucket of width "$\frac{1}{N}$" that we multiply by "N" through the "$\frac{1}{du}$", we simply in fact add up all the empirical values that we obtain for a certain range of distance "u" relative to the expected price of the moment. This is in practice this "sum" that will the "function" that now we would project on the basis of Hermite's functions. This is beyond the scope of this course but you can read the appendix below. One can combine then import properties like: the recursive relation between the Hermite's function, the fact that they are the eigen values of the linear operator "Fourier Transform", and the fact they all derive from the consecutive derivatives of the normal distribution.

This formula exhibits that what really matters is the "expected trading cost" as a fraction of the variance when and IF the option become at the money. This is really when the re-hedging matters since it induces costs. All this reverts back to the original equation:

$$df - \Delta dP = r(f - \Delta)dt - d\Delta(cost_{of_{trading_x}})$$

Above we can see that the portfolio made of the option PLUS the delta-hedge is more or less risk-neutral. We do add a term to the equation that is "make-up only" IF we keep the same rate "r". The effect is that either we actually need a breakeven volatility that is higher than what the usual volatility figure exhibits, or we actually operate in a market, for this delta-hedged option in particular, that is higher than what the treasury rates indicate. This means that if one invests the money on a costly option that carries a high trading cost, then one must think in terms of "equivalent rates of returns".

If we look at the first equation with the appropriate inverted time scale, we can see how the option price is built up and infer interesting conclusions about the volatility skew towards the puts mostly…

$$\partial_\tau g = +\frac{1}{2}\sigma^2[(1 - T_c)(\partial_x^2 g)]_{conv} + \left[\left(r - \frac{1}{2}\sigma^2(1 - T_c)\right)\partial_x g\right]_{conv} - [rg]_{conv}$$

If we plan to flatten the volatility level using the trading costs and consequences, we are disappointed here: we can see that, if we need to "flatten" the smile by increasing the implied volatility σ (relative to the one we get from the Black&Scholes model based on opion market prices) on higher strikes and by decreasing the implied volatility for lower strikes, the trading costs would have to grow with the strike , hence be lower when the strikes are lower, and even become negative…while the same σ^2 would be shared across the strikes in our model here….

At first sight it makes little sense that the trading costs diminish when markets crash and increase when markets are optimistic and bullish… unless we actually bear in mind that in the current markets conditions, even though the trading costs are notoriously higher for crashes, this concerns low delta portfolios and a gap would bring a once for all massive gain that would more than balance re-hedging costs. To be sure here, on this key point, one would likely execute once only to lock an otherwise massive and un-modelled gain within the pure diffusion process that is pictured so far. So, if we want to use locally constant trading costs values and rates values, we have to factor in the delta at least for the trading costs and the "gap probability modulo some expected gain then".

Combining the 2 equations, it would make sense then to set trading costs as a fraction of the delta*σ^2 value and adjust the rate to the trading cost in order to reflect that we have different levels of rates then…. But these trading costs follow a gradient that factors in the "gap risk" too. Typically this gap risk is mostly visible on the Put side, and is lower on the OTM calls. So there is a cross-current with the "trading costs" figure. It is spontaneously a negative value on ATM and on OTM calls or puts and the jump risk becomes clearly positive on OTM puts. So we have to find a fine balance: all the options carry implicitly a higher implied volatility due to the execution costs related to gamma hedging. They all have an embedded expected P&L linked to a possible sudden "jump" towards the down side. Relative to the current option price, the jump renders the OTM puts negative at times, ie they look like a "free option". The OTM calls, once adjusted for the "Jump" value, can be compared to the ATM calls and we should set the execution costs up to a level where the implied volatility is the same for both. The puts should match what is found. We then can tweak parameters to obtain the flattest possible vol curve across the strikes.

In other words, when markets crash the bid-offer widens and it is more expensive to trade the delta hedge indeed. On balance the jump in the realized variance is much bigger when we consider "gaps down". The equation above states that, although the trading costs, now put as a function of the second derivative that moves with the square of the volatility, the proportional cost is getting lower relative to the squared implied volatility that has increased. All the same, in a bullish case where actually the implied volatility tends to go lower the trading costs tend to the be equivalent: thus our "increase" in relative trading costs is sensible. This approach overall states that the trading costs fall in bear markets for option trading purpose and they increase in bull markets. This is fully consistent with the market behavior: indeed in bull markets the implied volatility tends to go lower and lower and this is often explained by the cost of trading the delta hedges. Likewise, in bear markets, despite the bad apparent conditions of execution, the option prices rise fast and sustainably beyond the "fear factor": this is just because in proportion the higher volatility better compensates for the wider bid-offer quotes. So, ultimately this model is perfectly consistent with the skew tendency of the markets!

In practice, how do we do that?!

The idea is to start with the standard Black&Scholes approach, the "rate" being set by the spread between the spot and the future price. An implied volatility is deduced but there is a skew that we want to correct. We would use the convolution product to factor the "jump" risk and infer the complementary value that is "pure diffusion". We introduce the trading costs as a mixture between the convexity term featured by σ^2 and the delta on the one hand and "probability weighted Jump risk" on the other hand. The execution costs, or "trading costs", are a function of the "gamma+vega" in general and their impact is felt through the delta of course . Hence there is a lower trading cost effect as we move away from the ATM level.

Thus we start with assuming a "jump" impact. We compute this jumps so that at least for the calls the implied volatility becomes flattish. We then obtain a flattish vol curve on the calls. What about the puts then? We may infer some weird outcomes on OTM puts that may have some negative premium. We next use the resulting option premiums on the put side to find the "good" trading cost % as $\frac{1}{2}\sigma^2 T_c$ versus $\frac{1}{2}\sigma^2$ so that the resulting implied vol remains as flat as flat as possible for puts apart from the deep OTM puts.

We will not know really what the implied vol is but we can assume that the trading costs on execution move inversely in proportion to the strike/spot ratio. Thus we could just set a gradient across the strike so that the OTM puts are further "hit" by these costs versu the OTM calls. So we would then infer a gradient on T_c as a % of σ^2 while σ^2 would actually be flat. That should work because the OTM puts are the ones that tend to have a negative premium due to the

jump scenario on the way down. This is appearing as a worsened picture due to the execution costs that are factored in our market price already. Hence we would correct that. As to the "base" execution cost it could be proxied from the current bid offer spreads on the spot underlying asset.

Ultimately, with this arbitrary choice to factor in the trading cost terms. By tweaking the trading cost across the strikes, we can flatten the curve of the value σ^2 at the cost of building a rate curve. What remains fully arbitrary is that we introduce the term $\frac{1}{2}\sigma^2 T_c$ and we will set a skew that mingles 'Gap' and "bid-offers".

The basics can be summed up as follows:

- The apparent ATM implied volatility is lower than the actual one that is required to lock the "risk-neutral" rate
- The trading costs are a product of the variance via the gamma in fact: the effect reduces the skew on the puts and increases it on the calls. The impact on the delta is spontaneous but overall it does not change the "optics"…
- The thinking must be looking at extreme cases: super big rally on the one hand and super crash on the other hand.
- On a big rally, it is prevalent that the execution costs become a fixed fraction of the underlying, thereby increasing in our models for further OTM calls. This explains why Vol declines as strikes go higher. This should be mitigated – as a refinement- somewhat as liquidity improves on bullish trends and declines in bearish trends.
- On a big crash, we could model the probability of say 90% collapse from current spot. This would be a fixed move but with a varying impact on the option price depending on the strike. The P&L includes the option and the current delta.
- The way to proceed is to estimate the "gap" gain from the most OTM put and the implied execution cost from the most OTM call and next refine so that it fits the observed curve of implied volatility. Once we have that we have to include a gradient of trading costs: they become lower on higher strikes and vice versa. Where to strike the balance between "higher crash% vs higher trading costs on puts"? We need the flattest possible implied volatility curve, never really knowing what the ATM execution costs are apart from looking at the bid-offer and the current realized volatility.

4- Expectation of the Max

Here we approach a concept that is often overlooked: if we hold a position over several time intervals that we consider "independent" on a "standard" longer period (for example: we have a position that is revalued day to day and we plan to hold it for a month, 3 months, may be a year and each day is independent from the others), what is the expected maximum gain that we should expect knowing that this is likely also the maximum loss that we should expect.

Here there is an approximation that is very useful that gives us key measures of the Maximum cumulated value taken by a string of consecutive price movements. The approximation works fine when n is large , ie in the thousands (not the quite the hundreds). But often in finance we wish we could optimize rather finely the "Maximum", at least its expectation, of a lower number of "returns" albeit random.

Also, if we have a portfolio of say 5 to 20 positions, we typically would like to know what the expectation of the "Maximum", be that a "gain" in order to optimize the "timing" to take profits on some positions, "reduce" or "increase", or be that a "loss" and then we would use it to define an ongoing "stop loss". So this concept lying around "oh I had a sudden big gain right now but what should I have expected when I started it a week ago? ", else, "Oh this position is bleeding money on and on, how significant is that? Is it just an accident that should reverse soon?"

We will see here how one obtains the generic approximation for the maximum of the Max of a string of random variables added together: either we care about the underlying time series of "one asset" over time and the "expected maximum gain" or "expected maximum loss" (that shapes the "stop loss" ahead of time), or this is the distribution of "n" assets that constitute a portfolio where we want to know in advance what a reasonable 'maximum expected return' we should expect given the weights among the assets and the individual distributions for each of these n assets.

We will start with proving the main approximation and next we will show how we can find the expected "Max" even when "n" is just higher than 5. The approximation for large "n" values works well because the distribution of the Max becomes a Dirac spike (called a Dirac measure). But this only happens to be significant when n is really large which, in financial real life, is rarely the case. The way the expectation of the Max if found is based on identifying the "expectation" with the "top" of the density of probability. This is true as you know with the normal distribution even if the variance is high. But this is not true if the distribution is not symmetrical around it spike, or if the distribution has more than 1 spike. As we can guess the distribution of the Max is skewed towards positive values even if the sum of the underlying variable is centered on 0. Is this inducing a distortion around the spike and is the expectation biased by that? How important is this when n is just >5 instead of being higher than say 5000? We see that in detail here.

So we will refine this approximation and first realize that the "top" is not quite what the generic approximation is when n is not really large. And next we will still observe that despite the skewness of the Max the distribution remains quite symmetrical but a little twisted still.

In contrast to the former chapter that spoke of "risk-neutral" rate whereby no asset class, on a risk-adjusted basis, is expected to deliver a higher return than another class, here we consider the actual "option" that people create for themselves. So far indeed we looked at a "price" , a "delta hedge", " position" as something that is "risk-neutral". But of course in real life, market players, companies, only think in terms of "creating options for themselves and opportunistically take gains or losses".

Let's take the case of an investor who either buys and option, or the underlying asset because the investor deems the timing is "right". How to define the "stop loss"? How to define the best windows to taking profits in a rosy outcome? What to do in the meantime before either it is "too late" or it is "too early"? Why should we "wait" when every day we can re-assess the relevance of our exposure? Should we just sit, wait, and finally realize that on average , doing nothing, we more or less earn a yield that we can get passively every 12 hours by taking actually almost no risk at all?

The current chapter shows the mathematical background for estimating the "expectation of the Maximum return" above the risk-neutral rate. Thus, as the prices fluctuate randomly, and deliver random returns day to day, hour to hour, minute to minute, we can monitor the performance and therefore pre-emptively manage the risks actively. One yardstick that is particularly useful is indeed to know, given the current recent distribution of past outcomes, what we should expect for the future in terms, not of "average returns" (will the risk-neutral rate anyway), but in terms of "volatility" and "opportunities". This is all about "timing" things with an appropriate benchmark.

This is what this chapter covers as simply as possible.

So we will first remind a simple way to compute the approximation for the expectation of the Max in this context. However the generally simple result is not accurate enough in general due to the advent a many more potential terms. So we will have to reinforce the core approximation process for the case of a normal distribution.

So first let's put in place the main blocks of the distribution of the max and its proxy expectation for a normal distribution of the underlying variables when "n", the number of samples, is high enough. We will define the first 2 orders but the method likely opens the way to finer estimates on the follow.

Let's therefore consider "n" iid (independent-identically-distributed) "normal" centered on 0 norm 1 variables $X_1, X_2, \dots X_n$. Let's note with $\varphi(x)$ the normal distribution and with $\Phi(x) = \int_{-\infty}^{x} \varphi(u)du$ the associated probability law. We simplify enormously from the start but any set of "n" random variables can safely ne reduced to that assumption thanks to the Central Limit Theorem.

We then see that the distribution of the "n_Max" is the first derivative of the probability law that itself derives from the fact that the n variables are independent as:

$$F_{M_{n_X}}(x) = \prod_{k=1}^{k=0} \Phi_{\mathrm{k}}(x_k)$$

As an exercise you should try to make the proof of that statement.

The notation is made more general on purpose for what follows next: of course in our core case we will assume that all the "n" variables have the same probability law but this expression above covers the more general situation, which is interesting as long as we have "independent" variables here, where the first derivative provides the density (ie distribution) which will be instrumental in computing the expectation of the Max on the follow. Recall that diagonalizing the covariance matrix allows to morph a group of "n correlated random variables" into a group "n uncorrelated eigen vectors" based on these canonical variables. So, provided the covariance matrix, that is symmetrical or else Hermitian anyway, can be processed, we really cover a very wide range of cases if "n" is high enough (thanks again to the CLT).

So here, for the purpose of our calculation, we will simplify the expression to :

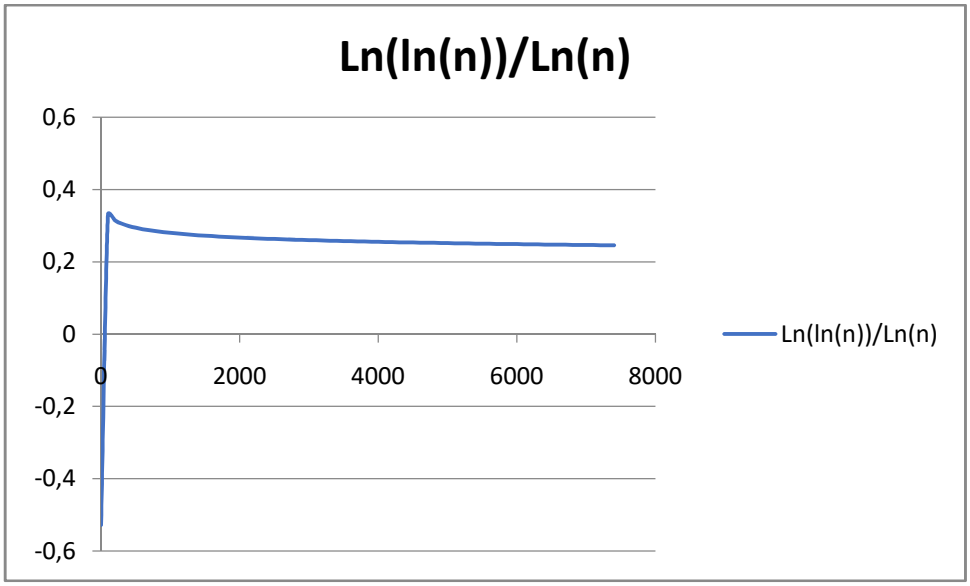
$$F_{M_{n_X}}(x) = \big(\Phi(x)\big)^n \; and \; therefore \; f_{M_{n_X}}(x) = n\varphi(x)\big(\Phi(x)\big)^{n-1} with \; \varphi(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}$$

We will now show that : $\boldsymbol{E\big[M_{n_X}\big] = x_{top} \approx \sqrt{2Ln(\frac{n}{\sqrt{2\pi}})} - \dots = \sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}(1 - \dots)}$

In the detail we will show that:

$$x_{top} \approx x_1 \left(\frac{1 + \frac{1}{2}x_0^2 + \frac{1}{2}x_1^2 - \ln(x_1)}{(1 + x_1^2)} \right) with \; x_1 \approx x_0 \left(1 - \frac{\ln(x_0)}{(1 + x_0^2)}\right) \dots x_0 = \sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}$$

One may see that for high values of "n" we are left with the main term $\sqrt{2Ln(n)}$, but for reasonably high values of "n" actually the second term is quite important, correcting down the main term by some 8%.



We will proceed step by step:

- Show the asymptotic behavior of the erfc function
- Infer the more generic approximation of the erfc function
- Compute the dominant term for the expectation of the Max
- Compute the second order term
- Open the method to obtain even finer estimates

Asymptotic behavior of the erfc function (a key element here!)

Let's consider $I(x) = \int_{x>0}^{+\infty} e^{-u^2} du = e^{-x^2} \int_0^{+\infty} e^{-2ux-u^2} du$

So we need to actually estimate, for "x" that is assumed to be large, the integral $\int_0^{+\infty} e^{-2ux-\ ^2} du \sim \int_0^{+\infty} e^{-2ux} du$ because indeed, when "x" is very large the only terms that would contribute to the integral sums are the ones where $u \approx 0\ ie\ e^{-u^2} \approx 1$ and therefore $\int_0^{+\infty} e^{-2ux-u^2} du \sim \int_0^{+\infty} e^{-2ux} du \approx \frac{1}{2x}$

We conclude then that $I(x) = \int_x^{+\infty} e^{-u^2} du \approx \frac{e^{-x^2}}{2x}$

We can here push the approximation further by considering the subsequent terms of the term in u^2 , ie e^{-u^2}: $e^{-u^2} \approx 1 - u^2 + \frac{1}{2}u^4 - \frac{1}{3!}u^6 + \cdots$

Another possible approximation could be:

$$e^{-u^2} \approx e^{-u}\left(1 + u\frac{d}{dx}\left(e^{-u^2+u}\right)[0] + \frac{1}{2}x^2\frac{d^2}{dx^2}\left(e^{-u^2+u}\right)[0] + \frac{1}{3!}x^3\frac{d^3}{dx^3}\left(e^{-u^2+u}\right)[0] + ..\right)$$

We introduce this simpler vanishing term "e^{-u}". This implies that

$$I(x) = e^{-x^2}\int_0^{+\infty} e^{-2ux-u^2}du$$
$$\approx e^{-x^2}\int_0^{+\infty} e^{-2(x+\frac{1}{2})u}\left(1 + u\frac{d}{du}\left(e^{-u^2+u}\right)[0] + \frac{1}{2}u^2\frac{d^2}{du^2}\left(e^{-u^2+u}\right)[0]\right.$$
$$\left. + \frac{1}{3!}u^3\frac{d^3}{du^3}\left(e^{-u^2+u}\right)[0] + ..\right)du$$

We then have a better approximation with $\frac{d}{dx}\left(e^{-u^2+u}\right)[0] = (-2u+1)e^{-u^2+u}[u=0] = 1$

Next $\frac{d^2}{du^2}\left(e^{-u^2+u}\right) = (-2 + (-2u+1)^2)e^{-u^2+u}[u=0] = -1$

Next: $\frac{d^3}{du^3}\left(e^{-u^2+u}\right)[0] = (-6(-2u+1)^1 + (-2u+1)^3)e^{-u^2+u}[u=0] = -5$

Next $\frac{d^4}{du^4}\left(e^{-u^2+u}\right)[0]$=$(12 - 12(-2u+1)^2 + (-2u+1)^4)e^{-u^2+u}[u=0] = 1$

Next $\frac{d^5}{du^5}\left(e^{-u^2+u}\right)[0]$=$(60(-2u+1)^1 - 20(-2u+1)^3 + (-2u+1)^5)e^{-u^2+u}[u=0] = 41$

So, $(x) = e^{-x^2}\int_0^{+\infty} e^{-2ux-u^2}du \approx e^{-x^2}\int_0^{+\infty} e^{-2(x+\frac{1}{2})u}\left(1 + u - \frac{1}{2}u^2 - \frac{5}{3!}u^3 + \frac{1}{4!}u^4 + \frac{41}{5!}u^5..\right)du$

Let's keep the most direct approximation for now…we assume that x>0 all along

Let's call the recursive many integrations by part here since they deliver the result almost directly:

$$v_k = \int_0^{+\infty} u^{2k}e^{-2ux}du = \left[-\frac{1}{2x}u^{2k}e^{-2ux}\right]_0^{+\infty} + \left(\frac{2k}{2x}\right)\int_0^{+\infty} u^{2k-1}e^{-2ux}du$$
$$= \left(\frac{2k}{2x}\right)\int_0^{+\infty} u^{2k-1}e^{-2ux}du = \left(\frac{2k(2k-1)}{2x2x}\right)\underbrace{\int_0^{+\infty} u^{2(k-1)}e^{-2ux}du}_{v_{k-1}}$$

Thus $v_k = \int_0^{+\infty} u^{2k}e^{-2ux}du = \frac{1}{2x}\frac{(2k)!}{2^{2k}x^{2k}}\ with\ of\ course\ x > 0\ and\ \int_0^{+\infty} e^{-2ux}du = \frac{1}{2x}$

Given that:

$$e^{-u^2} \approx 1 - u^2 + \frac{1}{2}u^4 - \frac{1}{3!}u^6 + \cdots + \frac{(-1)^k}{k!}u^{2k} + \cdots$$

So we actually can write that :

$$I(x) = \int_x^{+\infty} e^{-u^2}du \approx \frac{e^{-x^2}}{2x}\left[1 - \frac{1}{2x^2} + \frac{1}{2}\frac{4!}{2^4x^4} - \frac{1}{3!}\frac{6!}{2^6x^6} + \cdots\right]$$

$$I(x) = \int_x^{+\infty} e^{-u^2}du \approx \frac{e^{-x^2}}{2x}\left[1 - \frac{1}{2x^2} + \frac{3}{4x^4} - \frac{15}{8x^6} + \cdots\frac{(-1)^k}{k!}\frac{(2k)!}{2^{2k}x^{2k}} + \cdots\right]$$

If now we consider the normal distribution instead ie $\Phi(x) = \int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du = 1 - \int_x^{+\infty}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du$

In other words: $\Phi(x) = 1 - erfc\left(\frac{x}{\sqrt{2}}\right)$

Now: $if\ we\ set\ v = \frac{u}{\sqrt{2}}\ and\ erfc\left(\frac{x}{\sqrt{2}}\right) = \int_x^{+\infty} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}u^2} du = \int_{\frac{x}{\sqrt{2}}}^{+\infty} \frac{1}{\sqrt{\pi}} e^{-v^2} dv = \frac{1}{\sqrt{\pi}} I\left(\frac{x}{\sqrt{2}}\right)$

We can then define the erfc function as a function "y" and apply the approximation:

$$erfc(y) = \frac{1}{\sqrt{\pi}} \int_y^{+\infty} e^{-z^2} dz = \frac{1}{\sqrt{\pi}} \frac{e^{-y^2}}{2y} \left[1 - \frac{1}{2y^2} + \frac{3}{4y^4} - \frac{15}{8y^6} + \cdots \frac{(-1)^k}{k!} \frac{(2k)!}{2^{2k} y^{2k}} + \cdots\right]$$

We can verify then, with a change of variable where $y = \frac{x}{\sqrt{2}}$ that :

$$erfc\left(\frac{x}{\sqrt{2}}\right) = \frac{1}{\sqrt{\pi}} \int_{\frac{x}{\sqrt{2}}}^{+\infty} e^{-z^2} dz = \frac{1}{\sqrt{2\pi}} \int_x^{+\infty} e^{-\frac{1}{2}u^2} du = (1 - \Phi(x))$$

And therefore:

$$1 - \Phi(x) = erfc\left(\frac{x}{\sqrt{2}}\right) \approx \frac{1}{\sqrt{\pi}} \frac{e^{-\frac{1}{2}x^2}}{\frac{2x}{\sqrt{2}}} \left[1 - \frac{1}{2\left(\frac{x}{\sqrt{2}}\right)^2} + \frac{3}{4\left(\frac{x}{\sqrt{2}}\right)^4} - \frac{15}{8\left(\frac{x}{\sqrt{2}}\right)^6} + \cdots \frac{(-1)^k (2k)!}{2^{2k}\left(\frac{x}{\sqrt{2}}\right)^{2k}} + \cdots\right]$$

$$1 - \Phi(x) = erfc\left(\frac{x}{\sqrt{2}}\right) \approx \frac{1}{\sqrt{2\pi}} \frac{e^{-\frac{1}{2}x^2}}{x} \left[1 - \frac{1}{(x)^2} + \frac{3}{(x)^4} - \frac{15}{(x)^6} + \cdots \frac{(-1)^k}{k!} \frac{(2k)!}{2^k (x)^{2k}} + \cdots\right]$$

$$1 - \Phi(x) = erfc\left(\frac{x}{\sqrt{2}}\right) \approx \frac{1}{\sqrt{2\pi}} \frac{e^{-\frac{1}{2}x^2}}{x} \left[1 - \frac{1}{(x)^2} + \frac{3}{(x)^4} - \frac{3*5}{(x)^6} + \frac{3*5*7}{(x)^8} \ldots + (-1)^k \frac{\prod_{i=1}^{k-1}(k+2i)}{(x)^{2k}} + \cdots\right]$$

This opens the way to run an ever finer approximation of the expectation of the Max.

Indeed we have set the notations as follows:

$$F_{M_{n_X}}(x) = \left(\Phi(x)\right)^n\ and\ therefore\ f_{M_{n_X}}(x) = n\varphi(x)\left(\Phi(x)\right)^{n-1} with\ \varphi(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}$$

$$Therefore\ E\left[M_{n_X}\right]_X = \int_{-\infty}^{+\infty} x f_{M_{n_X}}(x) dx$$

We would use the approximation provided by the erfc function and we split the whole integral noticing that $\Phi(x_{\leq 0}) \approx \frac{1}{2}$ which is a part of the whole integral above that tends towards 0 when n grows large.

Let's check on that introducing "M=n" to define some upper bound for $\left(\Phi(x)\right)^{n-1}$ on $]-\infty, 0]$ defined as follows:

$$M_{<1}\left(\frac{1}{2}\right)^{n-1} \int_{-\infty}^{0} x \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2} dx = \int_{-\infty}^{0} x \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2} \left(\Phi(x)\right)^{n-1} dx$$

Does it make any sense? Well, "yes" if we look at the fact that $\Phi(x) \leq \frac{1}{2}$ on $]-\infty, 0]$:

We can indeed see that: $\frac{\int_{-\infty}^{0} x \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2} \left(\Phi(x)\right)^{n-1} dx}{\int_{-\infty}^{0} x \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2} dx} < \frac{\int_{-\infty}^{0} x \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2} \left(\frac{1}{2}\right)^{n-1} dx}{\int_{-\infty}^{0} x \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2} dx} < \left(\frac{1}{2}\right)^{n-1}$

This is what justifies the introduction of this value M that is strictly in]0;1[

Therefore:

$$E\left[M_{n_X}\right]_X = \int_{-\infty}^{+\infty} x f_{M_{n_X}}(x)dx = \int_{-\infty}^{0} x f_{M_{n_X}}(x)dx + \int_{0}^{+\infty} x f_{M_{n_X}}(x)dx$$
$$\approx \int_{0}^{+\infty} x f_{M_{n_X}}(x)dx + M_{<1}\left(\frac{1}{2}\right)^{n-1}\int_{-\infty}^{0} x\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}dx$$
$$\approx \int_{0}^{+\infty} x f_{M_{n_X}}(x)dx + \frac{M_{<1}}{\sqrt{2\pi}}\left(\frac{1}{2}\right)^{n-1}$$
$$see\ that:\ \int_{-\infty}^{0} x\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}dx = \frac{1}{\sqrt{2\pi}}$$

Next we would use the approximation that "n>>1" to dismiss the term $\frac{1}{\sqrt{2\pi}}\left(\frac{1}{2}(1-\varepsilon_{0\leq\cdots\leq 1})\right)^{n-1}$ that simply expresses the fact that the part of the expectation that refers the negative values for the Max tends towards 0 very fast when "n" grows. In practice the "Max" tends to strictly higher and higher positive values in expectation anyway. The second term turns out to be negligible based on the fact that the probability law of the normal distribution is inferior to ½ for any negative value and therefore $0 \leq [\Phi(x)]^n \leq \frac{1}{2^n}$.

If we consider the density of the "Max" it tends to spike towards a value where the probability law comes across an inflection point, ie the derivative of the density crosses "0".

Let's look at the function $g(x) = \frac{d\left[f_{M_{n_X}}(x)\right]}{dx} = \frac{d\left[n\varphi(x)(\Phi(x))^{n-1}\right]}{dx} = \frac{d\left[n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\left(\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du\right)^{n-1}\right]}{dx}$

$$g(x) = n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\left(\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du\right)^{n-2}\left[-x\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du + (n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]$$

$$Therefore\ g(x) = 0 \Leftrightarrow x\underbrace{\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du}_{\Phi(x)} = (n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2} \Leftrightarrow xe^{+\frac{1}{2}x^2}\underbrace{\left(1-erfc\left(\frac{x}{\sqrt{2}}\right)\right)}_{\Phi(x)} = (n-1)\frac{1}{\sqrt{2\pi}}$$

We can observe here that the left hand-side function is monotonic in « «x », ie $xe^{+\frac{1}{2}x^2}\left(1-erfc\left(\frac{x}{\sqrt{2}}\right)\right) = h(x)$ is strictly growing with « x » and it diverges towards infinity. So, while "n" will tend towards infinity, so will "x" albeit at a different pace.

So if now we use the approximation above for any "n" value we find that:

$$h(x) = xe^{+\frac{1}{2}x^2}\left(1-erfc\left(\frac{x}{\sqrt{2}}\right)\right) = \frac{n-1}{\sqrt{2\pi}}$$
$$\approx xe^{+\frac{1}{2}x^2}\left[1-\frac{1}{\sqrt{2\pi}}\frac{e^{-\frac{1}{2}x^2}}{x}\left[1-\frac{1}{x^2}+\frac{3}{x^4}-\frac{15}{x^6}+\cdots+\frac{(-1)^k(2k)!}{2^k x^{2k}}+\cdots\right]\right]$$

in a simplified way we get:

$$\frac{n-1}{\sqrt{2\pi}} \approx xe^{+\frac{1}{2}x^2} - \frac{1}{\sqrt{2\pi}}\left[1-\frac{1}{x^2}+\frac{3}{x^4}-\frac{15}{x^6}+\cdots+(-1)^k\frac{(2k)!}{2^k x^{2k}}+\cdots\right]$$

Let's keep in mind then the core equation :

$$\frac{n}{\sqrt{2\pi}} + \frac{1}{2\sqrt{\pi}}\left[-\frac{1}{x^2} + \frac{3}{x^4} - \frac{15}{x^6} + \cdots (-1)^k \frac{(2k)!}{2^k x^{2k}} + \cdots\right] \approx xe^{+\frac{1}{2}x^2}$$

Thus solving at first order we would say that:

$$\frac{n}{\sqrt{2\pi}} \approx xe^{+\frac{1}{2}x^2} \; ie \; for \; n \; large \; \ln(x) \ll x \; and \; thus \; x \approx \sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)} \; since \; Ln(x) + \frac{1}{2}x^2 \approx Ln\left(\frac{n}{\sqrt{2\pi}}\right)$$

At second order we can simply use the first result ie:

$$\frac{n}{\sqrt{2\pi}} \approx xe^{+\frac{1}{2}x^2}$$

This simplifies down to : $\frac{1}{2}x^2 \approx Ln\left(\frac{n}{\sqrt{2\pi}}\right) \; using \; \frac{1}{2}x^2 \gg Ln(x)$

We can go further down the road using the other terms on the follow and better approximate the expectation of the "Max" while "n" is lower and lower.

Now we should not forget the initial approximation that was made on the negative , still possible values that were made on the Max itself since this expectation stops being negligible if we lower "n". The expectation that we measure here reflects the emergence of some Dirac distribution at the expected value while the rest of the distribution converges towards 0. Indeed, we got rid of the "negative" range of the integral sum and we are left with the "positive". Here , on the "positive" part of the integral sum, when "n" grows higher and higher the underlying distribution converges towards a Dirac measure, ie a "spike".

If we compute at which value "x" we find the top of the density, we are close to the inflexion point for the probability law of the max we find (since the density is the first derivative of the probability law). So the density function for the "Max" is as we know:

$$f_{M_{n_X}}(x_{top}) = n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}^2}\left(\int_{-\infty}^{x_{top}}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du\right)^{n-1}$$

$$with \; at \; first \; order: \; \frac{n}{\sqrt{2\pi}} \approx x_{top}e^{+\frac{1}{2}x_{top}^2}$$

$$\int_{-\infty}^{x_{top}}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du = 1 - \int_{x_{top}}^{+\infty}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du$$

$$= 1 - \frac{1}{\sqrt{2\pi}}\frac{e^{-\frac{1}{2}x^2}}{x}\left[1 - \frac{1}{x^2} + \frac{3}{x^4} - \frac{15}{x^6} + \cdots + \frac{(-1)^k(2k)!}{2^k x^{2k}} + \cdots\right]$$

Again we can use the approximation provided by the erfc function and the approximation above :

$$f_{M_{n_X}}(x_{top}) \approx n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}^2}\left(1 - \frac{1}{\sqrt{2\pi}}\frac{1}{x_{top}e^{+\frac{1}{2}x_{top}^2}}\right)^{n-1} \quad with \; e^{+\frac{1}{2}x_{top}^2} \approx \frac{n}{x_{top}\sqrt{2\pi}}$$

We can notice that $x_{top}\sqrt{2\pi}e^{+\frac{1}{2}x_{top}^2} \approx n \; and \; x_{top} \approx \frac{n}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}^2}$

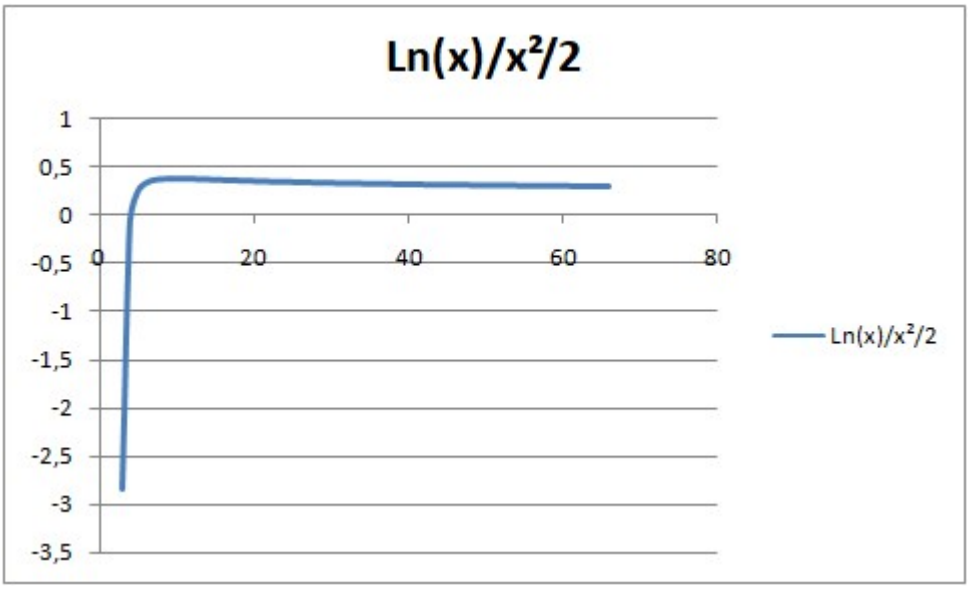
Considering that we can approximate x_{top} as explained above we can simplify the top of the distribution of the max with still a use of the first order that will serve to refine this approximation on the follow :

$$f_{M_{n_X}}(x_{top}) \approx x_{top}\left(1-\frac{1}{n}+\cdots\right)^{n-1} \; with \begin{cases} e^{+\frac{1}{2}{x_{top}}^2} \approx \dfrac{n}{x_{top}\sqrt{2\pi}} \\ x_{top} \approx \sqrt{2Ln\left(\dfrac{n}{\sqrt{2\pi}}\right)}+\cdots \end{cases}$$

We can notice that the formulation is very simple and that the top of distribution slowly grows as a function of "n" but the first order approximation error is in $\frac{1}{n}$.

Let's pause a little and look at the allure of the $x_{top} \approx \sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}$ function and at the approximation that we made to get this result: $\frac{1}{2}{x_{top}}^2 \gg Ln(x_{top})$

This approximation is not quite valid per se as the chart below shows: the approximation is close to 0.3 as a ratio for $\frac{Ln(x_{top})}{\frac{1}{2}x_{top}^2}$ due to the fact that $x_{top}\ is\ close\ to\ 2-3\ in\ value$



So we should consider instead a better estimation through an appropriate development of 'Ln(x)' from '$\sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}$' in order to keep the formulas simple. The formula shows that as we adjust the value for "x" then the approximations change too. So we will have to proceed recursively

Our core approximation formula is:

$$Ln(x)+\frac{1}{2}x^2 \approx Ln\left(\frac{n}{\sqrt{2\pi}}\right)$$

Let's write $x_0 = \sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}$. When shifting the value of "x" from x_0 to x_1 we will add an adjustment δ in a convenient way given the core equation that we deal with here.

We discounted "ln(x)" too much from the start with the first approximation and that was gross.

Let's therefore write a development of "Ln" based on $x_1 = (\delta + 1)x_0$ at the 2nd order to make sure that we miss as little as possible when we operate recursively one order after the other, ie $\ln(1+\delta) = \delta - \frac{1}{2}\delta^2$

$$Ln(x) + \frac{1}{2}x^2 \approx \ln\left(\sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}\right) + (\delta) - \frac{1}{2}\delta^2 + \frac{1}{2}2Ln\left(\frac{n}{\sqrt{2\pi}}\right)(\delta+1)^2 \approx Ln\left(\frac{n}{\sqrt{2\pi}}\right)$$

Thus:

$$\frac{1}{2}\ln\left(2Ln\left(\frac{n}{\sqrt{2\pi}}\right)\right) + \delta\left(1 + 2Ln\left(\frac{n}{\sqrt{2\pi}}\right)\right) + Ln\left(\frac{n}{\sqrt{2\pi}}\right) + \delta^2\left(Ln\left(\frac{n}{\sqrt{2\pi}}\right) - \frac{1}{2}\right) \approx Ln\left(\frac{n}{\sqrt{2\pi}}\right)$$

We now neglect the term in δ^2 (to be verified…)

$$\delta \approx -\frac{\ln\left(\sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}\right)}{\left(1 + 2Ln\left(\frac{n}{\sqrt{2\pi}}\right)\right)} \approx -\frac{\ln(x_0)}{(1+x_0^2)}$$

So rather than pick $x_0 \approx \sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}$, we had rather take $x_1 = (\delta + 1)x_0$

$$x_1 \approx \sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}\left(1 - \frac{\ln\left(\sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}\right)}{\left(1 + 2Ln\left(\frac{n}{\sqrt{2\pi}}\right)\right)}\right) \approx x_0\left(1 - \frac{\ln(x_0)}{(1+x_0^2)}\right)\ldots$$

We need to validate the approximation when we neglected the term in δ^2.

If we look at the values of: the ratio: $\frac{\delta^2\left(Ln\left(\frac{n}{\sqrt{2\pi}}\right)-\frac{1}{2}\right)}{\delta\left(1+2Ln\left(\frac{n}{\sqrt{2\pi}}\right)\right)} \approx \frac{1}{2}\frac{\frac{\ln(x_0)}{(1+x_0^2)}(x_0^2-1)}{(1+x_0^2)} \approx \frac{1}{2}\frac{\ln(x_0)(x_0^2-1)}{(1+x_0^2)^2}$ $with\ x_0 = \sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}$

The below below shows, from n>5 onwards how this ratio varies it is at best of the order 10^{-5}:

n	x_0	ratio	delta
5	1,175159	4,54841E-05	-0,06779
6	1,321227	7,58608E-05	-0,10146
7	1,433158	7,58588E-05	-0,11784
8	1,523485	6,58159E-05	-0,12677
9	1,598928	5,43242E-05	-0,13196
10	1,663518	4,40857E-05	-0,13509
11	1,719859	3,56646E-05	-0,137
12	1,769728	2,89409E-05	-0,13815
13	1,814393	2,36241E-05	-0,1388
14	1,854788	1,94221E-05	-0,13913
15	1,891619	1,60882E-05	-0,13923
16	1,925435	1,34271E-05	-0,13918
17	1,956668	1,1288E-05	-0,13902
18	1,985665	9,55584E-06	-0,13878
19	2,01271	8,14276E-06	-0,13848
20	2,038035	6,98164E-06	-0,13815
21	2,061836	6,0209E-06	-0,1378
22	2,084276	5,22066E-06	-0,13742
23	2,105496	4,54989E-06	-0,13704
24	2,125613	3,98427E-06	-0,13665
25	2,144732	3,50462E-06	-0,13626
26	2,162942	3,09571E-06	-0,13586
27	2,18032	2,74534E-06	-0,13547
28	2,196937	2,44369E-06	-0,13508
29	2,212852	2,18283E-06	-0,1347
30	2,22812	1,95627E-06	-0,13432
31	2,242788	1,75872E-06	-0,13395
32	2,256899	1,5858E-06	-0,13358
33	2,270493	1,43389E-06	-0,13322

So we make an appropriate approximation here. But we also notice that the delta remains high. Can we refine this further?

Then we should wonder what the residual error is now…

We are based on this approximation that we refine step by step: $Ln(x_{top}) + \frac{1}{2}{x_{top}}^2 \approx Ln\left(\frac{n}{\sqrt{2\pi}}\right)$

If we want to estimate the residual error we should write that $x_2 = x_1(1+\varepsilon)$

Replacing the expression we get $Ln(x_1(1+\varepsilon)) + \frac{1}{2}(x_1(1+\varepsilon))^2 \approx Ln\left(\frac{n}{\sqrt{2\pi}}\right)$

Running the approximation for the Ln function at 3rd order let's validate that our "epsilon" indeed is <<1…

$$Ln(x_1(1+\varepsilon)) \approx \ln(x_1(1+\varepsilon)) \approx \ln(x_1) + (\varepsilon) - \frac{1}{2}(\varepsilon)^2$$

Therefore , as in the former step, we use the core relation:

$$Ln(x_1+\varepsilon)+\frac{1}{2}(x_1(1+\varepsilon))^2 \approx Ln\left(\frac{n}{\sqrt{2\pi}}\right) \approx \ln(x_1)+(\varepsilon)-\frac{1}{2}(\varepsilon)^2+\frac{1}{2}x_1^2(1+\varepsilon)^2$$

We would find a similar equation next: but this time:

$$x_1 \approx x_0\left(1-\frac{\ln(x_0)}{(1+x_0^2)}\right)\ldots x_0=\sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}$$

$$\frac{1}{2}x_0^2 \approx \ln(x_1)+(\varepsilon)-\frac{1}{2}(\varepsilon)^2+\frac{1}{2}x_1^2(1+\varepsilon)^2$$

We would factorize: $\frac{1}{2}x_0^2-\frac{1}{2}x_1^2-\ln(x_1) \approx \varepsilon(1+x_1^2)+\frac{1}{2}(\varepsilon)^2(x_1^2-1)$

We can remain confident that we can pick the term of first order alone:

: $\varepsilon \approx \frac{\frac{1}{2}x_0^2-\frac{1}{2}x_1^2-\ln(x_1)}{(1+x_1^2)} < 0$

Running the verification we see now that the “epsilon” is very small which tells us that we can stop here: the order of magnitude for ε ie $6\,10^{-3}$ and the relative approximation is in the range of 10^{-3}. So the ultimate accurate expression will be $x_{top} \approx x_2 = x_1(1+\varepsilon) = x_0\left(1-\frac{\ln(x_0)}{(1+x_0^2)}\right)\left(1+\frac{\frac{1}{2}x_0^2-\frac{1}{2}x_1^2-\ln(x_1)}{(1+x_1^2)}\right)$

We can simplify a bit the expression:

$$x_{top} \approx x_1\left(\frac{1+\frac{1}{2}x_0^2+\frac{1}{2}x_1^2-\ln(x_1)}{(1+x_1^2)}\right) \; with\; x_1 \approx x_0\left(1-\frac{\ln(x_0)}{(1+x_0^2)}\right)\ldots x_0=\sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}$$

There is little more that we can do to simplify further the expression.

The table below gathers the details and we see that the approximation works fine from n=5.

n	x_0	ratio	delta	x_1	epsilon	ratio approx 2
5	1,175159	4,54841E-05	-0,06779	1,095497	-0,00035	-1,58333E-05
6	1,321227	7,58608E-05	-0,10146	1,187181	-0,00144	-0,000122026
7	1,433158	7,58588E-05	-0,11784	1,264273	-0,00259	-0,000297753
8	1,523485	6,58159E-05	-0,12677	1,330354	-0,00356	-0,000494875
9	1,598928	5,43242E-05	-0,13196	1,38793	-0,00434	-0,00068703
10	1,663518	4,40857E-05	-0,13509	1,438789	-0,00495	-0,000863496
11	1,719859	3,56646E-05	-0,137	1,484235	-0,00544	-0,001021238
12	1,769728	2,89409E-05	-0,13815	1,52524	-0,00582	-0,001160557
13	1,814393	2,36241E-05	-0,1388	1,562547	-0,00613	-0,001283046
14	1,854788	1,94221E-05	-0,13913	1,596731	-0,00637	-0,001390665
15	1,891619	1,60882E-05	-0,13923	1,628246	-0,00657	-0,00148535
16	1,925435	1,34271E-05	-0,13918	1,657457	-0,00673	-0,001568861
17	1,956668	1,1288E-05	-0,13902	1,684661	-0,00686	-0,001642737
18	1,985665	9,55584E-06	-0,13878	1,710101	-0,00697	-0,001708302
19	2,01271	8,14276E-06	-0,13848	1,733982	-0,00705	-0,00176668
20	2,038035	6,98164E-06	-0,13815	1,756473	-0,00713	-0,001818827
21	2,061836	6,0209E-06	-0,1378	1,777721	-0,00719	-0,001865553
22	2,084276	5,22066E-06	-0,13742	1,797847	-0,00723	-0,001907543
23	2,105496	4,54989E-06	-0,13704	1,816959	-0,00727	-0,001945384
24	2,125613	3,98427E-06	-0,13665	1,83515	-0,0073	-0,001979574
25	2,144732	3,50462E-06	-0,13626	1,8525	-0,00733	-0,002010542
26	2,162942	3,09571E-06	-0,13586	1,869079	-0,00735	-0,002038655
27	2,18032	2,74534E-06	-0,13547	1,88495	-0,00736	-0,002064231
28	2,196937	2,44369E-06	-0,13508	1,900169	-0,00737	-0,002087543
29	2,212852	2,18283E-06	-0,1347	1,914783	-0,00738	-0,002108832
30	2,22812	1,95627E-06	-0,13432	1,928837	-0,00739	-0,002128306
31	2,242788	1,75872E-06	-0,13395	1,942371	-0,00739	-0,002146147
32	2,256899	1,5858E-06	-0,13358	1,955419	-0,00739	-0,002162516
33	2,270493	1,43389E-06	-0,13322	1,968014	-0,00739	-0,002177553

Thus a proper estimate is obtained as soon as n>5. But this is only based on the “top” of the density. What if the distribution of the Max is skewed, which it is for the lowest values of “n”. In other words: “is the top of the density equal to the expectation?” in general this is not true if the density is not perfectly symmetrical, having only one relative top, around the top.

So we need to refine this estimate now based on that consideration. The idea is to find the highest slopes on the density around the top and use a “triangle” approximation to estimate the expectation of the Max distribution. We would indeed make a rather benign error with this triangle that would sketch the real distribution if the triangle joins the top to “0” via the maximum slope of the density function on the “left” and on the “right” of the top.

Let’s recall that our result comes from the derivation of the density of probability of the max that in nature is: $\frac{d\left[n\varphi(x)(\Phi(x))^{n-1}\right]}{dx} = \frac{d\left[n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\left(\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du\right)^{n-1}\right]}{dx} = 0$ which itself induces the equivalence below

$$x\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du = (n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2} \Leftrightarrow xe^{+\frac{1}{2}x^2}\left(1-erfc\left(\frac{x}{\sqrt{2}}\right)\right) = (n-1)\frac{1}{\sqrt{2\pi}}$$

Let's observe that the equation above can be generalized with other distributions since the law of the « Max » can be derived easily while the approximations remain rather straightforward if we project the distribution on the basis of Hermite's functions. Indeed we have above a linear differential equation and the connex terms are rather easily deducted recursively. The power laws, through the original derivation, would not alter the equation above:

$$x\int_{-\infty}^{x} Dens(u)du = (n-1)Dens(x)$$

$$with\ Dens(x) = \frac{1}{\sqrt{2\pi v^2}} e^{-\frac{1}{2}\frac{(x-x_m^2)^2}{v^2}} \sum_{k=0}^{+\infty} d_k \frac{A_k(-1)^k}{\sqrt{2\pi v^2}} e^{+\frac{1}{2}\frac{(x-x_m^2)^2}{v^2}} \quad \frac{d^k}{dx^k}\left[\frac{1}{\sqrt{2\pi v^2}} e^{-\frac{1}{2}\frac{(x-x_m^2)^2}{v^2}}\right]$$

$$and\ d_k = \int_{-\infty}^{+\infty} \frac{A_k(-1)^k}{\sqrt{2\pi v^2}} e^{+\frac{1}{2}\frac{(x-x_m^2)^2}{v^2}} \quad \frac{d^k}{dx^k}\left[\frac{1}{\sqrt{2\pi v^2}} e^{-\frac{1}{2}\frac{(x-x_m^2)^2}{v^2}}\right] Dens(u)du\ \ with\ A_k = \frac{1}{\sqrt{2\pi v^2 k!}}$$

We can see the "k!" divisor that greatly reduces the influence of the maximum of the density beyond the 2 or 3 first terms of the projection on the Hermite's functions basis.

So, we have covered so far the fact that the generally well knows estimate for the expectation of the Max, based on the "top" of the distribution (also named density of probability which is the first derivative of the probability law that governs the Max itself here), can be refined for n>5 very well. We can achieve any level of precision as far as the "top" is concerned. What about the inherent skewness of the distribution of the Max for a sum of variable that is otherwise symmetrical itself?

Indeed finding this top of the density does not guarantee that the "expectation" coincides with this maximum, especially if the distribution is skewed. However the "power law" off "n" will quickly reduce the impact of this skew anyway. How fast does it do it? What about the case when n=5 or 6?

This, taken in isolation, rather suggests that the higher moments per se do not bring looking forward this points to the magnitude of the higher moments in absolute value: they have to match with the "k!" scale to have a chance to weight on the "expected Max" that is actually seen so far as "the expectation of the maximum return" in our finance context.

Let's remind here what the framework and the premise are: we assume that returns average around some "0" in present discounted value terms and we look for the "best option" here that is defined by this "expected maximum return given a certain risk". Let's bear in mind that, should we look at the "downside" and therefore want to think in "stop loss" terms, the same approach prevails. Then we would want to define ahead of time the "expectation" of the maximum loss we should expect and this is again fully determined by this expectation.

We might as well want to define a standard deviation for the Max. The calculation is equally possible following the same method. It would be more tedious given that here we would focus on:

$$E\left[M_{n_{X^2}}\right]_X = \int_{-\infty}^{+\infty} x^2 f_{M_{n_X}}(x)dx = \int_{-\infty}^{+\infty} x\left(x f_{M_{n_X}}(x)\right)dx$$

This is the same approach but with a different proxy-density: instead of searching for the "top" of $f_{M_{n_X}}(x)$, we look for the top of $x f_{M_{n_X}}(x)$. This can be an interesting exercise to run.

Then, knowing both the expectation and variance of the Max, we can directly infer a sort of security bound that would add up say "expectation +1.5 standard deviation" as in Var measures in general. Or deploy even finer frameworks including "stress scenarios" and so on. All will based on "risks" that we compound

numerically in order to guide our intuition. The risk as such can only be a positive value. But actually, if we are to run a recursion that would actually factor higher and higher moments this would consist in defining a volatility regime within which there would be returns that depend on some inner volatility. We then again would look for a “maximum expected” return but through the lens of some 2D volatility: one defining the general volatility average regime, and the second defining an inner volatility conditioned by this broader regime that would back the fact that even the volatility is random in what yet is a constant regime for us. At the end of the day we would define an efficient surface. So many further developments are based on this method that we depict here.

The application in day to day trading or investing matters as a generic framework. And we also hinted at the fact that, people looking to diversify their risks, face the same problem with regards to the “n” assets or positions that they hold. How long should they hold each of them and all? What should the valuation triggers be once some outstanding change in P&L has occurred? Mathematically speaking we can observe that there is not much of a difference between “adding up random returns through the days” or “adding up random returns within a portfolio”.

Let’s just keep in mind then that we speak of “finance”, “profit” we speak of “maximum” versus “risk” , or put in different words “risk/reward” or else “Maximum expected gain” versus “stop loss”. The background here is that “we do not know what will happen” and we have to prepare for contingency plans if only not to lose ourselves. Of course the question is always double sided: “how much risk should I take now?” “How good is the current result? Should I lock some gains? Should I lock some losses and move on?” We are permanently walking along an “efficient path” for us and knowing the “efficient frontier” for our decision making process is central.

The general crucial concept consists in saying that there is a “fuel” that brings up some “expected maximum return” in a world where the expected return as such is “0 from a risk neutral” standpoint. Another way to put it “why the hell should I take risks if I cannot secure that this is worth it?” Of course, market driven economies rely on the fact that everyone has a different view on that question. The ground base indeed is to say that the expected return is the same across the asset classes if one does “nothing” but sits on the risk that is being taken. But, of course, doing nothing becomes stupid once we assume the existence of this “volatility” because of course our P&L will fluctuate and surprise us more than we ever wished for anyway. We here say that there are “regimes”, no “normal distribution” underneath the “expected returns” unless conjectural ones, and therefore we have to take action from time to time.

How to take the best course of action? This is when the “efficient frontier” theory becomes useful in showing that it all depends on the risk level that we target and what we can expect “at best but reasonably so too” from the risk we have undertaken so far. Then the “risk” exhibits the varying regimes that convey varying expectations locally in time. The higher the risk, the higher the expectation of the maximum return that we can lock. This view alone amounts to favor the boldest investors if and only if they make appropriate decisions on the way. Thus a “risk”, ie a “volatility”, is a fuel for the expectation of some “Maximum return”. In that it is itself an expectation of a maximum return while this underlying return might be negative as well, depending on the instance. So, the “volatility” is also a way to frame a “regime” and therefore we can define for each such “regime” a range of maximum expected returns that would be conditioned by some underlying sub-regime of volatility. This is always made “possible” because the volatility is inherently a random variance that should be zero if we had really a normal distribution underneath . But it is not that. It therefore has a distribution that is best captured as a “return” in fact.

Now that we can in theory always morph some volatility “back” into a distribution of conditioned returns , the condition being a given “regime of ambient volatility”, and now that we can finely estimate the Dirac-like point for the expectation of the Max where we would spot the top of the distribution of this max, we

have to wonder whether this inflection point for the probability law is also the expectation of the probability law.

Knowing the top of the distribution, what would the best proxy distribution be?

We can figure here that this distribution looks like some triangle with a top and 2 edges. The mid points of the edges on the left and the right of the top can be proxied with the fact that the density of Max will rise from "zero" towards the top, its derivative accelerating towards a maximum value and decelerating when reaching the top to be worth exactly 0 at the top. Next, rolling towards the right of the top, the density declines back to 0 accelerating on the way down but ultimately staying positive. This mandates that the slope of the density reaches a minimum in the negative territory and comes back to 0 itself.

This points to 2 inflection points where the second derivative of the distribution of the Max is nil.

We can use the erfc function for the negative value in order to proxy the whole integral. As to the positive values and the fact that we approximate the integral in the positive territory for "x" with a Dirac distribution when it is never really the case but we are "close enough" for most n values.

Note:

How can we empirically infer the distribution of the Max?

We know that the « Max » is the limit of $\left[\sum_{k=1}^{k=n} X_k^m\right]^{\frac{1}{m}}$ *when* $m \to +\infty$ *with each* X_k *having its own distribution. So the distribution extracted from this formula will converge towards the distribution of the Max itself.*

What would happen if we used the approximation ?

$$1 - \Phi(x) = erfc\left(\frac{x}{\sqrt{2}}\right) \approx \frac{1}{\sqrt{2\pi}} \frac{e^{-\frac{1}{2}x^2}}{x}\left[1 - \frac{1}{x^2} + \frac{3}{x^4} - \frac{15}{x^6} + \cdots \frac{(2k)!}{2^k x^{2k}} + \cdots\right]$$

Reminder:

Into the formula, that expresses the approximations, for the part where x>0:

$$E\left[M_{n_X}\right]_X = \int_{-\infty}^{+\infty} x f_{M_{n_X}}(x)dx = \int_{-\infty}^{0} x f_{M_{n_X}}(x)dx + \int_{0}^{+\infty} x f_{M_{n_X}}(x)dx$$

$where\ f_{M_{n_X}}(x) = n\varphi(x)\big(\Phi(x)\big)^{n-1} with\ \varphi(x) = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}\ and\ \Phi(x) = \int_{-\infty}^{x} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}u^2} du$

The problem now is, knowing the proxy value of the expectation of the Max as a function "n", ie $x \approx \sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)} \gg 1\ for\ n\ large\ enough$, we know that we can refine this proxy but we also need to include the fact that our integration itself assumed a Dirac like spike on this estimate which is not quite right.

We should consider again the function:

$$g(x) = n\frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}\left(\int_{-\infty}^{x} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}u^2} du\right)^{n-2}\left[-x\int_{-\infty}^{x} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}u^2} du + (n-1)\frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}\right]$$

We found the optimum by deriving once the distribution of the Max:

$$\frac{d\left[n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\left(\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du\right)^{n-1}\right]}{dx}$$

We know that the function of the distribution has a top and this function itself tends generally to be nil when "n" is high enough almost everywhere except in a shrinking neighborhood of the "top". This top itself is where the function "g" is nil swinging there from positive values to negative values but towards the extreme bounds, the function "g" gets back to 0 anyway. This implies, moving from $-\infty$ to $+\infty$, that the first derivative of "g" is nil on the extreme bounds, move to positive values and back to 0 at the "top" of "g", and next drop to negative values before reverting back to zero. So we should look at the zeros of the second derivative since we know this 1st derivative will reach a top from almost zero first, then cross 0 as we found out so far, and will reach a bottom before coming back to 0. This indicates that the second derivative will have 2 zeros that will bracket the 0 of the first derivative that we have already found.

$$\frac{dg}{dx}(x)=\frac{d^2\left[n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\left(\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du\right)^{n-1}\right]}{dx^2}$$

$$\frac{dg}{dx}(x)=\frac{d}{dx}\left[n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\left(\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du\right)^{n-2}\left[-x\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du+(n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]\right]$$

$$\begin{aligned}\frac{dg}{dx}(x)=n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}&\left(\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du\right)^{n-3}\left[-x\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du+(n\right.\\&-1)\left.\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]\left[-x\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du+(n-2)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]\\&+n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\left(\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du\right)^{n-2}\left[-\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du-\frac{x}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}-x(n\right.\\&-1)\left.\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]\end{aligned}$$

Hence we look for the x values that nullify the expression below:

$$\begin{aligned}&\left[-x\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du+(n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]\left[-x\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du+(n-2)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]\\&+\left(\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du\right)^{1}\left[-\int_{-\infty}^{x}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du-\frac{x}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}-x(n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]=0\end{aligned}$$

We can simplify:

$$\left[-x\Phi(x)+(n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]\left[-x\Phi(x)+(n-2)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]=\Phi(x)\left[\Phi(x)+xn\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]$$

$$\left[-x\Phi(x)+(n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]\left[-x\Phi(x)+(n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}-\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]=\Phi(x)\left[\Phi(x)+xn\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]$$

Next, considering that the distribution quickly becomes like a Dirac measure, we would simply consider that the zeros of the second derivative are close to the zero of the first derivative, ie "g(x)", and that the

computation of the expectation is run along a "triangle" that is defined by the zeros of second derivative as "mid-points" relative to the top of distribution itself (in first approximation again, assuming that n is large enough). We then expect to find the key points in this "triangle" that is drawn by the density of probability to be close to the initial gross estimate anyway…. When it will actually converge towards a Dirac measure when "n" increases. We here mostly care about the cases when "n">5 while not being really high: is the distribution then "symmetrical" relative to the "top" that we estimated so far for the density itself?

We should morph this expression based on the "top" of the density that actually nullifies the term $\underbrace{\left[-x\Phi(x)+(n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x^2}\right]}_{\delta}$ and consider the change in value for the derivative δ that corresponds to solving the following equation, where $x_\delta = x_{top}(1+\varepsilon) with\ |\varepsilon|<1\ in\ general$ as a working assumption:

$$[\delta]\left[\delta-\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}(x_\delta)^2}\right]=\Phi(x_\delta)\left[\Phi(x_\delta)+x_\delta n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}{x_\delta}^2}\right]$$

$x_{top}\approx\sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}\ (\dots) and\ x_\delta = x_{top}(1+\varepsilon).$

We cannot really assume here that $|\varepsilon x_{top}| \ll 1$ due to the diverging nature of x_{top} and due to the fact that, also, we want to consider the case of smaller values for "n" in the approximation. The whole purpose for the calculation that follows is to sketch a "triangle" that will be defined from the highest slopes (upward and downward) of the density of probability of the "Max" for the case of an underlying normal distribution. We know that the Max is not following a normal distribution but it quickly tends towards a Dirac measure as a density function. In the meantime, since we mostly care about the "expectation of the Max for n that is not necessarily large" we would shrink the density function down to some "triangle" shaped by the slope δ at some estimated point $x_\delta = x_{top}(1\pm\varepsilon_\pm)$. Computing the expectation would amount finding the intersection point between the 2 lines, plus the intersection of these lines with the abscis axis "x" and next run a simple "integral" like:

$\frac{1}{A}\int_{a=x=0\ \ for\ \delta>0}^{lines\ intersect} x(x-a)\delta_+dx+\frac{1}{A}\int_{lines\ intersect}^{b=x=0\ \ for\ \delta<0} x(x-b)\delta_-dx = E[Max]$ while

$$\frac{1}{A}\int_{a=x=0\ \ for\ \delta>0}^{lines\ intersect}(x-a)\delta_+dx+\frac{1}{A}\int_{lines\ intersect}^{b=x=0\ \ for\ \delta<0}(x-b)\delta_-dx=1$$

Once we know $x_\delta = x_{top}(1+\varepsilon)$ that we will infer from knowing $\delta\pm$ we can set the lines intersection points and sketch a rather accurate estimate for the expectation of the max on the grounds that the density function is well approached by this triangle since the differences will naturally offset one each other. But we might as well find out that the triangle is pretty much symmetrical around x_{top}…

So, when we have this triangle and we actually place the origin at x_{top}, if we want to compute the "expectation of the max" this will amount to the surface of the triangle PLUS x_{top}but this surface will then add up a negative half triangle on the left of x_{to} and a positive half triangle on the right. Since each left and right triangles are orthogonal in x_{top}each having $2\varepsilon_\pm$as basis and both having the top of the density in x_{top} as a shared "height", we infer that :

$$x_{exp_{max}} = x_{top}+x_{top}(\varepsilon_+ + \varepsilon_-)$$

This should anyway provide us with a reliable "function" of the expectation of the Max even when "n", that is in our case an auto-correlation induced factor that might not be very large, is not big. This will allow us to infer the Lagrangian constraint value that fits the efficient frontier.

We know that by definition we have:

$$\left[-x_{top}\Phi\left(x_{top}\right)+(n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2}\right]=0\ so\ x_{top}\Phi\left(x_{top}\right)=(n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2}$$

To be sure the top of the density is: $T_{op}=\ n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2}\left(\int_{-\infty}^{x_{top}}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}u^2}du\right)^{n-1}$

$$T_{op}=\ n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2}\left(\Phi\left(x_{top}\right)\right)^{n-1}=\ \frac{n}{n-1}x_{top}\Phi\left(x_{top}\right)\left(\Phi\left(x_{top}\right)\right)^{n-1}$$

$$T_{op}=\ \frac{n}{n-1}x_{top}\left(\Phi\left(x_{top}\right)\right)^{n}$$

We have also $\delta=-x_\delta\Phi(x_\delta)+(n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_\delta{}^2}$. Using the equation above and our assumption that $|\varepsilon|\ll 1$ anyway, we could write a first and second order estimate of δ:

$$\Phi(x_\delta)\approx\ \Phi\left(x_{top}\right)+\varepsilon x_{top}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2}-\frac{1}{2}\left(\varepsilon x_{top}\right)^2\frac{x_{top}}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2}$$

$$\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_\delta{}^2}\approx\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2}-\varepsilon x_{top}\frac{x_{top}}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2}+\frac{1}{2}\left(\varepsilon x_{top}\right)^2\frac{1}{\sqrt{2\pi}}\left[-e^{-\frac{1}{2}x_{top}{}^2}+x_{top}^2e^{-\frac{1}{2}x_{top}{}^2}\right]$$

By the "law" of the triangle we would look to set : $-2\delta x_{top}\varepsilon\approx T_{op}=\frac{n}{n-1}x_{top}\left(\Phi\left(x_{to}\ \right)\right)^{n}$

We start from our latest equation:

$$[\delta]\left[\delta-\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}(x_\delta)^2}\right]=\Phi(x_\delta)\left[\Phi(x_\delta)+x_\delta n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_\delta{}^2}\right]$$

Let's recall here that : $\left[-x_{top}\Phi\left(x_{top}\right)+(n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2}\right]=0$

Let's remember that this x_{top} value was rather well approximated with the help of the erfc function but in 2 stages at least. For the record and clarity, here is what we obtained:

$$x_{top}\approx x_1\left(\frac{1+\frac{1}{2}x_0^2+\frac{1}{2}x_1^2-\ln(x_1)}{(1+x_1^2)}\right)\ with\ x_1\approx x_0\left(1-\frac{\ln(x_0)}{(1+x_0^2)}\right)\ldots x_0=\sqrt{2Ln\left(\frac{n}{\sqrt{2\pi}}\right)}$$

We start with a gross assumption: $0<\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}(x_\delta)^2}\ll\delta$

Thus :

$$\delta^2=\Phi(x_\delta)\left[\Phi(x_\delta)+x_\delta n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_\delta{}^2}\right]$$

Also by keeping just the 1st order we obtain for the 1st term above:

$$\Phi(x_\delta)\approx\left(\Phi\left(x_{top}\right)+\varepsilon x_{top}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2}\right)$$

Likewise, keeping jus the first order for the other term: remembering $x_\delta=x_{top}(1+\varepsilon)$

$$\Phi(x_\delta)+x_\delta n\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_\delta{}^2}\approx\left(\frac{x_\delta n}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2}-\varepsilon x_{top}x_\delta n\frac{x_{top}}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2}\right)+\left(\Phi\left(x_{top}\right)+\varepsilon x_{top}\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2}\right)$$

$$\Phi(x_\delta) + x_\delta n \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_\delta}^2} \approx \left(\frac{x_{top}(1+\varepsilon)n}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_{top}}^2} - \varepsilon x_{top} x_{top}(1+\varepsilon)n \frac{x_{top}}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_{top}}^2}\right) + \left(\Phi(x_{top}) + \varepsilon x_{top} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_{top}}^2}\right)$$

$$\Phi(x_\delta) + x_\delta n \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_\delta}^2} \approx \left(\frac{x_{top}(1+\varepsilon)n}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_{top}}^2} + \Phi(x_{top})\right) + \left(\varepsilon x_{top} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_{top}}^2}\right)\left(1 - (1+\varepsilon)n \frac{x_{top}^2}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_{top}}^2}\right)$$

$$\Phi(x_\delta) + x_\delta n \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_\delta}^2} \approx \left(n \frac{x_{top}}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_{top}}^2} + \Phi(x_{top})\right) + \left(\varepsilon x_{top} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_{top}}^2}\right)\left(1 + n - (1+\varepsilon)n \frac{x_{top}^2}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_{top}}^2}\right)$$

Using $x_{top}\Phi(x_{top}) = (n-1)\frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_{top}}^2}$ we can remove the terms in exponential since:

$$\frac{x_{top}\Phi(x_{top})}{n-1} = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_{top}}^2}$$

The first multiplicand becomes:

$$\Phi(x_\delta) \approx \left(\Phi(x_{top}) + \varepsilon x_{top} \frac{x_{top}\Phi(x_{top})}{n-1}\right) \approx \Phi(x_{top})\left(1 + \varepsilon x_{top} \frac{x_{top}}{n-1}\right)$$

The second of the 2 terms become:

$$\Phi(x_\delta) + x_\delta n \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_\delta}^2} \approx \left(n x_{top} \frac{x_{top}\Phi(x_{top})}{n-1} + \Phi(x_{top})\right) + \left(\varepsilon x_{top} \frac{x_{top}\Phi(x_{top})}{n-1}\right)\left(1 + n - (1+\varepsilon)n x_{top}^2 \frac{x_{top}\Phi(x_{top})}{n-1}\right)$$

$$\Phi(x_\delta) + x_\delta n \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_\delta}^2} \approx \frac{n}{n-1} x_{top}\Phi(x_{top})\left(x_{top} + \frac{\mathrm{n}-1}{n x_{top}} + \varepsilon x_{top}\left(1 + \frac{1}{n} - (1+\varepsilon)x_{top}^3 \frac{\Phi(x_{top})}{n-1}\right)\right)$$

The root formula being ; $\delta^2 = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_\delta}^2} \Phi(x_\delta)\left[\Phi(x_\delta) + x_\delta n \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_\delta}^2}\right]$, ce qui donne :

$$\delta^2 \approx \Phi(x_{top})\left(1 + \varepsilon x_{top} \frac{x_{top}}{n-1}\right) \frac{n}{n-1} x_{top}\Phi(x_{top})\left(x_{top} + \frac{\mathrm{n}-1}{n x_{top}} + \varepsilon x_{top}\left(1 + \frac{1}{n} - (1+\varepsilon)x_{top}^3 \frac{\Phi(x_{top})}{n-1}\right)\right)$$

Therefore :

$$\delta^2 \approx \frac{n}{n-1} x_{top}\Phi(x_{top})^2\left(1 + \varepsilon x_{top} \frac{x_{top}}{n-1}\right)\left(x_{top} + \frac{\mathrm{n}-1}{n x_{top}} + \varepsilon x_{top}\left(1 + \frac{1}{n} - (1+\varepsilon)x_{top}^3 \frac{\Phi(x_{top})}{n-1}\right)\right)$$

$$\delta^2 \approx \frac{n}{n-1} x_{top}^2\Phi(x_{top})^2\left(1 + \varepsilon x_{top} \frac{x_{top}}{n-1}\right)\left(1 + \frac{\mathrm{n}-1}{n x_{top}^2} + \varepsilon x_{top}\left(\frac{1}{x_{op}} + \frac{1}{n x_{top}} - (1+\varepsilon)x_{top}^2 \frac{\Phi(x_{top})}{n-1}\right)\right)$$

$$\delta^2 \approx \frac{n}{n-1} x_{top}^2\Phi(x_{top})^2\left(1 + \frac{\mathrm{n}-1}{n x_{top}^2} + \varepsilon x_{top}\left(1 + \frac{\mathrm{n}-1}{n x_{top}^2} + \frac{n+1}{n x_{top}} - (1+\varepsilon)x_{top}^2 \frac{\Phi(x_{top})}{n-1}\right)\right)$$

So while we have, at first order, excluding the term in εx_{top}, $-2\delta x_{top}\varepsilon \approx T_{op} = \frac{n}{n-1}x_{top}\left(\Phi\left(x_{top}\right)\right)^n$

We should now consider that $\delta \approx \pm\sqrt{\frac{n}{n-1}}\mathrm{x_{top}}\Phi\left(x_{top}\right)\sqrt{1+\frac{\mathrm{n-1}}{nx_{top}^2}}$ and infer that: $x_{top}\varepsilon \approx -\frac{1}{2}\frac{n}{n-1}\frac{x_{top}}{\delta}\left(\Phi\left(x_{top}\right)\right)^n$

Let's quickly confirm that when $x_\delta \geq 1.2\ then\ \frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}(x_\delta)^2} = 0.19$..This is not always the case but it happens soon when "n" grows….

So, $x_{top}\varepsilon \approx -\frac{1}{2}\frac{\sqrt{\frac{n}{n-1}}\left(\Phi\left(x_{top}\right)\right)^{n-1}}{\sqrt{1+\frac{\mathrm{n-1}}{nx_{top}^2}}}$ which as such does not justify the approximation.

We need to apply a finer approximation from the formula below:

$$\delta^2 \approx \frac{n}{n-1}x_{top}^2\Phi\left(x_{top}\right)^2\left(1+\frac{\mathrm{n-1}}{nx_{top}^2}+\varepsilon x_{top}\left(1+\frac{\mathrm{n-1}}{nx_{top}^2}+\frac{n+1}{nx_{top}}-(1+\varepsilon)x_{top}^2\frac{\Phi\left(x_{top}\right)}{n-1}\right)\right)$$

And we would inject the prior estimate for $\varepsilon x_{top} = \left(\varepsilon x_{top}\right)_1 = \pm\frac{\frac{1}{2}\sqrt{\frac{n}{n-1}}\left(\Phi\left(x_{top}\right)\right)^{n-1}}{\sqrt{1+\frac{\mathrm{n-1}}{nx_{top}^2}}}$

based on $\delta_1 \approx \pm\sqrt{\frac{n}{n-1}}\mathrm{x_{top}}\Phi\left(x_{top}\right)\sqrt{1+\frac{\mathrm{n-1}}{nx_{top}^2}}$

We would infer :

$$\delta_2{}^2 \approx \frac{n}{n-1}x_{top}^2\Phi\left(x_{top}\right)^2\left(1+\frac{\mathrm{n-1}}{nx_{top}^2}\pm\frac{1}{2}\sqrt{\frac{n}{n-1}}\left(\Phi\left(x_{top}\right)\right)^{n-1}\left(1+\frac{\mathrm{n-1}}{nx_{top}^2}+\frac{n+1}{nx_{top}}-(1+\varepsilon_1)x_{top}^2\frac{\Phi\left(x_{top}\right)}{n-1}\right)\right)$$

Then $\left(\varepsilon x_{top}\right)_2 = \pm\frac{\frac{\frac{1}{2}\sqrt{\frac{n}{n-1}}\left(\Phi\left(x_{top}\right)\right)^{n-1}}{\sqrt{1+\frac{\mathrm{n-1}}{nx_{top}^2}}}}{\sqrt{\left(1+\frac{\mathrm{n-1}}{nx_{top}^2}\pm\frac{1}{2}\sqrt{\frac{n}{n-1}}\left(\Phi\left(x_{top}\right)\right)^{n-1}\left(1+\frac{\mathrm{n-1}}{nx_{top}^2}+\frac{n+1}{nx_{top}}-(1+\varepsilon_2)x_{top}^2\frac{\Phi\left(x_{top}\right)}{n-1}\right)\right)}}$

We can iterate further on:

$$\delta_n{}^2 \approx \frac{n}{n-1}x_{top}^2\Phi\left(x_{top}\right)^2\left(1+\frac{\mathrm{n-1}}{nx_{top}^2}\pm\left|\left(\varepsilon x_{top}\right)_{n-1}\right|\left(1+\frac{\mathrm{n-1}}{nx_{top}^2}+\frac{n+1}{nx_{top}}-(1+\varepsilon_{n-1})x_{top}^2\frac{\Phi\left(x_{top}\right)}{n-1}\right)\right)$$

$$\left(\varepsilon x_{top}\right)_n = \pm\frac{\frac{\frac{1}{2}\sqrt{\frac{n}{n-1}}\left(\Phi\left(x_{top}\right)\right)^{n-1}}{\sqrt{1+\frac{\mathrm{n-1}}{nx_{top}^2}}}}{\sqrt{\left(1+\frac{\mathrm{n-1}}{nx_{top}^2}\pm\left|\left(\varepsilon x_{top}\right)_{n-1}\right|\left(1+\frac{\mathrm{n-1}}{nx_{top}^2}+\frac{n+1}{nx_{top}}-(1+\varepsilon_{n-1})x_{top}^2\frac{\Phi\left(x_{top}\right)}{n-1}\right)\right)}}$$

We observe that 2 values are possible depending on whether we "add" the term $\left(\varepsilon x_{top}\right)_{n-1}$ or we subtract it.

In order to properly interpret what happens here: let's recall that we inferred the square of δ_n at first order, assuming correctly that the term $\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_\delta{}^2}$ is negligible relative to $\Phi(x_\delta)$.

We can see that a first gross approximation would consist in setting $\varepsilon x_{top} = \left(\varepsilon x_{top}\right)_1 \approx \pm\frac{1}{2}$

Therefore

$$\left(\varepsilon x_{top}\right)_n = \pm\frac{\frac{\frac{1}{2}\sqrt{\frac{n}{n-1}}\left(\Phi(x_{top})\right)^{n-1}}{\sqrt{1+\frac{n-1}{nx_{top}^2}}}}{\sqrt{\left(1\pm\frac{1}{2}+\frac{n-1}{nx_{top}^2}\pm\frac{1}{2}\left(\frac{n-1}{nx_{top}^2}+\frac{n+1}{nx_{top}}-\left(1\pm\frac{1}{2x_{top}}\right)x_{top}^2\frac{\Phi(x_{top})}{n-1}\right)\right)}}$$

Thus a gross estimate of the bias for the expectation can be estimated:

$$x_{Max} = x_{top}(1+\varepsilon_- +\varepsilon_+) \approx x_{top}\left(1-\frac{\frac{1}{2}\sqrt{\frac{n}{n-1}}\left(\Phi(x_{top})\right)^{n-1}\left(\sqrt{2}-\sqrt{\frac{2}{3}}\right)}{\sqrt{1+\frac{n-1}{nx_{top}^2}}}\right)$$

More finely we can write:

$$x_{Max} = x_{top}(1+\varepsilon_- +\varepsilon_+) \approx x_{top}\left(1 - \frac{1}{2}\sqrt{\frac{n}{n-1}}\left(\Phi(x_{top})\right)^{n-1}\left(\frac{\sqrt{2}}{\sqrt{\left(1+\frac{n-1}{nx_{top}^2}-\left(\frac{n+1}{nx_{top}}-\left(1-\frac{1}{2x_{top}}\right)x_{top}^2\frac{\Phi(x_{top})}{n-1}\right)\right)}} - \frac{\sqrt{\frac{2}{3}}}{\sqrt{\left(1+\frac{4}{3}\frac{n-1}{nx_{top}^2}+\frac{1}{3}\left(\frac{n+1}{nx_{top}}-\left(1+\frac{1}{2x_{top}}\right)x_{top}^2\frac{\Phi(x_{top})}{n-1}\right)\right)}}\right)\right)$$

We finally need to validate for $\underbrace{\left[-x_\delta\Phi(x_\delta)+(n-1)\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_\delta{}^2}\right]}_{\delta}$ that indeed for any n>5, we can say that

$$0 < \frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}(x_\delta)^2} \ll \delta \; ie \; 0 < \frac{1}{\delta_{>0}\sqrt{2\pi}}e^{-\frac{1}{2}(x_\delta)^2} \ll 1 \; with \; x_\delta = x_{top}(1+\varepsilon)$$

Let's review the formulas before we run any numerical estimates at first order:

$$\frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_\delta{}^2} \approx \frac{1}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2} - \varepsilon x_{top}\frac{x_{top}}{\sqrt{2\pi}}e^{-\frac{1}{2}x_{top}{}^2}$$

$$\delta^2 \approx \frac{n}{n-1} x_{top} \Phi(x_{top})^2 \left(1 + \varepsilon x_{top} \frac{x_{top}}{n-1}\right)\left(x_{top} + \frac{n-1}{n x_{top}} + \varepsilon x_{top}\left(1 + \frac{1}{n} - (1+\varepsilon) x_{top}^3 \frac{\Phi(x_{top})}{n-1}\right)\right)$$

$$-2\delta x_{top}\varepsilon \approx T_{op} = \frac{n}{n-1} x_{top}\left(\Phi(x_{top})\right)^n$$

$$\frac{x_{top}\Phi(x_{top})}{n-1} = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_{top}}^2}$$

We first infer that:

$$\frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_\delta}^2} \approx \frac{x_{top}\Phi(x_{top})}{n-1}\left(1 - \varepsilon x_{top} x_{top}\right) \approx \frac{x_{top}\Phi(x_{top})}{n-1}$$

Next we would infer that:

$$\delta^2 \approx \frac{n}{n-1} x_{top}\Phi(x_{top})^2 \left(x_{top} + \varepsilon x_{top}\left(1 - x_{top}^3 \frac{\Phi(x_{top})}{n-1}\right)\right) \approx \frac{n}{n-1} x_{top}^2 \Phi(x_{top})^2$$

Thus a first gross estimate would yield: $\frac{\left(\frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}{x_\delta}^2}\right)^2}{\delta^2} \approx \frac{x_{top}^2 \Phi(x_{top})^2}{(n-1)^2 \frac{n}{n-1} x_{top}^2 \Phi(x_{top})^2} \approx \frac{1}{n(n-1)} < \frac{1}{30} \ll 1$

This is the square so the approximation runs in $\frac{1}{n-1}$ at worst.

Conclusion: we can estimate well the expectation of the max for n>6 for a normal distribution. The fundamental equations to produce this estimates derive from the expression of the density of the Max that is always the same . Thus we can use on the follow the Hermite's function to proxy the real life distribution. The play of the consecutive power laws will make the formulas more intricate but the principles above remain.

So in the larger scheme, a maximum expected return being indexed by a "risk", this "risk" can be equivalently perceived as another "maximum expected return" that is obtained relative to some inner "risk" and so on and so forth. If therefore the original "risk" has actually a "distribution" because it matches with a distribution of "maximum expected returns" that has some expectation as such, we can refine an initial efficient frontier by defining a maximum expected return for a given bucket of the original distribution of the risk that we initially only defined through the expectation of this distribution. This is how we should add higher moments since they indirectly feature the permanent uncertainty that surrounds our measure of the "risk". We initially assume that the variance provides a good estimate of this risk. But it truly is random, even for the normal distribution. Yet we really care here about the deviation from the normal distribution: indeed the inclusion of the higher moments can make sense only if the CLT is not good enough to capture the "tail risk", the "skew risk" or some other non normal ass

5-Efficient Frontier

The starting point is the Markowitz efficient frontier: one would optimize the weights in a portfolio of assets based on the covariance and the expected returns of the individual assets one relative to the

others. Kristiaan Kerstens, for example, added elements that are based upon the higher statistical moments of the distribution of returns and a wider range tof criteria to measure the many possible investment strategies in order to pick “the optimal one”.

The Markowitz efficient frontier in a nutshell…

We consider a “portfolio” of assets or, more generally, a universe of possible asset classes that we can define through an expected return, a variance and more generally through a distribution of returns.

Let’s consider n assets x_k and a weighted combination made of weights w_k that would exhibit a return $r_w = \sum_{k=1}^{k-n} w_k r_k$

and a variance that would involve a correlation matrix, alternatively a covariance matrix (real symmetrical hence one that can be diagonal but not necessarily having all positive eigen values)

$$v_w = \sum_{k,l=1}^{k,l=n} w_k w_l \rho_{kl} \sqrt{v_k v_l} \ \ where \ \sqrt{v_k} = \sqrt{\frac{1}{N-1}\sum_{j=1}^{j=N}\left(r_{kj} - r_k\right)^2} \ \ with \ r_k = \frac{1}{N}\sum_{j=1}^{j=N} r_{kj}$$

in the expressions above we assume that we dispose of “N” empirical evidence of varying returns as measured through historical data or else Monte Carlo simulations. The key point being that “returns” follow a certain distribution that we have access to in part at least.

Depending upon the range of expected return, depending on the covariance, a weighted combination of the assets (assuming no leverage ie, long risk exposures only in general) will exhibit itself a distribution of returns and a range of expected returns. The “efficient frontier” concept implements the view that for every targeted “expected return” there is an optimal set of weights that will exhibit the lowest possible variance.

We should now express the characteristic of the portfolio in vectors and matrices since it will prove quite useful next.

We thus have a portfolio identified with a vector of weights “w” that would have an expected return based on a vector of individual expected returns (asset per asset) and we would look for the optimal vector of weights that would minimize the variance of the portfolio for a given expected return:

$$return \ r_w = w^T r, variance \ v_w = w^T C w \ C: covariance \ matrix$$

So if we write this into an “optimization” problem we have:

$$\text{Minimize } v_w = w^T C w \ under \ the \ constraints: \begin{cases} w^T r = r_{fixed} \\ w_i \geq 0 \\ w_1 + w_2 + w_3 + \cdots w_n = 1 \end{cases}$$

This problem is typical of a simplex problem that has a solution in general. We find in general a “risk aversion” pattern in historical data where the higher the target return we have the higher the increase in risk terms is going to be. In other words the efficient frontier is concave: if we target a 1% return we would be able to face say a 2% risk. If we target next a 2% return we would likely have to face a 5% risk instead of 2*2%=4%. And if we target a 4% risk, we likely will face a 12.5% risk instead of 10% ie 2*5% or even less so 8% as 4*2%.

When there is the possibility to generate financial leverage for some assets, then some players ,having more risk appetite because they can build an arbitrage on this risk aversion pattern by placing “long short” exposures through higher trading and risk management skills, will indeed put on

"trades" that will alter the shape of the efficient frontier since this will alter the historical distribution of returns. As a result, the efficient frontier cannot be reached for risk averse investors. And this efficient frontier should then result from a loser set of constraints, especially $w_i \geq 0$ and $w_1 + w_2 + w_3 + \cdots w_n = 1$, that should become (for example) $|w_1 + w_2 + w_3 + \cdots w_n| \leq Leverage_{ratio}$ and , $-w_{min} \leq w_i \leq w_{Max}$.

The nice feature here I that the constraints remain convex alongside the function that is to be optimized as long as the covariance matrix has all strictly positive eigen values. This last condition can be achieved in general through the method of perturbations.

Now if we assume a portfolio of "n" independent assets having each a mean return m_k and a variance v_k then the optimal weighting would secure a maximum ration $\frac{return}{standard_{deviation}}$

In other words we can look for the max of $\frac{\sum_{k=1}^{k=n} w_k m_k}{\sqrt{\sum_{k=1}^{k=n} w_k^2 v_k}} = R(w_1, w_2, .. w_n)$. We assume implicitly that the "risk" is best featured by the aggregate standard deviation of the return of the portfolio. To be sure, the "return" is the variation in price of the asset that has some "mean" value m_k over some fiwed period of time. It is not certain as such that we would realize this mean return unless we hold the asset up until some maturity time. The best example is for bonds. Another telling example is when one holds stocks for a long period of time: then the historical mean return tends to be realized. The "variance" that is used here reflect the statistical variance of the price. We do not need to assume that the prices follow a normal distribution. However most of the time we would rather consider the Ln(P) instead and consider the relative return on capital as a normally distributed variable. This gives more tractability to the model on the follow but imposes a different projection next for the P&L.

The optimum weighting secures that all the partial derivatives of this ratio are nil, given the constraint that $\sum_{k=1}^{k=n} w_k = 1$, hence we would only run on "n-1" independent coordinates ie "weights". Taking then $w_1 = 1 - \sum_{k=1}^{k=n} w_k$, we therefore have:

$$R(w_1, w_2, w_3 \ldots w_n) = \frac{\sum_{k=1}^{k=n} w_k m_k}{\sqrt{\sum_{k=1}^{k=n} w_k^2 v_k}}$$

Observe then that the weights appear as a square for the variance. It matters a lot for what follows. Indeed if the variance of a random variable X is "V", then the variance of the random variable "2X" will be "4V".

At the optimal weighting point using the 'n' dependent weights since the first is fully determined by them next we get for k=1,2,3….n

$$\frac{\partial R}{\partial w_k} = 0 = \frac{m_k \sqrt{\sum_{k=1}^{k=n} w_k^2 v_k} - \frac{1}{2}(2 v_k w_k) \overbrace{\frac{\sum_{k=1}^{k=n} w_k m_k}{\sqrt{\sum_{k=1}^{k=n} w_k^2 v_k}}}^{R_{opt}}}{\left(\sqrt{\sum_{k=1}^{k=n} w_k^2 v_k}\right)^2} = \frac{m_k - (v_k w_k) A}{\sqrt{\sum_{k=1}^{k=n} w_k^2 v_k}}$$

We infer then that $m_k = v_k w_k A \; with \; A = \frac{\sum_{k=1}^{k=n} w_k m_k}{\left(\sqrt{\sum_{k=1}^{k=n} w_k^2 v_k}\right)^2}$ and therefore $w_k = \frac{m_k}{v_k A}$

Do we have $\sum_{k=1}^{k=n} w_k = 1$?

Then $\sum_{k=1}^{k=n} w_k = 1 = \sum_{k=1}^{k=n} \frac{m_k}{v_k A}$ $and\ therefore\ A = \sum_{k=1}^{k=n} \frac{m_k}{v_k}$

As a result: $w_k = \frac{\frac{m_k}{v_k}}{\sum_{k=1}^{k=n} \frac{m_k}{v_k}}$

Then we could infer the optimal return based on these "n" independent assets that offer varying ratios $\frac{m_k}{v_k}$. Here the optimal ratio associated return and variances would respectively be:

$$m = \sum_{k=1}^{k=n} w_k m_k = \frac{\left(\sum_{k=1}^{k=n} \frac{m_k^2}{v_k}\right)}{\sum_{k=1}^{k=n} \frac{m_k}{v_k}} \ and\ v = \sum_{k=1}^{k=n} w_k^2 v_k = \frac{\left(\sum_{k=1}^{k=n} \frac{m_k^2}{v_k}\right)}{\left(\sum_{k=1}^{k=n} \frac{m_k}{v_k}\right)^2}$$

Thus the optimal ratio is going to be : $\frac{m}{\sqrt{v}} = \sqrt{\sum_{k=1}^{k=n} \frac{m_k^2}{v_k}}$. We can see that this solution takes the best of every component and beats every ratio $\frac{m_k}{\sqrt{v_k}}$ but it sets the level of "risk".

This calculation would only yield one "solution" for a portfolio of "n" assets where we would know in advance the expected return and associated variance.

What if, having a portfolio that carries 'n' different returns and variances, we wanted to know the optimal weights for a given target return? What if, instead, we wanted to know the best return for a fixed level of risk?

We need to identify the free variables of our problem then: the sum of the weights must be "1" so that we can compare the different possible configurations of the portfolio and the weighted return is fixed. Therefore $\sum_{k=1}^{k=n} w_k = 1\ and\ \sum_{k=1}^{k=n} w_k m_k = m$. Note that this works on the premise that indeed we can "know" for sure the ultimate return. The only case when this occurs is when we own "safe" bonds like government bonds AND we buy and hold them till they mature in full. Otherwise this premise is void of sense. But it is the most intuitive one to starts with.

We will set the assets by growing order of return and observe that only "n-2" weights are really free to fluctuate between 0% and 100%.

So the assets "1" and "2" will disappear in the optimization process, their respective weights being fully inferred from the other "n-2" assets.

We therefore state that : $w_1 + w_2 = 1 - \sum_{k=3}^{k=n} w_k\ and\ m_1 w_1 + m_2 w_2 = m - \sum_{k=3}^{k=n} w_k m_k$

We will note $\sum_{k=3}^{k=n} w_k = W,\ \sum_{k=3}^{k=n} w_k m_k = M\ and\ \sum_{k=3}^{k=n} w_k^2 v_k = V$ to simplify the resolution next.

Let's first "solve" $w_1\ and\ w_2$ relative to m,1,W and M:

$$\begin{cases} w_1 + w_2 = 1 - W \geq 0 \\ m_1 w_1 + m_2 w_2 = m - M \geq 0 \\ R = \dfrac{\sum_{k=1}^{k=n} w_k m_k}{\sqrt{\sum_{k=1}^{k=n} w_k^2 v_k}} \end{cases}$$

$$\begin{cases} -m_1w_1 - m_1w_2 = -m_1(1-W) \\ m_1w_1 + m_2w_2 = m - M \\ R = \dfrac{m_1w_1 + m_2w_2 + M}{\sqrt{w_1^2v_1 + w_2^2v_2 + V}} \end{cases} \quad ie \begin{cases} w_1 = (1-W) - w_2 \\ (m_2 - m_1)w_2 = m - M - m_1(1-W) \\ R = \dfrac{m_1w_1 + m_2w_2 + M}{\sqrt{w_1^2v_1 + w_2^2v_2 + V}} \end{cases}$$

$$\begin{cases} w_1 = (1-W) - \dfrac{m - M - m_1(1-W)}{(m_2 - m_1)} \\ w_2 = \dfrac{m - M - m_1(1-W)}{(m_2 - m_1)} \\ R = \dfrac{m_1w_1 + m_2w_2 + M}{\sqrt{w_1^2v_1 + w_2^2v_2 + V}} \end{cases} \quad ie \begin{cases} w_1 = \dfrac{m_2(1-W) - (m-M)}{(m_2 - m_1)} \geq 0 \\ w_2 = \dfrac{-m_1(1-W) + (m-M)}{(m_2 - m_1)} \geq 0 \\ R = \dfrac{m_1w_1 + m_2w_2 + M}{\sqrt{w_1^2v_1 + w_2^2v_2 + V}} \geq 0 \end{cases}$$

We find ultimately that “R” has a rather complex expression but one that is symmetrical for all the indices “k”:

$$R = \frac{m_1w_1 + m_2w_2 + M}{\sqrt{w_1^2v_1 + w_2^2v_2 + V}} = \frac{m_1\frac{m_2(1-W)-(m-M)}{(m_2-m_1)} + m_2\frac{-m_1(1-W)+(m-M)}{(m_2-m_1)} + M}{\sqrt{\left(\frac{m_2(1-W)-(m-M)}{(m_2-m_1)}\right)^2 v_1 + \left(\frac{-m_1(1-W)+(m-M)}{(m_2-m_1)}\right)^2 v_2 + V}}$$

We can simplify “R” first:

$$R = \frac{m_1w_1 + m_2w_2 + M}{\sqrt{w_1^2v_1 + w_2^2v_2 + V}} = \frac{m}{\sqrt{\left(\frac{m_2(1-W)-(m-M)}{(m_2-m_1)}\right)^2 v_1 + \left(\frac{-m_1(1-W)+(m-M)}{(m_2-m_1)}\right)^2 v_2 + V}}$$

We observe that the optimal point for “R” is exactly where “the total variance is nil”. So let’s simply look for this minimal total variance given that “m” is fixed anyway :

Let’s write $V_{ar} = \left(\frac{m_2(1-W)-(m-M)}{(m_2-m_1)}\right)^2 v_1 + \left(\frac{-m_1(1-W)+(m-M)}{(m_2-m_1)}\right)^2 v_2 + V$

The partial derivatives are therefore : knowing that $\frac{\partial W}{\partial w_k} = 1$, $\frac{\partial M}{\partial w_k} = m_k \; and \; \frac{\partial V}{\partial w_k} = 2w_kv_k$

Let’s write $V_{ar} = \left(\frac{m_2(1-W)-(m-M)}{(m_2-m_1)}\right)^2 v_1 + \left(\frac{-m_1(1-W)+(m-M)}{(m_2-m_1)}\right)^2 v_2 + V$

$$\frac{\partial V_{ar}}{\partial w_k} = 2v_1\left(\frac{m_2(-1) - (-m_k)}{(m_2 - m_1)}\right)\left(\frac{m_2(1-W) - (m-M)}{(m_2 - m_1)}\right) + 2v_2\left(\frac{-m_1(-1) + (-m_k)}{(m_2 - m_1)}\right)\left(\frac{-m_1(1-W) + (m-M)}{(m_2 - m_1)}\right) + 2w_kv_k$$

$$\frac{\partial V_{ar}}{\partial w_k} = 2v_1\left(\frac{m_k - m_2}{(m_2 - m_1)}\right)\left(\frac{m_2(1-W) - (m-M)}{(m_2 - m_1)}\right) + 2v_2\left(\frac{m_1 - m_k}{(m_2 - m_1)}\right)\left(\frac{-m_1(1-W) + (m-M)}{(m_2 - m_1)}\right) + 2w_kv_k$$

Which becomes :

$$\frac{\partial V_{ar}}{\partial w_k}$$
$$= 2\frac{\Big((1-W)\big(m_2v_1(m_k-m_2)-m_1v_2(m_1-m_k)\big)\Big)+(m-M)(v_2(m_1-m_k)-v_1(m_k-m_2))}{(m_2-m_1)^2}$$
$$+ 2w_kv_k$$

$$\frac{\partial V_{ar}}{\partial w_k} = 2\frac{\Big((1-W)\big(m_2v_1(-m_2)-m_1v_2(m_1)\big)\Big)+(m-M)(v_2(m_1)-v_1(-m_2))}{(m_2-m_1)^2}$$
$$+\frac{\Big((1-W)\big(m_2v_1(m_k)-m_1v_2(-m_k)\big)\Big)+(m-M)(v_2(-m_k)-v_1(m_k))}{(m_2-m_1)^2}$$
$$+ 2w_kv_k$$

$$\frac{\partial V_{ar}}{\partial w_k} = -2\underbrace{\frac{\left(\overbrace{(1-W)}^{C}(m_2^2v_1+m_1^2v_2)\right)-\overbrace{(m-M)}^{D}(v_2m_1+v_1m_2)}{(m_2-m_1)^2}}_{A}$$
$$+ m_k\underbrace{\frac{\left(\overbrace{(1-W)}^{C}(m_2v_1+m_1v_2)\right)-\overbrace{(m-M)}^{D}(v_2+v_1)}{(m_2-m_1)^2}}_{B}+2w_kv_k$$

We introduce values that are independent of "k" in order to "solve".

Since the optimum nullifies the partial derivatives for the free variables we infer that :

$$w_k = \frac{A-Bm_k}{v_k}$$

We will infer "C", "D" that we re-inject into A and B and finally find the values of A and B which will ultimately deliver the values of w_k and finally the values for w_1 and w_2

So: $C = 1-W = 1-\sum_{k=3}^{k=n} w_k = 1-\sum_{k=3}^{k=n}\frac{A-Bm_k}{v_k}$ and $D = m-M = m-\sum_{k=3}^{k=n} m_k\frac{A-Bm_k}{v_k}$

Therefore $\begin{cases} C = 1-A\sum_{k=3}^{k=n}\frac{1}{v_k}+B\sum_{k=3}^{k=n}\frac{m_k}{v_k} \\ D = m-A\sum_{k=3}^{k=n}+\frac{m_k}{v_k}B\sum_{k=3}^{k=n}\frac{m_k^2}{v_k} \\ A = \frac{\left(C(m_2^2v_1+m_1^2v_2)\right)-D(v_2m_1+v_1m_2)}{(m_2-m_1)^2}: \\ B = \frac{(C(m_2v_1+m_1v_2))-D(v_2+v_1)}{(m_2-m_1)^2} \end{cases}$

We should introduce then "known" values with $\begin{cases} S_0 = \sum_{k=3}^{k=n}\frac{1}{v_k} \\ S_1 = \sum_{k=3}^{k=n}\frac{m_k}{v_k} \\ S_2 = \sum_{k=3}^{k=n}\frac{m_k^2}{v_k} \end{cases}$ and $\begin{cases} r_2 = \frac{(m_2^2v_1+m_1^2v_2)}{(m_2-m_1)^2} \\ r_1 = \frac{(m_1v_2+m_2v_1)}{(m_2-m_1)^2} \\ r_0 = \frac{(v_2+v_1)}{(m_2-m_1)^2} \end{cases}$

The indices point to the power law of the means….

The system to solve for "A,B,C,D" then becomes:

$$\begin{cases} C = 1 - AS_0 + BS_1 \\ D = m - AS_1BS_2 \\ A = Cr_2 - Dr_1 \\ B = Cr_1 - Dr_0 \end{cases} \text{ therefore: } \begin{cases} C = 1 - AS_0 + BS_1 \\ D = m - AS_1 + BS_2 \\ A = r_2(1 - AS_0 + BS_1) - r_1(m - AS_1 + BS_2) \\ B = r_1(1 - AS_0 + BS_1) - r_0(m - AS_1 + BS_2) \end{cases}$$

$$\begin{cases} C = 1 - AS_0 + BS_1 \\ D = m - AS_1 + BS_2 \\ A = r_2 - r_1 m - A(r_2S_0 - r_1S_1) + B(r_2S_1 - r_1S_2) \\ B = r_1 - r_0 m - A(r_1S_0 - r_0S_1) + B(r_1S_1 - r_0S_2) \end{cases}$$

We finally solve the respective values for A and B:

$$\begin{cases} A = \dfrac{r_2 - r_1 m + B(r_2S_1 - r_1S_2)}{1 + (r_2S_0 - r_1S_1)} \\ B = r_1 - r_0 m - \dfrac{r_2 - r_1 m + B(r_2S_1 - r_1S_2)}{1 + (r_2S_0 - r_1S_1)}(r_1S_0 - r_0S_1) + B(r_1S_1 - r_0S_2) \end{cases}$$

$$\begin{cases} A = \dfrac{r_2 - r_1 m + B(r_2S_1 - r_1S_2)}{1 + (r_2S_0 - r_1S_1)} \\ B\left(1 - (r_1S_1 - r_0S_2) + \dfrac{(r_2S_1 - r_1S_2)}{1 + (r_2S_0 - r_1S_1)}\right) = r_1 - r_0 m - \dfrac{(r_2 - r_1 m)(r_1S_0 - r_0S_1)}{1 + (r_2S_0 - r_1S_1)} \end{cases}$$

$$\begin{cases} A = \dfrac{r_2 - r_1 m + B(r_2S_1 - r_1S_2)}{1 + (r_2S_0 - r_1S_1)} \\ B = \dfrac{r_1 - r_0 m - \frac{(r_2 - r_1 m)(r_1S_0 - r_0S_1)}{1 + (r_2S_0 - r_1S_1)}}{\left(1 - (r_1S_1 - r_0S_2) + \frac{(r_2S_1 - r_1S_2)}{1 + (r_2S_0 - r_1S_1)}\right)} \end{cases}$$

We can see that we can mechanically infer the weights because we can compute the values for A and B , the C and D, and ultimately the values for the weights for k≤ 3 with $w_k = \frac{A - Bm_k}{v_k}$ and infer finally the other weights with :

$$\begin{cases} w_1 = \dfrac{m_2(1 - W) - (m - M)}{(m_2 - m_1)} \geq 0 \\ w_2 = \dfrac{-m_1(1 - W) + (m - M)}{(m_2 - m_1)} \\ R = \dfrac{m_1 w_1 + m_2 w_2 + M}{\sqrt{w_1^2 v_1 + w_2^2 v_2 + V}} \geq 0 \end{cases} = \begin{cases} w_1 = \dfrac{m_2 C - D}{(m_2 - m_1)} \geq 0 \\ w_2 = \dfrac{-m_1 C + D}{(m_2 - m_1)} \\ R = \dfrac{m_1 w_1 + m_2 w_2 + M}{\sqrt{w_1^2 v_1 + w_2^2 v_2 + V}} \geq 0 \end{cases}$$

Now we could adopt an alternative approach saying: we fix the variance and look for the optimal expected return for this given aggregate variance.

The calculation is following the same principles. It has the advantage here that we do not need to explicitly assume that we “know” the ultimate return. As mentioned, this part can be quite problematic because the former premise that we would know the final return imposes that we only invest in fixed income products, that we ignore the default risk and that we hold the positions until maturity. Instead, assuming that we fix the variance and look for some optimal return then we introduce a more opportunistic part premising then that we only need to know the expectation of the return and next act accordingly all the way. Yet this introduces a review of the notion of

"expected return" and above all the "expectation of the maximum return". See the appendix for details on this approach that yet is more realistic.

But here we assumed that the variables were independent and normally distributed. Also we quite importantly we need to review what we should do when, and this is the case in general, the returns are more or less correlated. We can always reduce the covariance matrix with a basis of eigen vectors and therefore solve the problem in the eigen basis where, statistically, the variables now are uncorrelated which brings us back to the precedent development.

We will stick to the assumption that we "know" the expected return. Note that if we work with "expected maximum returns" all this optimization works equally well.

There is another way, more geometrical, to compute the frontier by defining first the eigen values and the eigen vectors. Indeed C, the covariance matrix being real and symmetrical, we can find a basis of eigen vectors ie we can transpose the problem to a case where we would only deal with n "independent" "eigen assets" based upon the coordinates of the eigen vectors that would diagonalize the empirical covariance matrix. Note that this approach can be generalized to higher order derivatives : we can either generate a gradient for odd degrees or we can combine pairs of degrees. We would not obtain always obtain perfectly diagonal matrices but, assuming that we always deal with smooth functions we would always be able to invert the order of derivation freely.

Sticking with the core model here in 2 dimensions we therefore could rephrase the problem in the eigen basis as: (note that we have a different set of factors for the last constraint that will depend upon the coordinates of the eigen vectors when each original weight would be expressed, as a vector, through a linear combination in the eigen basis).

Introducing the aggregate factors $(a_1, a_2 \dots a_n)$ that move us back from the eigen basis into the canonical basis, we look at the following optimization problem (our eigen weights may not match a real life situation where the weights in the canonical basis sum up to 1: $v_w =$

$$\sum_{k=1}^{k=n} v_{pk}\, w_{eige\ \ _k}^2 \; under\ the\ constraints: \begin{cases} {w_{eigen}}^T r_{eigen} = r_{fixed} \\ w_{eigen_i} \geq 0 \\ a_1 w_{eige\ \ _1} + a_2 w_{eigen_2} + a_3 w_{eige\ \ _3} + \cdots a_n w_{eign\ \ _n} = 1 \end{cases}$$

By changing the basis of representation one more time, moving towards $y_k = \sqrt{v_{pk}} w_{eigen_k}$ we land onto a very geometrical solution:

Minimize

$$R_y^2 = \sum_{k=1}^{k=n} y_k^2 \; under\ the\ constraints: \begin{cases} \frac{r_1}{r_{fixed}} \frac{y_1}{\sqrt{v_{p1}}} + \frac{r_2}{r_{fixed}} \frac{y_2}{\sqrt{v_{p2}}} + \frac{r_3}{r_{fixed}} \frac{y_3}{\sqrt{v_{p3}}} + \cdots \frac{r_n}{r_{fixed}} \frac{y_n}{\sqrt{v_{pn}}} = 1 \\ y_k \geq 0 \\ a_1 \frac{y_1}{\sqrt{v_{p1}}} + a_2 \frac{y_2}{\sqrt{v_{p2}}} + a_3 \frac{y_3}{\sqrt{v_{p3}}} + \cdots a_n \frac{y_n}{\sqrt{v_{pn}}} = 1 \end{cases}$$

So the optimal problem becomes a rather simple geometrical question in n dimensions : find the smallest sphere that intersects 2 hyperplanes as defined in the constraints, in the « positive quadrant ». We studied in detail how one can solve it explicitly but at the expense of tedious calculations. Here, once diagonalized, we can adopt a numerical resolution based on this geometrical view.

Then we could eliminate one coordinate so that we remain with only one condition on the "n-1" remaining coordinates and therefore only one hyperplane in some "n-1" space. The "squared radius"

function would become some elliptic function then. But we can always change one last time the coordinates so that to retrieve once more a squared radius equation and infer the radius of the tangent circle along with the coordinates of the tangent point

Let's consider a naïve portfolio that truly is the actual sum of the "n" native assets and let's see how the "dependence function" that should be independent from the sizes involved pops up from, first, the joint distribution, and second, from our usual target that is an expectation computed in the middle of a distribution of outcomes.

So the "closed formula" can be obtained once one diagonalizes the covariance matrix and expresses the native portfolio as a portfolio of "eigen" assets instead where one has to compute the associated expected return and expected variance, assuming on the follow that we have a normal distribution.

We can also adopt a more numerical resolution using a geometrical interpretation. But what if an investor is more risk averse and actually cannot really bear any level of risk for the sake of maximizing its risk reward? What if an investor is constrained by the range of risks it can take? How should we proceed? Is the variance actually the only "risk" for an investor. The answer is "no": an investor mostly fear the "tail risk", the "draw-down", where a lasting large loss has to be shouldered if the investor wants ultimately to be paid for the risk that was undertaken at the start of all this investment strategy. So we need to adopt a more comprehensive framework that would include higher statistical moments.

The short answer is we maximize a weighted return under a Lagrange constraint. Let's take a very theoretical approach as to what this entails.

So we would write the expected return of such portfolio as :

$$P(X_1 + X_2 \dots + X_n) = (x_1 + x_2 + x_3 \dots)$$

And we next should consider the many consecutive related moments since the n random variables can take many values as per their individual distributions and as per the joint distribution that binds the "n" independent variables together.

$$M_k(P) = \iiint (x_1 + x_2 + x_3 \dots)^k \prod_{k=1}^{k=n} p_k(X_k = x_k) dx_k$$

We know that the variables $X_1, X_2, X_3 \dots X_n$ are random and follow their own distribution p_k.

Remember: the formula above implicitly assumes that the variables are independent. If the variables are not independent, the theorem of Sklar introduces the idea that, should the n random variables be of same magnitudes on the first 2 moments, there is a "copula C" that allows us to define the joint distribution.

So, let's first write "the obvious" ie that "yes" we have a joint distribution even when the n random variables are not independent. Then the formula above should be written as follows:

$$M_{P_k}(X_1 + X_2 \dots + X_n) = \iiint (x_1 + x_2 + x_3 \dots)^k P_{joint}(X_1 = x_1, X_2 = x_2 \dots) \prod_{k=1}^{k=n} dx_k \dots$$

The theorem of Sklär allows us to write, quite importantly the following equivalence:

$$M_{P_k}(X_1 + X_2 \dots + X_n) = \iiint (x_1 + x_2 + x_3 \dots)^k \, C(\{p_k(X_k = x_k)\ k = 1,2, \dots n\}) \prod_{k=1}^{k=n} dx_k \dots$$

The very existence of this "Copula", at least from a pure theoretical standpoint that mostly is valid when we adhere to the existence of a unique class of joint distributions, has a very important consequence: since we would say that whatever the multiplier a_k that we may apply to the variable X_k we should certainly assume that $P(a_k X_k = a_k x_k) = P(X_k = a_k)$. Therefore the same copula would apply to any "weighting" that we may use on the native assets. Note however that the equality is not always valid if the "size matters" on some assets.

We should think of these variables less as "price" and more in terms of "returns" in relative values. Thus the expectation would be in the order of magnitude of a few % points in most cases but it could reach extreme values in exceptional circumstances. In this fashion we likely would get close enough to a normal distribution.

Intuitively, the primary target is the highest return, which in our framework points towards the maximum moment of first order. However, investors would have a "risk aversion" pattern that is expressed in the simplest form through the "risk" that they would undertake in "volatility" terms. This would generally correspond to the "variance", ie the squared distance relative to the expectation.

The variance, with our notations, would be expressed as follows:

$$V = \iiint \left(x_1 + x_2 + x_3 \dots - M_{P_1}(X_1 + X_2 \dots + X_n)\right)^2 C(\{p_k(X_k = x_k)\ k = 1,2, \dots n\}) \prod_{k=1}^{k=n} dx_k$$

Since $M_{P_k}(X_1 + X_2 \dots + X_n) = \iiint (x_1 + x_2 + x_3 \dots)^k \, P_{joint}(X_1 = x_1, X_2 = x_2 \dots) \prod_{i=1}^{i=n} dx_k$

When we would develop the "square" that is present under the integral signs, we would obtain a classical result:

$$\begin{aligned} V = \iiint (x_1 + x_2 + x_3 \dots)^2 \, C(\{p_k(X_k = x_k)\ k = 1,2, \dots n\}) \prod_{k=1}^{k=n} dx_k \\ - 2M_{P_1}(X_1 + X_2 \dots + X_n) \iiint (x_1 + x_2 + x_3 \dots)^1 \, C(\{p_k(X_k = x_k)\ k \\ = 1,2, \dots n\}) \prod_{k=1}^{k=n} dx_k + M_{P_1}(X_1 + X_2 \dots + X_n)^2 \end{aligned}$$

And we would infer that : $V = M_2(X_1 + X_2 \dots + X_n)^1 - M_{P_1}(X_1 + X_2 \dots + X_n)^2$

The investors being risk-averse would ponder the benefit against the "cost", ie the expected return against the risk that, because of the variance, they may go through a rough patch and then be forced to close prematurely their investment at a loss.

The most intuitive angle to it consists in trying to optimize the expected return with a penalty that would come from the associated value of the variance:

$$F(portfolio) = Max[M_{P_1}^2 - \beta M_{P_2}]$$

In our earlier calculations we made quite strong assumptions. First we assumed that an investor "knows" what the ultimate return will be. Well that is true for "buy and hold" investors who buy

bonds or else buy shares but over the very long term without changing much their strategy. This is actually the exception rather than the rule. Most investors have or fell the duty to actually "optimize" their money management, which implies that they monitor the events, the news, the price changes, the resulting P&L and returns and…make decisions to change the balance of risks in their portfolio. Doing so, they usually "cut losses" or "add risks" or else "take profits". In general they face constraints as to the envelope of risks that they are allowed to take by mandate. Therefore it would be more intuitive to actually look to maximize a return over a certain predefined period with the constraint that the "risk" is fixed. But is the risk "fixed" in essence? The answer is "no": the risk is volatile in essence. So, the framework is more the one where the risk is rather random and the return accordingly is more or less volatile. The whole now is to assume that "no" investors do not in general target the Sharpe ratio other than on rather short periods of time and under the constraint that they have a risk-aversion pattern. Thus investors look for an efficient frontier indeed but more in the way that we will picture in what follows.

We look to optimize the following function:

$$F(w_1, w_2, .. w_n) = Return(w_1, w_2, ... w_n) - \beta Risk(w_1, w_2, .. w_n)$$

The "risk" is typically measured with the standard deviation but it might include higher moments that feature the "tail risk" or else higher statistical moments with an additional weight.

We immediately note that the optimal configuration may depend upon the choice of our penalty β: the higher β is the lower the expected return. But what is the reasonable β given a certain level of risk? How is this β penalty changing when the risk aversion for the investor becomes lower than it was? We sense that for a given set of "n" assets, given their distributions, given their joint distribution, the penalty β varies. This means that this also must be re-assessed at regular intervals.

That would make sense since the best return that one can expect should grow along with the appetite for risk, the risk is varying and the investor's appetite for risk also varies independently so. This is mandated by the fact that, on balance, the higher the risk we take, the higher the probability that we undergo an underperformance in the end. In other words there should be no free-lunch: a risk averse investor should not be able to expect a higher return and in exchange for that he will get a more predictable but low ultimate outcome. What should we do, if we know the level of risk that we can accept, in order to spot the "best possible return", ie how do we choose the "best possible penalty through the choice of β?

This approach, if we consider that the choice of portfolio is made by picking the best weights among "n" assets, is close to the one that is implemented through the computation of the efficient frontier. In the latter case, we would spot the optimal set of weights for any given target return by searching for the configuration of the portfolio that would induce the smallest variance. So, once we would know the appetite for risk of an investor, we would look at this efficient frontier, and spot the maximum expected return we could get. And on that specific point of the efficient frontier, we would know the optimal weights that should deliver both the target return that we would inferred from the stated "tolerated variance" number.

Looking back at the last formula above, we could see the connection: if we look at the derivative of the formula relative to M_{P_2}, a risk neutral move would induce that $\partial_{M_{P_2}}[F(Portfolio)] = 0$. This would amount to say that $\partial_{M_{P_2}}\left[M_{P_1}^2\right] = \beta$ which is more or less the slope of the efficient frontier for a certain level of risk M_{P_2} . So, if we start wondering "what should be the optimal penalty that we should impose on risk given a give level if we want to achieve the best possible rate of return?", the answer would be: "take the slope of the efficient frontier".

We see here what the concept of “risk-neutrality” conveys: people may have varying risk appetite but a “risk-neutral” approach to risk itself would consist in moving along the efficient frontier. But, given that there are other determinants of the risk aversion profile of investors that may include other moments, and even more specific “path dependent” limitations, how should we model and compute (easily?!) the equivalent “efficient frontier”?

One quick review of the Sharpe ratio approach brings in a little more context. In a Sharpe ratio approach we actually consider the following optimization on the function:

$$S_{har} \quad (portfolio) = Max\left[\frac{M_{P_1}^2}{M_{P_2}} - \beta\right] = Max\left[\frac{M_{P_1}^2}{M_{P_2}}\right]$$

If we contrast the 2 formula, we observe a simple relation between the 2:

$$F(portfolio) = Max\left[M_{P_1}^2 - \beta M_{P_2}\right] = M_{P_2} S_{harp} \ (portfolio)$$

Clearly, the spontaneous concavity of an efficient frontier in general will induce a choice to target the lowest possible risks in variance terms and create next a maximum financial leverage effect over this tiny but ultra safe return. The ongoing concavity of the efficient frontier signals indeed that the slope of tangent to the efficient frontier can only decline. In more intuitive terms, this only says that any fixed increment of risk will induce a lower and lower gain in expected returns. As a result the best “slope” is found at “risk=0+”. Then a player might generate a multiplier effect by borrowing a lot of money in order to create a high multiple of the native return undertaking mostly counterparty risks.

This is what banks do and this is what tends to generate the so called “systemic risk”. The original reason why banks do that comes from the search of “safety” for the savings and deposits that induce very strict regulations. However, this need for “safety” does not fit with the conflicting need for banks to deliver the highest possible return on capital to their shareholders. They therefore mitigate the 2 contradicting objectives through this strategy that consists in targeting “AAA like” ie low returns but with the bare minimum risk. Since they need to deliver high returns banks borrow a maximum amount of money and build financial leverage around this arbitrage. At the other end of the spectrum we find “distress funds”, but also “pension funds” or “insurers”, or “corporate” that benefit in small parts from regulation exemptions and target the maximum risk and minimum financial leverage.

As such this leveraged approach could and should be modeled under a similar framework considering that “one asset” is the safe portfolio and another asset is the “ability to borrow collateralized money”. Of the 2 “assets” one is triggering financial crisis at times and thus one may as well opt to look for “diversified” sources of portfolios and “diversified sources of external” funding in order to alleviate the risk here. But so far our financial markets are centralized around one unique US$ ring of funding across the planet. Crypto currencies, the Yuan are recent alternatives but they do not have by far the same size and depth for now.

What if we aim to include other moments in this approach in order to express, for example, some “shortage” function?

Investors might first target the highest Sharpe ratio (expectation/standard deviation) while they actually may not whither properly some draw-downs or else stick to a strategy that has “won already”. The second alternative, ie when the portfolio stands at some outstanding gain, seems “too good” to be true: it may have perverse consequences actually. Indeed, as much as we can easily conceive the scenario where an investor faces temporarily a loss that is “much higher than expected”, ie a multiple of the standard deviation that points to some “ab-normally distributed”

outcome: in that case it would prompt an unwanted wind down of the investment and a fixed loss. We would consequently figure that the “super nice and unexpected profits” would be the reciprocal situation. Then an investor might “opt out” to secure these windfall gains. However, this situation induces choices on the follow that may have perverse consequences. The main issue then being “what to do next?”. Since the original strategy is exited, the investor would have to choose “something else”. Here this would be a deviation from the initial strategy that would be mandated by some opportunistic decision process. And, although the timing of the exit would prove to be right, whatever decision that would be made next would be an additional risk.

One can start e by including the “Skew” (moment of 3rd degree) and the “Kurtosis” (moment of 4th degree) of the distribution of returns in order to factor in these aspects. The concept underneath is the ” shortage function” that embodies this view that “mean” and “variance” do not fit with the actual context of an investor, irrespective of its “risk aversion” pattern. Indeed investors have varying patterns, aiming at different returns and therefore readying themselves to different levels of risk. However, their issue is, as much, about the sequencing of their investment process. Why is that? Because in reality the path that will be followed will matter, irrespective of whether these investors are happy to “take profits” or “stop out” or else “strategize” at regular intervals. They merely have to do it anyway, albeit in the form of some algorithmic investment process.

So the question now bears on the ability to define an equivalent “efficient space” that would embed more risk factors that the simple “variance” versus the “expected return”.

At first we need to clarify the relation between the copula and the traditional approach when all is based on the empirical covariance. Typically, once we have reduced the estimation of the moment of second order to the covariance matrix we often make this observation that this covariance itself can be split into a correlation matrix that is independent of the “sizes involved” and a vector of variances that will carry the “sizes involved”.

Can we generalize this approach that is typical once we assume that we deal with normal distribution and therefore a normal correlated joint distribution of “n” variables?

How is this possible with a copula? Is this a wise move?

First let’s clarify the core assumptions here…

The correlation matrix empirically is established as follows

$$C_{orr}(X_1, X_2, \dots X_n) = \left[C_{or\ ij} = \frac{\frac{1}{N-1}\sum_{k=1}^{N}\left(X_{i_k} - X_{i_{mean}}\right)\left(x_{j_k} - X_{j_{mean}}\right)}{\sqrt{\frac{1}{N-1}\sum_{k=1}^{k=N}\left(X_{i_k} - Xi_{mean}\right)^2}\sqrt{\frac{1}{N-1}\sum_{k=1}^{k=N}\left(X_{j_k} - X_{j_{mean}}\right)^2}} \right]$$

We can then define a vector-column of standard deviations:

$$S_{td} = \left[S_{td_i} = \sqrt{\frac{1}{N-1}\sum_{k=1}^{k=N}\left(X_{i_k} - Xi_{mean}\right)^2} \right]$$

And of course we would retrieve the covariance matrix through a simple formula:

$$C_{ov}(X_1, X_2, \dots X_n) = \begin{bmatrix} S_{td_1} \\ S_{td_2} \\ S_{td_3} \\ \dots \\ S_{td_n} \end{bmatrix} C_{orr}(X_1, X_2, \dots X_n) \begin{bmatrix} S_{td_1} & S_{td_2} & S_{td_3} & \dots . & S_{td_n} \end{bmatrix}$$

Observe that this is not an "outer product" but close to that: the column vector is n the right and the line vector is on the left which implies that there is no "summation", but just simple products. This is useful in programming terms. Thanks to the symmetrical nature of the correlation matrix we could express this in a basis of unit norm eigen vectors and there the correlation matrix would be a diagonal matrix. As such it is the reverse to what we usually use to compute a scalar from a bilinear form.

The expression has an elegant property: the "dependence" is expressed separately from the "size" of the components of the "sum of random variables" that is at play here.

Could we retrieve this elegant property from an expression like, knowing that the covariance matrix is the empirical estimate of the moments of order 2 in "n" dimension?

$$P_{joint}(X_1 = x_1, X_2 = x_2 \dots) = C(\{p_k(X_k = x_k)\ k = 1,2, \dots n\})$$

We need to formulate the standard deviation, the covariance matrix with the introduction of the standalone probabilities. We saw that the covariance element is, from its empirical definition towards its theoretical definition, defined as:

$$\frac{1}{N-1}\sum_{k=1}^{N}(X_{i_k} - X_{i_{mean}})(X_{j_k} - X_{j_{mean}}) \approx \int_{-\infty}^{+\infty}\int_{-\infty}^{+\infty}(x_i - x_{i_{mean}})(x_j - x_{j_{mean}})C(p_{x_i}, p_{x_j})dx_i dx_j$$

In order to better see the connection here, let's imagine that with the "N" empirical data we actually could split the samples into some p² sub-intervals that would cover the whole range of possible values for X_i on the one hand and for X_j on the other hand. We could then "count" the number of elements of each variable respectively in each sub-interval. Then we could write the approximation:

$$\frac{1}{N-1}\sum_{k=1}^{N}(X_{i_k} - X_{i_{mean}})(X_{j_k} - X_{j_{mean}}) \approx \frac{1}{p^2}\sum_{k_1=1}^{k_1=p}\sum_{k_2=1}^{k_2=p}\left(x_{i_{k_1}} - x_{i_{mean}}\right)\left(x_{j_{k_2}} - x_{j_{mean}}\right)p_{x_{i_{k_1}}, x_{j_{k_2}}}$$

We now can see where we come from applying the theorem of Sklär that states that there is a joint distribution that we name "copula" under rather loose conditions:

$$p_{x_{i_{k_1}}, x_{j_{k_2}}} \approx C\left(p_{x_{i_{k_1}}}, p_{x_{j_{k_2}}}\right)dx_i dx_j$$

The probability, or should we rather say the "density of probability" $p_{x_{i_{k_1}}, x_{j_{k_2}}}$, is what the copula embodies which points to the fact the Copula remains a function of the "bucket" that we focus on here even though the can set a function that purely depends on the the value of this density. We see here that IF the 2 variables were locally independent in a bucket we then ALSO could write that

$$p_{x_{i_{k_1}}, x_{j_{k_2}}} = p_{x_{i_{k_1}}}\, p_{x_{j_{k_2}}} \approx C\left(p_{x_{i_{k_1}}}, p_{x_{j_{k_2}}}\right)dx_i dx_j$$

Then we would infer that C is the identity density function as a function of the individual probabilities. In the more general case, we should write:

$$p_{x_{i_{k_1}},x_{j_{k_2}}} = \rho_{x_{i_{k_1}},x_{j_{k_2}}} p_{x_{i_{k_1}}} p_{x_{j_{k_2}}} \approx C\left(p_{x_{i_{k_1}}}, p_{x_{j_{k_2}}}\right) dx_i dx_j$$

The factor, at first order, feels very much like a "correlation factor" but it represents the dependence relation in a different fashion: it quantifies the chance that only the individual probabilities sufficiently describe the conjunction of co-presence in each bucket. We can observe that from a numerical standpoint we would only care about "p" which would be anyway independent of the eventual "weight" of any asset. We formerly hinted at the "density of probability"…Well here the density of probability is $C\left(p_{x_{i_{k_1}}}, p_{x_{j_{k_2}}}\right) \approx \rho_{x_{i_{k_1}},x_{j_{k_2}}} \frac{p_{x_{i_{k_1}}}}{dx_{i_{k_1}}} \frac{p_{x_{j_{k_2}}}}{dx_{j_{k_2}}}$

Of course the real density of probability would be "lower" as we grow the "granularity" of our bucketing with which we would "cut" the empirical ranges into sub-intervals.

Let's observe, and this is where the "copula" concept is appealing: this model assumes that the other terms do not matter ie $C\left(p_{x_{i_{k_1}}}, p_{x_{j_{k_2}}}\right)$ is NOT conditioned by the values that are taken by the other p_{x_m} with "m" differing from "j and i". We can see here how the concept generalizes what we usually assume with the normal distribution and a "fixed" dependence matrix. This simply comes from the fact that a normal distribution is fully determined by its mean and variance (maximum entropy distribution through a range of $]-\infty,+\infty[$ $for\ a\ known\ mean\ and\ variance$).

One powerful property of the Copula concept is that, if we can keep the same value for "p" irrespective of the "scale" of the weight that would be attributed to each asset, we then deal with constant values relative to the dependence effects.

It feels like ,so far, that we "only" deal with 2 variables at a time. This is not quite true since "C" defines the interdependence of "n" variables across their respective ranges. And the dependence going way beyond just "pairs" of variables, we must consider that $\rho_{x_{i_{k_1}},x_{j_{k_2}}}$ is a function of all the "n" buckets through the other variables. In other words, the theorem of Sklär states that if the dependence factor for 2 given variables depends upon the value of the others, only the probabilities would matter for the other variables since, as stated, this Copula function does exist that is a function of these probabilities alone, not the value of the variables themselves. This covers any distribution that is sufficiently well behaved for sets of variables that are sufficiently homogeneous, a thing that we can easily assume here.

Now, the whole issue is whether the empirical evidence is large enough and, more importantly, whether we can infer a reliable dependence factor numerically speaking IF we are not dealing with proxy joint normal distributions in the background. This is where we face a problem: only the normal distribution provides a dependence factor that can be estimated reliably from empirical evidence and that matches reliable interpolations or extrapolations for prediction purpose. However, assuming for example that we have a sum of normal distributions that have different "spikes", we can work with that and refine the correlation estimates by making them "conditional on the other probabilities of presence".

So from the usual empirical proxy that we apply with the covariance matrix, we actually "miss" the fact that the correlation factor may itself be a function of the other variables and that this sole aspect renders the empirical traditional estimate of the "variance" a misleading one. This is a problem that surfaced with credit derivatives and mandated the use of Copula based models. Even so, the calibration did not work well for other more fundamental reasons like: there is no "risk neutral" rate

and there is no market equilibrium. Still this approach is useful for portfolio risk management (even if it is not sufficient for pricing purpose).

How can we secure that standpoint and, further, how can we introduce in practice a standard deviation value that would permit to separate the "size" through this standard deviation from the "correlation pattern" seen now as a multi-dimensional "dependence pattern"?

Back to the root equation for order 2, we wrote initially:

$$\frac{1}{N-1}\sum_{k=1}^{N}\left(X_{i_k}-X_{i_{mean}}\right)\left(X_{j_k}-X_{j_{mean}}\right)\approx\frac{1}{p^2}\sum_{k_1=1}^{k_1=p}\sum_{k_2=1}^{k_2=p}\left(x_{i_{k_1}}-x_{i_{mean}}\right)\left(x_{j_{k_2}}-x_{j_{mean}}\right)p_{x_{i_{k_1}},x_{j_{k_2}}}$$

We simply stated above that the common "correlation" is some "average" of the dependence and this average itself is averaging other "averages" over the other distributions of the other variables entering in the aggregate joint distribution. Here, thanks to the theorem of Sklär, under rather loos constraints, we have a "weight" invariant measure of the dependence as long as we can "break" the range of possible values into a fix number of sub-intervals "scale-wise". Thus if we were to change the weights we would adjust automatically in proportion the size of the sub-intervals which would guarantee that the probability $p_{x_{i_{k_1}},x_{j_{k_2}}}$, but also the standalone probabilities $p_{x_{1_{k_1}}}\ and\ p_{x_{2_{k_2}}}$, would remain unaltered, and so would the correlation local factor $\rho_{x_{i_{k_1}},x_{j_{k_2}}}$.

Above we observe , that with these conventions in place, the "weights" effect associated to some "unit standard deviation" could be equally introduced through the terms $\left(x_{i_{k_1}}-x_{i_{mean}}\right)p_{x_{1_{k_1}}}$ since at the end of the day we could estimate a "unit +1" standard deviation based upon

$$S_{td_{x_i}}=\sqrt{\frac{1}{p^2}\sum_{k_1=1}^{k_1=p}\sum_{k_2=1}^{k_2=p}\left(x_{i_{k_1}}-x_{i_{mean}}\right)\left(x_{1_{k_2}}-x_{i_{mean}}\right)p_{x_{1_{k_1}}}\,p_{x_{1_{k_1}}}}$$

But here we would carry a very gross estimate of both the local standard deviation and the local correlation pattern that exists between the 2 standalone probabilities in ever bucket.

Yet we can now bridge the gap between the "local" view that is brought up by the concept of Copula and the standard empirical estimate of the "variance":

$$\frac{1}{p^2}\sum_{k_1=1}^{k_1=p}\sum_{k_2=1}^{k_2=p}\rho_{x_{i_{k_1}},x_{j_{k_2}}}\left(x_{i_{k_1}}-x_{i_{mean}}\right)p_{x_{1_{k_1}}}\left(x_{j_{k_2}}-x_{j_{mean}}\right)\,p_{x_{2_{k_2}}}=\rho_{x_i,x_j}S_{td_{x_i}}S_{td_{x_j}}$$

The emphasis on the weights being "specified" or not specified will show its importance later on when we will aim at shaping the "risk profile" of the investor n the one hand, and search the "optimal weights" next on the other hand. So far we have established how we could define "local correlation factors and how they relate the otherwise well chartered "linear correlation" factor. We above how this factor is a mere expectation, in 2 dimensions, of local standard deviations themselves averaged.

Can we extend this approach to other moments of higher order, like the "skew" , the "kurtosis" and beyond to any order? If "yes" is there a link towards the "moment generating" function in n dimensions?

We have now to fill the gap between this correlation pattern, including a copula or being just an average correlation factor across the range of variations, and the optimization target. The latter would most spontaneously show up an optimal return, risk adjusted. And here we could invoke the model of Harry Markowitz with the "efficient" frontier: for a given level of risk there is a maximum return that corresponds to a certain set of weights. These weights are inferred from the "slope" of the tangent line to the efficient frontier at the absciss that is equal to the risk level. The risk in itself is estimated with the variance against the squared return provided the variables are "independent". Equivalently we could optimize the expected return versus the standard deviation. The slope would change but the principle remains the same. We also saw that in general the variables are NOT independent. Yet we can use a correlation matrix, diagonalize it and revert to a formulation of the problem that is using independent "eigen" assets. We just saw that most of the time we can refine this approach further provided we have enough empirical data, define a Copula model and run this optimization conditioned upon the current probabilities of the other variables. It is consistent. But so far we only dealt with the first 2 statistical moments.

6- Time series , diagonalization and LLMs

This part deals with the basic techniques that handle time series and linear regressions.

The first part sketches the core issues for financial modeling and introduces the concept of "spectral analysis" where one focuses on the frequencies rather than the trends.

The second part puts an emphasis on the diagonalisation of matrices

The Last part summarizes the principles that govern LLMs like ChatGPT

The first part: trends and frequencies…..

Warm-up: exercises

1- given m and n 2 integers with a ppcm a and a pgcd b: find a simple expression of "ab" with m and n
2- Let a polynomial of 2nd order P. We know one root x and the value y where its derivative is 0: What is the value of the second root as a function of x and y?
3- What is the simple expression for $1 + x + x^2 + x^3 + \cdots . when\ |x| < 1$?
4- What is the value of $\sum_{k=1}^{N} k$? What is the value of $\sum_{k-1}^{N} k^2$? What is the value of $\sum_{k=1}^{N} k^3$?
5- Taylor-Young formula in 1 dimension: derivation, approximation… intuition….: any time series can be approximated with a polynomial with an arbitrary accuracy does it make sense?

Course

Executive summary:

A linear regression renders a simplified but powerful local "solution" for interpolation and extrapolation. It is useful but very soon it becomes frustrating and inefficient. Once one gets a relatively nicely correlated cloud of points, one should expect the CLT (Central Limit Theorem) to work its magic: if you assume that the dispersion from the mean ultimately should follow a normal distribution, then betting on the mean reversion mechanism should always work at one point in time. Unfortunately this is true only "at the end of times" and, in the meantime, the mean and the

variance themselves are random and they can go crazy at times…right when you would have thought "well this time it went too far, I am going counter to it because it must come back home…it must.." The fact is it does but the probability that you can hold on your position in the meantime is close to 0%.

The problem arises because the regression factors tend to fluctuate themselves but with a varying speed and magnitude. It is quite tempting to think then : " ok they also should converge towards a normal distribution…so why not pair the first deviation from the mean, variance adjusted, and the deviation of the mean itself from its own longer term mean, PLUS compound this with the distance of the variance relative to its own longer term expectation…Hmm… by the way is there a trend here?..." Making regressions over regression only piles up inertia and inaccuracies. The more you do that, the more you obtain a statistical significance that is reliable ONLY if the world has not changed…But these oscillations are precisely the sign that the world is changing…

The polynomial approach is only "better" since this is just a way to run a linear regression on derivatives first before spotting the optimal fitting constant. One salient issue is: any polynomial diverges towards infinity. You then would get a quite refined and accurate prediction tool that can only be relied upon in short periods of time. This is what hedge funds like Citadel or Renaissance do in essence with computer that trade every nanosecond! The structural divergence towards infinity is a core issue: since life has no room for infinity, we must infer that a polynomial is a misleading "good" approximation.

The convolution product would apparently morph a chaotic chart into a polynomial. But really it just offers a middle ground solution: it is not super accurate but it never diverges to infinity. That solution here could also be understood as a quasi-periodic function. Then the Fourier transform could be more efficient in catching the competing cycles that compound through the standalone signal. This is the main message of this lesson: rather than linear regression, use convolution products coupled with Fourier series. Actually most bot-trading engines rely on that approach. They also try to trade every nanosecond.

Main course….

We will begin with the most commonly known linear regression in 2D, in order to keep things simple. Imagine you have some points M(x,y) having an abscissa "x" and an ordinate "y". The points form what is called a "cloud of points" and we look for the optimal "slope a and a constant b" so that the aggregate distance of these points to the line "y=ax+b" is the smallest possible one.

If we index the "N" points M that we have as $M_k(x_k, y_k)$ we then would define a and b as:

$$D(a,b) = Min_{A,B}\left[\sum_{k=1}^{N}\left(y_k - (Ax_k + B)\right)^2\right] = Min_{A,B}\left[D(A,B)\right]$$

The function that we study here is $D(A,B) = \left[\sum_{k=1}^{N}\left(y_k - (Ax_k + B)\right)^2\right]$

The optimum for this function is where $\frac{\partial D}{\partial A} = \frac{\partial D}{\partial B} = 0$ ie when A=a, B=b and

$2\sum_{k=1}^{N} x_k\left(y_k - (ax_k + b)\right)^1 = 0$ and $2\sum_{k=1}^{N}\left(y_k - (ax_k + b)\right)^1 = 0$.

We can get rid of the factor "2". We introduce the average values for the x_k and for the y_k values:

$$x_m = \frac{1}{N}\sum_{k=1}^{N} x_k \ \ and\ y_m = \frac{1}{N}\sum_{k=1}^{N} y_k$$

From the equation $\sum_{k=1}^{N}\left(y_k - (ax_k + b)\right)^1 = 0 \quad we\ infer\ that\ Ny_m - aNx_m - Nb = 0$

We therefore obtain a first equation for a and b: $y_m = ax_m + b$

The second equation is a little more complex: $\sum_{k=1}^{N} x_k\left(y_k - (ax_k + b)\right)^1 = 0$

$$\sum_{k=1}^{N} x_k\left(y_k - (ax_k + b)\right)^1 = \sum_{k=1}^{N} x_k y_k - a\sum_{k=1}^{N} x_k^2 - Nbx_m \quad = 0$$

We need to introduce here the variances for the $x_k\ noted\ v_x$, the variance of the $y_k\ noted\ v_y$ and the correlation factor between the $x_k\ and\ the\ y_k\ noted\ \rho_{xy}$.

Here we have:

$v_x = \frac{1}{N-1}\sum_{k=1}^{N}(x_k - x_m)^2$, $v_y = \frac{1}{N-1}\sum_{k=1}^{N}(y_k - y_m)^2 \ \ and \rho_{xy} = \frac{1}{N-1}\sum_{k=1}^{k=N}\frac{(x_k - x_m)(y_k - y_m)}{v_x v_y}$

$$\rho_{xy} = \frac{1}{N-1}\sum_{k=1}^{k=N}\frac{(x_k - x_m)(y_k - y_m)}{v_x v_y} = \frac{1}{(N-1)v_x v_y}\sum_{k=1}^{k=N}(x_k - x_m)(y_k - y_m)$$

$$\frac{1}{(N-1)v_x v_y}\sum_{k=1}^{k=N}(x_k - x_m)(y_k - y_m) = \frac{1}{(N-1)v_x v_y}\left(\sum_{k=1}^{k=N} x_k y_k - Nx_m y_m\right)$$

We infer this result after developing: $(x_k - x_m)(y_k - y_m) = x_k y_k - x_k y_m - x_m y_k + x_m y_m$

Summing up over "N" we get $-Nx_m y_m - Nx_m y_m + Nx_m y_m - Nx_m y_m$

So we infer now that:

$$\rho_{xy} = \frac{1}{(N-1)v_x v_y}\left(\sum_{k=1}^{k=N} x_k y_k - Nx_m y_m\right) \rightarrow \sum_{k=1}^{k=N} x_k y_k = Nx_m y_m + (N-1)\rho_{xy} v_x v_y$$

From the second equation, we therefore infer that: from $\sum_{k=1}^{N} x_k^2 \quad = (N-1)v_x + Nx_m^2$

$$\sum_{k=1}^{N} x_k y_k - a\sum_{k=1}^{N} x_k^2 \quad - Nbx_m = 0 \quad hence\ Nx_m y_m + (N-1)\rho_{xy} v_x v_y - (N-1)av_x - aNx_m^2 - Nbx_m = 0$$

Which sums up to: $Nx_m y_m + (N-1)\rho_{xy} v_x v_y = (N-1)av_x + aNx_m^2 + Nbx_m$

We therefore built a system of 2 equations and 2 unknowns(a,b):

$$\begin{cases} y_m = ax_m + b \\ Nx_m y_m + (N-1)\rho_{xy} v_x v_y = (N-1)av_x + aNx_m^2 + Nbx_m \end{cases}$$

We can analyze a little more this system in order to explain "why", in the following part we will estimate the slope based on the correlation factor mostly…We can morph the equations a little bit first:

$$\begin{cases} ax_m = y_m - b \\ Nx_m(y_m - ax_m - b) + (N-1)\rho_{xy}v_x v_y = (N-1)av_x \end{cases}$$

$$\begin{cases} ax_m = y_m - b \\ \rho_{xy}v_y = a \end{cases}$$

Thus the slope as a first approximation is the average "slope" of "y" relative to "x" originating in (0,0). Note that when market participants "look" at moving averages that do implicitly just "that", ie they center their "time variable x" around 0. The moving average is, as such, well known for being "lagging" as an indicator to the trend.

The cornerstone here is "how can we combine convolution products and linear regression in order to make a more efficient algorithm?"

The mathematical background can help answering the question above. The scalar product is in fact a common feature here. We will show the main pitfall of the linear regression approach.

Let's take the simple case of a time series: we have a random variable x_k over the time index t_k. In order to make the case generic, we will assume that we may not observe the prices at regular time intervals. This is the case typically in illiquid markets or regular markets. This is more generally the case for all markets for, as far as the theory of "efficient markets" goes, the "intensity of time" is a direct function of the traded volumes. Indeed if a market is "open" but does not trade, it is as if "time" has not passed since no information has gone through. We frequently observe that phenomenon across the week-ends despite the fact the coupon is accruing, the dividends and profits accrue in quoted companies: yet, the price would have anticipated "that" by trading a bit more "towards the close" and "at the open".

So the volumes matter a lot and therefore the "time index" is different from the time on our clocks when market prices are concerned.

Now that we have an intuitive approach of our linear regression here: we want to find the trend over "time" of "x". In reality we have 2 random variables here: "x" and "t" and we actually want to know how their respective series line up.

If we define first, a target "slope" and a target "residual error", we want to find the average slope between x and t , given a certain "constant" that will minimize the "residual error" also known as the R^2 .

We would start by generating a "point 0" by computing the difference of the future records versus this very first one record : we then secure that the constant from the origin should be 0 although we know already that it will not be that in the general case. The reason is simple: unless the records exhibit a very regular trend, there is a higher chance of an acceleration, up, down, both in varying magnitudes… Then the constant will just "fit" this evolution.

So we would change the notations:

$$x_k \to z_k = x_k - x_0 \; and \; t_k \to T_k = t_k - t_0$$

Next, in the future, we will have pairs of (T_k, z_k) that will all exhibit a trend as a function of $T_{\cdot}$.. in general. The value for the "slope" ie the trend will be obtained through: $s_N = \frac{1}{N}\sum_{k=1}^{k=N}\frac{z_k}{T_k}$

Then we will want to "fit" at best the residual variance by finding a "constant value at the origin T=0" that may not be zero but that would minimize the variance between the "model" and the "empirical data". We therefore look for a value C such that the variance V(C):

$$V(C) = \sum_{k=1}^{k=N} (z_k - sT_k - C)^2 \ is\ minimal$$

Deriving V relative to C because the minimum must be where the derivative is nil we infer:

$$\frac{dV}{dC}(C) = \sum_{k=1}^{k=N} -2(z_k - sT_k - C) \ldots\ldots = 0\ when\ C\ is\ the\ minimum\ value$$

But developing the summation terms we realize the following: recall $s_N = \frac{1}{N}\sum_{k=1}^{k=N} \frac{z_k}{T_k}$

$$\sum_{k=1}^{k=N} (z_k - sT_k - C) = NE(z_k) - NsE(T_k) - NC\ and\ therefore\ C = E(z_k) - sE(T_k)$$

However: if now we set $s_k = \frac{z_k}{T_k}\ then\ s_N = E(s_k), z_k = s_k T_k\ and\ thus\ C = \frac{1}{N}\left(\sum_{k-1}^{k=N}(s_k - s_N)T_k\right)$

Let's observe the confirmation that if the different slopes average relative to s_N with certainty the weighted difference by T_k does not sum to 0 at all in general. We can see that the differences for the longest T_k will influence the most the value of C since this is when the multiplier effect of T is the largest.

Remark on the side:

If T_k is a quite simple function like $T_k = k$ as in the naïve approach of the markets where volumes do not matter, we can observe that, if s_k fluctuates around "s_N" then the fluctuation will be multiplied by "k". If the fluctuation is periodic so that s_k Is relatively often "higher" and "lower", the constant will NOT be 0 but still close to 0.

To see it let's assume for example that N=2P and that $s_k - s = +1\ for\ k \leq P\ and\ s_k - s = -1\ for\ k > P$, then $C = \frac{1}{N}\left(\sum_{k-1}^{k=N}(s_k - s)T_k\right) = \frac{1}{2P}\left(\sum_{k=1}^{k=P} k - \sum_{k=P+1}^{k=2P} k\right)$

Thus $C = s\frac{1}{2P}\sum_{k=P+1}^{k=2P}(k - (P + k)) = -s\frac{P}{2}$

The point of this remark is that, of course the constant, will highly depend upon the actual distribution of the intermediary slopes for an average that is the same. We need to envision that the bias is induced by the fact that for long periods the "-1" applies to the whole period ie the "underperformance" is really growing versus the main slope. Starting at (0,0) matters very little numerically on long time series but it makes the analysis simpler.

Having found the optimal constant, we can infer the residual variance now that is the variance formula that we wrote above:

$$V(C) = \sum_{k=1}^{k=N} (z_k - sT_k - C)^2 = \sum_{k=1}^{k=N} \Big(\big(z_k - E(z)\big) - s\left(T_k - E(T)\right)\Big)^2$$

What would the difference between these measures and the one inferred from the "local variations" be?

By “local variations”, we refer to the changes from one price to the next in the time series.

Let’s define them explicitly: $dx_k = x_{k+1} - x_k, dT_k = T_{k+1} - T_k$

We can now set a connection with the former variables: $z_k = \sum_{j=0}^{k-1} dx_j \, , and \; T_k = \sum_{j=0}^{l=k-1} dT_j$

If now we insert these new notations into “s” and “C”, we get:

$$s = \frac{1}{N}\sum_{k=1}^{k=N} \frac{\sum_{j=0}^{k-1} dx_j}{\sum_{j=0}^{l=k-1} dT_j} \; and \; C = \frac{1}{N}\left(\sum_{k=1}^{k=N}\sum_{j=0}^{k-1} dx_j - s\sum_{k=1}^{k=N}\sum_{j=0}^{k-1} dT_j\right)$$

We see that the first terms appear more often than the latest terms and we can group this effect as follows:

$$C = \frac{1}{N}\left(\sum_{k=0}^{k=N-1} (N-k)(dz_k - s dT_k)\right)$$

We can observe, on the follow of the remark above, that measuring the “trend” from local moves can become misleading. In other words the slope is dependent upon the time period that we look at. This observation as we saw right above, shows that the constant C value highly depends upon the variance of the slope itself throughout the whole period. Thus there is an optical bias that often appears among the markets participants when they tend to rely on moving averages in order to estimate the trend. They will be fine with the idea that “yes” the trend is also random. But they would rarely notice that while they follow the trend, if this trend is more volatile lately and an inch lower it means that the market is basing itself on lower and lower “constants” ie lower and lower market prices looking forward. The most counter-intuitive byproduct of that observation is when you turn this picture “upside down” and consider a bear market that has quieted down….this time the implied constant is getting higher and higher while the markets remain bearish paradoxically at a moment when , precisely, the overall markets anticipate higher and higher prices looking forward. The “timing” issue of course is much more dominant since the market players are normally short in capital after a market fall and therefore hesitate even more to bet for a rebound. They rarely look at the issue from the “constant C” standpoint, trying instead to read the tea leaves in the “trend” and its moving averages.

This implies that, unless we can frame this regression analysis into a cyclical pattern, nothing reliable can be inferred from a time series like this one….unless we can model safely the randomness of the slope itself…Which would imply a similar regression, that would induce a new constant, that itself would highly depend upon the variance of the gradient number for the slope itself….And so on and so forth. We could also increase the number of dimensions that describe the time series, but this would still require a periodic pattern.

Now what would a convolution product do?

First let’s observe that by definition of “s_N”, ie $s_N = \frac{1}{N}\sum_{k=1}^{k=N} \frac{z_k}{T_k}$,

and “C”, ie $C = \frac{1}{N}\left(\sum_{k-1}^{k=N}(s_k - s_N)T_k\right)$,

We apply sums of products, namely scalar products. This is also what convolution products do but in a different “spirit.” The convolution product would “roll” the regression . The whole being: once reduced to a periodic function, can we spot a regression with a reliable constant?

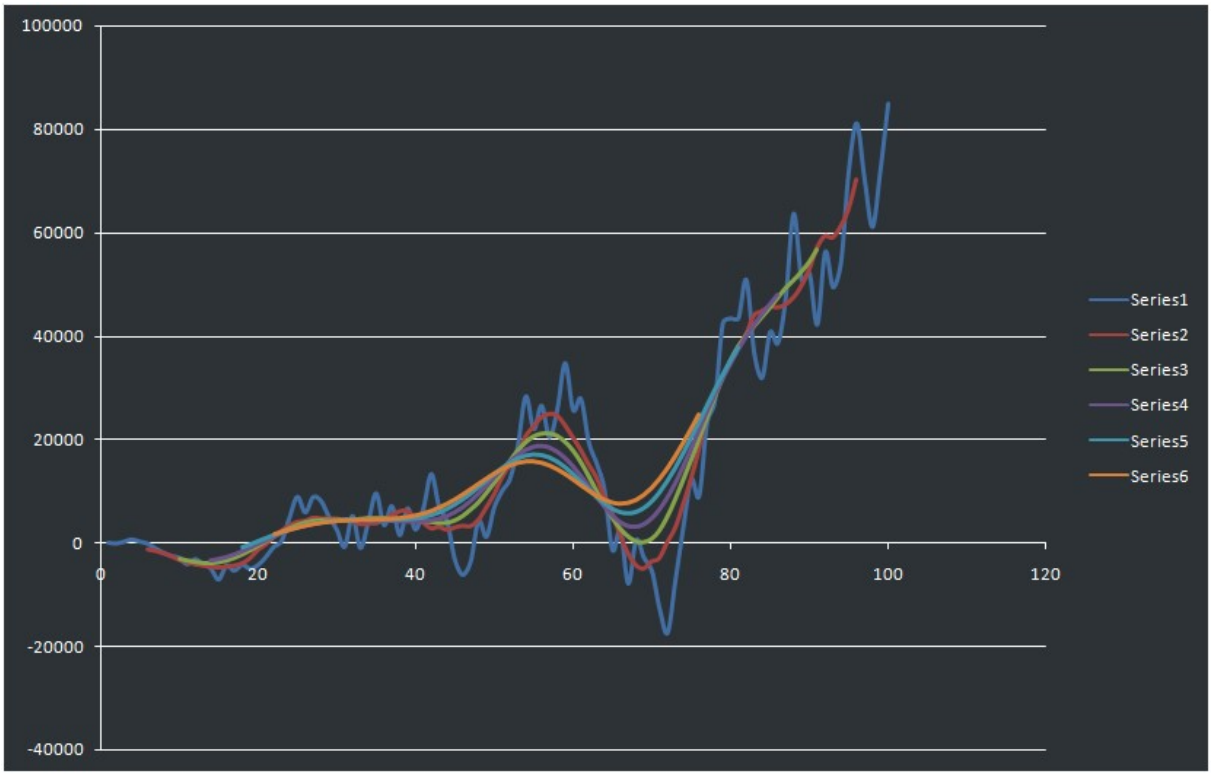
We can see that the “right “regression would fit at best the empirical data based on the “polynomial” that would arise from the compounded convolution products.

Now if we compound 4 or 5 convolution products with a distribution of weights that may NOT be uniform in the end, what would we get?. As a start, it will be uniform as pictured above. The purpose of shifting the distribution in the convolution product will be motivated and driven by what follows….

So, to prove the point, we will introduce the concept of convolution product: we introduce some “weights” noted a_k that will “digest” a point alongside its close neighbors and computes an weighted average that will replace the original value in a new series rank-wise. In other words, we take a native series of prices, we compute for a price at rank “n” in the time series a convolution product at the rank “k” in order to produce a new time series. But in this new time series, the “values” will be weighted average prices that will mix the price at rank “k” with its neighbors. The idea is to generate recursively several convolved series like this one based on another convolved series, all originating back to the native time series. Thus we would define the following compound convolved function:

$$y_{5,n} = \frac{1}{2p+1}\sum_{k=-p}^{k=+p} a_k y_{4,k} \ \ with\ y_{4,n} = \frac{1}{2p+1}\sum_{k=-p}^{k=+p} a_k y_{3,k} \ \ with\ y_{3,n} = \frac{1}{2p+1}\sum_{k=-p}^{k=+p} a_k y_{2,k} \ \ etc..$$

The chart below illustrates the effect of a convolution product that compounds itself:



We can notice the “smoothing” effect that is here because the weighting is based on “past” and “future” points relative to the rank “k”. So the convolution product would be systematically a lagging indicator of the current trend. These are weighted moving averages in layman’s terms. So the best we can do then is try to catch the periodicity of the main “cycles” that seem to appear.

Thus, we would define the Fourier series of this function as being bounded in an interval [-L,+L]:

$$f(n) = y_{5,n}\ and\ f_n = \frac{1}{2L}\int_{-L}^{+L} fx)e^{\frac{2in\pi x}{L}}dx$$

Look at the appendices for more information on the Fourier series

Have a glance at the other appendices….

APPENDICES

i- Convolution products elements

Let's consider a series x_n and the convolution product $y_{1n} = \sum_{k=-p}^{k=p} b_k x_k\ with\ \sum_{k=-p}^{k=p} b_k = 1$

The factors b_k are all >0 here but this is not mandatory in the more general case.

Let's assume that they are all equal and, of course n>p-1 so that this definition makes sense. We can then write $b_k = \frac{1}{2p+1}$ and $y_{1n} = \frac{1}{2p+1}\sum_{k=-p}^{k=p} x_{n+k}$

Let's define now a compound convolution product with $y_{2n} = \frac{1}{2p+1}\sum_{k=-p}^{k=p} y_{1,n+k}$

$$y_{2,n} = \frac{1}{2p+1}\sum_{k=-p}^{k=+p}\frac{1}{2p+1}\sum_{j=-p}^{j=p} x_{n+k+j}$$

We can see that some terms appear many times because "n+k+j" can sum up to very same number, assuming of course that n>2p-1. Then "n-2p" would appear once but "n-2p+1" would appear twice: once with k=-p and j=-p+1 and a second time for k=p-1 and j=p.

As to "n-2p+2" we have respectively for k and j the pairs (-p,-p+2)->(-p+1,-p+1)->(-p+2,-p) ie 3 combinations. We can generalize by saying that we would have "m+1" cases for "n-2p+m" up until m=2p where we would have still 2p+1 cases. But starting with m=2p+1 we then would have as many cases as when m=2p-1, ie "2p" cases ie "m-2+1=m-1". Likewise if m=2p+2, it is the same number of cases as when m=2p-2, ie 2p-1=m-3. Next will be "m-5" for m=2p+3 and more generally for m=2p+l we have "m+1-2l" cases, ie one case left alone for l=2p…As expected.

We therefore can write:

$$y_{2,n} = \left(\frac{1}{2p+1}\right)^2\sum_{k=-2p}^{k=+0}(k+1)x_{n-k} + \left(\frac{1}{2p+1}\right)^2\sum_{k=1}^{k=2p}(2p-k+1)x_{n+k}$$

We observe that the different elements of the initial series are weighed differently with an emphasis on the 'current spot'.

The weights are growing linearly ie regularly and declining next in the same fashion on the follow.

If now we look at the variation of 2 consecutive terms:

$$y_{2,n+1} - y_{2,n} = \left(\frac{1}{2p+1}\right)^2\sum_{k=0}^{k=2p} x_{n+k} - \left(\frac{1}{2p+1}\right)^2\sum_{k=1}^{k=2p} x_{n-k} = \left(\frac{1}{2p+1}\right)^2\sum_{k=0}^{k=+2p}(x_{n+k} - x_{n-k})$$

We can observe that the convolution product is having a smoothed "slope" for it takes the "average" of the variations spanning around the current spot.

Indeed let's assume now that we do have a constant value here.

Then $x_{n+k} - x_{n-k} = 2ks_{lope}$

$$y_{2,n+1} - y_{2,n} = \left(\frac{1}{2p+1}\right)^2 \sum_{k=0}^{k=+2p} (x_{n+k} - x_{n-k}) = s_{lope}\left(\frac{1}{2p+1}\right)^2 \sum_{k=0}^{k=+2p} 2k = \frac{s_{lope}(2p)}{2p+1}$$

ii- Fourier transform of a sequence

Let's consider a function f that is not zero on a some symmetrical range and its Fourier series: $f_n = \frac{1}{2L}\int_{-L}^{+L} f(x)e^{-n\frac{i2\pi x}{2L}}\ dx\ and\ if\ f\ is\ periodic\ or\ not\ f(x) = \sum_{-\infty}^{+\infty} f_n e^{+n\frac{i2\pi x}{2L}}$

The link to the usual Fourier series stands as follows:

$$a_n = \frac{1}{2L}\int_{-L}^{+L} f(x)\cos\left(n\frac{2\pi x}{2L}\right)\ dx = \frac{1}{2L}\int_{-L}^{+L} \frac{f(x)1}{2}\left(e^{-n\frac{i2\pi x}{2L}} + e^{+n\frac{i2\pi x}{2L}}\right)dx$$

$$b_n = \frac{1}{2L}\int_{-L}^{+L} f(x)\sin\left(n\frac{2\pi x}{2L}\right)\ dx = \frac{1}{2L}\int_{-L}^{+L} \frac{f(x)1}{2i}\left(-e^{-n\frac{i2\pi x}{2L}} + e^{+n\frac{i2\pi x}{2L}}\right)dx$$

$$f(x) = \sum_{k=0}^{+\infty} a_k \cos\left(k\frac{2\pi x}{2L}\right) + \sum_{k=0}^{+\infty} b_k \sin\left(k\frac{2\pi x}{2L}\right)$$

$$f(x) = \sum_{k=0}^{+\infty} \underbrace{(a_k - ib_k)}_{f_k} e^{+k\frac{i2\pi x}{2L}} + \sum_{k=0}^{+\infty} \underbrace{(a_k + ib_k)}_{f_k} e^{-k\frac{i2\pi x}{2L}}$$

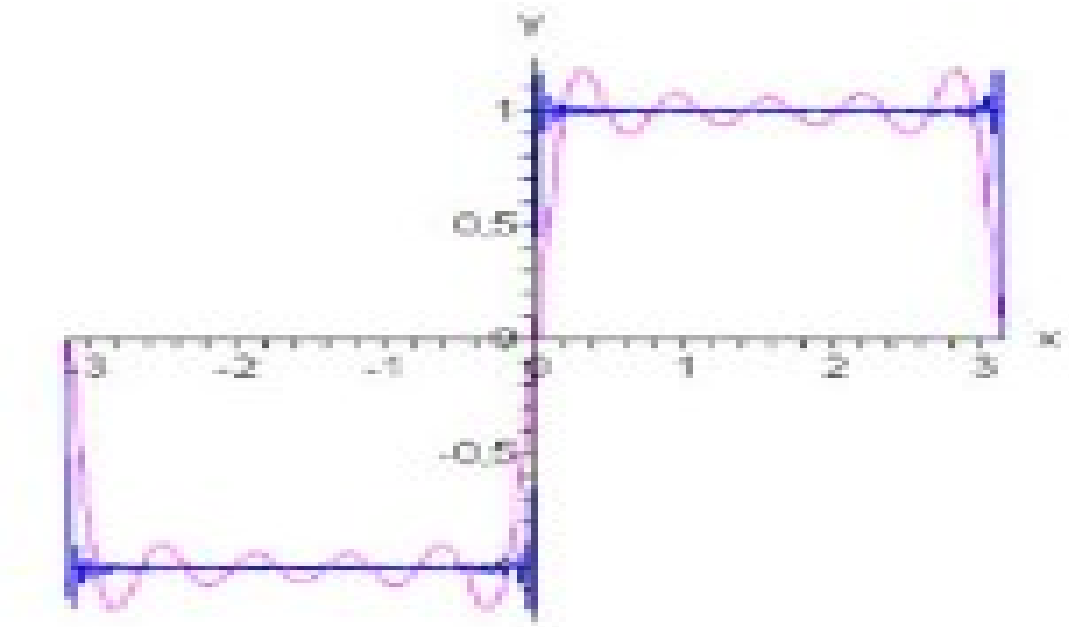
To be sure:

$$a_n - ib_n = \frac{1}{2L}\int_{-L}^{+L} f(x)\left(\cos\left(n\frac{2\pi x}{2L}\right) - i\sin\left(n\frac{2\pi x}{2L}\right)\right)\ dx = \frac{1}{2L}\int_{-L}^{+L} f(x)e^{-n\frac{i2\pi x}{2L}}\ dx$$

$$a_n + ib_n = \frac{1}{2L}\int_{-L}^{+L} f(x)\left(\cos\left(n\frac{2\pi x}{2L}\right) + i\sin\left(n\frac{2\pi x}{2L}\right)\right)\ dx = \frac{1}{2L}\int_{-L}^{+L} f(x)e^{+n\frac{i2\pi x}{2L}}\ dx$$

$$Therefore\ we\ have\ f_{-n} = \overline{f_n}\ with\ \ a_n = \frac{1}{2}(f_n + f_{-n})\ and\ b_n = \frac{1}{2i}(f_{-n} - f_n)$$

BY defining the many Fourier series factors we can set the most important frequencies
One salient useful case is the one of the sign function



If now we consider the discrete version….
If start with a function "f" that is piecewise constant, we can write a sequence:

$$Y_n e^{-n\frac{i2\pi x_n}{2L}} = \int_{x_n}^{x_{n+1}} f(x)dx$$

We then can write the Fourier coefficients with this sequence, provided we can split the interval [-L;+L] into sub intervals where the function f is piece-wise constant.

Then we would split [-L;+L] into sub-interval of length $\frac{2L}{N}$ $and\ Y_0 = \int_{-L}^{-L+\frac{2L}{N}} f(x)dx$. We would strip the intergral along the N sub-intervals $\left[-L;\ -L+\frac{2L}{N}\right], \left[-L+\frac{2L}{N}; -L+2\frac{2L}{N}\right]$ and so on up to $\left[+L-\frac{2L}{N}; +L\right]$...thus $x_n = -L + (n-1)\frac{2L}{N}$

$$f_n = \frac{1}{2L}\int_{-L}^{+L} f(x)e^{-n\frac{i2\pi x}{2L}}\ dx = \frac{1}{2L}\sum_{0}^{N} Y_k e^{-n\frac{i2\pi x_k}{2L}}\ and\ still\ f(x_p) = \sum_{-\infty}^{+\infty} f_n e^{+n\frac{i2\pi x_p}{2L}}$$

In a similar fashion, we can define a discrete version too for:

$$a_n = \frac{1}{2L}\int_{-L}^{+L} f(x)\cos\left(n\frac{2\pi x}{2L}\right)\ dx = \frac{1}{2L}\sum_{0}^{N} Y_k \cos\left(n\frac{2\pi x_k}{2L}\right)$$

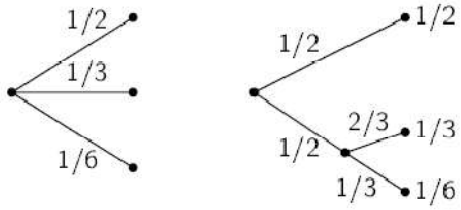
$$b_n = \frac{1}{2L}\int_{-L}^{+L} f(x)\sin\left(n\frac{2\pi x}{2L}\right)\ dx = \frac{1}{2L}\sum_{0}^{N} Y_k \sin\left(n\frac{2\pi x_k}{2L}\right)$$

$$f(x_p) = \sum_{k=0}^{+\infty} a_k \cos\left(k\frac{2\pi x_p}{2L}\right) + \sum_{k=0}^{+\infty} b_k \sin\left(k\frac{2\pi x_p}{2L}\right)$$

The discrete version is more often used in practice because we actually only have partial and anecdotal evidence of the underlying function "f". The more data we have the better the approximation of the whole function.

iii- Entropy of information: definition, concept

- The image below shows the case:



In such a situation,

$$H\left(\frac{1}{2},\frac{1}{3},\frac{1}{6}\right) = H\left(\frac{1}{2},\frac{1}{2}\right) + \frac{1}{2}H\left(\frac{1}{3},\frac{2}{3}\right)$$

This equation expresses that H does not depend on the "path" that was followed to land on the final distribution. The ½ factor above appears because the sub-choice only occurs half the time. To be sure again if we compounded "m" times some uniform distribution over says "n" equally weighted outcomes we would write $H(n^m) = mH(n)$. More generally, and this is how one actually proves the expression in general, when we face an arbitrary 2-stage distribution we write the entropy as:

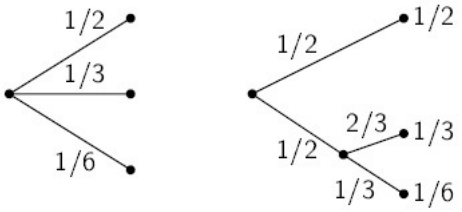
$H(\text{p=}\{p_i\})=\text{H}(\{p_i, n_i\}) = \sum_{i=1}^{n} p_i \underbrace{H\left(\frac{1}{n_i}, \dots, , \frac{1}{n_i}\right)}_{=Klog\left(\frac{1}{n_i}\right)}$.

We need to bootstrap the stage based errors for each configuration. Thus compounding now "2" consecutives stages, we can actually we compound one distribution with an another as the examples exhibited. We can see easily how the consecutive stages combine by conditional probabilities. How can we attribute an sub error to each intermediate stage?

We will use another form of this property : H(X,Y)=H(Z=X+Y)=H(X)+H(Y|X). And we should then specify that $H(Y|X) = \sum -p(x,y)Ln\left(\frac{p(x,y)}{p(x)}\right)$. If Y is the entropy, ie the loss, at stage n, then X is its counterpart at stage "n-1". The equation above is a ùore general illustration of the fact that $H(n^m) = mH(n)$, or that IF one ultimate distribution results from say p stages of conditional probabilities, then the ultimate entropy is just the sum of the resulting conditional entropies as expressed above.

Shannon defined 3 conditions for this measure "H" to address the question above:

- H should be continuous as a multi-variate function of the many p_i
- If all the probabilities are equal, ie $p_i = \frac{1}{n}$, then H is a growing monotonic function of n. This states that a 2 spike distribution has a much lower entropy than a uniform distribution…while the variances rank in the opposite order….
- If an ultimate choice requires an intermediary choice then H should be "constant" ie the H is the weighted sum of all the possible intermediary +final stages. Thus the "uncertainty" is solely representing the ultimate set of "choices" anyway NOT the "path" followed to arrive at this set of choices. The image below shows the case:



In such a situation,

$$H\left(\frac{1}{2},\frac{1}{3},\frac{1}{6}\right) = H\left(\frac{1}{2},\frac{1}{2}\right) + \frac{1}{2}H\left(\frac{1}{3},\frac{2}{3}\right)$$

This equation expresses that H does not depend on the "path" that was followed to land on the final distribution. The ½ factor above appears because the sub-choice only occurs half the time. Notice that "H" is just a function or more accurately a "measure" which is more generic than a function and has some monotonic properties that a function does not always have. In particular H has an arbitrary ability to be "undefined" on some spots like a Dirac measure for as many points as needed provided it is itself of "measure nil".

The proof now of the unique form that H can take….

Let $\left(\frac{1}{n},\frac{1}{n},\ldots,\frac{1}{n}\right) = A(n)$. Just a simplified notation of a uniform distribution….We will now focus on the 3rd property that H must verify…

Let's consider that we have "s" equally probable outcomes and that we make "m" consecutive choices among these "s" equally probably intermediate outcomes…That will produce a new distribution of s^m many outcomes that remain equally probable….

From the 3rd property, we can write that: $A(s^m) = mA(s)$. Indeed, if we look back up at the graph displaying the example, we "see" that we use one branch that diffuses "m" times the very same root uniform distribution. Imagine for example that "s"=2 and that the '1/2-1/2' distribution if diffused 2 times….we would get:

$$H\left(\frac{1}{4},\frac{1}{4},\frac{1}{4},\frac{1}{4}\right) = H\left(\frac{1}{2},\frac{1}{2}\right) + \frac{1}{2}H\left(\frac{1}{2},\frac{1}{2}\right) + \frac{1}{2}H\left(\frac{1}{2},\frac{1}{2}\right) = 2H\left(\frac{1}{2},\frac{1}{2}\right)$$

By recursion, if we go on like this for 3 choices, we get:

$$H\left(\frac{1}{8},\dots,\frac{1}{8}\right) = H\left(\frac{1}{2},\frac{1}{2}\right) + \frac{1}{2}H\left(\frac{1}{4},\frac{1}{4},\frac{1}{4},\frac{1}{4}\right) + \frac{1}{2}H\left(\frac{1}{4},\frac{1}{4},\frac{1}{4},\frac{1}{4}\right) = 3H\left(\frac{1}{2},\frac{1}{2}\right)$$

And so on and so forth….We could replace "2" with "s" indeed and "3" with "m"….accordingly.

In a similar fashion, we can consider now "t" equally probable "events" among which we make this time "n" consecutive choices and we thus would write that: $A(t^n) = nA(t)$. In essence we know that this function equation solves for 'Log' and Log only….

We consider integers and, given "t" and "s" set, we can always find "n" and "m" so that: $s^m \leq t^n < s^{m+1}$

Using the monotonic "growing" pattern of the logarithm:

$$\frac{m}{n} \leq \frac{\log(t)}{\log(s)} < \frac{m}{n} + \frac{1}{n} \; or \; \left|\frac{\log(t)}{\log(s)} - \frac{m}{n}\right| < \frac{1}{n} < \varepsilon \; \forall \varepsilon \; if \; n \; large \; enough$$

Using now the fact that "H", and therefore "A", are monotonic and growing when the distribution is uniform (properties 1 and 2 taken together), we can write first that :

$$A(s^m) \leq A(t^n) < A(s^{m+1})$$

We infer from that :

$$mA(s) \leq nA(t) < (m+1)A(s)$$

Assuming that A(s)>=0, we then infer that:

$$\left|\frac{\mathrm{A}(t)}{A(s)} - \frac{m}{n}\right| < \frac{1}{n} < \varepsilon$$

and therefore: $\left|\frac{\mathrm{A}(t)}{A(s)} - \frac{\log(t)}{\log(s)}\right| < 2\varepsilon$ for n "high" enough….

We thus have proved that :

$$\mathrm{A}(t) = K.log(t) \; due \; to \; its \; "continuity \; on \; (t,s)"$$

To be clear: this does NOT mean that the measure H is always continuous as a multivariate function since the distribution "p" is not continuous in general. But H is by design continuous when its observed individual probabilities change continuously. A Dirac case will be shown to be a well behaved "limit case"…

Now let's consider a more generic distribution that exhibits "n" distinct "events" or "buckets" of outcomes with respective probabilities p=$\{p_i\}$….

Let's assume that we can make a "count" ie that $p_i = \frac{n_i}{\underbrace{\sum_{i=1}^{n} n_i}_{N}}$

We have this 3rd property that secures that H is the same IF we look at this distribution as p=$\{p_i\}$…. Or if we look at the same distribution that actually has N possible many outcomes where it is obtained from a combined "choice" among the "n" probabilities first and next among a "uniform" choice of n_i equally probable cases. We thus can use the 3rd property to write that : H(p=$\{p_i\}$)=H($\{p_i, n_i\}$) = $\sum_{i=1}^{n} p_i \underbrace{H(\frac{1}{n_i}, \dots, , \frac{1}{n_i})}_{=Klog(\frac{1}{n_i})}$

Now we only have to "notice" that : $n_i = Np_i$. This leads to the well known formulation of the entropy as:

$$H(\{p_i\}) = Cst + \sum_{i=1}^{n} -Kp_i \log(p_i)$$

End of this proof! K and the Cst can be chosen arbitrarily as far as we only focus on the mathematical form of H. Obviously this "Cst" is a function of N and K implicitly.

The crucial aspect is to start from uniform distributions and notice that the 3 properties define a "log" inevitably. The fact that in practice any distribution is made by "buckets" within which we must assume a uniform distribution, this model is quite efficient.

iv- <u>If mean and variance are known, the max entropy distribution is the normal law</u>

Let's search for the maximum entropy distribution of a given continuous distribution given a "mean" and a "variance" that are fixed.

We can write the problem in Lagrange optimization format…We look to maximize a function 'L' of distributions "p":

$$L(p) = Entropy(p) - \alpha\left[\int pdx - 1\right] - \beta\left[\int (x-\mu)^2 pdx - \sigma^2\right]$$

In this context we aim to maximize L(p) given that the mean "μ" and the variance "σ^2" are fixed. To position this item versus the former one, let's say that we now study the kind of distribution $\{p_i, n_i\}$ that maximizes the measure 'H'. Thus, as H itself is continuous wrt to the vertex $\{p_i\}$ we may have quite erratic jumps from one bucket/event to the next. Let's notice as well that this is only the set of values that are taken by the distribution $\{p_i\}$ that matter for H, not the order.

So we are now going to consider using a continuum to derive H from a vertex of values represented by the distribution $\{p_i\}$. This vertex can be highly erratic with as many "jumps" as required by the experience….provided it has a converging integral with some "smooth" functions….we are in the world of distributions…

From the equation above, we can push all the terms under the integral sign:

$$L(p) = Cst\_p(\alpha, \beta)(\int p(x)\left[-log\big(p(x)\big) - \alpha - \beta(x-\mu)^2\right]dx)$$

Let's now consider a derivation as a differential of the vertex p that has "n" coordinates $\{p_i\}$: (p is chaotic and WE consider a continuous change ie small everywhere assuming this makes sense)

$$\delta L(p) = \vec{0} = \int \delta p(x)\big[-log\big(p(x)\big) - \alpha - \beta(x-\mu)^2\big]dx + \int p(x)\delta[-log(p(x)) - \alpha - \beta(x-\mu)^2]dx$$

Among these 2 terms, we observe that the second integral could be as arbitrarily small in any event compared to the first integral:

$\underbrace{-\int p(x)\left[\frac{[\delta p(x)]}{p(x)}\right]dx =}_{arbitrarily\ small\ anyway} \int -\delta p(x)dx$ is negligible in any event .

More it just adds up to the cst α when we factorize with the first term. The same comment applies to the other term in "x"….

This leads us to assume that we must have:

$$\int \delta p(x)\big[-log\big(p(x)\big) - \alpha - 1 - \beta(x-\mu)^2\big]dx = 0$$

$whatever\ \delta p(x) is\ going\ to\ be\ including\ open\ neighbor\ sets\ or\ setting\ local\ Dirac\ spikes$ ……

This integral should be interpreted as a scalar product between any "direction" proposed by $\delta p(x)$ and the "vector" $\big[-log\big(p(x)\big) - \alpha - 1 - \beta(x-\mu)^2\big]$ that has as many coordinates as we take distinct values for "x" actually.

Thus we just need to adjust the constant term to simply "α' " and we note that only $-log(p(x)) - \alpha' - \beta(x-\mu)^2 = 0$ secures that, at the optimal distribution point and in its neighborhood (assuming such neighborhood can exist) both terms are nil for sure whatever vector local change is operated on p(x) and this leads to the normal distribution.

This is not a complete proof but it shows one "optimal solution" that works as a distribution that is a smooth function. We note that since $p(x) = Ke^{-\alpha-\beta(x-\mu)^2}$ there is only one possible value for these parameters and we see "how" this happens. One key aspect here is that the support of our distribution is not compact and not bounded. The conclusion is quite different when the support is compact or when the support of the distribution has one bound as least.

v- <u>The DKL measure and its properties</u>

Now when we have some empirical distribution we would look to maximize the likelihood of our modeled distribution. For that matter, we assume that what we observed a proxy distribution of some random phenomenon that has explored all its choices at best. In other words our observation reflects an underlying distribution that HAS maximum entropy even though we only see part of it and , in itself, it a moving environment…anyway... Thus this quest amounts to maximize the entropy of our model under the constraint that we ALSO minimize the entropy differential against our empirical distribution. This objective can be expressed as follows:

$$\underbrace{Max}_{\substack{set\ of\ possible\\ distributions\ for\\ p_{mod}(x_{...})}} \left\{ E_{p(x)}[-Log(p_{mod}(x_{...})] - \alpha \underbrace{DKL(p_{mod}||p_{emp\ est})}_{=E_{p(x)}\left[Log\left(\frac{p(x)}{q(x)}\right)\right]} \right\}$$

$$DKL(p||q) = E_{p(x)}\left[Log\left(\frac{p(x)}{q(x)}\right)\right] = \underbrace{\int p(x)[Log(p(x) - Log(q(x))]dx}_{>0\ always\ thx\ to\ neg\ convextity\ of\ Log}$$

Quick proof:

$$DKL(p||q) = \int p(x)Log\left(\frac{p(x)}{q(x)}\right)dx = -\int p(x)Log\left(\frac{q(x)}{p(x)}\right)dx$$

Because Log is negatively convex the positively weighted average of many y values as $Log(y)$, where $\frac{q(x)}{p(x)}$ is a variable "y" for our case, is necessarily below the log of the weighted average of these y values. In simpler words, the concave curve of Log implies that the log of the average is above the average of the logs:

$$\int p(x)Log(y(x))dx \leq Log(\int p(x)y(x)dx)$$

(this is true as long as the weights, namely the p(x) values, are all >0)

Therefore $\int p(x)Log\left(\frac{q(x)}{p(x)}\right)dx \leq Log(\int p(x)\left(\frac{q(x)}{p(x)}\right)dx) = \log(1) = 0$

Thus $DKL(p||q) \geq 0$

Unlike the precedent case we have not restricted $p_{emp\ est}$ to one mean and one variance in particular. We yet have this parameter α to fit at some point in time with the constraint of the distribution that always integrates to 1. The constraint is that the modeled distribution maximizes its entropy of information in its shape and it fits at best with the empirical entropy that can only be an estimate.

We thus endeavor to optimize the following function under constraint using "n" given empirical points or "n" empirical buckets likewise:

$$L(p,n) = Cst + \int \dots \int -p(x)\left[log\big(p(x)\big) + \alpha\big[Log\big(p(x)\big) - Log(p_{emp\ est}(x))\big]\right]dx$$

We can notice than whatever the positive value taken by α, our modeled distribution shall have to mimic the empirical distribution and its entropy proxy value.

The empirical entropy would be first summarily estimated through a counting process run along some well chosen buckets. Now the issue would be that these bucket based measures are like Heavyside or Dirac measures.

This jumpy nature is just fine. The key take-away is that we always work on finite values and thus on a "sum" of uniform distributions actually, not normal distributions…in general….Yet the underlying is a normal distribution for every observation that is just one "sample"….

vi- The native Bayes theory

Representation issues for the error function that depends on X, but if we reduce it to a polynomial of degree2, the 3 coefficients too will vary with X. We can assume that they would vary randomly in a limited range independently. Therefore, we want to minimize the error and the entropy of information as well…

Let's consider a set of inputs having "n" coordinates $x_1, x_2 \dots x_n$ and a classifier random variable y. Each coordinates x_k and y can take any value but y will take a limited number of discrete values..

The starting point is the theorem of Bayes: $P(y|x_1, x_2, \dots . x_n) = \frac{p(y)p(x_1,x_2 \dots x_n|y)}{p(x_1,x_2 \dots x_n)}$

Above we express the probability that a given point identified by its coordinates $x_1, x_2 \dots x_n$ is in the group that itself is identified by $y = k$. We know that y can take say "p" distinct values that each feature one class. The problem is to set each point in one definite class among the "p" ones.

For that we make a naïve assumption: the different coordinates are independent in the many values that they take. In other words: $p\big(x_j\big|y, x_1 \dots . x_m \; diff \; from \; j\big) = p(x_j|y)$

Thus the probability of "y knowing the coordinates of the point In focus" becomes:

$$p(y|x_1, x_2, \dots x_n) = \frac{p(y)}{p(x_1, x_2 \dots x_n)} \prod\nolimits_j p(x_j|y)$$

Since we know from the empirical evidence $p(x_1, x_2 \dots x_n)$we will set an optimal "y" across the points through the distribution of $p(y|x_1, x_2 \dots x_n) ie \;\; Max_y[p(y) \prod_j p(x_j|y)]$

Thus each point will take a value of y in the possible range. As a result for any given value on every coordinate, we will be able to know the probability of any coordinate value depending upon the value of y. As a result every point, through its coordinates, has a probability for every possible value of y. There is one maximum value in probability for every point. So, at the end of the day every point defines one category with a given probability. To maximize the ultimate distribution among the many points we would look to maximize:

$$H\big(p(y_{Max}|x_1, x_2 \dots x_n)\big) = \sum\nolimits_{x_1,x_2 \dots x_n} -p(y_{Max}|x_1, x_2 \dots) Ln(p(y_{Max}|x_1, x_2 \dots))$$

This way we secure that the allocation provides each time a favored value of "y" for each point. The issue remains with the way the algorithm can maximize the entropy: we are then certain that overall the maximum probability is maximized even though some points will remain ambiguous. A certain number of tries are needed in order to find the gradient of the entropy around each point and the Hessian matrix. Clearly, in order to arrive at a fine tuning we should first use a recursive k-means algorithm in order to spot the main aggregates It will also be helpful in safeguarding against the naïve assumption.

The second part addresses the most efficient ways to diagonalize a matrix which helps get back in parallel to the treatment as pictured above….

Multi-dimensional analysis: Gauss Jordan, LU, QR , SVD

Most problems in real life embed many dimensions rather than one. This is particularly true in machine learning and even more so in AI. On the latter case, the number of dimensions is typically quite high which is what makes AI a universe that is devoted to massive data being featured through a high number of dimensions, with a dedicated purpose to perform a sophisticated task. AI problems are however expressed in a limited number of empirical dimensions for the inputs. However the real space of investigation concerns the weights, the biases and other hyper-parameters that are present overall in millions if not billions of dimensions. Mathematically this means billions of possible

scenarios and the robot, like us, may actually be wrong…And hopefully be still more often " right" than "wrong". More the robot has a program that learns from its mistakes.

We have observed the rise of impressive simulators of human intelligence through the many "chatbots" and in particular ChatGPT or BERT among others. If we consider Chatgpt, the input will be a sequence of "only" 1024 words picked in the dictionary of "about" 50 000 words. However to calibrate the system one needs to calibrate more than a trillion of parameters ie more than 1 000 000 000 000 (compared to 1024 and 50 000). The "problem" is to find the "right" point in a space of 1 000 000 000 000 dimensions. This is one reason why ChatGPT3, the most basic public version, requires about 285 000 laptops plus 50 000 GPUs to run "live" and synchronized.

This reality that pervades through all the places in AI and in finance as well mandates the use of matrices. There 2 problems arise: either one wants to invert the matrix or one wants to know the eigen vectors and eigen values in order to reduce the problem to many 1 dimension problems that are simpler to handle and solve

In "N dimensions" we deal with vectors and matrices. We will start to the inversion of a matrix which usually is required to solve a system of "n" unknowns through "n" equations. Bear in mind, from the comments above, that "n" can be as big as 100 000 or even more

Gauss Jordan

We start with the most direct and intuitive resolution of the set of linear equations.

Let's note a system of n equations with n unknowns:

$$\begin{cases} a_{11}x_1 & a_{12}x_2 & a_{13}x_3 & \dots & a_{1n}x_n & = b_1\ (1) \\ a_{21}x_1 & a_{22}x_2 & a_{23}x_3 & \dots & a_{2n}x_n & = b_2\ (2) \\ a_{31}x_1 & a_{32}x_2 & a_{33}x_3 & \dots & a_{3n}x_n & = b_3\ (3) \\ \dots & & \dots & \dots & \dots & \dots \\ a_{n1}x_1 & a_{n2}x_2 & a_{n3}x_3 & \dots & a_{nn}x_n & = b_n\ (n) \end{cases}$$

We will assume all along that the resulting diagonal element is non nil.

We will define a series of coefficients: $u_{0_{11}} = a_{11}, u_{0_{12}} = a_{12}\ and\ in\ general\ u_{0_{ij}} = a_{ij}$

We are going to make operations per lines to produce consecutive coefficients through the series "$u_{k_{ij}}$". How so?

The principle of the algorithm is that any non-nil linear combination of lines preserves the solution: in other words if we replace one line by a linear combination (non nil) of the other lines, the solution of the linear system that we aim to solve and that is right above, does not change. The solution remains the same as long as it exists and is unique.

As a first step, we will divide the first line by a_{11}:

$$\begin{cases} x_1 & \frac{a_{12}}{a_{11}}x_2 & \frac{a_{13}}{a_{11}}x_3 & \dots & \frac{a_{1n}}{a_{11}}x_n & = \frac{b_1}{a_{11}}\ \frac{(1)}{a_{11}} \\ a_{21}x_1 & a_{22}x_2 & a_{23}x_3 & \dots & a_{2n}x_n & = b_2\ (2) \\ a_{31}x_1 & a_{32}x_2 & a_{33}x_3 & \dots & a_{3n}x_n & = b_3\ (3) \\ \dots & & \dots & \dots & \dots & \dots \\ a_{n1}x_1 & a_{n2}x_2 & a_{n3}x_3 & \dots & a_{nn}x_n & = b_n\ (n) \end{cases}$$

The next step consists in replacing the lines below with a special linear combination that would involve the first line once it was modified. The purpose is to render all the coefficients of the lines that are below the first line to 0. For example the line (2) will become "$(2) - \frac{a_{21}}{a_{11}}(1)$". For the line (3), it will become "$(3) - \frac{a_{31}}{a_{11}}(1)$". We therefore obtain the next system:

$$\begin{cases} x_1 & \frac{a_{12}}{a_{11}}x_2 & \frac{a_{13}}{a_{11}}x_3 & \dots & \frac{a_{1n}}{a_{11}}x_n & = & \frac{b_1}{a_{11}} & \frac{(1)}{a_{11}} \\ 0 & \left(a_{22} - \frac{a_{21}a_{12}}{a_{11}}\right)x_2 & \left(a_{23} - \frac{a_{21}a_{13}}{a_{11}}\right)x_3 & \dots & \left(a_{2n} - \frac{a_{21}a_{1n}}{a_{11}}\right)x_n & = b_2 - \frac{a_{21}}{a_{11}}b_1 & (2) - \frac{a_{21}}{a_{11}}(1) \\ 0 & \left(a_{32} - \frac{a_{31}a_{12}}{a_{11}}\right)x_2 & \left(a_{33} - \frac{a_{31}a_{13}}{a_{11}}\right)x_3 & \dots & \left(a_{3n} - \frac{a_{31}a_{1n}}{a_{11}}\right)x_n & = b_3 - \frac{a_{31}}{a_{11}}b_1 & (3) - \frac{a_{31}}{a_{11}}(3) \\ \dots & & & \dots & \dots & \dots & \dots & \dots \\ 0 & \left(a_{n2} - \frac{a_{n1}a_{12}}{a_{11}}\right)x_2 & \left(a_{n3} - \frac{a_{n1}a_{13}}{a_{11}}\right)x_3 & \dots & \left(a_{nn} - \frac{a_{n1}a_{1n}}{a_{11}}\right)x_n & = b_n - \frac{a_{n1}}{a_{11}} & (n) - \frac{a_{n1}}{a_{11}}(1) \end{cases}$$

We then would inherit an 'n-1' x 'n' matrix where the coefficients will be:

$$u_{1_{11}} = 1, u_{1_{1,k}} = \frac{a_{1k}}{a_{11}}, \qquad anf\ for\ j > 1\ u_{1_{jk}} = \begin{cases} 0\ if\ k = 1 \\ \left(a_{jk} - \frac{a_{j1}a_{1k}}{a_{11}}\right) if\ k > 1 = a_{jk} - \frac{a_{j1}a_{1k}}{a_{11}} \end{cases}$$

We can then repeat the same operation considering the transition $a_{11} \to u_{1_{22}}$ over the remaining "n-1" lines and the remaining "n-1" columns

After "n-1" iterations we will have reduced to 0 all the sub diagonal elements. We therefore end up with a linear system of n linear equations but now the last equation solves very easily:

$$\begin{cases} u_{0_{11}}x_1 & u_{0_{12}}x_2 & u_{0_{1,n-2}}x_{n-2} & u_{0_{1n-1}}x_{n-1} & u_{0_{1n}}x_n & = & K_1 \\ 0 & u_{1_{2,2}}x_2 & u_{1_{2,n-2}}x_{n-2} & u_{1_{2,n-1}}x_{n-1} & u_{1_{2,n}}x_n & = & K_2 \\ 0 & 0 & u_{n-3_{n-2,n-2}}x_{n-2} & u_{n-3_{n-2,n-1}}x_{n-1} & u_{n-3_{n-2,n}}x_n & = & K_{n-2} \\ 0 & 0 & 0 & u_{n-2_{n-1,n-1}}x_{n-1} & u_{n-2_{n-1,n}}x_n & = & K_{n-1} \\ 0 & 0 & 0 & 0 & u_{n-1_{nn}}x_n & = & K_n \end{cases}$$

We can observe that each line would retain a certain batch of coefficients $u_{k_{xx}}$. Although this seems a complex formulation in traditional mathematical formulation, it is much easier to implement with a computer because we would just repeat the same operation from "n" lines, storing the first line, then do the same with the remaining "n-1" lines and "n-1" columns based on the new coefficients…save the n-1 th line, move to the "n-2" case and so on and so forth.

Please notice that if want to write the operation above it is just as if we had first multiplied our new matrix to retrieve the original matrix A Indeed let's write the intermediary matrix after the first operation as below: here we only have elements of the type $u_{1_{\dots}}$ below the 1st line

$$A_1 = \begin{bmatrix} u_{0_{11}} & u_{0_{12}} & u_{0_{13}} & \dots & u_{0_{1,n-1}} & u_{0_{1,n}} \\ 0 & u_{1_{22}} & u_{1_{23}} & \dots & u_{1_{2},n-1} & u_{1_{2n}} \\ 0 & u_{1_{32}} & u_{1_{33}} & \dots & u_{1_{3,n-1}} & u_{1_{3n}} \\ 0 & u_{1_{42}} & u_{1_{34}} & \dots & \dots & \dots \\ 0 & u_{1_{n-1,2}} & u_{1_{n-1,3}} & \dots & u_{1_{n-1,n-1}} & u_{1_{n-1,n}} \\ 0 & u_{1_{n2}} & u_{1_{n3}} & \dots & u_{1_{n-1,n-1}} & u_{1_{nn}} \end{bmatrix}$$

Multiplying this matrix by the matrix below, we retrieve the matrix A:

$$L_1 = \begin{bmatrix} 1 & 0 & 0 & & & \\ \frac{u_{0_{21}}}{u_{0_{11}}} & 1 & 0 & 0 & 0 & 0 \\ & & & 0 & 0 & 0 \\ \frac{u_{0_{31}}}{u_{0_1}} & 0 & 1 & 0 & 0 & 0 \\ & & & & & \\ \frac{u_{0_{41}}}{u_{0_{11}}} & 0 & 0 & 1 & 0 & 0 \\ & & & & 1 & 0 \\ & & & & 0 & 1 \end{bmatrix}$$

We need to "visualize" that probably….

$$\begin{bmatrix} 1 & 0 & 0 & & & \\ \frac{u_{0_{21}}}{u_{0_{11}}} & 1 & 0 & 0 & 0 & 0 \\ & & & 0 & 0 & 0 \\ \frac{u_{0_{31}}}{u_{0_1}} & 0 & 1 & 0 & 0 & 0 \\ & & & & & \\ \frac{u_{0_{41}}}{u_{0_{11}}} & 0 & 0 & 1 & 0 & 0 \\ & & & & 1 & 0 \\ & & & & 0 & 1 \end{bmatrix} \begin{bmatrix} u_{0_{11}} & u_{0_{12}} & u_{0_{13}} & \dots & u_{0_{1,n-1}} & u_{0_{1,n}} \\ 0 & u_{1_{22}} & u_{1_{23}} & \dots & u_{1_2,n-1} & u_{1_{2n}} \\ 0 & u_{1_{32}} & u_{1_{33}} & \dots & u_{1_{3,n-1}} & u_{1_{3n}} \\ 0 & u_{1_{42}} & u_{1_{34}} & \dots & \dots & \dots \\ 0 & u_{1_{n-1,2}} & u_{1_{n-1,3}} & \dots & u_{1_{n-1,n-1}} & u_{1_{n-1,n}} \\ 0 & u_{1_{n2}} & u_{1_{n3}} & \dots & u_{1_{n-1,n-1}} & u_{1_{nn}} \end{bmatrix} = A$$

Now let's observe that :

$$\begin{bmatrix} 1 & 0 & 0 & & & \\ \frac{u_{0_{21}}}{u_{0_{11}}} & 1 & 0 & 0 & 0 & 0 \\ & & & 0 & 0 & 0 \\ \frac{u_{0_{31}}}{u_{0_1}} & 0 & 1 & 0 & 0 & 0 \\ & & & & & \\ \frac{u_{0_{41}}}{u_{0_{11}}} & 0 & 0 & 1 & 0 & 0 \\ & & & & 1 & 0 \\ & & & & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & & & \\ 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & \frac{u_{1_{32}}}{u_{1_{22}}} & 1 & 0 & 0 & 0 \\ 0 & \frac{u_{1_{42}}}{u_{1_{22}}} & 0 & 1 & 0 & 0 \\ & & & & 1 & 0 \\ & & & & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & & & \\ \frac{u_{0_{21}}}{u_{0_{11}}} & 1 & 0 & 0 & 0 & 0 \\ & & & 0 & 0 & 0 \\ \frac{u_{0_{31}}}{u_{0_1}} & \frac{u_{1_{32}}}{u_{1_{22}}} & 1 & 0 & 0 & 0 \\ & & & & & \\ \frac{u_{0_{41}}}{u_{0_{11}}} & \frac{u_{1_{42}}}{u_{1_{22}}} & 0 & 1 & 0 & 0 \end{bmatrix}$$

And thus we realize "what" happens as we mute the matrix A one line after the next: we are building on the left side a lower triangular matrix that will be quite useful in the LU decomposition that will be more detailed further down….

We can operate on a line by line basis because the slots that are already at 0 will remain at zero on the subsequent operations. If we end up with an intermediate diagonal element that is 0, we have to rotate the indices of the unknown variables so that we retrieve a non zero diagonal element. Otherwise we would not be able to move since we would have to divide by 0, a thing that is not possible. But, once again, although difficult to picture in traditional maths notations, this rotation of indices is quite easy to achieve in Numpy for example with the function "numpy.roll".

In computing world what matters is that the algorithm requires the minimum number of elementary operations. Time is of the essence. This is not in the scope of this course to explain "how" this complexity calculation is done. The concept has been well defined by A Kolmogorov for those who are interested.

We will apply a proxy that has the merit of being simple and intuitive: every arithmetical operation counts for 1 operation. We therefore will NOT obtain the "exact" complexity but the order of magnitudes will be there. This is enough for the sake of this course.

Now if we estimate the complexity of the whole algorithm excluding the possibility of a nil diagonal element, we find out that with the "n" lines we will have done first n divisions on the first line(ie an inversion and a product ie 2 operations in our proxy estimation), next (n-1) times (n+1) products and (n-1) times (n+1) subtractions This sums up to 2(n-1)(n+1)+2n=2n(n+1+1)-2(n+1)=(4n-2)(n+1). We do that next (4n-6)(n) times with "n-1" lines, next (4n-10)(n-1) with "n-2" lines and so and so forth. As a total: we run approximately (for large values of "n") $4n(n+1)$. operations in first approximation, next 4(n-1)(n) etc…which, when summed up is equivalent to $\frac{4n^3}{6} = \frac{2}{3}n^3$. But this is not over yet…Then we obtain the upper triangular matrix that we need to invert line by line starting from the bottom: at line "n" we make a division. At line "n-1" we compute one product, run a subtraction and finally add a division. At line "n-2" we run 2 products, one subtraction and 1 division. So the total is $1 + (2+1) + (2+2) + (2+3) \ldots = 1 + 2(n-1) + \frac{n(n-1)}{2}$ so overall the complexity is in order of magnitude of $2n^2$. When "n" is high, this part becomes negligible versus the first part that grows in n^3.

Now if we consider inverting a matrix it is equivalent to solving "n" such equations ie the order of complexity is n^3. Let's observe that, from a pure computing point of view the operations are always identical on the left side involving the factors a_{ij} while we could apply the same calculations to any number fo vectors of the type $B_m = \begin{pmatrix} b_{m1} \\ b_{m2} \\ b_{m3} \\ \ldots \\ b_{mn} \end{pmatrix}$. All that can be vectorized and run in parallel. Each vector of the type $X_m = \begin{pmatrix} x_{m1} \\ x_{m2} \\ x_{m3} \\ \ldots \\ x_{mn} \end{pmatrix}$ is then found as the "m-th column" of the inverse matrix $A(a_{ij})$.

What is expensive in computation terms is the part where we generate the "0"s line after line. The actual resolution of the ultimate upper triangular matrix is easy in comparison.

LU decomposition

The LU decomposition (L for "lower", and U for "upper") is somewhat more complex but it turns out to be really useful when only the vector "b" varies while the "matrix A" remains constant , namely the factors a_{ij}. Here we go beyond the sole inversion of matrices. We start thinking in machine learning terms because we stop thinking in asking the machine to perform a fixed set of calculations and, instead, we alter the algorithm so that the machine autonomously repeats the calculation on varying conditions that we cannot anticipate in full. And yet we may as well think : " well this is just a more generic and abstract algorithm", which is exactly right.

The same matrix A now is going to be factorized into 2 matrices: one "lower triangular matrix" and one "upper triangular matrix" hence : $A = LU$.

At the start we try to solve the same equation $Ax - b = 0$

At the end of the Gaussian elimination process we end up with:

$$\begin{cases} u_{0_{11}}x_1 + u_{0_{12}}x_2 + u_{0_{13}}x_3 + \cdots + u_{0_{1n}}x_n = b_{0_1} = d_1 \\ 0 \quad + u_{1_{22}}x_2 + u_{1_{23}}x_3 + \cdots + u_{1_{2n}}x_n = b_{1_2} = d_2 \\ 0 \quad + \quad 0 \quad + u_{2_{33}}x_3 + \cdots + u_{2_{3n}}x_n = b_{2_3} = d_3 \\ \qquad\qquad\qquad\qquad \ldots \\ 0 \quad + \quad \ldots \qquad\qquad + u_{n-1_{nn}}x_n = b_{n-1_n} = d_n \end{cases}$$

Which we can express in matrix format as : $Ux - d = 0$

Multiplying now with an "L" matrix such that $L(Ux - d) = 0$ we actually would set:

$Ld = b \; and \; A = LU.$

We will develop on the remark that closed the part on the Gauss-Jordan elimination process. We observe here that the U matrix is inferred from the Gaussian decomposition above and the L matrix is the inverse matrix of L that morphs the original vector $(b_k) into \; (d_k) = (b_{k-1_k})$

We should notice that we do not need the inverse matrix of L at all because, being triangular we only need to shape a recursive resolution, knowing "b", to infer "d". Really the whole point here is to just store the factors impacting "b" into a separate matrix. The great advantage as we saw before is that this inversion is only "n²" in complexity terms. It does not matter when n<10 but it become quite important when n>1000. This implies indeed that one approach is 1000 times faster than the other!

The big point here is that if we know L and U. Then when "b" changes we would compute $L^{-1}b = d$ and next only solve $Ux = d$ that is rather fast, because, again, we do not need to invert L at all thanks to the fact that it is triangular. In practice, if L is some lower triangular matrix, we DO NOT have to compute the inverse of L, namely L^{-1}.We just have to "solve" a triangular matrix once again which is a complexity of n² not n^3. It is only sufficient to apply the matrix that comes along the transformation of b into d that is known from the Gaussian decomposition.

As an ultimate result, it turns out the matrix L is easy to infer from the matrix U.

If U is : $U = \begin{bmatrix} u_{0_{11}} & u_{0_{12}} & u_{0_{13}} & \dots & u_{0_{1,n-1}} & u_{0_{1,n}} \\ 0 & u_{1_{22}} & u_{1_{23}} & \dots & u_{1_{2},n-1} & u_{1_{2n}} \\ 0 & 0 & u_{2_{33}} & \dots & u_{2_{3,n-1}} & u_{2_{3n}} \\ 0 & 0 & 0 & \dots & \dots & \dots \\ 0 & 0 & 0 & 0 & u_{n-2_{n-1,n-1}} & u_{n-2_{n}-1,n} \\ 0 & 0 & 0 & 0 & 0 & u_{n-1_{nn}} \end{bmatrix}$

Then L is : $L = \begin{bmatrix} 1 & 0 & 0 & & & \\ \frac{u_{0_{21}}}{u_{0_{11}}} & 1 & 0 & 0 & 0 & 0 \\ \frac{u_{0_{31}}}{u_{0_1}} & \frac{u_{1_{32}}}{u_{1_{22}}} & 1 & 0 & 0 & 0 \\ \frac{u_{0_{41}}}{u_{0_{11}}} & \frac{u_{1_{42}}}{u_{1_{22}}} & \frac{u_{2_{43}}}{u_{2_{33}}} & 1 & 0 & 0 \\ & & & & & \end{bmatrix}$

For that matter, you just need to review "how" the Gauss Jordan elimination is performed: see how the original factors of the matrix A are retrieved from the elements of U….To make it simpler you can do it on the vector "b" since this is just one column.

QR decomposition

This decomposition follows a different purpose: rather than look for the inverse matrix, we would look for the eigen values and eigen vectors. The underlying motivation is larger than the sole inversion and goes in the direction of many more applications, once we have expressed the matrix in a different basis of representation. This is ubiquitously useful in finance, machine learning and AI. Among other things we will then be able to compute inverse values quite easily but not only that: we

can then reduce an n-dimensional problem where coordinates are correlated, ie in a space that is n² actually at least, to a problem of "n" independent linear problems of the same kind.

Here the target is a factorization of a matrix A as A=QR where Q is an orthogonal matrix ie $Q^T Q = I_d$ and R is an upper triangular matrix on which we will find the eigen values on the diagonal and next infer fast the eigen vectors.

This means that if we notice that the matrix Q has "vectors" to populate its columns, they are pair-wise orthogonal and of norm 1 each. To be clear here on the vocabulary: if we consider a vector X of n coordinates, like $X(x_1, x_2 \dots x_n)$, the vector X has norm 1 when $\|X\|^2 = \sum_{k=1}^{k=n} x_k^2 = 1$. Another way to state it is to say that Q is a transformation of a canonical basis into another canonical basis (orthogonal and unit 1): in less technical terms Q take the original basis that is made of vectors of norm 1 that are pair-wise orthogonal and generates a new basis that has also vectors of norm 1 that are pair-wise orthogonal too.

We apply the so-called Gramm-Schmidt process here:

We write the matrix A from its n columns that each features one vector a_k where:

$$A = [a_1, a_2, a_3 \dots a_n] = \left| \begin{bmatrix} a_{11} \\ a_{21} \\ a_{31} \\ \dots \\ a_{n1} \end{bmatrix}, \begin{bmatrix} a_{12} \\ a_{22} \\ a_{32} \\ \dots \\ a_{n2} \end{bmatrix}, \begin{bmatrix} a_{13} \\ a_{23} \\ a_{33} \\ \dots \\ a_{n3} \end{bmatrix} \dots \begin{bmatrix} a_{1n} \\ a_{2n} \\ a_{3n} \\ \dots \\ a_{nn} \end{bmatrix} \right|$$

In general if we have 2 vectors X and Y like $X(x_1, x_2, \dots x_n)$ and $Y(y_1, y_2, .., y_n)$ than the scalar product is $X.Y = \sum_{k=1}^{k=n} x_k y_k \; and \; \|X\|^2 = \sum_{k=1}^{k=n} x_k^2 \; and \; \|Y\|^2 = \sum_{k=1}^{k=n} y_k^2$

We will define on the follow a new basis with the vectors e_k:

$$u_1 = a_1 \; and \; e_1 = \frac{u_1}{\|u_1\|}$$

Next $u_2 = a_2 - a_2.e_1 \; and \; e_2 = \frac{u_2}{\|u_2\|} \; where \;\; we \; can \; check \; that \; e_1.e_2 = 0$ Here again to be sure:

$a_2.e_1$ is the scalar product of the vector a_2 with the vector e_1

On the follow, by the same process, we build the other vectors of the orthonormal basis:

$$u_k = a_k - a_k.(e_1 + \; e_2 + e_3 \dots + e_{k-1}) \; and \; e_k = \frac{u_k}{\|u_k\|} \; while \; still \; e_k.e_j = \delta_{jk}$$

In this new basis the matrix A becomes $\left| R = \begin{bmatrix} a_{11} \\ 0 \\ 0 \\ \dots \\ 0 \end{bmatrix}, \begin{bmatrix} a_2.e_1 \\ a_2.e_2 \\ 0 \\ \dots \\ 0 \end{bmatrix}, \begin{bmatrix} a_3.e_1 \\ a_3.e_2 \\ a_3.e_3 \\ \dots \\ 0 \end{bmatrix} \dots \begin{bmatrix} a_n.e_1 \\ a_n.e_2 \\ a_n.e_3 \\ \dots \\ a_n.e_n \end{bmatrix} \right|$

The transformation from the canonical basis (b_k) into the new basis (e_k) follows the matrix:

$$Q = \begin{bmatrix} e_1.b_1 & e_2.b_1 & e_3.b_1 & e_4.b_1 & \dots & e_n.b_1 \\ e_1.b_2 & e_2.b_2 & e_3.b_2 & e_4.b_2 & \dots & e_n.b_2 \\ e_1.b_3 & e_2.b_3 & e_3.b_3 & e_4.b_3 & \dots & e_n.b_3 \\ e_1.b_4 & e_2.b_4 & e_3.b_4 & e_4.b_4 & \dots & e_n.b_4 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ e_1.b_n & e_2.b_n & e_3.b_n & e_4.b_n & .. & e_n.b_n \end{bmatrix}$$

We can see above that the image of the vector that has 0 everywhere but on the k-th coordinate that is "1" then will give an image, after being multiplied by the matrix above on its left, that is e_k in the canonical basis (b_j).

We know, since this is a change of orthogonal basis of norm 1; that the lines of the matrix above give the coordinates of the canonical basis (b_j) into the new basis (e_j). You can check that in the matrix Q above by reading a line instead of a column: you see that the scalar products on one line indeed always refer to the same original vector b_k that is projected on the different vectors of the new basis that we just generated (e_j)

So if we consider a vector that is expressed in the old basis (b_k) and that is multiplied by R, then we see that it will be expressed in the new basis (e_k). We would therefore need to move the output back to the canonical basis (b_k) to get the proper operation …or we should just express first any vector of the original canonical basis into the new basis that we just created: and that would amount to multiply on the left the matrix R by the matrix Q.

Therefore we infer that $= QR$.

The complexity of this algorithm is about : $n^{3\,\prime}$

First vector: 1+n (squares)+n(sums) +square root+division+storing the vector (n)=3n+3

Second vector= 3n+3+ n products+n sums=5n+3

3rd vector=3+ 5n+2n for the additional scalar product

So, because we actually could do the scalar product by simply adding first the new basis vector, we should consider that the total number of operations is :

$$N_{umber} = 3n + (3 + 4 + 5 + n + 2)n = n(1 + 2 + 3 + 4 + 5 \dots + n + n + 1 + n + 2)$$
$$= n\frac{n(n+1)}{2} + n(2n+3) = \frac{1}{2}n^3 + \cdots$$

The inference of eigen values and eigen vectors

Now we are going to review the "QR method" that is directly inspired from the QR decomposition.

Up to now we have simply defined Q and R. But now we want to access in the fastest possible way the eigen value. If we could find a matrix P and an upper triangular matrix such that P is the matrix that makes us move from the canonical basis to some well chosen orthogonal unitary basis, then we can find the eigen values and the norm 1 eigen vectors altogether from R. We need to find "P":

$$A = P^T RP \; with \; P^T P = I_d$$

This is not quite the QR decomposition that we saw right before. Let's now make a few more operations to bridge the gap between the QR decomposition and what we want to achieve….

$$A = QR \; hence \; Q^T A = R \; and \; so \; Q^T AQ = RQ$$

Since Q is already a matrix that operates a change of an orthogonal basis into another, RQ has therefore the same eigen values as A. Beware: QR and RQ are different in general because the product of matrices is not commutative in general. However QR and RQ share the same eigen values….but not always the same eigen vectors that are associated with the eigen values in question.

To be sure, as much as we know the R matrix as an upper triangular matrix , the matrix RQ is not necessarily a triangular matrix too.

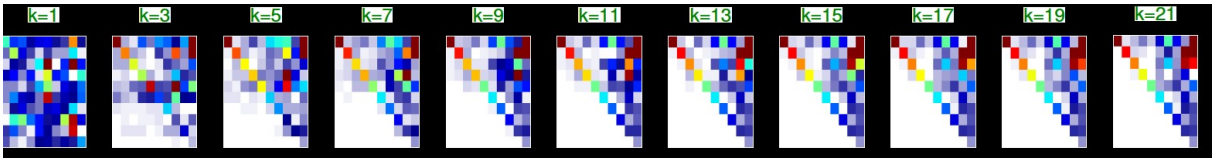
Since we know R and Q from the QR decomposition above, we will generate a matrix product with:

$$A_0 = A \rightarrow A_0 = Q_1R_1 \rightarrow A_1 = R_1Q_1 = Q_2R_2 \rightarrow A_3 = R_2Q_2 = Q_3R_3$$

Ultimately the matrix $A_{n+1} = R_nQ_n$ will be purely upper triangular matrix.

How comes?!!!

Below an illustration of the convergence process: the “white” points to “0”:



Proof of the convergence:

The point to make is that the lower triangle elements become all closer and closer to 0 through the iterations. Let’s have a look at one such element that lies below the diagonal:

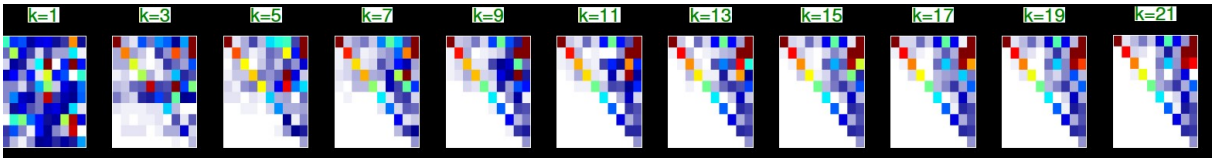
$$\overbrace{\begin{bmatrix} v_1 & & & & & \\ 0 & v_2 & & & & \\ 0 & 0 & v_3 & & & \\ 0 & 0 & 0 & v_4 & & \\ \dots & \dots & \dots & \dots & \dots & \dots \\ 0 & 0 & 0 & 0 & 0 & v_n \end{bmatrix}}^{R_k} \overbrace{\begin{bmatrix} q_{k_{11}} & q_{k_{12}} & q_{k_{13}} & q_{k_{14}} & \dots & q_{k_{1n}} \\ q_{k_{21}} & q_{k_{22}} & q_{k_{23}} & q_{k_{24}} & \dots & q_{k_{2n}} \\ q_{k_{31}} & q_{k_{32}} & q_{k_{33}} & q_{k_{34}} & \dots & q_{k_{3n}} \\ q_{k_{41}} & q_{k_{42}} & q_{k_{43}} & q_{k_{44}} & \dots & q_{k_{4n}} \\ \dots & \dots & \dots & \dots & \dots & \dots \\ q_{k_{n1}} & q_{k_{n2}} & q_{k_{n3}} & q_{k_{n4}} & \dots & q_{k_{nn}} \end{bmatrix}}^{Q_k} = \overbrace{\begin{bmatrix} v_1 & & & & & \\ & v_2 & & & & \\ & & v_3 & & & \\ & & & v_4 & Y_{ls} & \\ & X_{mp} & & & \dots & \\ & & & & & v_n \end{bmatrix}}^{A_{k+1}}$$

In order to grasp “why” the terms under the diagonal tend towards zero we have to see that the first column of the next matrix A_{k+1} is the image of the first basis vector that is in the first column of the matrix Q_k that is a matrix moving from one orthonormal basis to another, multiplied by the lines of the matrix R_k. Because the norm of the lines and the norm of the columns are all equal to 1in Q_k this means that all the terms are a fraction of 1 in absolute value. And you can see that because R_k has more and more zeros on the left of the diagonal as we move down along the lines of R_k, we can see that the lower we look on the lines for the vector image in the first column of A_{k+1} the less and less components of the first column of Q_k are used, hence a lower and lower value in module. The lower the line is, the less non zero terms are going to be used in the calculation of the new value in the next matrix. therefore the new coordinate will depend more and more on the ultimate terms of the vector that are taken into account.

Quite importantly, BOTH the non diagonal lower terms and the non diagonal upper terms are impacted which means that the basis tend to converge to themselves through the iterations since only the diagonal terms can remain non zero: they therefore can only be equal to 1 on the diagonal and zero elsewhere. Through the QR decomposition that follows, the normalization keeps lowering the non diagonal terms relative to the diagonal one. Now the matrix A in general does not have to to either orthogonal or of norm 1. The theorem of Jordan on the optimal decomposition of a matrix

shows that when the eigen values are multiple then the eigen vectors associated with the same eigen value may project on one each other. Thus the matrix A, in a basis of eigen vectors is not necessarily diagonal. More, and more frequently, the eigen vectors are NOT pair-wise orthogonal. However, the construction of a basis of eigen vectors will secure that the new eigen vector at rank "k+1" will only project onto itself and "k" former eigen vectors. This is why the subsequent product RQ is NOT going to be other than upper triangular.

We can observe the chart again and notice that the 0 is attained first on the lowest lines:



Now the effective convergence occurs through the "next QR" decomposition where, to recall, it generates a new basis based upon a new matrix A_{k+1} that has its first column tending to be like the first vector in the current basis. Likewise, when we define the second basis vector, we have the same effect …below the diagonal mostly, and so on and so forth: only the term on the upper triangle may not be nil. This is how we find the upper triangular matrix in the next QR decomposition. Yet, although the same effect might be felt on the upper side, above the diagonal, we have to have non nil projections at times precisely because A is not an orthonormal basis change like "Q" always is.

Remember the way we build the new basis: we start from a matrix A that is any matrix

$$A = [a_1, a_2, a_3 \dots a_n] = \left[\begin{bmatrix} a_{11} \\ a_{21} \\ a_{31} \\ \dots \\ a_{n1} \end{bmatrix}, \begin{bmatrix} a_{12} \\ a_{22} \\ a_{32} \\ \dots \\ a_{n2} \end{bmatrix}, \begin{bmatrix} a_{13} \\ a_{23} \\ a_{33} \\ \dots \\ a_{n3} \end{bmatrix} \dots \begin{bmatrix} a_{1n} \\ a_{2n} \\ a_{3n} \\ \dots \\ a_{nn} \end{bmatrix}\right]$$

We will define on the follow a new basis with the vectors e_k:

$$u_1 = a_1 \; and \; e_1 = \frac{u_1}{\|u_1\|}$$

If we recall the iteration above , if $A = A_{k+1}$,this vector $a_1$will have been generated from the eigen value and other terms that will have been minimized by the presence of 0s present in the former matrix R_k anyway. Thus, the normalization is always going to give more weight to the only area in the matrix that is NOT impacted by the zeros, namely the diagonal term. So the basis vector $e_{1_{k+1}}$ tends to be closer and closer to the former counterpart, namely e_{1_k} post the QR decomposition.

Next $u_2 = a_2 - a_2.e_1 \; and \; e_2 = \frac{u_2}{\|u_2\|} \; where \;\; we \; can \; check \; that \; e_1.e_2 = 0$

It is just as if we now did the very same thing but in the sub-space that covers the remaining "n-1" dimensions. Therefore apart from the very first coordinate that may well never be nil, all the coordinates below the diagonal follow the same evolution as pictured before, but in the sub-space of the dimension "n-1" that is orthogonal to $e_{1_{k+1}}$.

On the follow, by the same process, we build the other vectors of the orthonormal basis:

$$u_k = a_k - a_k.(e_1 + \; e_2 + e_3 \dots + e_{k-1}) \; and \; e_k = \frac{u_k}{\|u_k\|} \; while \; still \; e_k.e_j = \delta_{jk}$$

In this new basis the matrix A becomes $\left| R = \begin{bmatrix} a_{11} \\ 0 \\ 0 \\ \dots \\ 0 \end{bmatrix}, \begin{bmatrix} a_2.e_1 \\ a_2.e_2 \\ 0 \\ \dots \\ 0 \end{bmatrix}, \begin{bmatrix} a_3.e_1 \\ a_3.e_2 \\ a_3.e_3 \\ \dots \\ 0 \end{bmatrix} \dots \begin{bmatrix} a_n.e_1 \\ a_n.e_2 \\ a_n.e_3 \\ \dots \\ a_n.e_n \end{bmatrix} \right|$

The transformation from the canonical basis (b_k) into the new basis (e_k) follows the matrix:

$$Q = \begin{bmatrix} e_1.b_1 & e_2.b_1 & e_3.b_1 & e_4.b_1 & \dots & e_n.b_1 \\ e_1.b_2 & e_2.b_2 & e_3.b_2 & e_4.b_2 & \dots & e_n.b_2 \\ e_1.b_3 & e_2.b_3 & e_3.b_3 & e_4.b_3 & \dots & e_n.b_3 \\ e_1.b_4 & e_2.b_4 & e_3.b_4 & e_4.b_4 & \dots & e_n.b_4 \\ \dots & \dots & \dots & \dots & \dots & \dots \\ e_1.b_n & e_2.b_n & e_3.b_n & e_4.b_n & .. & e_n.b_n \end{bmatrix}$$

So, a number of iterations will bring up the following result:

$$A_N = R_N = Q_N R_N \; hence \; A_N = R_N = Q_N^T R_N Q_N = (Q_N Q_{N-1} .. Q_3 Q_2 Q_1)^T R (Q_N Q_{N-1} .. Q_3 Q_2 Q_1)$$

As a result we get also: $A = (Q_N Q_{N-1} .. Q_3 Q_2 Q_1) A_N (Q_N Q_{N-1} .. Q_3 Q_2 Q_1)^T$

This convergence is achieved and we inherit a final orthonormal basis change through the matrix:

$$(Q_N Q_{N-1} .. Q_3 Q_2 Q_1)$$

Then the eigen values can only be equal to the diagonal elements and we then know the eigen values of A. This means that, unless the matrix is not diagonal we can infer the eigen vectors too. In general we would need to repeat this operation "n" times at least and therefore the complexity is n^4

Bear in mind here the "cost" of a matrix product: it is 2n for each element and we have ,n² elements, therefore the complexity of one matrix product if $2n^3$ and we will have at least 2n such matrix products , hence a total complexity of $4n^4$.

Now that we have this decomposition rather fast. Let's recall that $Q^T Q = I_d$

Therefore $Q^T R Q = A$

If w recall that R is an upper triangular matrix we have $R = \begin{bmatrix} v_{p_1} & r_{12} & r_{13} \\ 0 & v_{p_2} & r_{23} \\ 0 & 0 & v_{p_3} \end{bmatrix}$

The matrix is deliberately taken on 3 dimensions so that the proof is simple.

We have this matrix R in a basis that we will call $B_0 = (b_{0_1}, b_{0_2}, b_{0_3})$ and we notice that

$$Bb_{0_1} = \begin{bmatrix} v_{p_1} & r_{12} & r_{13} \\ 0 & v_{p_2} & r_{23} \\ 0 & 0 & v_{p_3} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} = v_{p_1} \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

We have the first eigein vector. Now if we look at

$$Bb_{0_2} = \begin{bmatrix} v_{p_1} & r_{12} & r_{13} \\ 0 & v_{p_2} & r_{23} \\ 0 & 0 & v_{p_3} \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} r_{12} \\ v_{p_2} \\ 0 \end{bmatrix}$$

We infer that we could combine the 2 results to generate rather fast the second eigen vector:

$$B(b_{0_2} - ab_{0_1}) = \begin{bmatrix} v_{p_1} & r_{12} & r_{13} \\ 0 & v_{p_2} & r_{23} \\ 0 & 0 & v_{p_3} \end{bmatrix} \begin{bmatrix} -a \\ 1 \\ 0 \end{bmatrix} = \begin{bmatrix} r_{12} - av_{p_1} \\ v_{p_2} \\ 0 \end{bmatrix} = \begin{bmatrix} -av_{p_2} \\ v_{p_2} \\ 0 \end{bmatrix}$$

Then we set "a" such that : $r_{12} - av_{p_1} = -av_{p_2}\ ie\ a = \frac{r_{12}}{v_{p_1} - v_{p_2}}$

We finally get the 3rd eigen vector by the same process:

$$B(b_{0_3} + a_1 b_{0_1} + a_2 b_{0_2}) = \begin{bmatrix} v_{p_1} & r_{12} & r_{13} \\ 0 & v_{p_2} & r_{23} \\ 0 & 0 & v_{p_3} \end{bmatrix} \begin{bmatrix} a_1 \\ a_2 \\ 1 \end{bmatrix} = \begin{bmatrix} a_1 v_{p_1} + a_2 r_{12} + r_{13} \\ a_1 v_{p_2} + r_{23} \\ v_{p_3} \end{bmatrix} \begin{bmatrix} a_1 v_{p_3} \\ a_2 v_{p_3} \\ v_{p_3} \end{bmatrix}$$

We then need to solve for the pair $a_1 and\ a_2$

So we have to invert a matrix that is rather straightforward with the complexity of k² for the k-th eigen vector in a row. Adding up this to the n eigen vectors we reach a complexity of n^3 to a former complexity of $4n^4$, ie this part is not the main issue.

With this approach above we have found both the eigen values and the eigen vectors of "R" that is the matrix A but expressed in a different basis. Since we have $Q^T RQ = A$ we easily infer that if a vector v_{R_k} is an eigen vector associated with the eigen value V_k then $Q^T v_{R_k} = v_{A_k}$ is the corresponding eigen vector for A. Indeed $Av_{A_k} = Q^T RQQ^T v_{R_k} = Q^T Rv_{R_k} = V_k Q^T v_{R_k} = V_k v_{A_k}\ QED$!

There is a much more efficient way to obtain the upper triangular matrix through the use of Hessenberg matrices and Householder reflectors. But this is beyond the scope of the course. The current process has a complexity of n^4 at least. The Hessenberg+Householder method has a complexity of n^3. The idea is to use the Householder reflectors to generate an Hessenberg matrix that has only the sub-diagonal line non-zero, the lower part being 0 already. Then the QR method goes much faster on the follow.

Householder reflector:

$$let\ u\ a\ norm\ 1\ vector\ and\ define\ P = I_d - 2u.u^T,\ then\ P^T = P = P^{-1}$$

Important lemma: let x be a non zero vector,

define $a = \pm\|x\|\ and\ u = \frac{x - ae_1}{\|x - ae_1\|}\ a, d\ P = I_d - 2u.u^T$, then $Px = ae_1$

The Hessenberg matrix is constructed in n-1 steps: the first "x" vector is made with the elements of the first column of A starting from the line 2 downward**.**

Reminder: in finance we mostly deal with symmetrical matrices and the eigen value and eigen vector decomposition is crucial if we want to reduce a multidimensional correlated problem down to n independent problems. However the "correlation" may not remain stable. Likewise, when we deal for example with structured product and stochastic equations (Black & Svholes model) we may also come across ill defined derivatives, ie non symmetrical Hessian matrices for second order partial derivatives.

Remark

For smaller matrices there is a direct algorithm that you will find in the appendix and is much simpler to implement. However it works only on symmetrical matrices. But we will see below with the SVD

algorithm (appendix too: not mandatory) that we can always put ourselves into this configuration. We will by the way see how the Householder reflectors and the Hessenberg matrices play an important role into reducing the complexity of the operation. This is in the appendix, ie not mandatory for this course

APPENDICES

vii- The Householder reflector and how to reduce a matrix

All is based on the so-called "Householder reflectors": $P = I_d - 2u\,u^T \; with \; \|u\| = 1$. Here we see a vector as a matrix of one column and the transpose is a matrix of one line. Thus $u\,u^T$ is a symmetrical n x n matrix.

The Householder reflectors have quite useful properties:

- P is symmetrical: $P^T = I_d - 2(u\,u^T)^T = I_d - 2u^{T^T}\,u^T \; and \; u^{T^T} = u$
- $P^2 = I_d$.

Indeed, $P(PX) = (I_d - 2u\,u^T)\,(X - 2u\,u^T X) = X - 2u\,u^T X - 2u\,u^T(X - 2u\,u^T X)$

Hence: $P(PX) = X - 4u\,u^T X - 4u\,(u^T u)u^T X \; but \; u^T u = \left||u|\right|^2 = 1, so \; u\,(u^T u)u^T = u\,u^T \; .QED$

Beware though….above cannot say in general that AB=BA when we think of matrices instead of scalars. But here we only need to perform the sum products that come along with the product of matrices in any "order" ie $4u\,(u^T u)u^T = 4(u\,u^T uu^T) = 4(u\,u^T)(uu^T)$=$4(u\,u^T u)(u^T)$ etc…

- If we know "u", PX can be performed with a complexity "n" instead of "n²"
  Which is the key here in our endeavor to reduce the complexity of calculation
  Indeed $PX = X - 2u(u^T X) \; where \; u^T X = \sum_{k=1}^{k=n} u_k x_k \; requires$ n products + n $sums$
  Next $-2u\,(u^T X) requires$ n$products \; and \; the \; final \; result \; requires \; another \; 2 \; sums$
  At the end of the day, instead of "n+n²" operations we only need to run "4n+1"

Lemma: consider a non zero vector "x" and coefficient $= \pm 1 \; and \; a = \rho\|x\|$, then we define $u = \frac{x - ae_1}{\|x - ae_1\|} = \frac{z}{\|z\|} \; and \; we \; we \; can \; say \; that \; P = I_d - 2u\,u^T \; is \; a \; householder \; operator$

$$and \; Px = ae_1$$

proof the vector u has norm 1! And $Px = x - 2\frac{z}{\|z\|}\left(\left(\frac{z}{\|z\|}\right)^T x\right) = x - 2\left(\left(\frac{z}{\|z\|}\right)^T x\right)\frac{z}{\|z\|}$

if we focus on $\left(\frac{z}{\|z\|}\right)^T x$ that is a scalar, hence the permutation of the terms above we get:

$$\left(\frac{z}{\|z\|}\right)^T x = \frac{1}{\|z\|}(x - ae_1)^T x = \frac{1}{\|z\|}(\|x\|^2 - ax_1)$$

Now $\|z\|^2 = (x - ae_1)^T(x - ae_1) = \|x\|^2 - 2ax_1 + a^2 = 2(\|x\|^2 - ax_1)$

So, we can combine the results here and conclude that :

$$\left(\frac{z}{\|z\|}\right)^T x = \frac{1}{\|z\|}(\|x\|^2 - ax_1) = \frac{1}{\|z\|}\frac{1}{2}\|z\|^2 = \frac{1}{2}||z||$$

Therefore: $Px = x - 2\frac{z}{\|z\|}\left(\left(\frac{z}{\|z\|}\right)^T x\right) = x - 2\left(\left(\frac{z}{\|z\|}\right)^T x\right)\frac{z}{\|z\|} = x - 2\frac{1}{2}\|z\|\frac{z}{\|z\|} = x - z = ae_1\ QED$

In order to be more efficient on the coming error rounding, we will choose $\rho = -Sign(x_1)$

Note that the basis vector is any unit basis vector.

Now we are going to use this lemma because this Householder reflector basically "projects" the whole vector "x" onto the first basis vector e_1 while remaining a bijection transformation of the space given the properties of "P" that is equivalent to a convenient change of orthogonal unit normed basis.The Householder projectors are going to be used now in conjunction with the column vectors of the matrix A. Let's focus on the very first column of the matrix A that therefore gather the factors $[a_{11}, a_{21}, a_{31}, \dots a_{,n1}] where\ A = [a_{ij}]$.

We are in a space of n dimensions and we have a canonical basis that we call $(e_1, e_2, e_3 \dots e_n)$. Let's confine ourselves to the sub space S_2defined by the basis $(e_2, e_3, \dots e_n)$ which is the space that is orthogonal to the first basis vector noted e_1 in the present conventions.

We know that if we use the basis vector e_2 within $S_2$to build an Householder projector within S_2 we would need first a vector "u" having "n-1" coordinates. Let's take the "n-1" last coordinates of the first column of A: $u_2 = (a_{21}, a_{21}, a_{31} \dots a_{n1})$ and build $P_2 = I_d - 2u_2u_2^T$.

We know that if we complement this reflector with the identity along the first eigen vector e_1, we have built a operator that realizes an orthogonal unit basis change. Indeed the matrix this operator P_{n_2} is $P_{n_2} = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & & & \\ \dots & & I_{d_{n-1}} - 2u_2u_2^T & \\ 0 & & & \end{bmatrix}$.

Then if we consider the product of the matrices

$$\text{“}P_{n_2}A = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & & & \\ \dots & & I_{d_{n-1}} - 2u_2u_2^T & \\ 0 & & & \end{bmatrix}\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} & .. & a_{1n} \\ a_{21} & a_{22} & a_{23} & a_{24} & .. & a_{2n} \\ a_{31} & a_{32} & a_{33} & a_{34} & .. & a_{3n} \\ a_{41} & a_{42} & a_{43} & a_{44} & \dots & a_{4n} \\ \dots & \dots & \dots & \dots & \dots & .. \\ a_{n1} & a_{n2} & a_{n3} & a_{n4} & \dots & a_{nn} \end{bmatrix} = \begin{bmatrix} a_{11} & x & x & x & .. & x \\ \rho\|u_2\| & x & x & x & .. & x \\ 0 & x & x & x & .. & x \\ 0 & x & x & x & \dots & x \\ \dots & x & x & x & \dots & x \\ 0 & x & x & x & \dots & x \end{bmatrix}$$

The notation "x" indicates that there are values that in general are NOT 0. However the first column would have zeros for sure on all the lines but for the 2 first ones. Why is that?

If we focus on the first column of the resulting matrix we see that for the first term, on the first line this is naturally a_{11}. For the second line we can develop the calculus: $0a_{11} + p_{2_{11}}a_{21} + p_{2_{12}}a_{32} + \cdots$

which is the projection of the vector $P_{n_2}u_2$ onto e_2 on the second line, and next the projection of the vector $P_{n_2}u_2$ on the vector e_3 on the third line…and so on…the k-th line on the first column will be the projection of $P_{n_2}u_2$ on the vector e_k. But we proved in the lemma before than then $P_{n_2}u_2 = \rho\|u_2\|e_2$. Hence the certainty that the elements below the 2nd one can only be 0.

Now, to ascertain that conclusion, we need to explain "how" indeed the product $P_{n_2}A$ has its column ending up providing the "image" by P_{n_2} of the first column of A.

Let's consider the simple product by A of the first basis vector e_1:

$$Ae_1 = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} & .. & a_{1n} \\ a_{21} & a_{22} & a_{23} & a_{24} & .. & a_{2n} \\ a_{31} & a_{32} & a_{33} & a_{34} & .. & a_{3n} \\ a_{41} & a_{42} & a_{43} & a_{44} & ... & a_{4n} \\ ... & ... & ... & ... & ... & .. \\ a_{n1} & a_{n2} & a_{n3} & a_{n4} & ... & a_{nn} \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ ... \\ 0 \end{bmatrix} = \begin{bmatrix} a_{11} \\ a_{21} \\ a_{31} \\ ... \\ a_{n1} \end{bmatrix}$$

Therefore if now we consider the first column of A alone we realize the the image of this first column is :

$$\begin{bmatrix} 1 & 0 & ... & 0 \\ 0 & & & \\ ... & & I_{d_{n-1}} - 2u_2u_2^T & \\ 0 & & & \end{bmatrix} \begin{bmatrix} a_{11} \\ a_{21} \\ a_{31} \\ ... \\ a_{n1} \end{bmatrix} = \begin{bmatrix} a_{11} \\ (I_{d_{n-1}} - 2u_2u_2^T) \begin{bmatrix} a_{21} \\ ... \\ a_{n1} \end{bmatrix} \end{bmatrix} = \begin{bmatrix} a_{11} \\ 0 \\ 0 \\ 0 \end{bmatrix} + \rho\|u_2\|e_2 = \begin{bmatrix} a_{11} \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ \rho\|u_2\| \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} a_{11} \\ \rho\|u_2\| \\ 0 \\ 0 \end{bmatrix}$$

The only really difficult part is right above: the operator simply handles separately the first dimension from the 'n-1' other ones.

Next, we use a crucial property of such an operator ie that $P_{n_2}^T = P_{n_2}$. This again comes from the fact that the matrice of the Householder reflector has the same property natively.

This means that we actually made a basis change where:

$$\begin{bmatrix} a_{11} & x' & x' & x' & .. & x' \\ \rho\|u_2\| & x' & x' & x' & .. & x' \\ 0 & x' & x' & x' & .. & x' \\ 0 & x' & x' & x' & ... & x' \\ ... & x' & x' & x' & ... & x' \\ 0 & x' & x' & x' & ... & x' \end{bmatrix} = P_{n_2}AP_{n_2}$$

This becomes clear when we put the 2 matrices one next to the other:

$$\begin{bmatrix} a_{11} & x & x & x & .. & x \\ \rho\|u_2\| & x & x & x & .. & x \\ 0 & x & x & x & .. & x \\ 0 & x & x & x & ... & x \\ ... & x & x & x & ... & x \\ 0 & x & x & x & ... & x \end{bmatrix} \begin{bmatrix} 1 & 0 & ... & 0 \\ 0 & & & \\ ... & & I_{d_{n-1}} - 2u_2u_2^T & \\ 0 & & & \end{bmatrix} = \begin{bmatrix} a_{11} & x' & x' & x' & .. & x' \\ \rho\|u_2\| & x' & x' & x' & .. & x' \\ 0 & x' & x' & x' & .. & x' \\ 0 & x' & x' & x' & ... & x' \\ ... & x' & x' & x' & ... & x' \\ 0 & x' & x' & x' & ... & x' \end{bmatrix}$$

Now the expression $P_{n_2}AP_{n_2} = \begin{bmatrix} a_{11} & x' & x' & x' & .. & x' \\ \rho\|u_2\| & x' & x' & x' & .. & x' \\ 0 & x' & x' & x' & .. & x' \\ 0 & x' & x' & x' & ... & x' \\ ... & x' & x' & x' & ... & x' \\ 0 & x' & x' & x' & ... & x' \end{bmatrix}$ $with\ P_{n_2} = P_{n_2}^* = P_{n_2}^{-1}$ shows that we moved from one orthogonal to a new one and nullified all the terms in the first column below the 2nd line. Next we just have to keep the first 2 basis vectors in this new basis and repeat the operation in S_2 ie the reference 1 st vector in "n-1" dimension is the second basis vector in the next basis that is implemented through P_{n_2}.

Then we repeat the same construction and we will end up with a matrix of this kind:

$$\begin{bmatrix} a_{11} & x_2{}' & x_2{}' & x_2{}' & .. & x_2{}' \\ \rho\|u_2\| & a_{2_{11}} & x_2{}' & x_2{}' & .. & x_2{}' \\ 0 & \rho\|u_{2_3}\| & x_2{}' & x_2{}' & .. & x_2{}' \\ 0 & 0 & x_2{}' & x_2{}' & ... & x_2{}' \\ ... & 0 & x_2{}' & x_2{}' & ... & x_2{}' \\ 0 & 0 & x_2{}' & x_2{}' & ... & x_2{}' \end{bmatrix}$$

In order to avoid any confusion in the matrix above: the notation x_2' does NOT mean that the terms are all equal: it is only meant to specify that these terms are not zero in general, they are not alike anyway. The sub index "$.._2$" indicates that these various terms are NOT alike the "x'" of the former matrix. This notation generally states "you have there non zero values in general".

Over the repetitions on these tests, notice that one need the resulting matrix to obtain the diagonal element and the element below in the same column. Notice as well that at each step a new matrix "A" appears. At the end of the day we can write that:

$$P_{n_3}P_{n_2}AP_{n_2}P_{n_3} = \begin{bmatrix} a_{11} & x' & x' & x' & .. & x' \\ \rho\|u_2\| & a_{2_{11}} & x' & x' & .. & x' \\ 0 & \rho\|u_{2_3}\| & x' & x' & .. & x' \\ 0 & 0 & x' & x' & ... & x' \\ ... & 0 & x' & x' & ... & x' \\ 0 & 0 & x' & x' & ... & x' \end{bmatrix}$$

And ultimately, after n-1 steps we obtain the Hessenberg marix:

$$P_{n_n}. ... P_{n_3}P_{n_2}AP_{n_2}P_{n_3} ... P_{n_n} = \begin{bmatrix} a_{11} & x' & x' & x' & .. & x' \\ \rho\|u_2\| & a_{2_{11}} & x' & x' & .. & x' \\ 0 & \rho_2\|u_{2_3}\| & a_{3_{11}} & x' & .. & x' \\ 0 & 0 & \rho_3\|u_{2_3}\| & a_{4_{11}} & ... & x' \\ ... & 0 & 0 & \rho_4\|u_{4_3}\| & ... & x' \\ 0 & 0 & 0 & 0 & ... & a_{n_{11}} \end{bmatrix} = H$$

We can move the stage 2 thanks to which we will secure a better convergence without reducing the complexity though…

Noting the unitary matrix U such that $U = P_{n_2}P_{n_3} ... P_{n_n}$ $then$ $A = UHU$ $since$ $UU^T = UU = I_d$

The complexity of the generation of the Hessenberg matrix is n^3.

viii- THE SVD APPROACH
(beware: this is a first draft…there might be errors!)

SVD: Singular Value Decomposition

The SVD approach is a benchmark that addresses the same topics as before but in a more general case for any m x n matrix, ie the matrix has "m" lines and "n" columns. We will find an orthonormal basis change "U" in $\mathbb{R}^m$and an orthonormal basis change "V" in $\mathbb{R}^m$ where $A = U^T DV$ where "D" is a diagonal matrix having m lines and n columns, like A.

Let's set the rank of A as "r". We will build a larger space of "n+m" dimensions and therefore we will have to complete our matrix A with nil matrices here and there. Then 4 spaces matter: the set of

$u_1, u_2, \ldots u_r$ the "column space", next $u_{r+1}, u_{r+2} \ldots u_m$ the "left null space", then $v_1, v_2 \ldots v_r$ the line space, and finally $v_{r+1}, v_{r+2}, \ldots v_n$ the right null space.

The main point here is that the usual methods to compute eigen values are plagued with convergence issues whenever an eigen value is close to 0 or when 2 eigen values are very close.

The present method offers the advantage that for any matrix A, if you add enough dimensions, where you put "0"s you will have increased the dimensions but also you then can define an orthogonal basis of "singular values" that in the end play the same role as the eigen values and "singular vectors" that are equivalent to eigen vectors in the native matrix…but in a different basis that is larger than the original one. On balance; you will obtain a rather fast and stable convergence at the expense of a higher dimension. SO you reduce the complexity bu your increase de "n".

This part holds 3 sub-sections:

1- The definition and role of singular vectors and singular values
2- The general approach that steps through a symmetrical matrix and positive eigen values
3- The Golub Kahan algorithm (1986) that offers an elegant and "simple" approach

Singular vectors and singular values

Consider a matrix A that has m lines and n columns again: picture that it is a collection of "m" vectors having each "n" coordinates. Picture too that these vectors could be represented as "arrows" all starting from the origin and therefore creating a bundle of arrows. These arrows may point all in the same direction on average. And next, the residual part, being defined as the orthogonal vector to this average vector also tend to point, on average, to a shared vector. And next, we could again look at the next residual and run the same search for a "common" complementary vector.

We also could impose that, at every stage of this process, we look for an optimal "common optimal" vector that is orthogonal to the former ones. Then we will defined a new basis of representation for the "m" original vectors that we can make "unit norm". As that moment we would have defined a basis from the "singular vectors" that feature the matrix A.

The chart below, in 2 dimensions, helps "visualize" the matter.

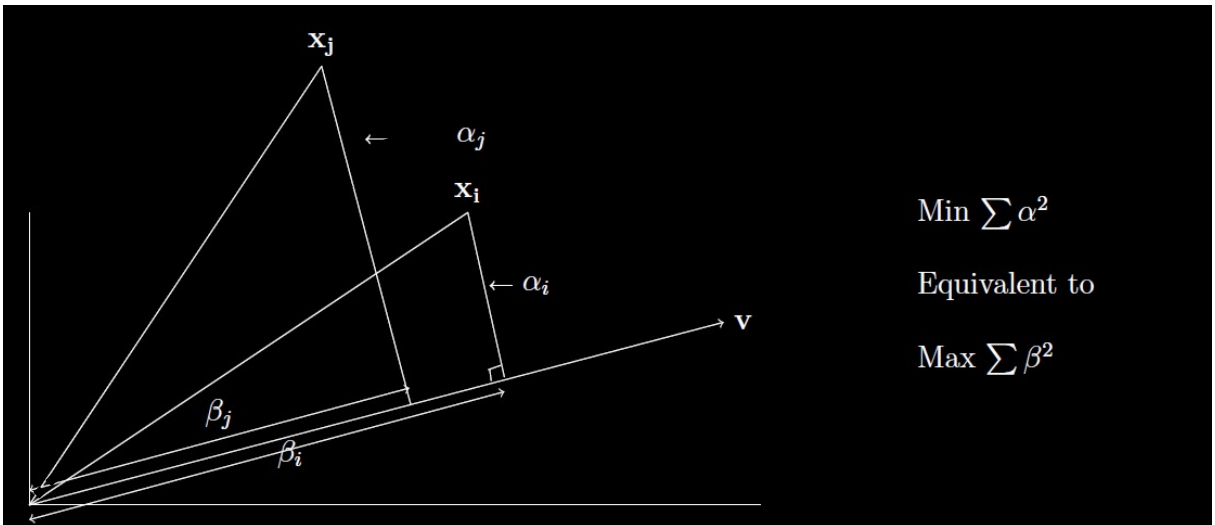


Figure 1.1: The projection of the point $\mathbf{x_i}$ onto the line through the origin in the direction of $\mathbf{v}$

Above, the different vectors, with therefore n=2 here, are the $x_i \; and \; x_j$ and we can see that we look for a vector "v" but one that would make the projections overall having a maximum absolute value. We can see above that, as far as $x_i \; and \; x_j$ are concerned, the current vector "v" is not optimal . We can guess that the dotted fat vector would be a better fit is only these 2 vectors are involved.

Intuitively we observe that the ideal vector "v" minimizes the complementary distance in the spirit of the theorem of Pythagoras.

This chart above can be generalized if you imagine many more dimensions: each time a "center of inertia" of the current "point" that can be represented like this with the use of the origin, one can construct a complementary space where the extremity of the projection of each vector on the vector v becomes the source of the complementary vectors. Each line-vector of the matrix A would have a different projection on "v". So we would define a vector of norm 1 that is collinear with the same direction as the optimal "v". We would compute the residual vector that would be then:

$v_{a_k} - v_{a_k}.v_{opt}$. By definition this residual vector will be orthogonal to v_{opt} . Then one defines a space to warehouse these new residual vectors, use the origin to set new "points" but with one less coordinate. Then we look again for the center of inertia, define a new optimal vector "v". And move on to the next stage until we get to the dimension 2 case as set above.

Let's now look at the main definition and properties of these singular vectors. Basically a singular vector is the vector of norm 1 that has the direction of the "optimal" vector of projections "v"

Let's consider this matrix A as a collection of "m" lines of "n" columns and, let's consider a vector "v" of "n" coordinates. If now we notice that the "projection of one vector-line of A on the vector v" is the scalar product of this line with v, ie for a line "k" among the "m" ones that constitute A, the projection on the vector "v" for this line "k" is $a_{k\ldots}.v = a_{k1}v_1 + a_{k2}v_2 + \cdots . a_{kn}v_n = \beta_k$

If we look at all the projections that we care about here, we can group them as the coordinates of one vector and observe that $Av = \begin{bmatrix} \beta_1 \\ \beta_2 \\ \beta_3 \\ \cdot\cdot \\ \beta_m \end{bmatrix}$

We then would define the optimal vector v_1 that is the first singular vector for the matrix A as :

$$v_1 = \; Max_{\|v\|=1}\left(\sum_{k=1}^{k=m} \beta_k^2\right) = Max_{\|v\|=1}(\|Av\|^2)$$

And we would define the next singular vector as : $v_2 \; = Max_{\begin{cases}\|v\|=1 \\ v.v_1=0\end{cases}}(\|Av\|^2)$

Then , notice the added condition on "v" since we want to make sure that the optimal vector v_2 remains orthogonal with v_1 through the search for an optimal direction.

We then would define the "singular value" as $\sigma_1 = \|Av_1\|^1 \; and \; \sigma_2 = \|Av_2\|^1$

Iterating like this at the most "m" times: $v_{k+1} \; = Max_{\begin{cases}\|v\|=1 \\ v.v_1=0 \\ \ldots \\ v.v_k=0\end{cases}}(\|Av\|^2)$

We generate the "k+1" singular value knowing the singular vector above as $\sigma_{k+1} = \|Av_{k+1}\|^1$

Notice here that we have NOT dealt with the computation and the algorithm that would efficiently find these directions and values. Just bear in mind the change of optics here. We actually look to "maximize" the projection on one axis, which, by the effect of Pythagoras theorem, is equivalent to "we minimize the residual distance" which also can be phrased as "we look for the least squared errors". The latter expression is ubiquitous BOTH in finance and in AI. Take a moment here to realize that whenever you have "points" based on which you want to make "predictions", you will apply a "model" and optimize this "model" based on the "errors" and usually you will target a minimization of the "least squared errors". Doing so, your "singular values" will the ones that will shape your "model" and therefore your future predictions. Thus, behind this approach, there is a way to bring a hierarchy in the way to shrink the errors. The "singular" values will give you the "average residual variance" relative to your model. Each "line" here is an "input point". In other words, we have shifted from the classical context of the symmetrical Hessian matrix or a covariance matrix to a more typical machine learning, AI approach.

Remember, however, the introductive remarks that what follows enable us also to "solve" the traditional problems involving symmetrical matrices in a more convenient way!

We know that having "m" lines at the most we would define a space of "n" dimensions since we "only" have "n" dimensions. But, in a nice world where actually these "lines", these "empirical points" eventually, are well behaved and actually describe a space of 2 dimensions, only, or maybe 3 or more…who knows….! Let's call the dimension "r" and let's say then that our matrix A is of "rank r".

We then infer that once we have set the first "r" singular vectors and singular values we would have defined an orthogonal basis that fully represents the vector space that is covered by our initial "m" vectors of "n" dimensions that also feature the matrix A. We know for sure the "r" is lower or equal to "n".

It turns out that proceeding in this way we will find an optimal basis that will maximize on average absolute value terms the representation of the native "lines" of A. in other words, this basis captures the maximum value that the m lines can take as a group when they are projected on the most representative axis.

Now let's review the most important properties of the singular vectors and values. So far we defined the "right" singular vectors ie, back to our target representation $A = U^T DV$ we actually defined the matrix "V" for now with the singular vector v_k being the "k-th" column of the matrix V that is an n x n matrix.

We should note that we could run the very same approach "from the left", ie define A now as a matrix of "n" columns having each "m" coordinates. Observe then that we would a scalar product with now a vector "u" of "m" coordinates as , with the column "j" of the matrix A:

$$\alpha_j = u_1 a_{1j} + u_2 a_{2j} + u_3 a_{3j} + \cdots u_m a_{mj}$$

In a similar construction we would end up defining a singular vector from the "left side " of the matrix A as : $u_1 = \ Max_{\|u\|=1}\left(\sum_{j=1}^{j=n} \alpha_j^2\right) = Max_{\|u\|=1}(\|u^T A\|^2)$

We then could define a second, and a 3rd and up to the rank "r" of the matrix A again:

$$u_{k+1} = \ Max_{\|u\|=1}\left(\sum_{j=1}^{j=n} \alpha_j^2\right) = Max_{\begin{cases}\|u\|=1 \\ u.u_1=0 \\ \dots \\ u.u_k=0\end{cases}}(\|u^T A\|^2)$$

What is connection between the “right singular vectors v” and the “left singular vectors u”? Do they appear in the same number ie is the rank of as a collection of m vectors of n coordinates the same as the rank of A seen now as a collection of “n” vectors of m coordinates?

Some quite important properties of the singular vectors and singular values

Let’s observe that there can only be 1 optimal direction for the “u” vectors anyway and since they are of norm 1, there can only be ONE such vector. In other words, the basis of left singular vector is unique, orthogonal and norm 1.

Let’s define the left singular vectors differently: $\sigma_1 u_1 = Av_1$ ie u_1 is simply the norm 1 “direction” of the image by A of our first “right singular vector”.

Let’s define the matrix: $B = A - \sigma_1 u_1 v_1^T \; with \; \sigma_1 = \|Av_1\|^1 \; and \; \|v_1\|^1 = 1 \; and \; u_1 = \begin{bmatrix} u_{11} \\ u_{12} \\ u_{13} \\ \cdots \\ u_{1m} \end{bmatrix}$

To be clear: the vectors are column vectors and therefore $u_1 v_1^T$ is an m x n matrix, not a scalar product

Thus the matrix B that we just defined is an m x n matrix very much like A is.

If we consider the first singular vector of B that we will note $v_{B_1} \; then \; v_{B_1}.v_1 = 0$

Indeed $Bv_1 = Av_1 - \sigma_1 u_1 v_1^T v_1 \; but \; v_1^T v_1 = \|v_1\|^2 = 1 \; hence \; Bv_1 = Av_1 - \sigma_1 u_1 = 0$

Then consider the first singular vector of B, noted v_{B_1} for which we reach the maximum: $\left\|Bv_{B_1}\right\|$

If we have actually $v_{B_1}.v_1 \neq 0 \; then \; \left\|\frac{B(v_{B_1} - (v_{B_1}.v_1))v_1}{\|(v_{B_1} - v_{B_1}.v_1)\|}\right\| = \frac{\|Bv_{B_1}\|}{\|(v_{B_1} - v_{B_1}.v_1)\|} > \left\|Bv_{B_1}\right\|$ because we then have $\left\|\left(v_{B_1} - v_{B_1}.v_1\right)\right\| < 1$. This is in contradiction with the fact that v_{B_1} is the vector where we reach the optimum. Clearly here, if v_{B_1} and v_1 are not orthogonal, we reach a strictly higher value than the stated optimum. Therefore it must be that $v_{B_1}.v_1 = 0$. Once we get this “next” maximum singular vector based on the maximum singular value, we can generate a new “matrix B” under the same principle and repeat the reasoning…Note however that for A the dimension set by v_1 was defining one dimension in the image of A, it does not for B. We therefore can go on like this until there is no more dimension to define through the singular vectors.

For any vector v that is orthogonal to v_1, that includes of course all the coming singular vectors of next B matrices, we have $Bv = Av \; and \; thus \; Bv_{B_k} = Av_{B_k}$. This applies ALSO to all the other singular vectors of A itself that are orthogonal to v_1. In the orthogonal space that is defined as the complementary space of v_1 we then have B=A in fact! Therefore the 2 matrices have the very same singular vectors and singular values (right values). We infer from that that B is the restriction of A to the sub-space that complements the one dimension that is set by v_1 in the image of A.

We will prove now by recursive thinking that the many “images” of the right singular vectors are orthogonal too. Consider that it must be that $v_{B_1} = v_2$.

So this means that “B” is alike A in the complementary space. We can proceed like this dimension per dimension: each time we generate a new matrix B based on the latest “higher” singular value, we look for the “next one” and we have reduced the rank by 1 while going into an orthogonal space relative the former ones where we looked for the “higher” singular values.

More rigorously. We start with A and the max singular vector v_1 and define the matrix $B_1 = A - \sigma_1 u_1 v_1^T$. If the matrix A has rank r, then the matrix B_1 has rank "r-1". We know also that the max singular vector will then be v_2 and we can define a new matrix $B_2 = B_1 - \sigma_2 u_2 v_2^T$ that will have a rank of "r-2". After "r" such iteration we get a matrix $B_r = B_{r-1} - \sigma_r u_r v_r^T = 0$. We therefore "unwind" all the sequence: $B_{r-1} = \sigma_r u_r v_r^T, then\ B_{r-2} = B_{r-1} + \sigma_{r-1} u_{r-1} v_{r-1}^T$ ie

$$B_{r-2} = \sigma_r u_r v_r^T + \sigma_{r-1} u_{r-1} v_{r-1}^T$$

We can rewind like this back to A and B_1 and obtain:

$$A = \sigma_r u_r v_r^T + \sigma_{r-1} u_{r-1} v_{r-1}^T + \cdots + \sigma_2 u_2 v_2^T + \sigma_1 u_1 v_1^T$$

We infer therefore that:

$$A = \sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots \sigma_r u_r v_r^T$$

let's now construct $A^T A = (\sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots \sigma_r u_r v_r^T)^T (\sigma_1 u_1 v_1^T + \sigma_2 u_2 v_2^T + \cdots \sigma_r u_r v_r^T)$

If we look at the standard term: $(\sigma_k u_k v_k^T)^T (\sigma_m u_m v_m^T) = \sigma_k \sigma_m v_k u_k^T u_m v_m^T = \delta_{mk} \sigma_k \sigma_m\ v_k v_m^T$

We can see that only the diagonal term will remain: $A^T A = \sigma_1^2 v_1 v_1^T + \sigma_2^2 v_2 v_2^T + \cdots + \sigma_r^2 v_r v_r^T$

Let's look simply at a scalar product between 2 "images by A " of the right singular vectors:

$$\sigma_i \sigma_j u_i . u_j = u_i^T u_j = (A v_i)^T A v_j = v_i^T A^T A v_j$$

Using the formula above: $\sigma_i \sigma_j u_i . u_j = v_i^T (\sigma_1^2 v_1 v_1^T + \sigma_2^2 v_2 v_2^T + \cdots + \sigma_r^2 v_r v_r^T) v_j = \sigma_j^2 v_i^T v_j = \sigma_j^2 \delta_{ij}$

The scalar product is 0 each time I and j differ while it is worth 1 when i=j

So, given that the singular values are strictly positive it must be that $u_i . u_j = \delta_{ij}$

In order to visualize what happens, the charts below show the process that we are in now..

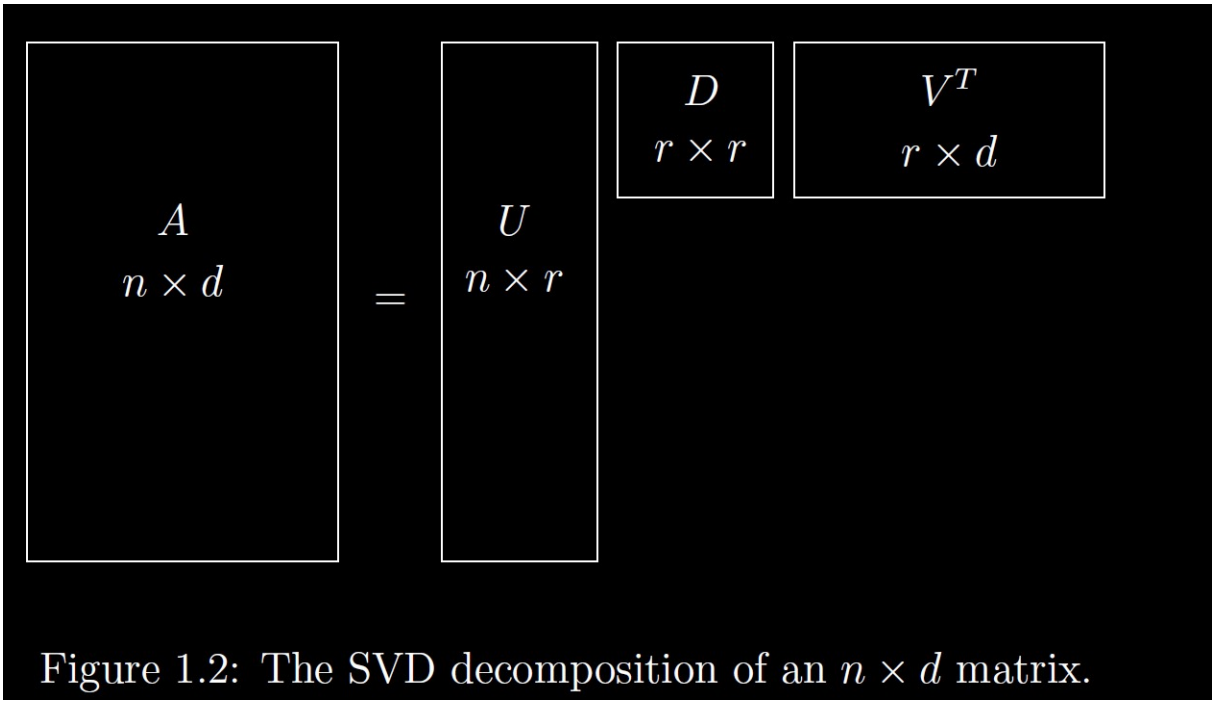


Figure 1.2: The SVD decomposition of an $n \times d$ matrix.

We realize here that the matrix has the same rank in "m" dimension and in "n" dimensions. This matches with the fact that, among the "m" lines or among "n" columns we can pick "r" among them

and define an independent system that can be reduced to an rxr matrix actually, the determinant of which is not 0. The notion of "rank=r" is equivalent to say "we can find r independent n-lines or r independent m-columns but we can find r+1 independent lines or columns".

Another was to observe the equivalence is to notice that we only worked on r dimensions:

More than just orthogonality, these basis vectors diagonalize the matrix A:

"A is diagonalized" $Av_1 = \sigma_1 u_1$ $Av_2 = \sigma_2 u_2$ ... $Av_r = \sigma_r u_r$ (1)

Those **singular values σ_1 to σ_r** will be positive numbers: σ_i *is the length of* Av_i. The σ's go into a diagonal matrix that is otherwise zero. That matrix is Σ.

(using matrices) Since the $\boldsymbol{u}$'s are orthonormal, the matrix U with those r columns has $U^{\mathrm{T}}U = I$. Since the $\boldsymbol{v}$'s are orthonormal, the matrix V has $V^{\mathrm{T}}V = I$. Then the equations $Av_i = \sigma_i u_i$ tell us column by column that $\boldsymbol{AV_r = U_r\Sigma_r}$:

$$(m \text{ by } n)(n \text{ by } r) \quad (m \text{ by } r)(r \text{ by } r)$$
$$\boldsymbol{AV_r = U_r\Sigma_r} \qquad A \begin{bmatrix} & & \\ v_1 & \cdots & v_r \\ & & \end{bmatrix} = \begin{bmatrix} & & \\ u_1 & \cdots & u_r \\ & & \end{bmatrix} \begin{bmatrix} \sigma_1 & & \\ & \ddots & \\ & & \sigma_r \end{bmatrix}. \quad (2)$$

This is the heart of the SVD, but there is more. Those $\boldsymbol{v}$'s and $\boldsymbol{u}$'s account for the row space and column space of A. We have $n - r$ more $\boldsymbol{v}$'s and $m - r$ more $\boldsymbol{u}$'s, from the nullspace $\boldsymbol{N}(A)$ and the left nullspace $\boldsymbol{N}(A^{\mathrm{T}})$. They are automatically orthogonal to the

As is explained above we complete the basis "v" and the basis "u" respectively to make and "n xn " orthogonal basis transition matrix U and an "m x m" orthogonal transition matrix V and thus write;

$$A = \sigma_r u_r v_r^T + \sigma_{r-1} u_{r-1} v_{r-1}^T + \cdots + \sigma_2 u_2 v_2^T + \sigma_1 u_1 v_1^T = U^T D V$$

We then infer that $A^T A = U^T D D V$ *and therefore that the* v_l *singular vectors of* A *are the eigen vectors of* A!

This implies that the singular values of A are the positive square roots of the eigen values of $A^T A$

In light of the remarks made before: we can see that we really need to compute the singular values of either a rectangular matrix A or some symmetrical matrix. We can see that we have not yet define a proper to compute either the eigen values or the singular values yet. This comes with the Golub Kahan alforithm that is quite sophisticated!

Golub Kahan algorithm

You should have a look at key background information about the Householder reflectors and the Hessenberg matrices. This algorithm leverages the principles described in the appendix i below.

The algorithm and, compared to the simplicity of the algorithm explained in the appendix ii- below, the motivations for this algorithm lie on only one problem: the convergence towards the eigen values can be often impaired either because an eigen value is very low or because 2 eigen values are very close. We could observe so far that we reasoned on "maximums" and this is all very nice but we have no means to safely compute them. The algorithm provide a safe approach but a sophisticated one.

As a reminder the stakes are big because in finance when we want to handle correlation risks or when we want to price complex risks through the heat equation and a Hessian matrix of second order partial derivatives vis a stochastic equation (ie all the time!) we have to find eigen values and eigen vectors. And there we may not get reliable estimates In due time. The same issue occurs in AI whenever we want to minimize the "means squared error", ie what any machine learning program will try to do anyway (even through the maximization or minimization of the entropy of information).

Let's also recall that to actually diagonalize efficiently a symmetrical matrix it may be astute to morph the problem into finding the singular vectors and values of a rectangular matrix.

The core objective is indeed to define a "unitary" pair of matrices.P and Q where, for a given m x n matrix A we have $P_{mm}P_{mm}^T = I_{d_{mm}}, Q_{nn}^T Q_{nn} = I_{d_{nn}}$ $and\ A = P^T JQ$

With

$$A = PJQ^*,$$

where P and Q are unitary matrices and J is an $m \times n$ bidiagonal matrix of the form

$$J = \begin{pmatrix} \alpha_1 & \beta_1 & 0 & \cdot & \cdot & \cdot & 0 \\ & \alpha_2 & \beta_2 & 0 & \cdot & \cdot & \cdot \\ & 0 & \cdot & \cdot & \cdot & & \\ & & & \cdot & \cdot & \cdot & \\ & & & & \cdot & \cdot & \\ & & & & & \cdot & \beta_{n-1} \\ & & & & & & \alpha_n \\ & & & 0 & & & \end{pmatrix} \Big\} (m-n) \times n$$

We are going to construct a QR method but rather than run a process like

$$A_k = Q_k R_k \rightarrow A_{k+1} = R_k Q_k$$

If you remember as a iterations go, the unitary matrix Q_k tends towards the identity matrix while, by design the sub-diagonal terms converge towards 0. But we saw that the upper triangle not being very much "0" induces quite a lot calculations on the follow and some instability when eigen values are close to 0 or similar.

We instead are going to generate 2 unitary matrices that will have quite precious properties inherited from the Householder reflectors (see appendix I for details and proofs)

Thus we use 2 sequences of Householder reflectors and process in "semi-steps".

We indeed define the following sequence:

$$\begin{cases} A_{k+\frac{1}{2}} = P_k A_k \\ A_{k+1} = A_{k+\frac{1}{2}} Q_k \end{cases} where \begin{cases} P_k = I_d - 2x_k x_k^T \\ Q_k = I_d - 2y_k y_k^T \end{cases} are\ Householder\ reflectors$$

We can observe the change by combining the 2 equations above: $A_{k+1} = P_k A_k Q_k$

We are going to define $P_k\ so\ that\ a_{k+\frac{1}{2}ik} = 0\ for\ i = k+1, k+2, \ldots . m$. Thus the matrix $A_{k+\frac{1}{2}}$ will be bi-diagonal like J above but only up to the column k and line k.

We will also define Q_k so that $a_{k+1_{kj}} = 0 \; for \; j = k+2, k+3, \ldots . n$

The picture below shows how the matrix A_{k+1} looks like after the 2 relfectors have been applied to A_k: In appendix, it is shown "how" this objective can be achieved in part. Indeed, selecting the 'n-1' coordinates of the first line of the matrix A, we build a vector "x" like the one that is used in P above. That has the effect of generating 0 below in the first column of the "image" the is indexed as "k+1/2". The next operation is the same but in a transposed format. Below is the explanation of "why" this couple works. First, below, the "target" next matrix A that we want to have at stage "k+1" from the stage "k"

$$A^{(k+1)} = \begin{pmatrix} \alpha_1 & \beta_1 & 0 & \cdot & \cdot & & & & & \\ 0 & \alpha_2 & \beta_2 & 0 & \cdot & & & & & \\ \cdot & 0 & \cdot & \cdot & \cdot & & & 0 & & \\ \cdot & \cdot & \cdot & \cdot & \cdot & & & & & \\ & & & & \alpha_k & \beta_k & & & & \\ & & & & & x & x & \cdot & \cdot & \cdot \\ & & 0 & & & x & x & \cdot & \cdot & \cdot \\ & & & & & \cdot & \cdot & \cdot & \cdot & \cdot \\ & & & & & \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix}.$$

You have to understand that at each step "k" to wards "k+1" we actually operate on the remaining dimensions and it is as if we now dealt with a matrix A or rank "r-k". The operation is same always and only the number of coordinates change along with their values at the current stage.

As it was shown in the appendix, after a selection of an appropriate , we reduce the first column fo the running matrix A:

$$\begin{bmatrix} 1 & 0 & \ldots & 0 \\ 0 & & & \\ \ldots & & I_{d_{n-1}} - 2u_2u_2^T & \\ 0 & & & \end{bmatrix} \begin{bmatrix} a_{11} \\ a_{21} \\ a_{31} \\ \ldots \\ a_{n1} \end{bmatrix} = \begin{bmatrix} a_{11} \\ (I_{d_{n-1}} - 2u_2u_2^T) \begin{bmatrix} a_{21} \\ \ldots \\ a_{n1} \end{bmatrix} \end{bmatrix} = \begin{bmatrix} a_{11} \\ 0 \\ 0 \\ 0 \end{bmatrix} + \rho\|u_2\|e_2 = \begin{bmatrix} a_{11} \\ 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0 \\ \rho\|u_2\| \\ 0 \\ 0 \end{bmatrix} = \begin{bmatrix} a_{11} \\ \rho\|u_2\| \\ 0 \\ 0 \end{bmatrix}$$

The other columns are impacted but we cannot tell "how".

$$\begin{bmatrix} a_{11} & x' & x' & x' & .. & x' \\ \rho\|u_2\| & x' & x' & x' & .. & x' \\ 0 & x' & x' & x' & .. & x' \\ 0 & x' & x' & x' & \ldots & x' \\ \ldots & x' & x' & x' & \ldots & x' \\ 0 & x' & x' & x' & \ldots & x' \end{bmatrix}$$

Now, in order to understand how the second "transposed" operation is going to have the required effect, let's call the matrix above $\begin{bmatrix} a_{11} & x' & x' & x' & .. & x' \\ \rho\|u_2\| & x' & x' & x' & .. & x' \\ 0 & x' & x' & x' & .. & x' \\ 0 & x' & x' & x' & \ldots & x' \\ \ldots & x' & x' & x' & \ldots & x' \\ 0 & x' & x' & x' & \ldots & x' \end{bmatrix} = B^T$

Then the second stage amounts to, in a transposed format, compute at first:

$$QB = \begin{bmatrix} 1 & 0 & \dots & 0 \\ 0 & & & \\ \dots & I_{d_{n-1}} - 2yy^T & & \\ 0 & & & \end{bmatrix} \begin{bmatrix} a_{11} & \rho\|u_2\| & 0 & 0 & .. & 0 \\ x' & x' & x' & x' & .. & x' \\ x' & x' & x' & x' & .. & x' \\ x' & x' & x' & x' & \dots & x' \\ \dots & x' & x' & x' & \dots & x' \\ x' & x' & x' & x' & \dots & x' \end{bmatrix} = \begin{bmatrix} a_{11} & \rho\|u_2\| & 0 & 0 & .. & 0 \\ \rho\|u_2\| & x' & x' & x' & .. & x' \\ 0 & x' & x' & x' & .. & x' \\ 0 & x' & x' & x' & \dots & x' \\ \dots & x' & x' & x' & \dots & x' \\ 0 & x' & x' & x' & \dots & x' \end{bmatrix}$$

We obtain here a tri-diagonal matrix, not yet a bi diagonal matrix. Since we look at the transpose we would like to reduce the term in the first column and on the second line to 0 without losing the other 0s that we already have. The is possible with an astute set of equations since as we will see this solves naturally below…

So, for the first Householder reflector $P_k = I_d - 2x_k x_k^T \; we \; pick \; x_{k_i} = 0 \; for \; i = 1,2, \dots k-1$

Since the operator P_k is unitary and orthogonal we have $|\alpha_k|^2 = \sum_{j=k}^{j=m} \left|a_{k_{jk}}\right|^2$ which in other words means that at a certain stage "k" BEFORE we morph the terms that are not yet 0 to zero, the transformation operated by $P_k \; (and \; Q_k)$ does not alter the norm of the vector that all these coordinates represent.

Thanks to the fact $P_k = P_k^T = P_k^{-1}$ we also can write that $P_k A_{k+\frac{1}{2}} = A_k \; with \; P_k = I_d - 2x_k x_k^T$

Thus, in a somewhat counterintuitive manner we have : $\begin{cases} \left(1 - 2\left|x_{k_k}\right|^2\right)\alpha_k = a_{k_{kk}} \\ -2x_{k_i}\overline{x_{k_k}}\alpha_k = a_{k_{ik}} for \; i \geq k+1 \end{cases}$

We actually "solve" the system above since we look for the appropriate values for the x_k. Note that we here make a much more sophisticated choice of coordinates for the vector that will shape the Householder reflector at this stage. It happens to be possible. We will see that it remains doable for the vector y_k despite imposing more zeros. The explanation above provide an intuition as to why it is possible.

Thus $\begin{cases} \frac{1}{2}\left(1 - \frac{a_{k_{kk}}}{\alpha_k}\right) = \left|x_{k_k}\right|^2 \\ x_{k_i} = -\frac{1}{2}\frac{a_{k_{ik}}}{\overline{x_{k_k}}\alpha_k} \; for \; i > k \quad 0 \; otherwise \end{cases}$ In light of the first formula, since we are only constrained by: $|\alpha_k|^2 = \sum_{j=k}^{j=m} \left|a_{k_{jk}}\right|^2 \; we \; will \; choose \; to \; have \; \alpha_k = -\frac{a_{k_{kk}}}{\left|a_{k_{kk}}\right|}\sqrt{\sum_{j=k}^{j=m} \left|a_{k_{jk}}\right|^2}$

With this choice $\left(1 - \frac{2a_{k_{kk}}}{\alpha_k}\right) = \left|x_{k_k}\right|^2 = \frac{1}{2}\left(1 + \frac{\left|a_{k_{kk}}\right|}{\sqrt{\sum_{j=k}^{j=m}\left|a_{k_{jk}}\right|^2}}\right) > 1 \; in \; general$

It works for the terms of the same columns that lie below the diagonal. But we have no means yet to impose anything on the lines. This will come thanks to the reflector Q_k. In the intuitive explanation above we showed that this corresponds to the "column" in a transposed manner. Here we know that the solution is quite easy to find. So the hardest part is done already when it is about to shrink the matrix to 2 diagonal lines only.

We can therefore summarize the resolution of the sequence of Householder reflectors P_k:that do solve the reduction to 2 diagonal lines problem that we flagged above.

$$\begin{cases} s_k = \sqrt{\sum_{j=k}^{j=m} \left|a_{k_{jk}}\right|^2} \\ \alpha_k = -\frac{a_{k_{kk}}}{\left|a_{k_{kk}}\right|} s_k \\ x_{k_i} = 0 \; for \; i < k \\ x_{k_k} = \sqrt{\frac{1}{2}\left(1 + \frac{\left|a_{k_{kk}}\right|}{\sqrt{\sum_{j=k}^{j=m} \left|a_{k_{jk}}\right|^2}}\right)} \\ x_{k_i} = -\frac{1}{2}\frac{a_{k_{ik}}}{\overline{x_{k_k}}\alpha_k} = \frac{1}{2}\frac{a_{k_{ik}}}{\overline{x_{k_k}}\frac{a_{k_{kk}}}{\left|a_{k_{kk}}\right|}s_k} \; for \; i > k \end{cases}$$

In quite a similar manner since, this is the same operation in a transposed basis, we find for the $sequence\ Q_k$, ie we invert the indices and use "n" instead of "m":

$$\begin{cases} t_k = \sqrt{\sum_{j=k+1}^{j=n} \left|a_{k+\frac{1}{2}kj}\right|^2} \\ \beta_k = -\frac{a_{k+\frac{1}{2}k,k+1}}{\left|a_{k+\frac{1}{2}k,k+1}\right|} t_k \\ y_{k_i} = 0 \; for \; i < k+1 \\ y_{k_{k+1}} = \sqrt{\frac{1}{2}\left(1 + \frac{\left|a_{k+\frac{1}{2}kk}\right|}{\sqrt{\sum_{j=k+1}^{j=n} \left|a_{k+\frac{1}{2}kj}\right|^2}}\right)} \; real \; by \; choice \\ y_{k_i} = -\frac{1}{2}\frac{a_{k+\frac{1}{2}ik,+1}}{\overline{y_{k_{k+1}}}\beta_k} = \frac{1}{2}\frac{a_{k+\frac{1}{2}ik,+1}}{\overline{y_{k_{k+1}}}\frac{a_{k+\frac{1}{2}kk}}{\left|a_{k+\frac{1}{2}k,k+1}\right|}t_k} \; for \; i > k+1 \end{cases}$$

We can "compare" the bi-diagonal solution is more complex than the tri-diagonal one where we pick the coordinates of the next vector "x" for P_k and the next vector X for Q_k much more simply since these are respectively the "n-k-1" coordinates of the line "k" on A_k and next they will be the "m-k-1" coordinates of the line k in the transpose obtained after the former transformation. But we would not have a direct access yet to the singular values and even less so access to the singular vectors.

But we are close now!

Consider a bi-diagonal matrix like this one $\begin{bmatrix} a_1 & b_2 & 0 & \dots & 0 & 0 \\ 0 & a_2 & b_3 & \dots & 0 & 0 \\ 0 & 0 & a_3 & \dots & 0 & 0 \\ 0 & 0 & 0 & \dots & b_{r-1} & 0 \\ \dots & \dots & \dots & \dots & a_{r-1} & b_r \\ 0 & 0 & 0 & \dots & 0 & a_r \end{bmatrix}$

If we look for an eigen vector here using only the first 2 basis vectors, if we try to find an "x" such that $\begin{bmatrix} a_1 & b_2 & 0 & \dots & 0 & 0 \\ 0 & a_2 & b_3 & \dots & 0 & 0 \\ 0 & 0 & a_3 & \dots & 0 & 0 \\ 0 & 0 & 0 & \dots & b_{r-1} & 0 \\ \dots & \dots & \dots & \dots & a_{r-1} & b_r \\ 0 & 0 & 0 & \dots & 0 & a_r \end{bmatrix} \begin{bmatrix} 1 \\ x \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} = V \begin{bmatrix} 1 \\ x \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$ then we have $\begin{cases} a_1 + b_2 x = V \\ a_2 x = Vx \end{cases}$ We set

$x = \frac{a_2 - a_1}{b_2}$ *when* b_2 *not zero*

$a_2 = V$ *otherwise the first eigen vector is* e_1

Looking now at the second and 3rd column we can do the same with just a switch of indices. So we find "n-1" eigen vectors. We only need to find last one: it clearly is the first basis vector! QED!.

We might wonder whether the eigen vectors that we find are the appropriate singular vectors. They are because they happen to be the eigen vectors of the matrix $A^T A$ and thus they can only the vectors v_k. The reduction to only 2 lines made the resolution simple. However 3 diagonals would have been hardly more difficult: we would have to solve consecutive systems of 3 equations instead of 2.

Let's check on that

$$\begin{bmatrix} a_1 & b_2 & 0 & \dots & 0 & 0 \\ c_1 & a_2 & b_3 & \dots & 0 & 0 \\ 0 & c_2 & a_3 & \dots & 0 & 0 \\ 0 & 0 & c_3 & \dots & b_{r-1} & 0 \\ \dots & \dots & \dots & \dots & a_{r-1} & b_r \\ 0 & 0 & 0 & \dots & c_{n-1} & a_r \end{bmatrix} \begin{bmatrix} 1 \\ x \\ y \\ 0 \\ 0 \\ 0 \end{bmatrix} = V \begin{bmatrix} 1 \\ x \\ y \\ 0 \\ 0 \\ 0 \end{bmatrix} \; implies \begin{cases} a_1 + b_2 x = V \\ c_1 + a_2 x + b_3 y = Vx \\ c_2 x + a_3 y = Vy \end{cases}$$

We can solve for $V = a_3$ and move on to the 4th line and finally find n-2 eigen vectors like this. Finally, imposing $V = a_1 and\, V = a_2$ we complete the picture/

So the Golub-Kahan algorithm is very sophisticated but simpler ones would work just exploiting the same tricks.

We can retain however that the Golub-Kahan process is close to optimal when we think of the theorem of decomposition of Jordan.

ix- The direct evaluation of eigen values and eigen vectors on a symmetrical real matrix

There is a direct way to both determine the eigen values and eigen vectors, under some rather non restrictive conditions that we will see below.

Take a matrix A: $A = (a_{ij})$

Start with a norm 1 vector $X_0\ such\ that\ \left||X_0|\right| = \sqrt{\sum_{k=1}^{n} X_{0_k}^2} = 1$

We know that we can build a basis of eigen vectors but in the set of complex numbers only $\mathbb{C}$. This means that for the sake of the proof we will have speak in "modules".
Let's now consider the possible expression of our vector X_0 in this basis "orthonormal" but in $\mathbb{C}$ this time.

Then, to be clear we still have $\left||X_0|\right| = \sqrt{\sum_{k=1}^{k=n}\left|X_{0_k}\right|^2} = \sqrt{\sum_{k=1}^{k=n} X_{0_k}\overline{X_{0_k}}}$

We are going to show that **under mild conditions**, then a simple iteration and normalization will have us find the eigen vector and the eigen in question quite easily.
Let's call $X_1 = \frac{AX_0}{\|AX_0\|}$. Let's now note $B = (b_1, b_2, \dots b_n)$ the original canonical basis and let's note $E = (e_1, e_2 \dots e_n)$ the basis made of eigen vectors. We should now express $X_0\ in\ the\ E\ X_0 = X_{0_{e_1}} e_1 + X_{0_{e_2}} e_2 + \cdots X_{0_{e_n}} e_n$
Then $AX_0 = V_{p_1} X_{0_{e_1}} e_1 + V_{p_2} X_{0_{e_2}} e_2 + \cdots V_{p_n} X_{0_{e_n}} e_n$
And therefore $\|AX_0\|^2 = \sum_{k=1}^{k=n}\sum_{l=1}^{l=n} V_{p_k}\overline{V_{p_l}}\, e_k.\overline{e_l}$

Then if we factorize by this maximum eigen value in module, from the formula below:

$$X_1 = \frac{AX_0}{\|AX_0\|} = \frac{V_{p_1}}{\|AX_0\|} X_{0_{e_1}} e_1 + \frac{V_{p_2}}{\|AX_0\|} X_{0_{e_2}} e_2 + \cdots \frac{V_{p_n}}{\|AX_0\|} X_{0_{e_n}} e_n$$

We can observe that IF $\|AX_0\| > \left|V_{p_k}\right|$ then the new coordinate for X_1 will be strictly lower than the corresponding one for X_0.
Thus all the coordinates would tend to zero in module except the one for which we may have then $\|AX_0\| = \left|V_{p_k}\right|$. This situation will occur each time for the highest eien value in module as we repeat the iteration above. Indeed the other coordinates that would NOT be associated with this eigen value that dominates in module, would all decline towards 0 leaving even more predominance in the module $\|AX_0\|$ to the dominant eigen value.
If, at an early stage more eigen values offset each other through the different scalar products, they others will anyway vanish, leaving then only the most important ones. Then again the biggest eigen value in module will ultimate dominate in the recursive sequence.
So after a certain number of iterations we would obtain: $X_{N+1} = \frac{AX_N}{\|AX_N\|} \approx X_N$. We would then have found BOTH the eigen vector with X_{N+1} and the eigen value that is the largest in module with $V_{p_M} = X_N.\frac{AX_N}{\|X_N\|^2}$
How to find the other eigen values then, namely the ones with lower module?
Easy: generate the same recurrence but based on unit on vectors that have no projection on the first eigen vector axis ie in an orthogonal space.
We thus start now with $X_{1_0} = \frac{X_0 - X_0.e_1}{\|X_0 - X_0.e_1\|}$ where the vector X_0ie generated randomly.
Next we apply the sequence : $X_{1_{N+1}} = \frac{AX_{1_N} - AX_{1_N}.e_1}{\|AX_{1_N} - AX_{1_N}.e_1\|}$
It matters to see that , quite importantly, thanks to the fact that the eigen vectors will form an orthonormal basis, we must find the "next" eigen vector and eigen value in module, since they are "there" somewhere in the space that is orthogonal to the first eigen vector. This is not the case in general because the eigen vectors form a basis but not an orthogonal one.

Of notice: in finance we mostly use either covariance matrices or Hessian matrices for the partial second derivatives. In both cases we speak of symmetrical matrices.
In the general case, for the k+1-th eigen value, we would launch the following sequence:

$$X_{k_{N+1}} = \frac{AX_{k_N} - AX_{k_N}.(e_1 + .e_2 + ..e_k)}{\left\| AX_{k_N} - AX_{k_N}.(e_1 + .e_2 + ..e_k) \right\|}$$

x- Jordan theorem on matrices and diagonal

Let's consider a matrix A that has all its eigen values in $\mathbb{C}$
Let's first show that the eigen sub-spaces must be orthogonal pair-wise.
Consider 2 distinct eigen values $\gamma_1 \; and \; \gamma_2$ along with one unit norm eigen vector, namely $u_1 \; and \; u_2$. We do not know yet whether they are orthogonal. But we are certain that they define 2 distinct dimensions since their eigen values are distinct.
Indeed if they were collinear this would mean that the vectors would be equal and therefore nil since we would have $Au_1 = \gamma_1 u_1 = Au_2 = \gamma_2 u_2 \rightarrow (\gamma_2 - \gamma_1)u_1 = 0 \rightarrow u_1 = 0 \; because \; \gamma_1 \neq \gamma_2$
Note: we could have assumed that they were both of norm 1 but in opposite direction. But then, taking a factor "-1" we would have ended to the same conclusion.

Therefore $Au_1 = \gamma_1 u_1 \; and \; Au_2 = \gamma_2 u_2$ with $\gamma_1 \neq \gamma_2 \; and \; u_1 \neq u_2$
So we can say that the whole space can be decomposed into sub spaces that form a partition of the whole. We can therefore define a basis of eigen vectors across the many eigen sub-spaces that will cover the whole space.
If now we focus on one eigen sub-space of order "p", which means that the eigen value is of order "p" in the characteristic polynomial defined as :

$$P(x) = Det(A - xI_d)$$

Consider now 1 eigen vectors in the sub-space as a starter:

$$Au_1 = Vu_1$$

If now we consider $(A - VI_d)(A - VI_d)u_2 = 0$ and $(A - VI_d)u_2 \neq 0$
It can only means that $(A - VI_d)u_2 = au_1$
Then we can always the norm as $u_2 \rightarrow \frac{u_2}{a} \; since \; a \; is \; not \; nil$
Likewise, expanding to the 3 rd degree
$(A - VI_d)(A - VI_d)(A - VI_d)u_3 = 0$ and$(A - VI_d)\,(A - VI_d)u_3 \neq 0$
It must be that $(A - VI_d)u_3 = a_{31}u_1 + a_{32}u_2$ and
$(A - VI_d)\,(A - VI_d)u_3 = a_{32}u_2$

$$we \; then \; define \; u_3 \rightarrow v_3 = u_3 - yu_2 \; so \; that \; (A - VI_d)v_3 = a_{321}u_2$$

$$we \; can \; see \; that \; if \; y = a_{31} \; only \; a \; term \; in \; u_2 \; remains$$

We then normalize and move to the 4th dimension if there is one… and so on
To be sure here…Let's go one degree higher….We consider initially a vector u_{40} that verifies:
$(A - VI_d)(A - VI_d)(A - VI_d)(A - VI_d)u_{40} = 0$
but $(A - VI_d)\,(A - VI_d)(A - VI_d)u_{40} \neq 0$
We infer that $(A - VI_d)(A - VI_d)(A - VI_d)u_{40} = a_0 u_1$ and therefore:

$$(A - VI_d)(A - VI_d)u_{40} = a_{12}u_2 + a_{11}u_1$$

$$And\ so, (A - VI_d)u_{40} = a_{23}u_3 + a_{22}u_2 + a_{21}u_1$$

We look for a combination of these vectors like:

$$u_{41} = c_{40}u_{40} + c_3u_3 + c_2u_2\ such\ that\ (A - VI_d)u_{41} = u_3$$

$$(A - VI_d)u_{41} = c_1(a_{23}u_3 + a_{22}u_2 + a_{21}u_1) + c_3u_2 + c_2u_1$$

$$The\ equality\ above\ results\ from\ the\ former\ choices$$

$$(A - VI_d)u_{41} = c_1a_{23}u_3 + (c_1a_{22} + c_3)u_2 + (c_1a_{21} + c_2)u_1 = u_3$$

$$We\ therefore\ infer\ that\ c_1a_{23} = 1, a_{22}c_1 + c_3 = c_1a_{21} + c_2 = 0$$

Picking $c_1 = \frac{1}{a_{23}}\ and\ c_3 = \frac{-a_{22}}{a_{23}}\ and\ c_2 = -\frac{a_{21}}{a_{23}}$ we get the appropriate new vector. This can be done through the same approach recursively, once the former degrees are done. QED

This is not necessarily useful in computing because the matrix is non diagonal only if there is at least one eigen value that is not "simple". Otherwise the eigen vectors provide a basis and therefore the matrix is always diagonal.

And lastly the build-up of LLMS and chatGPT in particular that are the gate towards "interpreting" financial markets time series as a proxy language…

From computing to machine learning and AI

This last part is meant to show how from the 1-D techniques, to diagonalization in n-dimensions we land on new issues despite the fact that we may efficiently reduce the problems in n dimensions to n independent 1 dimension problems.

In finance and more generally in the economy, we always use tools to optimize a function. And for that matter we model a "profit function" also called "utility function" under constraints. We customarily reduce the function and the constraints to linear expressions. Thanks to that we can extensively use the simplex method. But even then we have to make assumptions on the lines of "we assume that everything else is held equal…." Which we know is really unrealistic.

So far we would have to make the same assumption all the way: the data that we have belong to one regime that we can approximate well enough with the data that we have. But what would happen is actually our environment was in constant adaptation? What if actually we were never really in a steady equilibrium but we were in the illusion of such equilibrium that was already gone by the time we collected the data? Could we use a linear regression to spot the evolution the best we can? Of course, if the world is changing, like the economy does, like the markets do, we still have constant patterns. But how to flag them? How to make sure that we do not miss something?

Machine learning algorithms started with that perspective with regards to operate "optimal control". Good examples historically were given by the automatic pilots on planes. Other automated "human like tasks" were featured in "program trading". These days the most profitable hedge funds are exactly on that business line: they buy computer for hundreds of millions of $ that can trade in a nanosecond with the organized market place on a future, an option or any other financial instrument that can trade without human instervention.

Another salient example of machine learning is all the self-adaptive software that come to action on back-office but also risk management or automated insurance payments. The most salient example of them is called the "crypto currencies" that are in the process of integrating AI fully since 2022. One less known example of machine learning regards "reading signatures" on checks, or contracts. Else

universities and Human resource departments “profile” candidates based on machine learning too. This is not per se yet what is called “Intelligence” but it gets closer and closer every year fast now. The one most obvious example is ChatGPT, or BERT or else LLMs. In the domain of Q-learning we know how to train a machine that will beat any human being on Chess, on Go or any other typical intellectually intensive game…and therefore any intellectual challenge….

How do they work, all these machine learning algorithms? How do they eventually catch a potential regime change before a human eye could? Could a machine make a better decision on the follow?

Fundamentally machine learning or AI or even the most sophisticated part of machine learning, it is all about classification: are we in the current regime or another one? Is the current sentence alike a former one, or is the context altering the “next word”?

We will review briefly here the transformer model that gave birth to the main LLMs, ie Large Language Models. Of interest though many other types of neuron networks are simpler to implement even though they are less impressive: the MLP (the simplest by far), the Convolution Neural Network, the auto-encoder and the Variational auto-encoder, the Q-learning methods, the LSTM, the stable diffusion models, and finally the transformers.

4 parts:

1- Introduction of the fundamental concept: Claude Shannon (1958!)
2- The transformer model with a focus on the GPT family
3- How to use transformers’ models on time series (short..up to you to develop)
4- How to use transformers models to financial data (short..up to you to develop further)

1- The main concepts: we human copy on and on and…slightly distort

Back in 1958, Claude Shannon published a small book that would be a reference for the decades to come: ‘ A mathematical theory of communication”. There he explained the concept of “entropy of information” to simply put a theory in mathematical terms that was meant to “correct transmission errors on a message”. His problem at the time was that the transmission channels were, and still are, prone to lose some “bits” some “digits” among the billions that they convey from one point to another. Blame the climate, blame the apparels, blame whatever else, the distortion, the pollution occurs potentially at every single nano-second but at random…Does it matter?. His problem was not to prevent that from happening. His problem was: “how to use what we got, that can be considered as “self-consistent” in order to infer with the best likelihood what may have been corrupted and, on the follow, correct it in the most sensible way given the data that we have?”

He based his theory on one key assumption: “ if a message is conveyed and therefore received, it must mean something. If it means something it must be that its contents, even encoded, store some shared knowledge that is expressed in one language. And this language , to be understood by many people in quite diverse contexts and environments as carrying strictly a meaning that is independent of the context in general, this language must be following rules of construction that will show in the watermarks of the message itself”.

This means that there must be “keys” like “letters” or “grammar rules” and “words” too that all band together in “sentences” in a certain “order” that is not arbitrary. A message, for all its flaws and distortion, will for sure be an empirical evidence of all these rules and as such it carries this information at least.

If we want to translate this into today's description of ChatGPT for example, people would say that the transformers blocks in the Large Language Model (LLM) would "learn about the context where each word appear through the sequence that is being analyzed by the neural network…." Ok…this remains cryptic…but we will review these terms in detail and at a second reading you will get it much better…

These rules, that shape the context in the watermarks of any sensible text, have in common that they happen with a certain frequency. And actually they combine into conditional frequencies, many conditional frequencies. Claude Shannon knew that…

How many letters do we have? 26. How many grammar rules, how many verb spelling rules do we have ? Well, many probably a couple of thousands. How many words do we use ? about 50 000. These are the order of magnitudes.

But hey here we start seeing the issue: there are rare rules, rare words and people do not always "speak" well. Who actually speaks any language perfectly? Nobody probably…However, we do our best each time so that we can be understood and therefore the rules certainly tend to appear rather than disappear in any text. This is the working assumption here when Shannon defines the concept of "entropy of information": the more certain to come in a fixed form a message is the lower the entropy of the underlying message is. In other words a simple message conveys little meaning, requires few digits and therefore their small combinations are few and stereotyped. The entropy of information, namely how large is the dispersion of such groups, is small. On the contrary, if the message is complex then the short sequences of digits will be numerous and their distribution will spread through a large spectrum. In physical terms, the expression of message will exhibit an large "exploration" of many possible configuration of digits in smaller groups than the message itself. Then the entropy of information is large, many interpretations coulbe found, and the message itself might be quite difficult to understand.

Of course the higher the entropy is, the more uncertain the future message will be and therefore the richer it will be in "meaning" terms. Behind this dispersion that the entropy of information grossly measures there are rules, there are "conditions". Over many and many messages, going through a given channel of communication, Shannon inferred with his model that the flaws of the channel could be flagged and corrected over time with these sole assumptions and that would "work". It did! Really the idea was to compare short sequences in their spectrum of frequency of appearance, spot the discrepancies, compute the "expectation", correct with this expectation and check back to "see" whether that made sense. Given, indeed, that all these message did convey information, the reconciliation was actually quite simple. But Claude Shannon looked for a solid mathematical foundation for this.

The most interesting part for us here bears on a little experiment he made himself in English. He wondered, noticing that his assumptions fit well with natural languages: "how many consecutive conditions would be needed in order to replicate a meaningful sentence in English but from a random starting point?"

He first theorized a bit on the sequence of letters, considering punctuation and spaces as "special letters", no more. Thus started with an alphabet of 26 letters plus the punctuations and the spaces of different kinds.

Consider a letter "b" that is preceded by another letter "a": what is the distribution of "b" given that we know that 'a' is right before? And he would follow with the case of a letter "c" that would be preceded by a letter "b", itself being preceded by a latter "a"….Could he estimate all this and how so? He started with a simple approach that is quite generic.

He recorded a long text in English. He measured first the probability of occurrence of each letter in the text. He called the letter a “token” ie an integer that was attached to this letter and no other. If you consider the alphabet that he built he may have given the token ‘0’ to a space, the token ‘1’ to the letter “a”, the token “2” to the letter “b” and so on and so forth. He therefore noted the probability of occurrence in a mathematical format:

$$for\ i = 0; 1; 2; 3; \dots n\ we\ have\ p(i) = P_{rob}(letter\ x\ in\ the\ text)$$

Next he wondered of course what the probability of a given pair of tokens was in this text. He noted this probability $p(i,j)$ and he wondered : “are the individual properties of I and j enough to find the probability $p(i,j)$?’. To his surprise he found that “yes almost!”. This means that the letters appeared not randomly so much since the text was “well thought and meaningful”. So at first we would have thought that a given letter “called” the next one, and so on and so forth. When we read a text we would easily admit that the letters must be following a “path” that is fully devoted to the “meaning” and that our language was not built on rules but rathe on some quite empirical tries and errors that led to texts. It would be only later, as our intuition goes, that “teachers”, “parents”, “censors” would erect “rules”and “laws” to entice us into a language that we rather dislike just because of that. But the experiment that comes unearthed another side of the story: these rules are not arbitrary. The letters have their fundamental probability to occur and this is it mostly! Next, of course, it is more complex in the sense that in the context of an “a”, the distribution for the next letter differs form the distribution of the next letter if we start from a “b” instead. So, “yes” the former letters matter, but only from a pure statistical standpoint. A text is written like we play a game actually not like some text was carved in stone thousands years ago and we built aribitrary rules based on that text that everyone should understand. Ok there are rules we all know that but they are not capable of mapping all the variety of texts and meanings that a language can convey….We are smarter than that right?! We rely on a set of fixed distributions…How many of them?

Well it is true fundamentally that human beings are “smarter” than the language that they use and smarter than the consensual thoughts that drive our societies. But , as chatgpt exemplifies so saliently the illusion is almost perfect! It turns out that Claude Shannon discovered that, ie that machines could emulate any known level of human capabilities….faster cheaper ever better than humans could ever do. Without humans machines cannot, not yet, make progress in full autonomy but, as far as human activity is concerned, we can only get understood by others if we follow enough rules. And machines do better and better by the day.This is what the experiment that comes exhibited albeit in a very coarse fashion as early as 1958.

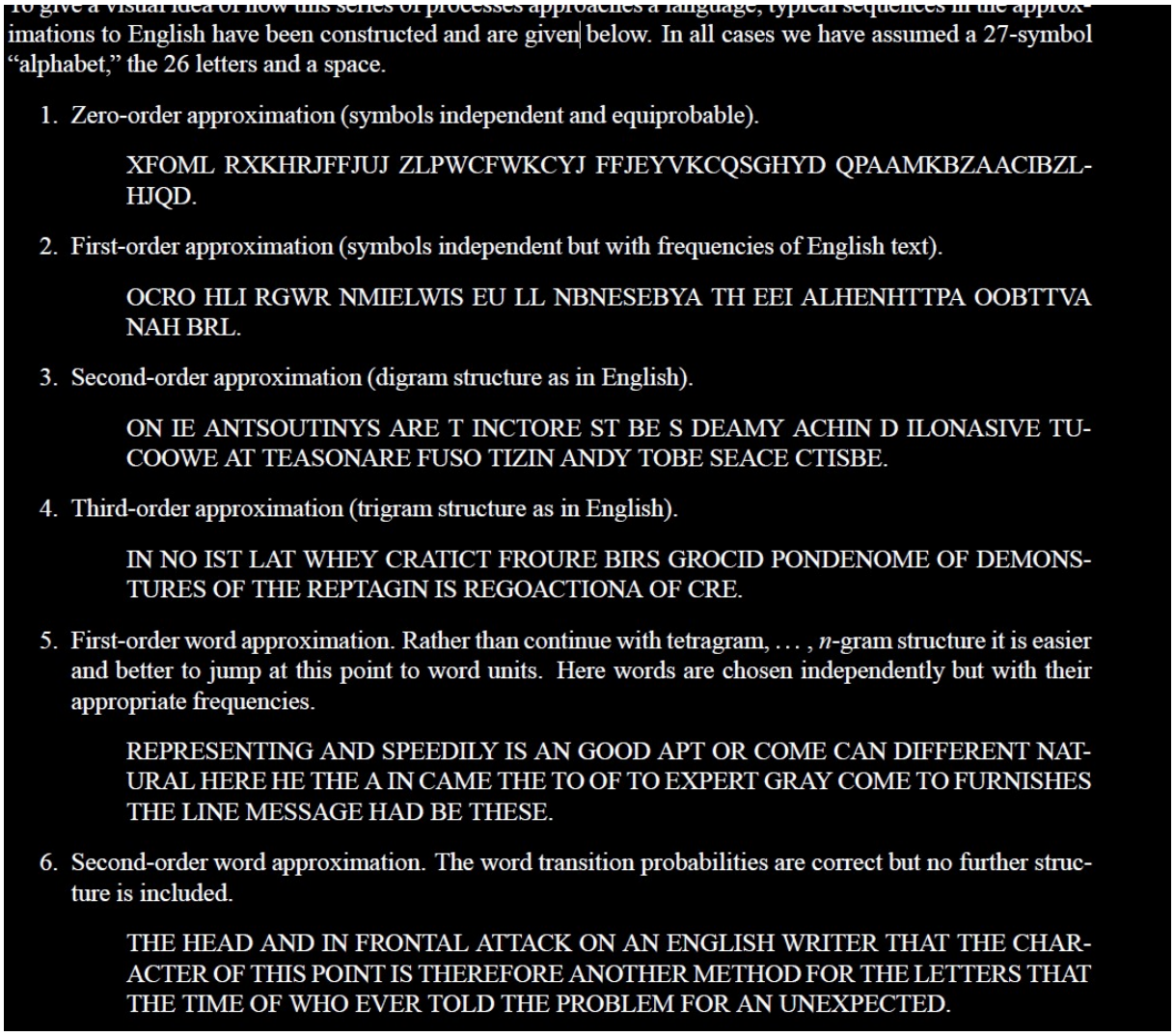
Claude Shannon took the letters, measured the probability of occurrence of each letter (26 plus spaces ..) And here is what he got after including next the probabilities conditioned on the prior letter and finally conditioning on the prior 2 letters one by one, ie never assuming any inherent correlation between letters…..He was half satisfied. So he made the same experiment with words directly…

He realized that to get “words” from letters he would need a lot more conditioning and he did not have, in 1958, a computer that was powerful enough. But we do today if we use python or rust or C++.

What he did is what the designers of ChatGpt and any other LLM will do 60 years laters with machines that would about a billion times more powerful than what he had then, in 1958: he will look at the conditional probabilities of “words” directly. And then, the magic came already in 1958!

This phenomenon is known as a “Markov” process: there is a game underneath the generation of a language, underneath the generation of human thoughts in their most common realization. It is very much like, surprise surprise, playing games between us actually! The rules mix up with a part of randomness that is well contained and channeled. Thus whatever the path that will be followed the “game” will unfold in a meaningful way for all of us despite the fact that we will say “gosh I do not get it! Why me,!” But this is actually all the fun right?! This is what life is all about: ups and downs and recoveries…until we get tired of it.

Our languages do not differ: they follow rules and any word can potentially be followed by any other word. In other terms the probability of occurrence in any context is not specifically tied to the specific words in particular, or this specific context, other than the “next” word follows a distribution that only depends on the “rules”. We learn some of them like the grammar rules. But there are other even more important rules that make most people understand and be understood even when they do not respect “all” the rules. Where the “magic” coming from? There is a magic and stupid rull all the same. This additional “rule” is called “redundancy”: in most sentences, if you remove up to 50% of the words, a person will still get the main meaning that the sentence conveys! From a statistical standpoint this shows “how” a language is effective while it “only” stores a limited number of conditions that are fixed for all the possible texts: the probabilities are repeated through this redundancy, therefore ascertained, confirmed…etc... This is a pattern that make some games quite popular: they use few rules, come through the same point often but the outcome while predictable always carries an element of surprise. What this means for machines is that they just need to catch the rules, especially the redundancy one, and they will sound like us the humans.



To give a visual idea of how this series of processes approaches a language, typical sequences in the approximations to English have been constructed and are given below. In all cases we have assumed a 27-symbol "alphabet," the 26 letters and a space.

1. Zero-order approximation (symbols independent and equiprobable).

   XFOML RXKHRJFFJUJ ZLPWCFWKCYJ FFJEYVKCQSGHYD QPAAMKBZAACIBZL-HJQD.

2. First-order approximation (symbols independent but with frequencies of English text).

   OCRO HLI RGWR NMIELWIS EU LL NBNESEBYA TH EEI ALHENHTTPA OOBTTVA NAH BRL.

3. Second-order approximation (digram structure as in English).

   ON IE ANTSOUTINYS ARE T INCTORE ST BE S DEAMY ACHIN D ILONASIVE TU-COOWE AT TEASONARE FUSO TIZIN ANDY TOBE SEACE CTISBE.

4. Third-order approximation (trigram structure as in English).

   IN NO IST LAT WHEY CRATICT FROURE BIRS GROCID PONDENOME OF DEMONS-TURES OF THE REPTAGIN IS REGOACTIONA OF CRE.

5. First-order word approximation. Rather than continue with tetragram, . . . , *n*-gram structure it is easier and better to jump at this point to word units. Here words are chosen independently but with their appropriate frequencies.

   REPRESENTING AND SPEEDILY IS AN GOOD APT OR COME CAN DIFFERENT NAT-URAL HERE HE THE A IN CAME THE TO OF TO EXPERT GRAY COME TO FURNISHES THE LINE MESSAGE HAD BE THESE.

6. Second-order word approximation. The word transition probabilities are correct but no further structure is included.

   THE HEAD AND IN FRONTAL ATTACK ON AN ENGLISH WRITER THAT THE CHAR-ACTER OF THIS POINT IS THEREFORE ANOTHER METHOD FOR THE LETTERS THAT THE TIME OF WHO EVER TOLD THE PROBLEM FOR AN UNEXPECTED.

Of course this is just a "hint": after just 2 conditions over about 50 000 words that are learned from just one book. Any random generator would create a "text" that we see is not far from "suggesting a sense". Because whenever we read a text, we would instinctively try our best to find a "meaning". How many "layers" of conditioning do we need to be "good"? Answer about 60 years later: "you need about 1024 conditions, not 2", ie 2^{10}! And more details that we will see. However and not so surprisingly, chatgpt can speak ALL the languages provided it could be trained with enough material.

This strongly supports the breakthroughs of ethnologists, philologists, linguists that all exhibited a common thread underneath the generation of any language, that is statistical and based on redundancy.

Over 60 years; most people were in open disbelief that this was "possible"….Others said "well, it is like the blackholes right; on paper we see it but if we really see it we are finished!"

2- The transformer's model in the guts

The transformers model like the one implemented in ChatGPT is based upon an encoder-decoder structure, like a VAE. However, unlike a VAE or an auto-encoder, in a transformer's model like

chatGPT the encoder and the decoder part do NOT store “layers”…They actually compound neural networks themselves. The encoder , alike the decoder, is a series of neural networks that actually combine 2 neuron networks. The part that we see in this paper is the “encoder” that is made of so-called “transformer” blocks. With an encoder, once it has learned the “context” as we pictured it in the former part, we can “plug” different decoders: one would generate a “prompt”, then a “chat” for an “interactive talk”. But the decoder could also be a “translators” and even a summarizer. In any even, encoders and decoders compound “transformers blocks” that in essence themselves are “a network of networks”.

It is noticeable that all the main public achievements of AI so far involve a “network” of networks. But this is mostly caused by the will to hit the public opinion. So, anyway, encoder, decoder, chatGPT, BERT and the likes are networks of transformer blocks. Such encoder or decoder compounds “transformer blocks” (8 or 16…) that are placed in series: one transformer block receives the real life “inputs”, produces an “output” that will be the “input” of the next transformer block and so on and so forth. Very briefly the former transformer block feeds the current transformer block. The first block received the real life inputs. The last transformer block provides an output, and some some “queries” that will be used next by a “decoder” in order to process a “next word” in the context of the same language, another language, a summary a development etc…. This is the decoder that will need training then, using also queries, values in many transformer blocks too. The difference mostly lies in the fact that the first transformer block will borrow the queries of the last transformer block of the encoder.

In one standalone transformer block we have several “heads” (like 8 or 16) that each tackle one facet of a text or a sentence of some 1024 to 2048 words. So the description below mostly focuses on just one transformer block and one head in this block most of the time. We do observe power laws of 2 due to the scale of the model that requires that all the numbers are digital among other things….

The transformer block itself is made of a “transformation” part, a ‘loop part’, a “normalization” part, an “MLP” part, a “loop” part and one more “normalization part. Typically the sequence is either “transformation ->loop->normalization-> MLP ->loop-> normalization” or “normalization-> transformation->loop->normalization->MLP->loop”.

The principle is that a word is known as a “token” ie an integer to which some “coordinates” are known: we call them the “embeddings”. They are of 2 kinds “meaning embeddings” and “positional embeddings”. All the words/tokens share 3 common matrices: a key matrix K, a query matrix Q and a Value matrix V.

Below comes a short explanation about “how” the matrices are theoretically expected to indeed capture the meaning of a word that happens to appear in a given position in a given sequence: do not forget that if we consider a token it often has 512 coordinates (or 1024) to picture its meaning in a certain “optic” and 512 coordinates (or 1024) to picture its position in a sequence of 512 words (or 1024). This number of “512” is just an example. These positional embeddings, one for each “head” in one transformer block, actually are sine functions that picture where in a cycle a word appears relative to all the others in the same sequence. You have to “see” here that in the operations that will be explained below the positional embeddings have an impact. Thus a given word like “the” will have “meaning embeddings” that would not change between 2 different sentences, but its positional embeddings will. If we pause on a word like “the” and we look at it relative to a word like “beef”, we see that “the” may refer many things: it has a very abstract meaning, which is not the case fo “beef”. We can see here why the “positional” embeddings for a word like “the” are crucial in a sentence

where “beef” is also present. Because “beef” is going to convey a meaning that is unlikely to change whatever its positional embedding but “the”, depending on its positional embedding relative to “beef” is going to greatly enrich the sense that one can extract from this sentence. And If “the” has a different positional embedding then the meaning might, or might not be different and, more, there might be a subtlety that would only be suggested through the position of “the” actually…leaving the door open to many competing interpretations, which in many cases readers actually enjoy. More, the other words, will actually have a role with their own positional embeddings. As a result, although the explanation focuses on one token ie one word in a sequence where the all the words are computed in parallel, the end result is that all the positions relative to all the words are also factored in. This comment sounds a bit cryptic for now. But it will be complemented later on: just bear in mind that positional embeddings are very important.

First, here is a short summary set of the formulas that compute the “Attention” ie permit each word to sort of “self-identify” in a sequence: in that respect this word here will catch the words that are the most important to this word in the same sequence: this word is itself identified by the token “x” that uses the matrices Q, K and V to set its “key” but also its “query” over the many “keys” that are in the sequence, generated by the other words, in order to process the “Value” as a distribution among the many words of the sequence. In the end the word, token “X”, will issue a “pseudo meta word” that will have the same number of embeddings than the native token “X” but with different values.

The word/token “X” will have weighed all the possible “next word” that is the most relevant to itself. It is colloquially said that the word “attends” to the other words of the sequence, spot the most “probable ones” and choose one in one way or another. The concept of “attention” relies on the fact that this “distribution”, implemented through the SoftMax function after the token X query questioned all the “keys” in the sequence, will emphasize only a few words in the current sequence as “worth of attention for this word here”. This has the consequence, down the line towards the next transformer block to have every word “focus” on the meaning and position of the “ words that are friends to that one”.

Here’s a final set of equations for computing self-attention for a single self-a output vector $\mathbf{a}_i$ from a single input vector $\mathbf{x}_i$, illustrated in Fig. 10.3 for the calculating the value of the third output $\mathbf{a}_3$ in a sequence.

$$\begin{aligned} \mathbf{q}_i = \mathbf{x}_i\mathbf{W}^{\mathbf{Q}}; \mathbf{k}_i &= \mathbf{x}_i\mathbf{W}^{\mathbf{K}}; \mathbf{v}_i = \mathbf{x}_i\mathbf{W}^{\mathbf{V}} \\ \textit{Final verson:} \quad \text{score}(\mathbf{x}_i, \mathbf{x}_j) &= \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} \\ \alpha_{ij} &= \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \;\; \forall j \leq i \\ \mathbf{a}_i &= \sum_{j \leq i} \alpha_{ij}\mathbf{v}_j \end{aligned}$$

In order to “visualize” what the “heads” mean, the following text summarizes once more how the matrices line up in parallel within one transformer block. Bear in mind here that the embedding and the QKV matrices are all adjusted together during the training. The existence of the many “heads” means that the “convergence” will vary from one “head” to the other. The point will be to generate a competition between the many heads during the training. The way it is achieved is through

initializing the matrices and the embeddings in a “mapping” manner so that the many heads start as remotely as possible one from the other.

In multi-head attention, as with self-attention, the model dimension d is still used for the input and output, the key and query embeddings have dimensionality d_k, and the value embeddings are of dimensionality d_v (again, in the original transformer paper $d_k = d_v = 64$, $h = 8$, and $d = 512$). Thus for each head i, we have weight layers $\mathbf{W}_i^Q \in \mathbb{R}^{d \times d_k}$, $\mathbf{W}_i^K \in \mathbb{R}^{d \times d_k}$, and $\mathbf{W}_i^V \in \mathbb{R}^{d \times d_v}$, and these get multiplied by the inputs packed into $\mathbf{X}$ to produce $\mathbf{Q} \in \mathbb{R}^{N \times d_k}$, $\mathbf{K} \in \mathbb{R}^{N \times d_k}$, and $\mathbf{V} \in \mathbb{R}^{N \times d_v}$. The output of each of the h heads is of shape $N \times d_v$, and so the output of the multi-head layer with h heads consists of h matrices of shape $N \times d_v$. To make use of these matrices in further processing, they are concatenated to produce a single output with dimensionality $N \times hd_v$. Finally, we use yet another linear projection $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d}$, that reshape it to the original output dimension for each token. Multiplying the concatenated $N \times hd_v$ matrix output by $\mathbf{W}^O \in \mathbb{R}^{hd_v \times d}$ yields the self-attention output A of shape $[N \times d]$, suitable to be passed through residual connections and layer norm.

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_i^Q \; ; \; \mathbf{K} = \mathbf{X}\mathbf{W}_i^K \; ; \; \mathbf{V} = \mathbf{X}\mathbf{W}_i^V \tag{10.17}$$

$$\mathbf{head}_i = \text{SelfAttention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \tag{10.18}$$

$$\mathbf{A} = \text{MultiHeadAttention}(\mathbf{X}) = (\mathbf{head}_1 \oplus \mathbf{head}_2 \ldots \oplus \mathbf{head}_h)\mathbf{W}^O \tag{10.19}$$

For now we have just reviewed the main tools of the “attention process”: 1- a word generates a query with the matrix Q; 2- this query will “question” all the keys of the sequence that will have come from all the words using the matrix K, 3- a scalar product of the query with each key will generate a vector values; 4- this vector is morphed into a distribution of probabilities with the softMax function, 5- an algorithm decides to pick or many of these probabilities, build a vector that will be used with the value matrix V in order to generate new embeddings for what looks like a “word/token Y”. This process is done for each “head of attention”. The many “new words” are added up together and next are added, through their embeddings to the initial word. All this can done in parallel for all the words of the sequence, a thing that is very powerful in computation terms.

What is the “intuition” behind all that?! Where is this idea of using “embeddings”, “heads”, “query”, “keys”, ‘values”.

Some intuitive bbackground behind the Attention mode, the Query, Key, Value matrices…

Let’s now dive into the mathematical structure of the transformer block from a “meaning” standpoint mostly. Consider as a first step that we only speak of what we commonly understand as “meaning”, ie the definition in the dictionary. Let’s recall here if we lookup for some words like “the”, although they do not mean much per se, the definition is going to be quite lengthy and almost unsettling: after we have read it all we often think that we do not know at all what this word actually “means”. We emphasized already the importance of the positional embeddings in that regard. But we emphasize more the fact that a “meaning” may be much more than just a definition. The explanation below works for the positional embeddings too but it is more abstract. Say we have a sequence of n words that each are described by “m” embedding coordinates. We therefore would

define a matrix V having n lines with m columns: $V = \begin{bmatrix} v_{11} & v_{12} & ..v_{1n} \\ .. & .. & .. \\ v_{m1} & v_{m2} & ..v_{mn} \end{bmatrix}$ where each column is one word vector made of m embeddings. As we described before, for each word an attention coefficient, dedicated to a given word, will be generated against every one of the n words, including the word itself that originated all these attention coefficients. Thus the attention vector will have n coordinates, one for each word that is "seen" from the standpoint of a given word that "queries" all the n words through its key and their keys..

Let's now note the attention vector as follows: $a = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \\ ... \\ a_n \end{bmatrix}$ displaying as a column the "n" attention coefficients that reflect the affinity of a given word with the "n" words of the dictionary.

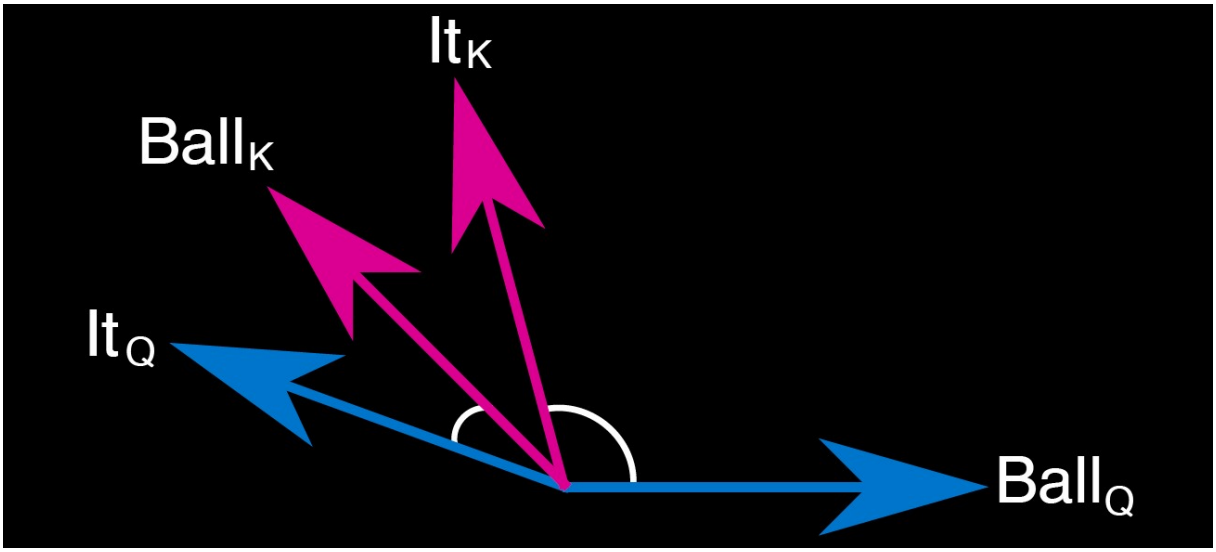
We can then conceive that the "next word" is going to be, at one ultimate stage, some weighted combination of the n words modulo the coordinates of the attention vector through the formula: $Va = \begin{bmatrix} v_{11} & ... & v_{1n} \\ ... & ... & ... \\ v_{m1} & ... & v_{mn} \end{bmatrix} \begin{bmatrix} a_1 \\ ... \\ a_n \end{bmatrix}$ that is going to produce a pseudo_word having "m" coordinate that are its embeddings. It is only a pseudo-word because the Value matrix is expected to give a specific "value" to every word… But the very existence of the value matrix V is precisely to predate the fact that attention might favor a word that has its embeddings already but that also would appear post the attention process as mixed up with other words. So the value matrix is a tool that permit the model to "fall back" on the native embeddings…but not in full may…for what matters the most is that the subsequent transformers block get what they need. And what they need may not be what we needed initially to generate the embeddings of the words. Think about it: if say "dog" has value 1 and "cat" has value 0 and this lovely attention thing tells to watch after "0.6666"…what do you do? You try to melt 2/3rds of a dog with 1/3 of a cat….in terms of "meaning" that will get soon Sci-fi. And then which was the "way" through which we would have obtained a quite typical outcome? Would it be "better" to simply choose the dog here? This is the whole point of building a "distribution" with the softmax function where we had rather obtain a "score" that would be "higher" for the dog than for the cat…Then the choice becomes much more intuitive: we would automatically favor the word or the words that are the most probable without butchering anything in the process. This is why the matrix V conveys "values" indeed that are midway between the native "meaning" in the embeddings and the "self-attended" meaning that the transformer block requires for its own self-consistency. Typically, to the great dismay of those who like cats but not dogs, or the opposite, the machine might care most about the bugs…

Let's observe that here we express the case for one word of the pack relative to all the words of the dictionary. This is the principle. But In the encoder, the attention will focus solely on the other words of the sequence. It is the decoder, tasked with finding the "next word", that will blend the query of the current word with the keys and values of ALL the words that are present in the dictionary. As we can guess, the decoder might have to be a butcher at one stage…More the decoder, unlike the encoder that would be trained with full sequences, the decoder would only use the "past" words to predict the next one…So, the decoder, in order to train itself, would only use the words in a sequence up to the word "n-1", if the next word if the word "n". In contrast the encoder would use all the words for the generation of the keys and values. But, in the end, the error function for the decoder is focused on the very last word 'N+1' once the decoder has used all the former words in the sequence. So the decider deliberately blinds itself about the "future words" in the sequence, but only judges its

efficiency from its ability to predict the “word after the current sequence”. In contrast again, the encoder would train itself by using all the keys and minimize the error rate about predicting the “next word” across the full sequence. Given that the “next word” at a position “k” happens to be known as the word “k+1” in the very same sequence, this is why we usually say that the encoder learns the “context” while the decoder really learns, for example, to “generate text”.

So the value matrix of the decoder will differ in essence. The purpose here is to make the encoder “encode” the specific context of a given word and that can only involve the words that are in the sequence since they are precisely the elements that define the context. So, the encoder “learns” the context by relying on the embeddings of the tokens, the shared matrices Q,K and V and will transmit its findings after consecutive “transformer” blocks will have indeed “attended to words that were attended to words coming from the former transformer block…the latter having itself used “words” that either were native “inputs” or “outputs” that resulted from the attention process of the former transformer block.

The chart below exhibits the effect or running scalar products between keys and a given query specific to a given word, based on the embeddings of this word in particular. We can see that depending on whether the word “it” is the one querying the word “ball”, or , from a different context, it would be “ball” querying the word “it”, the scalar product would give a different value. We see that “it” through the matrix Q, has a “query” vector. The same word “it” through the key matrix would have, while using the same embeddings, delivered a key vector that would be very different. As a result in general “it” will NOT attend much to “it”…or if it does, there is a very high chance that another word in the sequence will attend much better, ie deliver a much higher value through the scalar product. Cats and dogs…Note here that we do not consider the positions so much: we just take the “meaning embeddings” and say that, through the texts, the neural network will learn that a word rarely repeats itself. The chart shows “why” we have a Query matrix on one end and we have a Key matrix on the other end.



Let’s extract a little more intuition of the process and “why” we would need many heads, ie “directions to interpret the context itself”… The graph above exhibits one key aspects of the “transformers” and the “attention” concept: a word like “ball” would be quite similar for its key to a word like “it”. The reason is simple: a ball is a “it” somewhere in the text quite often. However “it” may refer to many other names in the “query” world. Similarly if we think about the antonyms “hot” and “cold” they are both adjectives but they represent opposite effects. Thus, on one perspective they are quite close, in terms of them being adjectives that tend to provide a complete world of

descriptions since they are antonyms. However, as to their meaning they are opposite. Thus they might appear opposite one to the other under a certain context and quite close in another. This is what motivates the different heads: “it” and “ball” are grammatically different and so 2 heads may suffice. However, “it” is so generic and ubiquitous that if we want to “find” the right associate here we probably need to run many “heads” in parallel in search for the “best contender”. More, with “hot” and “cold” we see that, being grammatically identical, the “search for the best contender” would not work…hence the need for more heads.

But as we can guess our languages have to remain simple and efficient: we struggle in reading abstract texts precisely because we often struggle to secure what the use of this word, here in this sentence, meant like, here in this paragraph…. This is the case for philosophical books. Likewise somehow but in an opposite fashion, we would struggle to read maths. This time the issue is not the many possible interpretations or angles to a given word that is the issue. It is in fact quite the opposite: the text is certainly abstract due to the many symbols, signs,neologisms, definitions, conventions that govern the text so specifically. Here, if the math text is really good, there is only one possible interpretation….but we often cannot even comprehend what this interpretation might be… Common language speakers position themselves as far as possible from the 2 antagonistic extremes, even favoring slang, colloquialism and oversimplifications. But this arbitrage “works” because many “angles” remain possible and “efficient” still, explaining in a different way why “redundancy” is such an efficient rule.

As a way of summary, we see with the example of “ball” vs “it” or “hot” vs “cold” that the key and query of one given word/token X convey different perspectives relative to any given word: the key carries the meaning while the query will uncover the influence of the context that a given word comes across. We remember here the experiment of Claude Shannon in 1958: he showed that the sequencing of words were close to Markov processes. As a consequence, it makes sense to assume that a “query” matrix coupled with a “Key” matrix might actually lead the way to conditional probabilities. Actually these matrices are more abstract than that: their efficiency, proven today with ChatGPT among others, shows that all the words follow the same “abstract rules” of setting the context for the other words locally in a sequence. If we want to “unearth” these shared abstract rules we only need to split the “embeddings” of each word relative to the others. More, the example now of “hot” versus “cold” uncovers the need to approach the meaning of a word at least from a pure grammatical standpoint and from pure semantic standpoint a well….at least. This pair of examples illustrates the need for several heads of attention in parallel, each one offering a dual view over a word for the role it conveys in a text. On top of that we can identify the syntax that takes into consideration the “style” or more generally the level of language.

We saw through the comments on philosophy or maths texts that with each one of the heads we cannot hope to reach a great level of abstraction and therefore “meaning” in essence. A head will take an “angle” and of course it will be coarse one, not a subtle ambiguous one…so boring, boring….However with the many heads we generate a melting pot of frequent interpretations that, for one at least would make sense to a reader. So the many heads permit to generate a text that is “normally redundant”, providing the reader a “way” to give a sense to this text autonomously. Again we do not like “sharp texts”, be that on the high level conceptual sense or in the mathematical sense: they are hard to decipher. We by far prefer to read “inspiring” stories, ie text that “clearly speak to us” so that we do not have to question ourselves too much on our ability to “get it”.

We might consider as well the punctuation and more generally the sentence structure, the paragraph structure. We notice that a prior training is required so that each head starts form a differing perspective before they are put in parallel for the final training stage. As it was explained the

inception of the many heads will be as far apart as possible, from a pure topological standpoint, and the point is to secure an early convergence towards configurations that differ a lot from one head to the other. Then the heads will be merged together into what will be the transformer block that we have been studying.

We need to review now “how” the attention coefficients are processed.

Let’s therefore imagine that we encoded the words as “tokens”, ie a series of digits, that represent intelligence on every word in terms of meaning, grammar, syntax, style, position. They have in common that they define the context of an expression of a language that must have some inherent way of producing “sense”, ie something that others will get when they read the sequence with some ambiguity but not much in general. The context is all and the make up for the rest. In order to crystallize this “context” for this language we would look to fit 3 matrices: 1 for the keys, 1 for the query and 1 for the value. Intuitively again, the key matrix and the query matrix belong to a certain universe of formal relations that carry little outright meaning other than self- consistency rules to which all the words would abide.

All this model stands a chance to be “generally successful” because the generation of words in a language follows a Markov process that, as in popular games, relies on a limited number of rules to consistently generate a very large variety of scenarios that all would “potentially make sense”. The “path” followed would only matter in that it will shrink the range of probabilities to very few contenders, the most likely among them being generally a “valid follow up”. As explained before if a text that is not at least a little “ambiguous” ie open to different interpretations, it becomes a “boring” an “abscond” text: no one cares, no one looks forward to it. Instead texts, like the bible, that mostly talks in “types”, “classes”, “hyperboles” where anyone can “see” whatever anyone wants to see, become often famous…famous for their implacable ambiguity. This is at odds with “numbers” and “n-th digit accuracy”. This is all about classes and in every classes there is some randomness…

If we chart a typical basic Markov process as Shannon pictured in 1958 we would get the chart below:

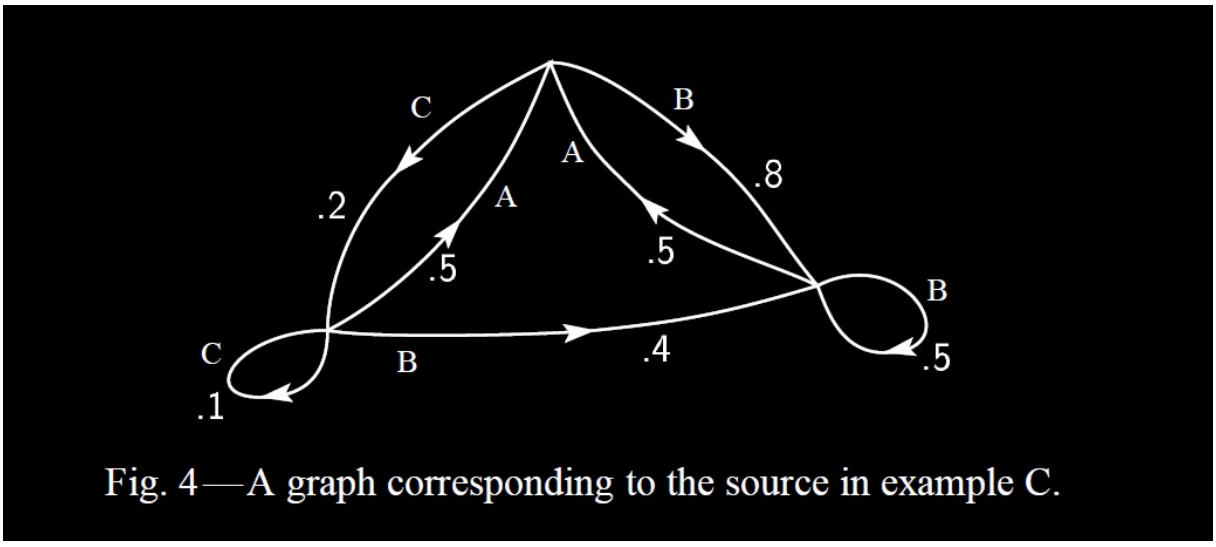


Fig. 4—A graph corresponding to the source in example C.

We can see how a “node” , ie a token ie a word, can attend to itself, or 2 other words with a permanent set of probabilities….given the context… Weighing together all of them in the context of all the other words being present in a given sequence, one becomes “most likely” than others. Picking greedily the most likely for every word and training the network to get the “next word” right by this greedy choice, it happens to “work”. The Attention calculation is here for that purpose: draw

a similar chart among 1024 words in a sequence that would “shift” every word in every position to process the “sequence shifted by one word”. The matrices Q, K and V are here to secure that, within the numerical vectors that we have with the embeddings. So it is just “as if” a higher level “game” was being played here since the very same matrices have to “work” for other sequences containing other words or the same words but in a different order.

In the attention process that we describe now, we saw that the Query matrix and the Key matrix generate embeddings related vectors and the attention will use the scalar products of these vectors. The Attention is almost done: we will use the different probabilities that we get with the SoftMax function in order to generate an “attention” score to each word of the current sequence. Usually a greedy algorithm will pick the highest attention value as the “next word”. But this will induce a rather boring, overly consensual text. It is preferred to pick challengers and randomly select one of them among those that have the highest attention scores anyway. And it “works” even better for us, making CHatgpt capable of grasping “humor” or other emotions.

So the key and query matrix are “words” focused. In contrast the value matrix concentrates the meaning of every word relative to the others through some “new generation embeddings”. If now we want to discriminate the respective roles that are being played then by the key on the one hand and the query on the other hand we should think of the key as a token for the grammatical role of a word while the query had rather focus on the syntax. The Value would be here a pure abstraction of the current context saying somewhat: “ hey it depends on the context, let me see….”. There is no much “meaning” in all that except may on the “shift” that will occur through the embeddings in entry and the embeddings in exit.

But this is only the “intuition” because in practice the several heads are actually devoted to unearth orthogonal aspects of the language as we said before but in a pure numerical perspective as far as chatGPT is concerned. So if we adjust the intuition to say a triplet key-query-value that is devoted to the “meaning” side of the context handling, we then should see the “value” as indeed the meaning carrier. Again, like “cats” and “dogs”, the value matrix V convey a shift in some “sense” that the former embeddings carried with them natively. As to the “key” it would refer to a certain class of associated meanings while the “query” would feature the relationships that may exist between classes of meanings. Thus in general the “value” conveys a “sense” in certain facets of what defines the context, the “key” defines the class for a word or, rather its coordinates among the many possible classes that are just “types of sense”. Ultimately the “query” part features the bridges that one must find in a sentence between the classes , ie the “types” of significations.

Thus these 3 matrices, for any given “head” ie topic that sketches the context as a whole from a pure numerical standpoint that is just “orthogonal” to the others, define the shared knowledge of the words interacting together to convey some abstract “meaning” that anyone would more or less grasp in the same way.

So we are going to make a little roundabout with the notations related to the attention coefficients.

Let’s therefore consider the “n” words in their native embeddings having “m” dimensions while the value vectors will have “n” dimensions as per the former formulas. We are now to apply the transposed version for the attention vectors and consider a treatment in parallel for all the words taken altogether.

Let’s call M the matrix that gathers the n words having each “m” coordinates as columns. Remember that the words are columns in M now. The purpose of this transposition is to go deeper in the understanding of the mathematical operation that indeed can be parallelized.

So M is some m x n matrix. K is some d x m matrix. Q is also an d x m matrix. V is some m x m matrix. Thus we would consider an "d x n" matrix KM for the generation of the keys, likewise an "d x n" matrix QM for the generation of the queries. To be clear here: the words have "m" embeddings and they will be used to produce for each word a "key" of "d" coordinates and a query of "d" coordinates too. and therefore an "m x n" matrix VM, ie n columns, and n lines giving the m coordinates for each word. In this format the attention process will generate an attention matrix A where each line now will correspond to one word, picturing "n" attention factors. Thus the attention matrix will be an n x n matrix.

Then looking at each word individually it has a given "query vector" of "d" coordinates. Take for example the word in the sequence that has rank "k": its query vector is the "k-th" column in the matrix QM. Let's note this vector Q_k.We will generate the attention vector of this word ranked "k" in the sequence by, first computing a first "line" vector from $Q_k^T KM = (\ z_{1_k} \quad z_{2_k} \quad z_{3_k} \quad \cdots \quad z_{n_k}) = Z_k$(ie every word in the sequence has a key, that is multiplied with the query of the k-th word, and that generates one coordinate. In total we have n words and therefore n coordinates on a line now). We can run this calculation for all the words in one go actually: since we picked here the column k to morph it into a line, we realize that we can generate the same kind of line with taking QM rather than just the k-th column of QM. This is made possible because we will "transpose" the whole matrix ie moving all the columns into lines for the generation of the "attention lines".

Thus all the words are used through the following formula: $(QM)^T KM$ .We then would obtain an n x n matrix, each "line" being representative of the word in the k-th" column of the matrix M originally will generate the k-th line in the n x n attention matrix A. Using the fact that $(QM)^T = M^T Q^T$ we can reduce the formula above to: $M^T\ (Q^T K)M$ which defines a nice quadratic form. But our original convention was that "words are columns" initially and we end up with "lines". What can we do to preserve the convention?

Initially we could have run the whole process the other way round…Rather than transpose our "query vector" for the k-th word, we could equally keep as a column but multiply it on the left by the transpose of the "key" matrix namely KM. Thus we would have generated: $(KM)T\ QM = M^T K^T QM$.

But, and this is the convention in the most well known papers these days, we could also opt to say: "the words are lines , not columns". And then the formula becomes $M(QK^T)M^T$ We then obtain with $A = QK^T$ some n x n matrix that for each word will provide a vector-line of n coordinates for the attention for this word in particular. So we end up with the same profile for the core attention factors: the k-th word has "n" factors on the k-th line of the attention matrix A. The numbers are "gross" for now ie they can be anything in numerical value terms. What we really care about is to give preeminence to the highest values since they would flag the best "alignment" as it was explained before. The more 2 vectors are aligned, the higher their scalar product is going to be. Before we use the "value" matrix to actually process the "most likely embeddings for the next word" we need to "scale" these gross attention values in a convenient way. This is why we will use hez SoftMax function: this function takes any series of real numbers and morph the series into a distribution where the highest numerical values will carry the highest density of probability. Applying this normalization first and using the normalized matrix values with the value matrix V we will obtain for each word a value vector of "m" coordinates, but in a transposed format now. Let's see "why" this transposition happens…

Take again now this word of "m" coordinates in the matrix M that, by convention now imposed by the publications, is a line. This means that M is some n x m matrix. TWe can briefly "explain" why this convention is chosen: most of the times we would obtain "data" as lines having "m" coordinates.

Thus the “database” typically appears as a very long table having say millions of lines and much fewer columns for the embeddings. This is what justifies this convention here.

Then taking the line “k” in M we produce a query that is a line (not a column) of “d” coordinates, Q being an m x d matrix now. K also is an m x d matrix: m lines and d columns. Thus MK will be some n x d matrix. We would generate the gross attention values through the product $Q_k(MK)^T$. MK is also an n x d matrix and therefore its transpose is an d x n matrix. As a result the product $Q_k(MK)^T$will be a vector line of n coordinates that correspond to the attention gross factors (before morphing them into a distribution) of this word that therefore ranks in the“ k-th” position within the n-words-sequence that is analyzed here. We also have a “value” matrix V that follows the same rules: the k-th word lands on thek-th line and it has “m” coordinates like M_k. The idea here is that the attention coefficients, once normalized, should point to a “word” ie a “line of m coordinates” that should look like the original words of the sequence.

The matrix as such V is some m x m matrix. So, the matrix MV is some n x m matrix where each line flags a given word among the others….while the attention vector is a line of “n” coordinates being typical of the most likely words that the word “k” should pay attention to. So, if we want to see again how “aligned” the word k is with others we need to use directly the value matrix. Then we will generate a series of “m” alignments of this word “k” relative to he n words that constitute the sequence. Thus running the product: $Q_k(MK)^TMV$ we will obtain a new “line” having, m coordinates. Of course the matrix V will also be trained alongside the embeddings matrix, the Q matrix, the K matrix so that its own components are properly adjusted with the others. We’ can understand better now what matrix V is doing: it predates the twisted rotation and sort of “re-calibrate” so that the ultimate “word” is a usable word for the next stage. We can see here that although V looks like a matrix that generates a word from another word, it actually mostly serve as a rebalancing tool with regards to the filter that the attention process has engineered: we will pick one or a couple of attention factors only and the combination is more like a meta word that captures a bit of the context of the next word. We can see as well that this twist in the native word will actually be added to the native word itself alongside the outputs of many other heads through a basic mathematical “sum”. There is nothing sophisticated here. So, aiming at producing a “word” in the end, we see the value matrix here for what it is in reality: it generates a “shift” ie a marginal evolution of the native word, much more than a “new word” alone. This observation flows directly from the fact that we would add the 8 to 16 similar “new values” across the m embeddings to the native word before normalizing all this! There would be very little logic in adding 16 genuine words numerically before normalizing the total brutally and hoping that this would provide some monster that hopefully could be a word ultimately. But there is a complete logic in seeing he matrix V as a matrix a rather small values compared to all the native embeddings, given then the favor to this word in this head, and to this other word in this other head, and adding up all the drifts to infer what is a “monster” indeed but a more natural monster compared to the native word. Indeed, this new “meta word” may not be readable for us but it would feature its context with a wealth of numerical details. We can interpret the consecutive attention blocks as compounding subsequent “observations and rotations” of the very same native word aggregating the context of a context of a context….a certain number of time while never going far away from the original “next word” in fact.

We observe here that, again, we can pile up all the words as lines and therefore generalize the expression as follows: $[MQ(MK^T)]_{norm}MV$

One last “detail” that actually matters a lot: the usual way the easy descriptions of the model do all this is by summarizing the model by removing “M” assuming we always speak about one word taken in isolation….However we miss the fact that the words in the same sequence are going to influence

the values of the matrices MQ,M K and MV line by line…This is the very astute feature of the attention models: they run words in parallel, which is a very efficient way to address the dimensionality problem in terms of computation time, and yet they also factor in the sequencing of the words thanks to the positional embeddings.

The standard simplified known formula includes a softmax function: $SoftMax(x_i) = \left\{\frac{e^{x_i}}{\sum_i e^{x_i}}\right\}$

$$Attention\ (Q, K, V) = SoftMax\left(\frac{QK^T}{\sqrt{m}}\right)V$$

The formula above exhibits a $\sqrt[2]{m}$ that is a safety factor meant to avoid numerical overflows. We only care here about the highest values, not that one probability would have to be really high. Please remember that the embeddings are going to be "normalized" too through the training. We will see later "why" this matters actually a lot.

Embeddings look alike byproducts of some PCA regression analysis. Actually we are closer to a convolution product…

An interesting aspect of this approach is that the attention calculation is a simplified form of a convolution kernel. This remark will likely induce the design of much better models than ChatGPT in the future. Note that PCA analysis subsequent projections to define the embeddings is not incompatible with the convolution products. Indeed let's consider the attention coefficients as being the components of the convolution kernel that will express this word in the current context through the fixed matrix of values (in a transposed format still):

We have the basic expression for a given dimension (among the m ones in the space of values Let's assume that we focus on one word and that we have processed its attention factors relative the words of the sequence. We are now processing the m embeddings that emanate from the Valur matrix. Then we pick the k-th that is:

$$[a_1\ a_2 \dots a_n]\begin{bmatrix} v_{1k} \\ v_{2k} \\ \dots \\ v_{nk} \end{bmatrix} = a_1 v_{1_k} + a_2 v_{2_k} + a_3 v_{3_k} + \cdots a_n v_{n_k}$$

$$= Val_k \sim \int f(u) convol(x-u) du \ \ for\ the\ word\ in\ question \dots$$

Now, since we deal with "n" dimensions in parallel, we would speak of a "convolution kernel" that would be a matrix of some sort, where the convolving density (ie the "function" that is used alongside the attention factors) could be specific to each dimension actually. To be sure here: the computer runs a sum of products in a discrete number of times, like we sum "n" terms here to get each of the "m" embeddings that derive from the value matrix. But we could consider this sum of "n" terms as the integral housing the product of 2 functions that are piece-wise constant. We realize here that the "convol" function is shared through all the many words and their associated attention vectors of "n" coordinates. So, with this modeling based on convolution kernels, here the attention model applies one attention set of factors throughout the "n" dimensions of the V matrix common to all the words. Which is what here? The values could be labeled as a distribution too for this would not impair at all the ability to segregate the words on the follow.

The convolution approach seems better suited to the role that the value matrix is expected to play: it would smooth finely the possible randomness and increase the generality of the fitting. The attention model is, in that perspective, a very gross approach.

Thus, if now we consider again running all the words in parallel, either we do the attention model and the "convol" function is the same for all the points….Or we define a genuine convolution kernel where there is a value matrix for each word. The attention model offers the advantage that, sharing the value matrix, some long term dependencies between words are captured because the value matrix does not require too much resources. But the "re-balancing" that we explained before is rather gross each time. On balance, the convolution kernel approach will refined much better the adjustments of the current context but it would require many ore parameters and a more delicate training. So we could summarize the remark like this: if we note $\begin{bmatrix} v_{1k} \\ v_{2k} \\ \dots \\ v_{nk} \end{bmatrix} = V_k \ \ and \ A_{\dots}^T = [a_1 \ a_2 \dots a_n] \ then$

$$\begin{cases} A_{\dots}^T V_1 \\ A_{\dots}^T V_2 \\ \dots \\ A_{\dots}^T V_m \end{cases} while \ we \ could \ define \ actually \ \text{n} \ distinct \ vectors \ of \ attention$$

This summarizes the pros and cons between convolution networks and attention based protocols: the former are more refined BUT given the recourse to square matrices they have a limited ability to run on long term dependencies between words OR they mandate a correspondingly low number of dimensions in value terms if we do not want to be overwhelmed by computation issues. The latter, the attention based process, is not bound by "n" or even "m" BUT it forces one size fits all attention parameters across the dimensions.

Yet if we assumed that we had enough computing capabilities, nothing would preclude the application of some convolution network calibration for the value part AND for the query part and for the key part…which would induce many more keys and query matrices actually, one per dimension of the value space actually. And this is how we can grasp once more the interest of having even more heads of attention running in parallel and heading into actually the same common matrix of values. We then would NOT any longer "grossly emulate" a convolution product that therefore can identify a "sign" among others, but we would be closer to that convolution principle, running then across many and many more heads.

In other words: from the standpoint of seeing the attention process as such as a convolution process, rather than a linear regression model PCA-style, the "heads" seem to be another gross approximation of a convolution process that has been reduced to a bare minimum. It obviously "works" as far as we look for the "as if" standards. The training would be quite easy to expand since we would just have to replicate the current value matrix, query matrices and key matrices for each slot: here we would slide a matrix, in each slot…The process would remain the same as summarized below…

$$\mathbf{q}_i = \mathbf{x}_i\mathbf{W}^{\mathbf{Q}}; \mathbf{k}_i = \mathbf{x}_i\mathbf{W}^{\mathbf{K}}; \mathbf{v}_i = \mathbf{x}_i\mathbf{W}^{\mathbf{V}}$$

$$\textit{Final verson:} \quad \text{score}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}}$$

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{x}_i, \mathbf{x}_j)) \ \forall j \leq i$$

$$\mathbf{a}_i = \sum_{j \leq i} \alpha_{ij}\mathbf{v}_j$$

On the follow the normalization part that looks benign… Actually it is a part that is even more crucial to the success of the training. Without the normalization and the MLP all that was explained above has no chance to succeed. The "theory" is right so far, testimony of the findings of Claude Shannon back 1958. But the real modern findings comes now: how to make all these heads, layers and parameters harmoniously converge? This goes way beyond a mere series of empirical tricks to make it converge….

vector normalized. Thus layer norm takes as input a single vector of dimensionality d and produces as output a single vector of dimensionality d. The first step in layer normalization is to calculate the mean, μ, and standard deviation, σ, over the elements of the vector to be normalized. Given a hidden layer with dimensionality d_h, these values are calculated as follows.

$$\mu = \frac{1}{d_h}\sum_{i=1}^{d_h} x_i \tag{10.20}$$

$$\sigma = \sqrt{\frac{1}{d_h}\sum_{i=1}^{d_h}(x_i - \mu)^2} \tag{10.21}$$

Given these values, the vector components are normalized by subtracting the mean from each and dividing by the standard deviation. The result of this computation is a new vector with zero mean and a standard deviation of one.

$$\hat{\mathbf{x}} = \frac{(\mathbf{x} - \mu)}{\sigma} \tag{10.22}$$

Finally, in the standard implementation of layer normalization, two learnable parameters, γ and β, representing gain and offset values, are introduced.

$$\text{LayerNorm} = \gamma\hat{\mathbf{x}} + \beta \tag{10.23}$$

Putting it all together The function computed by a transformer block can be expressed as:

$$\mathbf{O} = \text{LayerNorm}(\mathbf{X} + \text{SelfAttention}(\mathbf{X})) \tag{10.24}$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{O} + \text{FFN}(\mathbf{O})) \tag{10.25}$$

Or we can break it down with one equation for each component computation, using $\mathbf{T}$ (of shape $[N \times d]$) to stand for transformer and superscripts to demarcate each computation inside the block:

$$\mathbf{T}^1 = \text{SelfAttention}(\mathbf{X}) \tag{10.26}$$

$$\mathbf{T}^2 = \mathbf{X} + \mathbf{T}^1 \tag{10.27}$$

$$\mathbf{T}_3 = \text{LayerNorm}(\mathbf{T}^2) \tag{10.28}$$

$$\mathbf{T}^4 = \text{FFN}(\mathbf{T}^3) \tag{10.29}$$

$$\mathbf{T}^5 = \mathbf{T}^4 + \mathbf{T}^3 \tag{10.30}$$

$$\mathbf{H} = \text{LayerNorm}(\mathbf{T}^5) \tag{10.31}$$

The FFN, read Feed Forward Network, is NOT linear: it is an MLP with one or two hidden layers that have twice the size of the input: typically the input contains 8-16 x 1024 coordinates (8 to 16 heads) where each coordinates in the output is the SUM of a “meaning embedding “ and a “positional embedding” for each word ie , as said above, each word has a “meaning embedding” number of 1024.

Let’s now look batter at “what” these blocks are collectively doing to a native input vector of embeddings that feature a word/token…..

The combination now of consecutive blocks steps through the “loop” and “normalization” and “transformation” and “feed forward network” parts to, in fine, run a matrix operation on the native input vector: it is as if we start with an input X, we finally produce, at the output gate of one network in the chain, some vector $(I_d + A_X)X$. If we assume that more or less the same matrix operation A occurs at every block we ultimately $obtain\ in\ the\ end\ (I_d + A)^n X$. It is a polynomial: it could rapidly diverge or collapse to 0….We perceive here the explosive nature of the combination of these blocks under this very gross representation….if we do not normalize at each step on the way….

So we see the need already to “normalize” the vectors ie make sure that the norm remains at 1 and nowhere else so that there is neither a collapse towards 0 or a divergence towards infinity. It is not so much that the number of blocks is so high that indeed we would land to very low or, alternatively very high values, in absolute terms. What matters is that, since we start from words and produce “distributed words”, we want the output to be as unbiased as possible in essence: it must remain “words” ie close to the initial ones. In order to make sure the core attention principle works on a standalone basis, it was mentioned already that the embedding vectors are, as much as possible, of norm 1 and centered around zero if we look at their 1024 or 2048 coordinates as randomly distributed values for the layman’s eye to see. Hence the anchoring to “1”is the golden rule here: we start with embeddings that for each word look like a normal distribution centered on 0 and of norm 1. We transform these embeddings through an attention block…in order to rebuild a word having the same properties…And we will go on like this with the merging of the many heads in a basic “sum”, the normalization, the pushing the original X through, and MLP and normalizing again before sending…making the former input loop and produce ultimately a “meta word” that also will have the same number of embeddings to profile it and the coordinates, again, will seem to be distributed around 0 with norm 1. As it was mentioned there is a fundamental reason to proceed like this. And we will briefly see what this reason is…

If now we refine the model, we should observe that the matrix A is actually a function of X. If you recall the attention is based on XQ, XK and XV for each word X. So the transformation of X into AX is made with a matrix that itself is a function of X. But, thanks to the normalization, we can safely assume that at the first order $A_X = K_X + A_1 X$ where A_1would be some shared matrix among the words. Then, using this overly simplified version of what happens (since C also is a function X) we can define a specific matrix $A_1$for each block that would sum up the transformation. After say n blocks we would produce, excluding normalization for now:

$$(X + A_1 X) \to X + A_1 X + A_2(X + A_1 X) \ldots = (I_d + A_1)(I_d + A_2) \ldots (I_d + A_n)X$$

Note here that this is really to show what mainly “happens” here that we make this gross approximation that A_1 for example is common to all the words. But as long as the matrix V is going to add small values to the native embeddings then all the normalizations can be approximated by their first order Taylor-Young series and therefore the approximation that we make above is valid.

The normalization per block takes all its significance, since it should be done. Intuitively the operator $T_{race}(\overbrace{M^T}^{\prime}M)$ would provide the sum of the squared eigen values modules.

We start seeing why the normalization of X is advised: given the product of matrices that compound through the many neural networks that constitute a transformer block and the many blocks along side. Once we diagonalize the compound matrix we will have n eigen values that will embody the "size" of the output vectors. Given that the compound matrix will result from the product of other matrices that have eigen values too, we come across again the problem that such diagonal products may either diverge or collapse to 0. If all but 2 among the matrices are having eigen values close to 1, and the 2 other matrices have very low eigen values, then the ultimate compound matrix will have very low eigen values as a result. The same reasoning, assuming instead that the 2 matrices have very high eigen values, then the compound matrix also will have very high eigen values. All the intermediary scenarios are possible of course but the "products" have this property to depend mostly on the extreme values that constitute them.

So we see now that the more we make the product of matrices, the more we risk having a "compound effect" that randomly pushes the ultimate matrix close to much higher values or much lower values than "1"in absolute terms. This is why it matters that we keep control of the unit norm all the way.

In the "details" also, we should wonder what the role of the feed-forward network is: it takes say the 1024 coordinates and spreads them along 2048 neurons, to generate a broader but digital description of "X" at one stage, post the transformation: it therefore emphasizes some of the coordinates that appeared through the attention weighting the value matrix elements to manufacture m new coordinates. The MLP has this peculiarity that it uses a Relu function. So, a neuron would nullify any value that arrives below a certain threshold and, otherwise the neuron would simply "pass" the value in extenso if it is above the threshold. We can see what happens here: the MLP will only keep what matters. In order to determine what matters it does not look at the output of the attention block only. It does more. It uses say the 1024 embeddings of the attention output, runs it through 2048 neurons having a relu function, that would here and there nullify some coordinates. Next the MLP will generate a new set of 1024 emdeddings using again a Relu function that will filter the weighted sum of the 2048 new values inherited from the hidden layer of neurons…and there will be 1024 different ways to actually combine these 2048 values alongside with 1024 thresholds. So the MLP morphs the attention output quite in depth before it actually shrinks back to only 1024 coordinate. Next these brand new 1024 coordinates are again normalized …

So we can see the formulation above as indeed a series of matrices, either being a transformation matrix or else a feed-forward MLP style matrix. The normalization each time operates in the fashion of the SoftMax function. We start with an input, we apply a linear transformation at first. But next we impose a normalization ie a return to vectors that will have a norm equal to 1 and coordinates centered on 0. If we try to imagine geometrically what happens in 2 dimensions, we start with a vector having 2 coordinates and being on the circle of radius 1. The first linear transformation generates, from this vector, a new vector that may be larger (or smaller) than 1 in norm, that may have rotated and that even may have been translated. But the normalization will take this vector back to the unit circle in a way that is not linear but in a way that always has the same, continuous and homogeneous "role" from one vector to the next. If we look at the MLP alone, we take the new norm 1 vector being on the perimeter of the unit circle and we actually generate a vector that is on the hypershere in double the dimensions. The latter probably acts as a "ratcheting" mechanism that tends to underline some tentative drifts in 4 dimensions…It is a little bit as if we were unpacking a

box but in many different ways under many different angles all at the same time. What do we know about that? How can we just picture what happens? We simply cannot and we do not even know whether the new 4D vector has norm 1. It does not matter: the Relu effect will eradicate some coordinates making them nil and preserve the others. Next this crippled 4D vector will be projected back in 2D but NOT on the unit circle. There might be an angle or 2 that will drive this projection and on the way some coordinates might become nil again….At last we make a translation-rotation-homothety to return to the unit circle. The original vector has mutated without being really different. This is the purpose of all this: make it mutate a bit like we all age and our face becomes a reflection of what we are deep inside, what we lived…Over the years it will have changed quite a lot but still people might recognize the face decades later…or not….

We can sense that the MLP is tiering out the features among the 1024 attention coordinates that actually "make sense" for the next attention block to look at. We can sense as well that all this is "fine" while not being linear at all if the shifts, the mutations are benign. Indeed, it is easy to differentiate 2 straight lines, ie 2 linear impacts because 2 lines cannot be confounded unless they are identical. But with curbed lines, ie non linear changes, we may have many confusions….unless each curve is far away enough from the other ones and yet mutable. We cannot figure that out in 1024 dimensions since already in 3 dimensions we can fool ourselves (see the Klein's bottle if you are curious or the ribbon of Möebius…)

The interesting part now concerns the underlying mechanism that secures, most likely, the convergence towards a nice "solution" to the problem: the large language generation. The whole idea relies on a theorem: the roots of a unit polynomial that has normally distributed coefficients are converging towards the unit circle. The proof is in appendix but it is difficult and beyond the scope of the course. Let's just admit it and use it.

We need to refine the perception of what the transformer blocks do along the series of consecutive blocks and what happens when the decoder is merely supposed to produce the "next word" alone.

We start with a vector X of embeddings, typically 1024 meaning embeddings and 1024 positional embeddings. The computation is run in parallel throughout the 1024 words of a sequence that therefore is represented as a matrix of 1024 lines (one line per word) and 1024+1024=2048 columns.

The matrix is multiplied with a Key matrix that is going to produce inner coordinates, typically 2048 or 4096 coordinates per word. The same structure applies with a Query matrix this time. What is important to understand is : the core Key matrix and Query matrix will therefore generate a vector of keys and a vector of queries. So this will generate 2 matrices that this time will be specific to the current sequence. The positional embedding provides the position of a given word relative to the others in the sequence and of course that "position" is not a simple "rank". Many reasons apply: the vector must remain of norm 1 and we want to have the word "see" the relative position of the other words in the sequence from its standpoint. As a result, the keys and queries will factor in all this.

The next operation will test, for each line of the query matrix that was produced as pictured above will compute its alignment with every key, including the one of the word in question. But this is a first "rotation" that occurs, the query vector and the key vector reflect the current sequence under 2 directions: first the actual keys and queries embody the current words in the current sequence, second the position embeddings for the same words but eventually placed in a different order are going to deliver an additional relative positioning element.

Quite shortly an explanation on the positional embeddings: the word needs to have a position relative to all the words in the sequence if we want the network to capture elementary refinements of a language where the "orider" of the words matter to convey one nuance or another. However, numerically if a given word happens to be at the very first position in a sequence or the very last position in another sequence, moving forward from the latter, where the word is the last one to the "next sequence" where the same word would be the first one should NOT have much of a numerical impact. Otherwise we would face a consistency issue when we start building a new sequence by rolling the words of a former sequence from "the last position" in the former sequence to the "first position" in the new sequence. There we need these words to "see" each other in a similar numerical position since their "meaning embeddings" would not change anyway. In other words, the closest words should have the same positional embeddings one relative to the others whether they appear at the end of one sequence or at the end of the former sequence. The way to address these conflicting objectives, namely that we want differing values between words but we also want the positional embedding to be symmetrical relative to the middle of the sequence, consists in assigning positional embeddings as per sine functions. Thus say the sine function is worth 0 in position1 of the current sequence and it returns to 0 in the last position of the next sequence, independently of the word meaning. Thus across the 1024 positions of the sequence the positional embedding for a word that is the first one in the sequence will picture the relative position of all the other words along a sinusoidal cycle. Every word sees its own position as a "0" across a sinusoidal cycle.

We know now that even if this positional embedding matrix is rather fixed for all the sequences, it has parameters still. Indeed we really need a periodic function but the frequency can be adjusted through the training. Indeed the position really matters for the closest words. And one words being always influenced by its closest neighbors, it will indirectly be influenced but the most remote words too. That, for example, we have several cycles running through the 1024 positions is not necessarily a problem because only the closest words really matter. You thus can picture the positional embedding as a set of "waves" that are offset by one part of the cycle when we move our sight from one word to the next word. It is the same cycle almost, although each position in the sequence likely will apply a different set of frequencies for its cyclicality.

We need first to study the possible application of the theorem on the polynomials that have coefficients following a centered normal distribution.

First of all let's assuùe a very simplistic situation where instead of having 1024=2^{10} dimensions in the native inputs, we only have 2 of them. And let's imagine that our chatGPT here compiles attention blocks with feed-forward neural network of some MLP style with normalization and loops as we sketched them before. So, it is a chatgpt but one that only deals with 2 dimensions. We then would apply our approximation and infer then that we will picture our input with a complex number z.

Depending on the "word/token", this complew number z will have some peculiar real part and some peculiar imaginary part. We would try to secure that all the words "z" that we are to deal with are of module 1, ie norm 1. Then we would use our linear approximation for the attention and MLP blocks assuming that the output of our chatgpt for a "z" complex number that picture our words in only 2 dimensions is polynomial of the form $output - z) = z + P_{n_z}(z)$ where, as we described earlier the small values in the matrix V along with the many loops and normalization would ensure that the polynomial P_{n_z} would be a polynomial of degree "n" and its coefficients would be almost "normally distributed" ie their mean would around 0 and their variance would be finite. Let's assume that the variance is 1, that the average is 0 and that the coefficients are "random" because from one vector to the other, from one context to the other, we only know that they are random and "normally distributed" in general. If "n" is higher and higher the polynomial will tend to adopt the cyclotomic

values associated to “n” ie $e^{\frac{i2\pi k}{n}}\ with\ k = 0\ 1 \ldots n-1$. We can see then that this polynomial will step through 0 more and more often as “n” tends towards infinity.

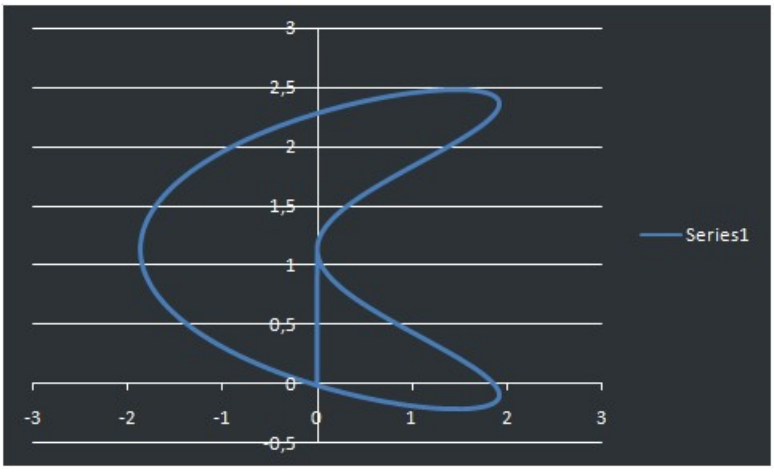
So whatever the “word/token” that we now represent with z that is on the unit circle as a complex unmber (z is of norm = 1), no matter where z is, the outcome is always the same, the added part of the chatgpt is a polynomial that twist the original z in a way where this polynomial steps back to zero, diverges, and come back to 0, and diverges, and so on an so forth. This means that, as we pictured before, the transformation never goes very far from the inception point: in other words, when we project the output back to the unit circle to “guess” the “next word” we cannot fall far from a word that indeed exists and this will not result from a wild ultimate distortion of the raw output vector.

This is fine but our “n” is more likely to be in the range of 8*2=16 or may be more but not much much more.

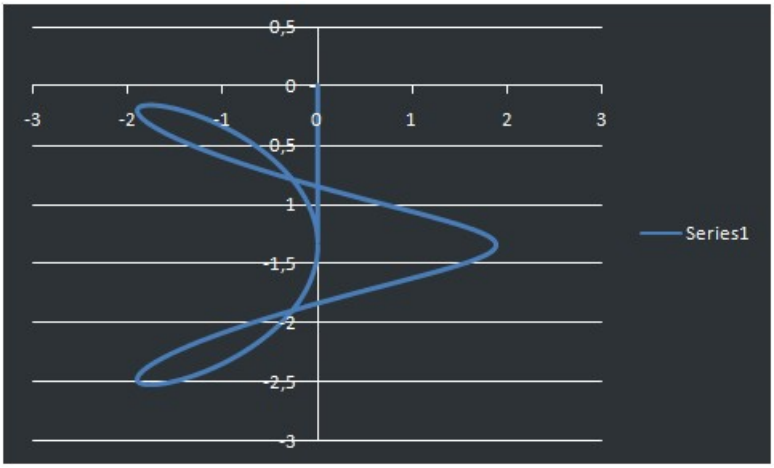
The charts below provide some charts that picture how, when we randomize the coefficients in the conditions that were described so far what the path of this polynomial P_{n_z} is….See that the randomness comes from the fact that we deal with 50 000 different words that can appear in some 1024 different positions ie 1024! Ie a number that no normal computer can even estimate grossly since you have at least 900 time 10^2 ie 10^{1800} (excel 64 bis blows up after the first 150 highest numbers of the factorial). Yes we have these many possible contexts! And each pair word-context would induce a peculiar set of coefficients for this polynomial. So “z” can be anywhere on the unit circle and the coefficients , each time specific to the situation, can really tae any value we can imagine in a way that just random.

So, if say we decided to make a very dumb basic chatgpt with just 1 attention block and one MLP what kind of profiles should we expect?

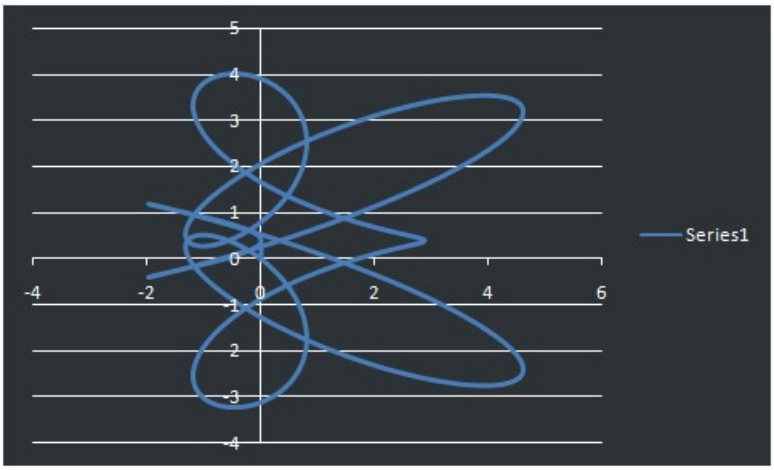
Below a few samples for the polynomial value un-normalized. All the chart are unnormalized so you need to divide by “n” in your mind to picture how much the, that pictures the curve that the polynomial value follows when “z” goes runs through the unit circle itself…

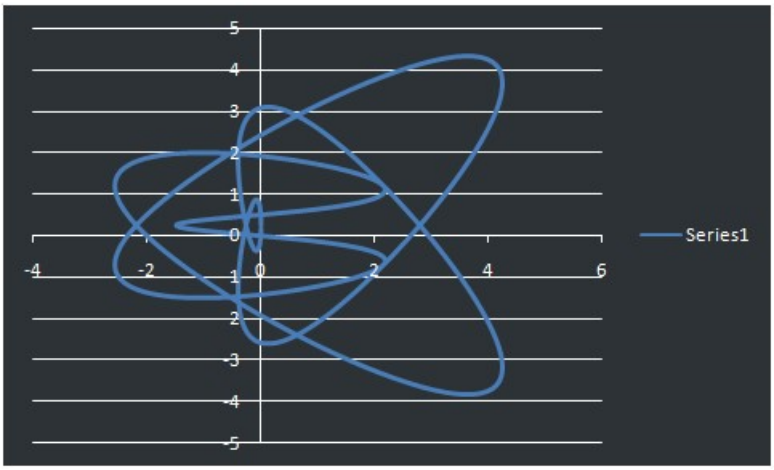


The chart above and the chart below show that, well the polynomial can take rather high values when z describes the unit circle then the projection on a “word” may be quite distorted.



What would happen if the put, as it was observed experimentally, 3 transformers' block in aseries and there build a "residual" polynomial with "n=6"? The 2 charts below give 2 samples…

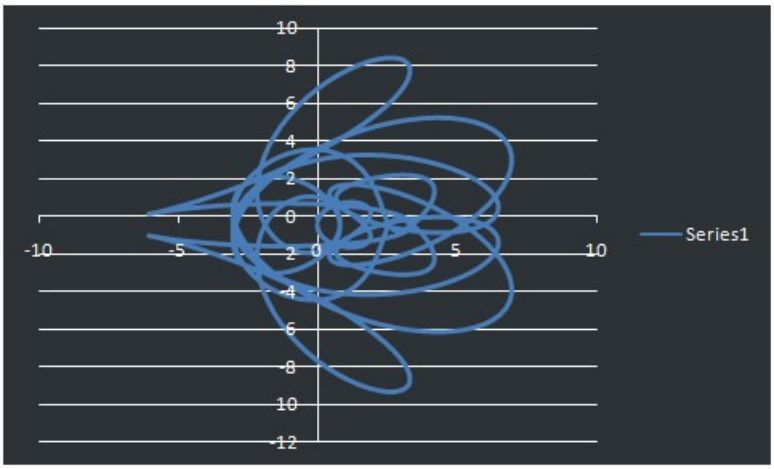


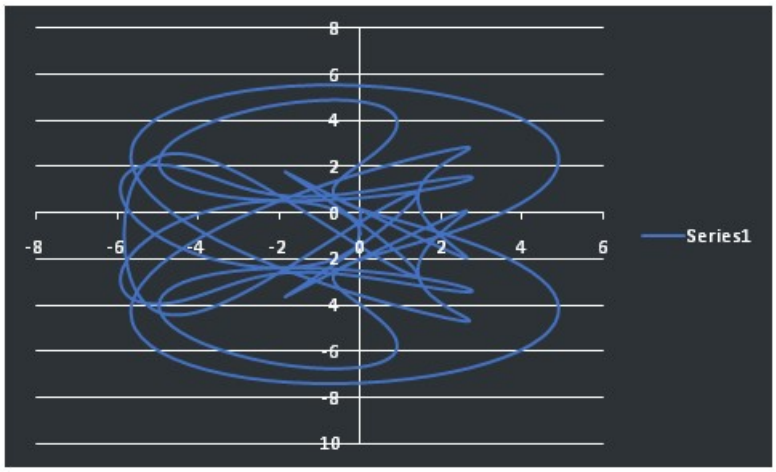


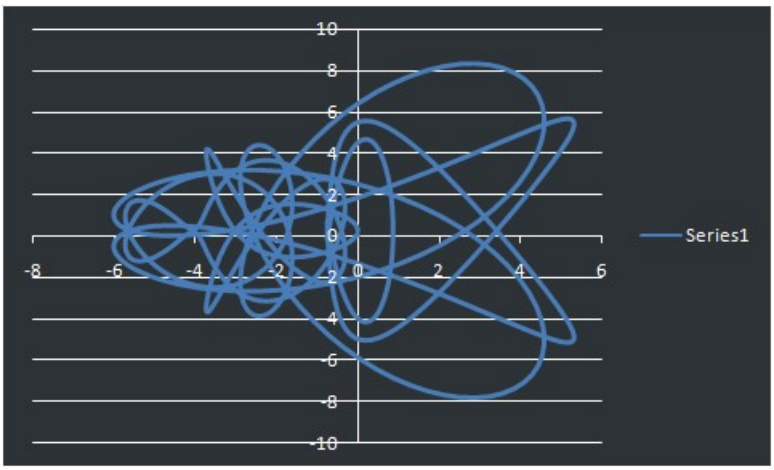
The chart above and the chart below cover the area with many more "in and outs" around 0.

But then what happens when we use the current formats? Here we should consider at least a degree n of 16 ie 8 x2 …(2 because in each block you have an attention part and and MLP part)

The 3 charts below provide 3 samples of tokens/words “z” in just 2 dimensions:



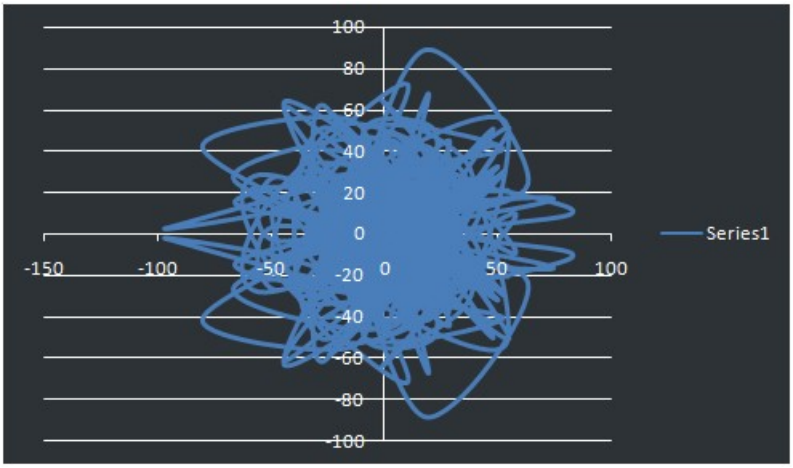


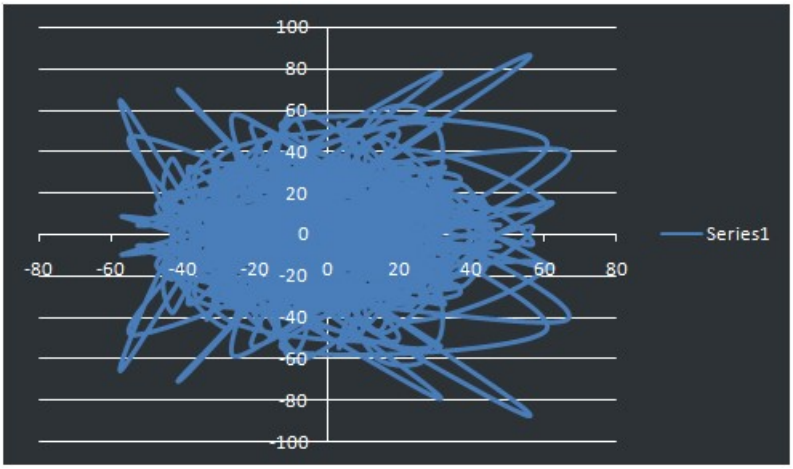


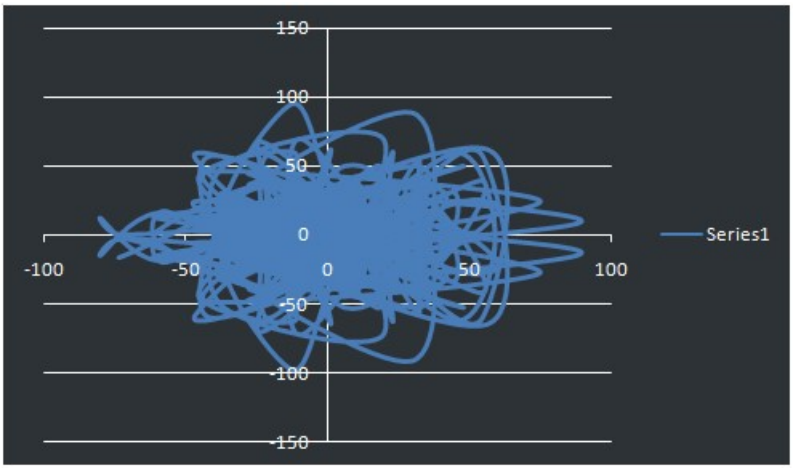
We diverge a little many times and we come back….

Just for illustration, how does it look when we actually put like a high degree as n=15 000 for example?

The charts below exhibit that the graph draws a fat point that, once normalize will look more and more like a smaller and smaller point as n goes ever higher



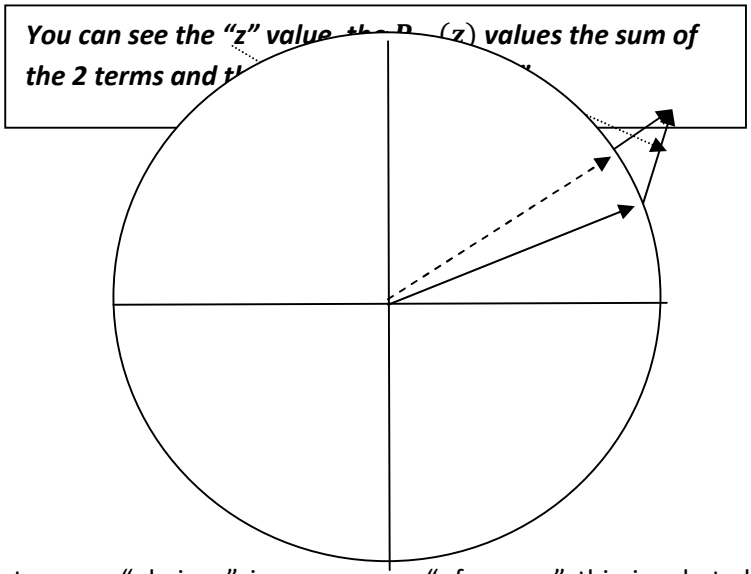




We realize here that this very degree “should” be beneficial but this would also render the tokens less distinguishable on the unit circle….unless we actually increase the “depth” and reduce the number of embeddings. The higher degree would be more involved into solving the context issues. In other words, if one grows the number of blocks but keeps the same number of dimensions then one is probably overfiiting too much that the “next word” might fail to be spotted when we project back to the unit circle in 2D. Now is one shrinks the number of dimensions in the embeddings and the matrices and all with the view to grow the degree “n”, ie the number of consecutive blocks, one needs to have a well organized training plan and some knowledge about how the many contexts may embed themselves into broader contexts….Which implies a much more complex training strategy.

So, you can see then that the "z" value will induce a "residual" added part that will "look" like this if only the randomness is fixed into polynomial parameters of degree 16 and we wonder then, "but what is the path that chatgpt would follow for this context for this word here and how the residual transformation compounds in the end to point towards the "next word"?"

Remember that is "z" is the input, ie the word/token, the ultimate output of the encoder is going to be $z + \varepsilon_V P_{n_z}(z)$. Here you need to picture a "big" norm 1 vector, ie "z", to which we add a rather benign distorting vector with the polynomial that is going to be $\varepsilon_V P_{n_z}(z)$ where the term ε_V reminds us that the role of the matrix V plus the normalization after re-introdducing the "input" at each stage has the effect of adding a small coefficient. So the sum of these 2 values would initially step out of the unit circle where "z" is. And, when looking to a normalized version that is expected to be "another word/token" as the "next word" we will do a return to the unit circle that is summarily sketched below (the residual term is made bigger than it is for the sake of clarity):



This chart seems "obvious" in some way: "of course" this is what should happen but you should picture that over 1024 dimensions instead of 2, and through 50 000 different words, the calibration had to go through many progressive steps. The training began with pieces of text that we small, used the most commonly used words and the most commonly understood sentences. The heads were pre-trained individually and next constrained to remain orthogonal pair-wise through their respective matrics Q,K and V.

Tentative explanation of the role that this theorem plays in the success of ChatGPT:

Here we will review all the formulas one by one and illustrate in more details how the polynomial is building up through the transformers' blocks:.

First we start with an input that is expressed in "d_{emb}" embedings: $X_k = \left(x_1, x_2, \ldots \quad x_{d_{emb}}\right)$

We apply that actually to a matrix of "n" words that constitute a sequence "S", ie we speak here of a matrix that has in fact "n" lines and d_{emb} columns. We call the matrix M_X and we use a query matrix Q $(d_{emb} lines\ and\ d\ columns)$ and a ley matrix $K(d_{emb} lines\ and\ d\ columns)$

We will generate through a matrix product 2 times: "keys" and "queries":

$$Q_{X_S} = M_X Q\ and\ K_{X_S} = M_X K$$

The query and key matrix is shared for all the possible sequences S and they have d_{emb} lines with "d" columns. Therefore the matrices that are generated here for a sequence are of "n" lines and "d"

columns. We can see here that for every word we generate a query and a key. Let's not forget that in the embeddings we have information on the position of each word in the sequence.

Now we will query the keys matrix for each word: in practice each word will use the d coordinates that come from its line in the native sequence matrix M_X post the products above initially and run a scalar product with ALL of the "n" lines of the key matrix to produce each time one scalar, called the attention factor. We realize that we actually work on the transpose of the matrix K_{X_S}, namely $K_{X_S}^T$ and we can summarize the whole operation with :

$A_{ttention_{matrix}} = Q_{X_S} K_{X_S}^T$ that is an nxn matrix. In other words each line delivers on "n" projections of the query of the word on line "k" relative to all the "key" axis, one axis per word in the sequence. Therefore the Attention matrix has "n" lines, each line representing the "n" Attention factors specific for querying word.

Now these are numbers that can be positive or negative on each line. Through the many steps in the gradient descent, without a normalization, they could take any value. Without the re-injection of the input plus the constraining on the matrix V to keep rather low values in absolute terms, this would not work well.

Before we go farther, let's see what we did and what we get for now:

$A_{ttention_{matrix}} = M_X Q K^T M_X^T$. For a given word, the actual operation is: $X_k Q K^T M_X^T$

This is not linear, rather a sort of bilinear operator that projects on each word next. But we therefore may not have numbers that "fit well" even though, having the embeddings normalized, the query columns normalized, the key columns normalized to 1, the "signature" on the key axis is unlikely to remain of norm 1 as a vector. Does it matter? Well this is the role of the theorem on the polynomial thathaving randomly distributed coefficients around 0 to show that "yes" it will matter here.

The aim is to spot a word, or 2 or more, that may likely be "the next word" for all the words in the sequence…Tentatively….The next operation will consist in morphing the attention matrix to "normalized" distributions: we apply the softmax function for that matter word by word, line by line….because we do not know the general range of the outputs as they are so far…

From the chart that you saw, you can now picture what would happen if we do not constrain this polynomial to remain small relatively to the initial input all along the "transformation".

We normalize on every line as follows, to generate an attention matrix of a second type:

$$A_{tt_{Ma\ Norm}} = \left[\left[\frac{e^{M_{kj}}}{\sum_{m=1}^{m=n} e^{M_{km}}}\right]\right]$$

Note that the "normalization" is NOT that the norm of the line is 1….The normalization consists in morphing the factors into a distribution. This guides the intuition here: now we will use a 3rd shared matrix for this current transformer's block, the "Value matrix" that has "n" lines and d_{emb} columns: $V\ such\ that\ V_{X_S} = M_X V$.

Through this operation we have generated on each line a "value" vector that is typical, in this sequence, of the word that is in position "k" in the sequence "S" that we work on now. Observe that the Value matrix aims at the generation of "new words", or we had rather say "meta words" or else "mutant words".

Therefore for each attention vector we have now a "probability" attributed to each of the "n" words that lie in the sequence "S" that we look at. Running the line "k" with its "n" coordinates through the

Value matrix by the left as done before for keys and queries, we generate for this line d_{emb} weighted values that "look like" a word similar to the original ones but in a "tiny" format most likely. But we are not there, not quite. How can we make sure for example that we have not introduced a "drift" with these weights? Recall the "vectors" above and the inherent "craziness " of the polynomials in general? Are we really dealing with "words" with all these numbers? We most likely are NOT generating "words" per se, rather "derivations from words"…

We know for sure we cannot be sure about that. So the idea is that the embeddings originally are probably overall centered on 0 (as much as possible for each word, hence the need to have sufficient coordinates and probably use linear regression to define the initial coordinates). And the embeddings will set to be "nom 1 vectors that we hope are pair-wise orthogonal as often as possible". But bear in mind here that with 1024 meaning embeddings and say 50 000 we only 1024 dimensions and therefore on 1024 orthogonal axis, not 50 000. So this "objective" of setting the embeddings on words so that the 50 000 tokens are "orthogonal" in 1024 dimensions is plain impossible. But, if they simply are "separable" at the maximum, this will be good enough. That alone will define the embeddings as axis based distances from a centroid that is shared across all the points. In the context of large language generation models like BERT of ChatGPT, there is a massive linguistic work to perform in order to encompass the projection space at least in the theory. More we will want to have the embeddings for the points as a group as vectors of unit norm too…As much as possible. The reality of the training however is that the initial embeddings, side by side with the parameters of the Q,K and V matrix and the MLP weights and biases will all be initiated at random. So, this is really the set of penalties that will guide the training that are essential. Here it matters much more to impose constraints on, for each word, the norm being one, the sum of the embeddings being 0, the scalar product between 2 embeddings being 0 , the heads being orthogonal for their respective matrices through the different stages of the training…all these constraints will be instrumental. In other words, the theory only matters for an initial design purpose, not for the training itself. This means that the "embeddings" are pure numerical artefacts of the gradient descent!

Thus we will do this normalization too for the output that we pictured before…so that it has quite similar characteristics: the embeddings are centered on 0 and the vector has unit norm…as much as possible… This implies the introduction of some constant term that is induced by centering back to 0 all the way.

All in all we can observe that if we look at the transpose that defined the attention matrix and forget, just this time, about the normalizations above we actually "did": $X_k Q K^T M_{X_S}^T M_{X_S} V$.

We can see that the whole matrix appears as its "squared norm" but , once we have "normalized" it is as if we generated a " weird rotation" of our initial word in the embeddings: $X_k Q K^T M_{X_S}^T M_{X_S} V \sim C_{st_{X_S}} + X_k R_{\leftrightharpoons_{X_S}}$ that is linear in first approximation although on each coordinate the "angles" change a bit randomly from one word of the sequence to the next.

Notice here the presence of the term $M_{X_S}^T M_{X_S}$: if the embeddings are vectors of d_{emb} coordinates that are of norm 1 and if it happens that the number of tokens in the sequence, ie "n" is equal to d_{emb} then the matrices above are nxn square matrices. If we can engineer through the training process that the tokens are actually vectors of norm 1 that are pair-wise orthogonal then the matrix $M_{X_S}^T M_{X_S} = I_d$ is the identity matrix. It would imply then that this whole operation would merely be a rotation of the initial token. This implies that the normalization would still take place but it would then be a soft non linear touch. Therefore the words would be rotating along some 1024 angles in a well ordered manner, securing that no confusion is induced between.

One way to achieve that goal without inducing too much excess calculation consist in minimizing the error function against a penalty and thus try to minimize the following function:

$$F(w_i) = \sum \|prediction - target\|^2 - \alpha \left\| M_{X_S}^T M_{X_{S_{I_d}}} \right\|^2 - \beta \sum \left| \left\| M_{X_{S_k}} \right\|^2 - 1 \right|$$

How to choose the weights? Well they can estimated at an early stage where the training runs in parallel through an array of pairs (α, β) including of course the point (α, β) =(0,0) ie the case where no constraint is imposed. Then the optimal point lies where the the constraints grow in value but the error only creeps up slowly….until the moment when, the constraints being dominant the training looks more and more inefficient. We catch here the highest curvature point on the "surface" that is defined by $F_{trained}(\alpha, \beta)$: this point shows the maximum efficiency of the constraints. Of course the point "0,0" matters a lot since it would show whether these constraints are actually useful. This analysis would be done at an early stage of the training when the first periods of training rather involve the most frequent sentences and words.

The subscript in $X_k R_{\leftrightharpoons}$ points to the fact that we could approximate this to a linear change in every coordinate as long as the changes remain small and shared across the words of the sequence. But every coordinate follows its own agenda here., that in essence depends on the word X_S To be clear here , every word has its own rotation approximate change even though all the words share the same matrices. This is due to the double normalizations that we mentioned 'one distribution' and "one unit norm'. So at first order all the words share the same "weird rotation" but at second order they depend upon the word itself.

Next we would do a seemingly benign thing but that is one thing that matters a lot: we re-inject the native input to the recent normalized output before we feed the feed-forward MLP:

$$X_k \to X_k + C_{st_{X_S}} + X_k R_{\leftrightharpoons} = X_k (I_d + R_{\leftrightharpoons}) + C_{st_{X_S}}$$

For every word, we would do that, run the MLP (2 layers, one unique hidden layer and n_{emb} final neurons based on a Relu function). The MLP would actually have 2*n_{emb} neurons before shrinking them back to n_{emb} Relu based outputs. Here some embeddings will have been reduced to 0. But then, once again we run a "layer norm" ie "center on zero and unit norm" mutation.

Now, as commented before, the "weird rotation" is specific to each word X_k,and at first order, ie as long as the output remains "residual" relative to the native input, we can say that the rotation is a linear move relative to the word itself, ie $R_{\leftrightharpoons} = R_S X_k$

For a similar reason, we might as well state that the subsequent mutation through the MLP is likewise a linear operator relative to the word itself. Let's bear in mind that this "second order" impact, residual hopefully so, is due to the squared norm of M_{X_S} for the attention part and it is due to the Relu function for the MLP. We work here extensively under the assumption that "yes" the values are residual all the way.

Thus, if we sum up the operations: $X_k \to X_k (I_d + R_{\leftrightharpoons}) + C_{st_{X_{S_{attention}}}} \to (X_k (I_d + R_{\leftrightharpoons}) + C_{st_{X_{S_{attention}}}}) \left(I_d + MLP_{Relu_X}\right) + C_{st_{X_{S_{MLP}}}}$

Once again, at first approximation order, we can assume that $MLP_{Relu_X} = MLP_S X_k$ post the normalization…as long as the attention impact remains residual numerically relative to the initial embeddings.

Expressing all this with the linear approximation, due to the second order dependence upon the word itself, we can write :

$$X_k \to X_k(I_d + R_{\leftrightharpoons}) + C_{X_A} \to (X_k(I_d + R_{\leftrightharpoons}) + C_{X_A})\left(I_d + MLP_{Relu_X}\right) + C_{X_{MLP}}$$
$$\approx X_k(I_d + R_S X_k)(I_d + MLP_S X_k) + terms\ and\ a\ constant$$

We can see that the "constant" terms in first approximation are the result of innumerable calculations that we cannot really comprehend. And they will change with every word in every configuration for this word. They are quite random values.

It matters to note that the layer norm that centers around 0 and unit norm the aggregate outputs before they become an "input" for the next neural network block has the effect of always making the approximation credible. It therefore has an effect on the linear matrices above, namely $R_S\ and\ MLP_S$, are NOT to be zeros but rather "small rotations along 1024 angles" since we always morph a vector into a twisted but homogeneous new vector anyway. The constants play a peculiar role, each one being introduced by the centering on 0 at layer norm stage.

Combining the encoder blocks we just build up a polynomial of higher and higher degree here. Making sure all along that the product of these matrices remains "centered on X_k" by always re-introducing X_k and maintaining the next vector to a constant norm anyway, we definitely randomize the factors in some unpredictable way for the rotations that we highlighted above. But the end results remains "close to 0 outside of the identity" in general.

We can observe that we retrieve then the original input word plus a polynomial of random factors that are symmetrically distributed around 0 and have a constrained variance. This is enough to use the theorem on random polynomials.

One last point on the training of these models. It turns out that the embeddings are the parameters that tend to converge faster than the parameters of the K,Q,V matrices or the parameters of the MLP neurons. All this comes from what is known as the "chain rule". The principle is that, to infer "how much" every weight should be adjusted, we start from the error, use the derivative of every inner "output" against each weight and combine in a product all these derivative to trace back the error towards every weight. The closer a weight is to the native input, the farther it is from the neuron that produced the final output that induced the error. As a result the closer this weight is to the native input the higher the number of intermediate derivatives in the chain rule will be involved. The adjustment of the weight in question will present itself as a product of intermediate derivatives pplus its own. If in the product at least one intermediate derivative is close to zero, so the product will be and the weight will be made motionless. This is due to the fact that when, in the middle of this deep neural network some intermediate derivatives go to 0 or close the changes of the embeddings will be neutralized by the tranches that have these small derivatives anyway. You can figure what the "chain rule" is and how the product of derivatives build up when you derive a function that is the composition of many inner functions. They make the last layers "blind" towards what the former layers do, including the layer of embeddings. The last layers, beyond the low derivatives level in the network will only "see" very small changes and will therefore have to apply large changes on their own parameters to correct the error.

As a result, when an error persists, the deeper layers, ie the ones that are close to producing the final "output" of the network (ie the ones that are farther from the input ie the embeddings) tend to shoulder all the correction. Even so, if next with another token the derivatives are all fine, the impact of the embeddings changes will come to unsettle the recent adjustments in the deeper layers…that will have then to "correct back". The consequence of that is that through phases during the training

when there are so-called “vanishing gradients”, the embeddings are “frozen” and next resume their adjustments without fail. In the meantime the deeper layers will have overcompensated first because of the vanishing gradients that left all the job to them alone. And next, when the gradients become normal again, they will have to reverse course most likely. The deeper layers are therefore going through a more volatile and uncertain path towards convergence.

A contrary situation may occur too where the gradients are “exploding” then it means that this time the deeper layers are neutralized and the embeddings are being unsettled. But is it their “turn” in this configuration to converge more slowly than the deeper layers? Not quite… Why is that? First the embeddings are softly constrained to be as “linear” and “smooth” and “centered around 0” and ”normalized to 1” as possible. More they are also preserved from that outcome by the permanent normalization. If we think about it: the recurrent normalization is mostly going to induce values that are low in absolute terms and therefore the chance that their derivative explodes is way much less than the probability that its value being glued somewhere and its derivatives on values that are close to 0. This finishes explaining why embeddings are trained faster and more easily than the other parameters. This is why also they usually have learning rates that decline less faster than the other subsequent layers.

3- How to use transformers’ models on time series

We will now sketch innovative uses of financial data. It is up to you to go farther….

We will start with what looks the most intuitive since it seems that a time series is mostly a 1-d , read 1 dimensional, dataset.

Well this is not quite true if you see all the risk factors and other “moving averages” and “correlations” and “auto-correlations” and “volumes” and many other “technical analysis” measures that are very popular in the media and social networks. We kind of “get” that idea that underneath a “time series” many dimensions actually compete.

Let’s see how a model like ChatGPT could make sense with a much lighter implementation than the one that you know already….

To train a transformer model in order to “predict” the “next day” in a daily time series for example, we need to define the “letters”, the “words”, the sentences, the language the best we can. This is just an exercise…

The “letters” could be the price, the “words”, non overlapping vectors of “prices”, the sentences could be “periods” of varying length etc…but, as we saw, what matters is the identification of Markov processes and thereafter the definition of conditional probabilities that follow distributions that are not path dependent. The prices would exhibit a potentially infinite number of different paths, but in doing so they actually define a set of rules that were fixed from the start. We cannot really get an explicit formulation of all the rules, as is the case in languages.

Now we need to conceive the “embeddings”, the “queries”, the “keys”, the “heads” the correlations that would bring meaningful attention coefficients.

The way chatGPT is trained is “all in one go ie embeddings, positional embeddings, query, key, value matrices, MLP…” And the point is always to revert back to a sort of similar vector of prices.

To keep it simple we know that the volumes are important but we will do without them. Just to focus on one data that we have in a very long time series. Really we should look at pairs ‘price, volume’ and

potentially more coordinates that appear contemporaneously. We will see now that, anyway, the "1 dimension only" is a plain illusion.

We already mentioned that, even with only the price, the precedent records matter. Thus we had rather consider vectors of consecutive prices that are not overlapping. These vectors themselves, like words appear many times, as in a language…oops! Is it the case really? Well not quite right because If we look at enough decimal places we might could end up concluding that every historical vector is "unique" in the history that we have. On the other hand, we also would easily concede that if we divide the full historical range into enough sub-intervals we may actually be able to group the many historical records in sub-groups and we would have lost very little sense for each vector. Indeed we know that a fixing price is just an average value and as such any rounded number would be as good.

So, one first arbitrage that we should do in this endeavor to apply a language "logic" is that we should actually round the vector values to appropriate averages and thus define groups of vectors. These groups would define classes of vectors that would be the equivalent of words. Another to look at it is to say that a word like "the" looks always the same, but it does not revert to the same thing at all depending on the context. So the "numbers with decimals" that we get are partial "snapshot" of the word in its context…and we want to flag the "words" knowing that the context, as a premise in this modeling that consists in saying "hey there is a language underneath these decimals and many moves", provides some "make-up" on the raw word. And we want the "word" really because we will build the rules of the context as a "drift", or a residual "difference". We also know that the decimals here would be the "tree that hides the forest" of underlying and undisclosed rules. But, hey, chatgpt learns a language without ever learning any grammar rule or vocabulary…so….it might work in financial time series as well…

Take for example 3 vectors like $\begin{bmatrix}1.2345\\2.3789\\3.5217\\4.8975\\5.1275\end{bmatrix}\begin{bmatrix}0.9852\\2.0002\\2.9852\\3.7585\\4.9999\end{bmatrix}$ and $\begin{bmatrix}1.1111\\2.2222\\3.3333\\4.4444\\5.5555\end{bmatrix}$ they all look close to $\begin{bmatrix}1.1\\2.2\\3.3\\4.4\\5.2\end{bmatrix}$

If the other records display significantly different profiles that this one like $\begin{bmatrix}-1.4\\2.2\\3.3\\4.4\\5.2\end{bmatrix}$ and we end up with a thousand different vectors and we will be equipped with enough of a "language" for our purpose to flag a lot of the major market configurations that we go through in general. Because, at the end of the day, we actually have a very basic need to know whether to "buy" or to "sell".

Yet, despite the extreme simplicity of our ultimate conclusion, we still need to have a refined way to arrive to this conclusion. But the "1000 words" tag is just a first guess.

So this means that the goal should be look at "points" actually. And we should run a k-means clustering algorithm in order to spot the 1000 barycenters that will define our groups. Some, like words, will be well populated, others will have very few members….like rare words…

If we want to define an optimal number we should compute the entropy of information of the distribution of points that we get, assuming a normal distribution of the members around their barycenters, and use the variance of each group to infer the entropy of information. Why is that?

Because if one computes the entropy of information of a variable x that is normally distributed on a mean "m" along a variance "v" its probability density if $p(x) = \frac{1}{\sqrt{2\pi v}} e^{-\frac{\frac{1}{2}(x-m)^2}{v^2}}$ and the entropy of information of this variable x is

$$E(x) = \int_{-\infty}^{+\infty} -p(x)Ln\big(p(x)\big)dx = \int_{-\infty}^{+\infty} \frac{1}{\sqrt{2\pi v}} e^{-\frac{\frac{1}{2}(x-m)^2}{v^2}} \left(Ln\big(\sqrt{2\pi v}\big) + \frac{\frac{1}{2}(x-m)^2}{v^2} \right) dx$$

$$E(x) = Ln\big(\sqrt{2\pi v}\big) \int_{-\infty}^{+\infty} \frac{1}{\sqrt{2\pi v}} e^{-\frac{\frac{1}{2}(x-m)^2}{v^2}} dx + \int_{-\infty}^{+\infty} \frac{1}{\sqrt{2\pi v}} e^{-\frac{\frac{1}{2}(x-m)^2}{v^2}} \left(\frac{\frac{1}{2}(x-m)^2}{v^2} \right) dx$$

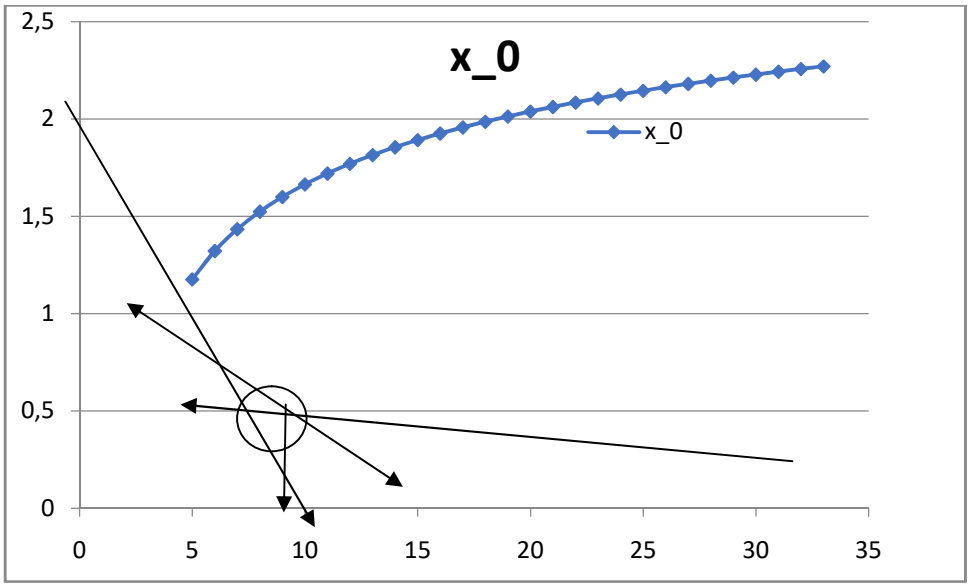
$$E(x) = Ln\big(\sqrt{2\pi v}\big) + 1$$

Thus if we model our group of points as a weighted sum of such normally distributed groups around their respective barycenters the entropy of information is well approximated with

$$E(vectors) = \sum_{k=1}^{k=N_{groups}} \frac{n_k}{N} \big(1 + Ln\big(\sqrt{2\pi v_k}\big)\big) \; with \; n_k \; number \; of \; vectors \; in \; group \; "k"$$

The variance of each sub group of vectors is v_k.We can see here that the more groups we allow to have through a k-means clustering the lower each variance v_k and therefore the lower the entropy.

Inversely if we start reducing the number N_{groups} in the k-means clustering algorithm then the points will be classified in groups that will have on average a larger variance and therefore will sum up to higher entropy values. If we start with a k-means based on a high number of groups N_{groups}, we save the barycenters, record the entropy of information for this classification, and use them now with $N_{groups} - 1$….the k-means clustering will converge fast. We then would measure again the entropy and store it. We would reduce again a by "1" the number of groups, run the k-means and record again the entropy. We could then draw a chart of the entropy of information and quite likely we would observe a phenomenon like this if we plot the entropy against the number of groups:



We can observe that the circle defines an area where the entropy decline as we increase the number of groups and soon after the decline of the entropy tends to vanish: this indicates that adding more

groups is just overfitting with the empirical data that we have. This area provides an order of magnitude for the appropriate number of subgroups that we should pick: below we do capture enough of the diversity that is not a pure noise, and beyond we artificially group the points amid actual “empirical noise”. The number is not hard coded: it depends on the data that we use. The interest in finding this optimum lies in the fact that, then, we will be able to find a maximum number of points for each group without “over-describing” the data with too many groups.

Once we have that we can start considering the kind of “attention” we want to compute, the kind of “rules” we have for our language.

To go on by yourself!....

4- How to use transformers models to financial data

Let’s now take a different angle. We want again to “build” words and a language based on financial data. Here we will very likely NOT have very long time series. So this cannot be the baseground for building “words” and emulating a “language” from these “words”. However, the financial reports usually carry thousands of numbers in generic accounting categories. More there are also hundreds of pages of plain text with “warnings”, “disclaimers”, “strategies”, “positives” “negatives”. So, now we should actually digitalize these words in numbers ie tokens and morph a financial report in a very long “digitalized text”. Again we would look to generate “words” in a “text”. This “text” would have a different shape: through this series, alike a time series now, a “story” would be conveyed. So we would have to order the many numbers in a sequence that would “tell” the financial story that lies underneath this report.

Here it matters to define a narrative: is it a “story” about “financing a going concern?”, is it a story about “M&A”, or is it a story about “strategy and EBITDA”?...for example…depending upon the optics the “series” will follow a certain guideline through the financial reports or another…. It is probably a good idea next to define “heads” of attention based upon these various perspectives in order to develop deeper a more comprehensive financial analysis. It is all about projections in the end. So, different from the times series case before, here we also need to define the “arrow of projection” and this is not “time” in a direct manner here. It has to be defined too before we can build “words” around it that will be “vectors of numbers” anyway.

7- FFT

The starting point is we assume that our time series is a sum of periodic functions the period being a multiple of the core time increment. Thus if we have a time series that is a minute by minute record, we assume that the “signal” adds up periodic functions having a period of “k-minutes”=$\frac{N}{n}$ or a frequency $\frac{n}{N}$. To be clear here: “n” is an integer, the “implied period” is not always an integer since it is $\frac{N}{n}$ and “n” may not be a divisor of “N”. But the only thing that matters is that “n” is an integer.

We thus look for a proxy formula like : $x(n)$ being a relative weight of the frequency $\frac{N}{n}$

$$f(t) = \sum_{n=0}^{N-1} x(n) e^{-\frac{2i\pi n}{N}} \ the\ period\ being \frac{N}{n}\ and\ the\ \text{time}\ being\ \text{"t"}\ in\ minutes$$

"N" is the number of observations that we have. The period is counted as a "multiple" N/n of the time unit (1 minute here). So we do NOT cover regularly the "periods" but we do cover uniformly the frequencies that are multiplied by our "time t" key variable.

Quick technical comment:

the more intuitive version would consist in considering a representation in "sin" and "cos" like $f(t) = \sum_{n=0}^{N-1} a(n)\cos\left(\frac{2\pi nt}{N}\right) + b(n)\sin\left(\frac{2\pi nt}{N}\right)$

But with our representation we would likely obtain complex values for $x(n)$.

If now we set $x(n) = a(n) - ib(n)$ and if we develop our initial definition that was:

$$f(t) = \sum_{n=0}^{N-1} x(n)e^{-\frac{2i\pi nt}{N}} = \sum_{n=0}^{N-1} x(n)e^{-\frac{2i\pi nt}{N}}$$

Thanks to the nice properties of the Fourier transform we can compute the factors 'x(n)' as:

$$x(n) = \frac{1}{N}\sum_{n=0}^{N-1} f(t)e^{+\frac{2i\pi nt}{N}} = \sum_{n=0}^{N-1}\left(a(n) - ib(n)\right)\left(\cos\left(\frac{2\pi nt}{N}\right) + i\sin\left(\frac{2\pi nt}{N}\right)\right)$$

$$f(t) = \sum_{n=0}^{N-1}\left(a(n)\cos\left(\frac{2\pi nt}{N}\right) + b(n)\sin\left(\frac{2\pi nt}{N}\right)\right) + i\left(a(n)\sin\left(\frac{2\pi nt}{N}\right) - b(n)\cos\left(\frac{2\pi nt}{N}\right)\right)$$

So if our signal $f(t)$ is made of real values the pure imaginary part is nil as a whole, or, in practice it will be quickly negligible when N becomes large.

This comment only emphasize the equivalence of the 2 representations and actually its broader spectrum of applications since it also works for holomorph functions (ie functions of some complex z). Back to the mainstream course..

So this seems an "easy" thing to do, namely compute all the "x(n)" factors one by one. But it may become quite time consuming when say you deal with millions of "times". We notice indeed that to obtain "N" factors like "$x(n)$" with a real part and an imaginary part we speak of N factors that each require an N scalar product ie N products and N sums of complex numbers: $x(n) = \sum_{t=0}^{N-1} f(t)\frac{1}{N}e^{-\frac{2i\pi nt}{N}}$. This also involves cosine and sine which are NOT simple functions. In total this means a double pair-wise sum from 4 products per pair-wise products of 2 complex numbers ie N(each x(n))*N (main sum)*N (main products)*(4+4)=$8N^3$

One way to speed up the calculation consists in using vertices and matrices. We notice that all the factors use all the same factors $e^{+\frac{2i\pi nt}{N}}$.

So if we call X the vertex that gathers our factors 'x(n)' and F the vertex of our empirical data seen as a function having T instants of representation: F being a column vector having "T" coordinates. We could then define a matrix M(N,T) such that $M_{n,t} = \frac{1}{N}e^{+\frac{2i\pi n}{N}}$. Then, assuming N=T, we can write synthetically that X=MF. M is some N^2 matrix, hence potentially a huge one that would mobilize a lot of RAM and CPU time.

The issue arises because we would have here N² factors for the complex exponentials. We will use actually the symmetries of these sine and cosine functions to reduce this computation burden. The problem arises on CPU and most importantly on RAM since this requires loading many numbers.

Let's consider indeed that N is even ie that N=2p. Doing this choice of N=T we would not expect the lowest frequency to have much weight. But we at least cover all the lowest frequencies anyway.

Then we can split the sum in 2 parts and use the nice properties of $e^{-\frac{2i\pi nt}{N}}$ so that we are left with much smaller matrices all along….splitting the total N into 2 ie the even group and the odd group

$$x(n)=\sum_{t=0}^{N-1} f(t)\frac{1}{N}e^{-\frac{2i\pi n}{N}}=\sum_{\substack{t=0\\ step=}}^{N-1} f(t)\frac{1}{N}e^{-\frac{2i\pi n}{N}}+\sum_{\substack{t=0\\ step=2}}^{N-2} f(t+1)e^{-\frac{2i\pi(t+1)n}{N}}$$

$$x(n)=\sum_{t=0}^{N-1} f(t)e^{-\frac{2i\pi}{N}}=\underbrace{\sum_{\substack{m=0\\ step=}}^{p-1} f(2m)e^{-\frac{2i\pi 2m}{N}}}_{f_{1,even}(m)}+\underbrace{\sum_{\substack{m=0\\ step=}}^{p-1} f(2m+1)e^{-\frac{2i\pi(2m+1)n}{N}}}_{f_{1,odd}(m)}$$

$$x(n)=\sum_{t=0}^{N-1} f(t)e^{-\frac{2i\pi n}{N}}=\underbrace{\sum_{\substack{m=0\\ step=2}}^{p-1} f(2m)e^{-\frac{2i\pi 2m}{N}}}_{f_{1,even}(m)}+\underbrace{\sum_{\substack{m=0\\ step=}}^{p-1} f(2m+1)e^{-\frac{2i\pi(2m+1)n}{N}}}_{f_{1,odd}(m)}$$

If instead we have an odd number N, we strip the odd part and run again the split in even parts. The odd index is fully computed subtracted to the original function. Let's keep our "N" base and assume now that N=2p+1:

$$x(n)=\sum_{t=0}^{N-1}\frac{f(t)}{N}e^{-\frac{2i\pi n}{N}}$$

$$=\underbrace{\sum_{\substack{m=0\\ step=}}^{p-1}\frac{f(2m)}{N}e^{-\frac{2i\pi(2m)n}{N}}+\sum_{\substack{m=0\\ step=1}}^{p-1}\frac{f(2m+1)}{N}e^{-\frac{2i\pi(2m+1)n}{N}}}_{typical\ former\ case\ of\ an\ even\ number}+\underbrace{\frac{f(N-1)}{N}e^{-\frac{2i\ (2m+1)n}{N}}}_{odd\ term}$$

We thus compute straight $f(N-1)$ alone and we switch the estimation from f(t) to:

$$x(n)\ \underbrace{-\frac{f(N-1)}{N}e^{-\frac{2i\pi(N-1)n}{N}}}_{odd\ term}=\underbrace{\sum_{\substack{m=0\\ step=}}^{p-1}\left(f(2m)+f(2m+1)e^{-\frac{2i\pi n}{N}}\right)e^{-\frac{2i\pi(2m)n}{N}}}_{typical\ former\ case\ of\ an\ even\ number}$$

We then observe that we have shrunk the number of dimensions of our matrix by 2 on the "column" side: we then define a Nxp matrix instead of some NxN matrix $M_{n,m}=\frac{1}{N}e^{-\frac{2i\pi(2m)n}{N}}$. We can write then that actually our vertex based expression is:

$$X_1=X-\frac{f(N-1)}{N}e^{-\frac{2i\pi(N-1)n}{N}}Id_n=\underbrace{M_{np}}_{nxp}\underbrace{F_{2m..}}_{p-vertex}+\left[\underbrace{e^{-\frac{2i\pi(2n)}{N}}}_{n\_vertex}\right]Id_n\underbrace{M_{np}}_{nxp}F_{2m+1\ldots}.$$

So beware here since the lines of N are multiplied by a value that depends on the factor still. But in numpy this is a very easy thing to do. This is a "new axis" basic product in numpy. This opens the ways to many vectorizations. So the total number of operations has not really declined but it can be run in parallel: indeed we have twice less operations with matrices but we have to run them twice. However they can be done in parallel and each require reduces the load in memory by 2.

If it turns out that if p is again some even number then we should divide by 2 again, reducing further the size of the matrix involved. As a general formulation, using a "k" power law of 2 before p becomes again some odd number, we can write (recall that N=2^kp+1):

$$x(n) - \frac{f(N-1)}{N} e^{-\frac{2i\pi(N-1)n}{N}} = \underbrace{\sum_{m=0}^{p-1}}_{step=} \underbrace{\sum_{j=0}^{2^k-1} \left[e^{-\frac{2i\pi(j)n}{N}}\right] f\left(2^k m + j\right) \quad e^{-\frac{2i\pi(2^k m)n}{N}}}_{typical\ former\ case\ of\ an\ even\ number}$$

$$X \underbrace{- \left[e^{-\frac{2i\pi(N-1)n}{N}}\right] \frac{f(N-1)}{N}}_{odd\ term\ n-vectorized} = \sum_{j=0}^{2^k-1} \left[e^{-\frac{2i\ (j)n}{N}}\right] Id_n\ MF\left(2^k \ldots + j\right)$$

$$M\ is\ now\ some\ nxp\ matrix$$
$$F\ is\ some\ p-vertex$$

Of course if "p" is then some odd number we can reduce even further the calculation by swinging the indexing once more but this will introduce a more complex term then, introducing a smaller matrix M that would then be nxp_1:

$$x(n) = \frac{f(N-1)}{N} e^{-\frac{2i\ (N-1)n}{N}} + \sum_{j=0}^{2^k-1} \left[e^{-\frac{2i\pi(j)n}{N}}\right] f\left(2^k(p-1) + j\right)$$
$$+ \underbrace{\sum_{m=0}^{p_1-1} \sum_{j_1=0}^{2^{k_1}-1} \left[e^{-\frac{2i\pi(j_1)n}{N}}\right] f\left(2^{k_1} m + j_1\right) \quad e^{-\frac{2i\pi(2m)n}{N}}}_{step=1\ typical\ former\ case\ of\ an\ even\ number}$$

In vectorized form this becomes:

$$X = \underbrace{\underbrace{\left[e^{-\frac{2i\pi(N-1)n}{N}}\right] \frac{f(N-1)}{N}}_{odd\ term\ n-vectorized} + \sum_{j=0}^{2^k-1} \underbrace{\left[e^{-\frac{2i\pi(j)n}{N}}\right] F\left(2^k(p-1) + j\right)}_{n-vertex\ X\ a\ scalar}}_{we\ ha\quad here\ 2^k-1+1\ n-vertice\quad multiplied\ by\ scalars}$$
$$to\ produce\ one\ n-vertex \ldots with\ some\ einsum\ product$$
$$EF_{sel}\ whe\quad E\ is\ some\ nx2^k\ matrix\ and\ F_{sel}\ some\ 2^k\ vertex$$
$$+ \sum_{j=0}^{2^{k_1}-1} \left[e^{-\frac{2i\pi(j)n}{N}}\right] Id_n M_1 F\left(2^{k_1} \ldots + j\right)$$
$$M\ is\ now\ some\ nx\ _1\ matrix$$
$$F\ is\ some\ p_1-vertex$$

So while we would divide the burden of matrix products by 2^k we still would have to run it at least 2^k and we would have to add some subtractions on the way. The fact that we would use less memory is a net positive though because in general we would not have enough capacity to handle a million long record. Imagine that the initial memory would be 1 TeraOctet at least! So this astute split instead removes this limitation at the expense of a very benign overload in the number of operations.

But there is an even more astute way to use the FFT….

The Fourier transform of a function "f" is meant to actually interpolate and extrapolate any function that is only known empirically. All that follows can be easily extended to any dimension. The whole point is about getting the Fourier factors or even eventually directly interpolate or extrapolate from the empirical data….something which is all AI stuff….

Theoretical comment:

We can observe that when we solve the Fourier factors "x(n)" we actually determine the coefficients of a polynomial in $X(t) = e^{\frac{2i\pi t}{N}}$. Then we may also say that this is all about finding the "polynomial roots" to these data, namely some prime factors that factor them all in the end. Here we land into the Galois groups since we would end up solving for the zeros of some polynomial of a very high degree. This is where "periodic" functions and "polynomials between their zeros" are kind of the same thing.

If we aim to predict a "function" we always end up computing a polynomial anyway (even for exponentials or logarithm in practice) and we need the trend-line of the zeros since, up to a finite number of factors this boils down to solving a polynomial with its roots. The question of the Galois theory becomes central since we have to be sharp on the roots values in any event. The Fourier series, unlike Lagrange polynomials or the associated Hermite's polynomials (different from the Hermite's functions issued for the Gaussian law derivatives) transpose the problem in full into the "periodic" or "pseudo periodic" functions that would assume an infinity of zeros…that still we could define sharply so ie rationally.

Back to the main course…

Below the algorithm shows that a basis 2 is making wonders in speeding up the computation of the factors and uncovers "issues" around the extrapolation….

We thus look again for a proxy formula like:

$$f(t) = \sum_{n=0}^{N-1} x(n) e^{-\frac{2i\pi nt}{N}} \ the\ period\ being \frac{N}{n}\ and\ the\ \text{time}\ being\ \text{"t"}\ in\ minutes$$

"N" is the number of observations that we have. The period is counted as a "multiple" N/n of the time unit (1 minute here). So we do NOT cover regularly the "periods" but we do cover uniformly the frequencies that are multiplied by our "time t" key variable.

Thanks to the nice properties of the Fourier transform we can compute the factors 'x(n)' as:

$$x(n) = \frac{1}{N} \sum_{n=0}^{N-1} f(t) e^{+\frac{2i\pi nt}{N}}$$

So this seems an "easy" thing to do, namely compute all the "x(n)" factors one by one. But it may become quite time consuming when say you deal with millions of "times". One way to speed up the calculation consists in using vertices and matrices. We notice that all the factors use all the same factors $e^{+\frac{2i\pi nt}{N}}$.

So if we call X the vertex that gathers our factors 'x(n)' and F the vertex of our empirical data seen as a function. We could then define a matrix M such that $M_{n,t} = \frac{1}{N} e^{+\frac{2i\pi nt}{N}}$. Then we can write synthetically that X=MF. M is some N^2 matrix, hence potentially a huge one that would mobilize a lot of RAM and CPU time.

What if we supposed that N=2^L and what if we "completed" "f" with the most likely "periodic" complement?

Let's look into the breakdown expansion if indeed we are in a "perfect" situation where each time we can split into 2 parts…Let's resume to the initial stage where :

$$x(n) = \sum_{\substack{m=0 \\ step=1}}^{p-1} \left(f(2m) + f(2m+1)e^{-\frac{2i\pi n}{N}}\right) e^{-\frac{2i\pi(2m)n}{N}}$$

$$= \sum_{\substack{m=0 \\ ste}}^{2^{L-1}-1} \left(f(2m) + f(2m+1)e^{-\frac{2i\pi n}{N}}\right) e^{-\frac{2i\pi(2m)n}{N}}$$

We can surely set a "new" function that is: $f_1(m_1) = f(2m) + f(2m+1)e^{-\frac{2i\pi n}{N}}$

And we realize then that computing the x(n) values amounts to:

$$x(n) = \sum_{\substack{m=0 \\ step=}}^{2^{L-1}-1} \left(\underbrace{f(2m) + f(2m+1)e^{-\frac{2i\pi n}{N}}}_{=\widehat{f_1(m)}}\right) e^{-\frac{2i\ (2m)n}{N}}$$

$$= \sum_{\substack{m_1=0 \\ step=}}^{2^{L-2}-1} \left(f_1(2m_1) + f_1(2m_1+1)e^{-\frac{2i\pi n}{N}}\right) e^{-\frac{2i\pi(2m_1)n}{N}}$$

So this process can go on and on since we surely can define then:

$$f_2(m_2) = f_1(2m_2) + f_1(2m_2+1)e^{-\frac{2i\pi n}{N}}$$

So we get next:

$$x(n) = \sum_{\substack{m_1=0 \\ step=}}^{2^{L-2}-1} \left(f_1(2m_1) + f_1(2m_1+1)e^{-\frac{2i\pi n}{N}}\right) e^{-\frac{2i\ (2m_1)n}{N}}$$

$$= \sum_{\substack{m_2=0 \\ step=1}}^{2^{L-3}-1} \left(f_2(2m_2) + f_2(2m_2+1)e^{-\frac{2i\pi n}{N}}\right) e^{-\frac{2i\ (2m_2)n}{N}}$$

So here we have shrunk the size of the matrix down to 8 times on each dimensions, hence we have 64 times less elements! On balance we are based now on f_2 that requires some preliminary computation….What is the expression of f_2 against "f" the original input?

We have to look at the term in total to figure this out:

$$\left(f_2(2m_2) + f_2(2m_2+1)e^{-\frac{2i\pi n}{N}}\right) e^{-\frac{2i\pi(2m_2)n}{N}}$$

$$= \left(f_1(4m_1) + f_1(4m_1+1)e^{-\frac{2i\pi n}{N}}\right) e^{-\frac{2i\ (4m_1)n}{N}}$$

$$+ \left(f_1(4m_1+2) + f_1(4m_1+3)e^{-\frac{2i\pi n}{N}}\right) e^{-\frac{2i\pi(2m_1+2)n}{N}}$$

So, we have done no "miracle" since, again, we have not changed the number of operations :

$$\left(f_2(2m_2) + f_2(2m_2+1)e^{-\frac{2i\pi n}{N}}\right) e^{-\frac{2i\pi(2m_2)n}{N}}$$

$$= \left(f_1(4m_1) + f_1(4m_1+1)e^{-\frac{2i\pi n}{N}} + f_1(4m_1+2)e^{-\frac{2i\pi 2n}{N}} + f_1(4m_1+3)e^{-\frac{2i\pi 3n}{N}}\right) e^{-\frac{2i\pi(4m_1)n}{N}}$$

This is "how" we end up with a certain number of "k" reductions like this to:

$$x(n) = \sum_{\substack{m=0 \\ step=}}^{2^{L-k}-1} \underbrace{\left[\sum_{j=0}^{2^k-1} \left[e^{-\frac{2i\pi(j)n}{N}}\right] f\left(2^k m + j\right)\right]}_{we\ repeat\ k\ times\ the\ algorithm} e^{-\frac{2i\pi(2^k m)n}{N}}$$

So, if we now we invert the summation order we can see the balance:

$$x(n) = \underset{step=}{\sum_{j=0}^{2^k-1}} \left[e^{-\frac{2i\pi(j)n}{N}}\right] \underbrace{\left[\sum_{m=0}^{2^{L-k}-1} e^{-\frac{2i\pi(2^k m)n}{N}} f(2^k m + j)\right]}_{this\ is\ the\ CPU\ intensive\ part\ now}$$

We morph it to a scalar product and we have not really reduced the number of operations for a given "x(n)". On the initial expression we set:

$$x(n) = \frac{1}{2^L} \sum_{n=0}^{2^L-1} f(t) e^{+\frac{2i\pi n}{2^L}}$$

So we really run initially 2^L products and run 2^L additions. This sums to 2^{L+1} operations. Since we would likely need to do that on some 2^L different "n" values, this totals up to 2^{2L+2}. This would lock up a lot of memory since we would have to, in theory, store as much different numbers.

With the most recent formula, namely:

$$x(n) = \underset{step=}{\sum_{j=0}^{2^k-1}} \left[e^{-\frac{2i\pi(j)n}{N}}\right] \underbrace{\left[\sum_{m=0}^{2^{L-k}-1} e^{-\frac{2i\pi(2^k m)n}{N}} f(2^k m + j)\right]}_{this\ is\ the\ CPU\ intensive\ part\ now}$$

Here we run 2^{L-k} products, 2^{L-k} additions, and this we do it 2^k times which amounts to $2^{L-k+1+k} = 2^{L+1}$.

So, on the face of it, there is no gain whatsoever apart from the fact that, thanks to the recursive algorithm, we actually only need to perform the following operation "k" times only:

$$x(n) = \underset{step=}{\sum_{m=0}^{2^{L-1}-1}} \underbrace{\left(f(2m) + f(2m+1)e^{-\frac{2i\pi n}{N}}\right)}_{f_1(m)} e^{-\frac{2i\pi(2m)n}{N}}$$

$$= \underset{step=}{\sum_{m_1=0}^{2^{L-2}-1}} \underbrace{\left(f_1(2m_1) + f_1(2m_1+1)e^{-\frac{2i\pi n}{N}}\right)}_{f_2(m_1)} e^{-\frac{2i\ (2m_1)n}{N}}$$

$$= \underset{step=}{\sum_{m_2=0}^{2^{L-3}-1}} \underbrace{\left(f_2(2m_2) + f_2(2m_2+1)e^{-\frac{2i\pi n}{N}}\right)}_{f_3(m_2)} e^{-\frac{2i\pi(2m_2)n}{N}}$$

$$\ldots = \underset{step=}{\sum_{m_2=0}^{2^{L-k}-1}} \underbrace{\left(f_{k-1}(2m_{k-1}) + f_{k-1}(2m_{k-1}+1)e^{-\frac{2i\pi n}{N}}\right)}_{f_k(m_{k-1})} e^{-\frac{2i\pi(2m_{k-1})n}{N}}$$

So the idea is that we first build the functions $f_k(m_{k-1})$ that only require 2^{L-1} slots to start with and decrease over time, each process dividing the size by 2. We only need to process a 2x2 scalar product by

the very same vector $\left(1, e^{-\frac{2i\pi n}{N}}\right)$, which can be vectorized by placing the even indices in the first colum, and the odd indices in the second colum, multiplying then by this vector as a column vector. To make sure we actually retrieve 2 columns for the next operation we need a generic way to strip the even from the odd indices all along. But there is an even faster way to compute it because we do not need to store all the factors while at the end of the day these are only products and sums.

So looking back at the formula we can see more:

$$x(n) = \underset{step=}{\sum_{j=0}^{2^k-1}} \left[e^{-\frac{2i\pi(j)n}{N}}\right] \underbrace{\left[\sum_{m=0}^{2^{L-k}-1} e^{-\frac{2i\ (2^k m)n}{N}} f\left(2^k m + j\right)\right]}_{this\ is\ the\ CPU\ intensive\ part\ now}$$

We can see that we would strip the empirical data into subsets:

$$\left[\sum_{m=0}^{2^{L-k}-1} e^{-\frac{2i\pi(2^k m)n}{N}} f\left(2^k m + j\right)\right]$$

One vector is independent of "n" and that is $\left[f\left(2^k m + j\right)\right]$ for a number of "j" values that ranges from 0 to $2^k - 1$. Next the scalar product is made first with $\left[e^{-\frac{2i\ (2^k m)n}{N}}\right]$ $over\ 2^{L-k}\ power\ laws\ on\ "m"$

Thus we only need really $e^{-\frac{2i\ (2^k)n}{N}}$ and $e^{-\frac{2i\pi n}{N}}$ for the rest of the calculation where we would not store any other vector than the original empirical data one.

We can deduce "x(n+1)" from "x(n)" and we note the following:

$$x(n+1) = \underset{step=}{\sum_{j=0}^{2^k-1}} \left[e^{-\frac{2i\pi(j)(n+1)}{N}}\right] \underbrace{\left[\sum_{m=0}^{2^{L-k}-1} e^{-\frac{2i\pi\left(2^k m\right)(n+1)}{N}} f\left(2^k m + j\right)\right]}_{this\ is\ the\ CPU\ intensive\ part\ now}$$

$$x(n+1) = \underset{step=1}{\sum_{j=0}^{2^k-1}} e^{-\frac{2i\pi(j)}{N}} \left[e^{-\frac{2i\ (j)n}{N}}\right] \underbrace{\left[\sum_{m=0}^{2^{L-k}-1} e^{-\frac{2i\pi\left(2^k m\right)}{N}} e^{-\frac{2i\ \left(2^k m\right)n}{N}} f\left(2^k m + j\right)\right]}_{this\ is\ the\ CPU\ intensive\ part\ now}$$

We actually would infer all the "x(n)" solely from $e^{-\frac{2i\ (2^k)m}{N}}$ and $e^{-\frac{2i\pi j}{N}}$ so in other words: $e^{-\frac{2i\ (2^k)}{N}}$ and $e^{-\frac{2i}{N}}$. We would apply embedded loops straight based upon "j", then "m", then "n". All the "x(n)" will be computed in parallel as one long vector. We can see that the empirical data needs to be re-ordered first so that the values (or else we use a numpy table and filters) after rewriting the index in basis 2^k. Once we have the 2^{L-k}vectors on 'f' featured by $f\left(2^k m + j\right)$ we multiply each one by $e^{-\frac{2i\pi(2^k)}{N}}$ "n" times and store the result each time to build a factor that starts at 1 for m=0. A "m" grows we multiply these factors again once for each increment of "m" and add up the product with the associated "f" value. We start at "j"=0 and then grow the sum….So we need to store a vector of weights, a vector of sub sums and the vector for the many "x(n)" values. The weights are incremented along with "m", "j" and "n". The indices, "j","m", and "n" start from 0 and rise up to their respective upper bounds: n first for each m and j, m next for each j and finally j. The arbitrage occurs at the RAM level while the number of operations for the CPU is unchanged anyway.

The trick here will be to first complete the "missing" values to match 2^L with "0"s, spot a first approximation for the "x(n)", complete de "0"s with what this first approximation suggests, record the "error", make a second estimation, record the "error" that should be lower…and so on until the "error" gets below a certain level.

APPENDICES

APPENDIX: about quickly estimating the implied vol from ,the B&S closed form solution

The typical formula comes as follows:

$$C(x,T) = SN_{Prob}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r+\frac{1}{2}\sigma^2\right)T\right]}{\sigma\sqrt{T}}\right) - Ke^{-rT}N_{Prob}\left(\frac{Ln\left(\frac{K}{S_0}\right)+\left(r-\frac{1}{2}\sigma^2\right)T}{\sqrt{\sigma^2 T}}\right)$$

We can notice that we have 2 functions that we can develop at first order:

$$SN_{Prob}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r+\frac{1}{2}(\sigma+\delta)^2\right)T\right]}{(\sigma+\delta)\sqrt{T}}\right) \sim SN_{Prob}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r+\frac{1}{2}(\sigma)^2\right)T\right]}{(\sigma)\sqrt{T}}\right) + \delta\left[\frac{-\left[Ln\left(\frac{K}{S_0}\right)+(r)T\right]}{\sqrt{T}(\sigma+\delta)^2}+\frac{\sqrt{T}}{2}\right]SN_{dens}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r+\frac{1}{2}(\sigma)^2\right)T\right]}{(\sigma)\sqrt{T}}\right)$$

As to the "other" leg of the Call option price we have:

$$Ke^{-rT}N_{Prob}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r-\frac{1}{2}(\sigma+\delta)^2\right)T\right]}{(\sigma+\delta)\sqrt{T}}\right) \sim Ke^{-rT}N_{Prob}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r-\frac{1}{2}(\sigma)^2\right)T\right]}{(\sigma)\sqrt{T}}\right) + \delta\left[\frac{-\left[Ln\left(\frac{K}{S_0}\right)+(r)T\right]}{\sqrt{T}(\sigma+\delta)^2}-\frac{\sqrt{T}}{2}\right]Ke^{-rT}N_{dens}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r-\frac{1}{2}(\sigma)^2\right)T\right]}{(\sigma)\sqrt{T}}\right)$$

Thus:

$$C(x,T,\sigma+\delta) \sim SN_{Prob}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r+\frac{1}{2}(\sigma)^2\right)T\right]}{(\sigma)\sqrt{T}}\right) + \delta\left[\frac{-\left[Ln\left(\frac{K}{S_0}\right)+(r)T\right]}{\sqrt{T}(\sigma+\delta)^2}+\frac{\sqrt{T}}{2}\right]SN_{dens}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r+\frac{1}{2}(\sigma)^2\right)T\right]}{(\sigma)\sqrt{T}}\right) - \left[Ke^{-r}\ N_{Prob}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r-\frac{1}{2}(\sigma)^2\right)T\right]}{(\sigma)\sqrt{T}}\right) + \delta\left[\frac{-\left[Ln\left(\frac{K}{S_0}\right)+(r)T\right]}{\sqrt{T}(\sigma+\delta)^2}-\frac{\sqrt{T}}{2}\right]Ke^{-rT}N_{dens}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r-\frac{1}{2}(\sigma)^2\right)T\right]}{(\sigma)\sqrt{T}}\right)\right]$$

$$C(x,T,\sigma+\delta)\sim C(x,T,\sigma)+\delta\left[\left(\frac{-\left[Ln\left(\frac{K}{S_0}\right)+(r)T\right]}{\sqrt{T}(\sigma+\delta)^2}+\frac{\sqrt{T}}{2}\right)SN_{dens}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r+\frac{1}{2}(\sigma)^2\right)T\right]}{(\sigma)\sqrt{T}}\right)\right.$$
$$\left.-\delta\left[\left[\left(\frac{-\left[Ln\left(\frac{K}{S_0}\right)+(r)T\right]}{\sqrt{T}(\sigma+\delta)^2}-\frac{\sqrt{T}}{2}\right)Ke^{-r}\ N_{dens}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r-\frac{1}{2}(\sigma)^2\right)T\right]}{(\sigma)\sqrt{T}}\right)\right]\right.$$

Thus if we want to get a better picture of the approximation formula we need to write some intermediary ones:

$$N_1(\sigma)=N_{Prob}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r+\frac{1}{2}(\sigma)^2\right)T\right]}{(\sigma)\sqrt{T}}\right)$$

$$N_2(\sigma)=N_{Prob}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r-\frac{1}{2}(\sigma)^2\right)T\right]}{(\sigma)\sqrt{T}}\right)$$

$$d_1(\sigma)=N_{dens}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r+\frac{1}{2}(\sigma)^2\right)T\right]}{(\sigma)\sqrt{T}}\right)$$

$$d_2(\sigma)=N_{dens}\left(\frac{\left[Ln\left(\frac{K}{S_0}\right)+\left(r-\frac{1}{2}(\sigma)^2\right)T\right]}{(\sigma)\sqrt{T}}\right)$$

$$C(x,T,\sigma)=SN_1(\sigma)-Ke^{-rT}N_2(\sigma)$$

$$C(x,T,\sigma+\delta)-C(x,T,\sigma)\sim$$

$$\delta\left[\frac{\sqrt{T}}{2}[Sd_1(\sigma)+Ke^{-r}\ d_2(\sigma)]+\left[\frac{-\left[Ln\left(\frac{K}{S_0}\right)+(r)T\right]}{\sqrt{T}(\sigma+\delta)^2}\right][Sd_1(\sigma)-Ke^{-rT}d_2(\sigma)]\right]$$

So:

$$\delta\sim\frac{[C(x,T,\sigma+\delta)-C(x,T,\sigma)]}{\left[\frac{\sqrt{T}}{2}[Sd_1(\sigma)+Ke^{-r}\ d_2(\sigma)]-\left[\frac{\left[Ln\left(\frac{K}{S_0}\right)+(r)T\right]}{\sqrt{T}(\sigma)^2}\right][Sd_1(\sigma)-Ke^{-rT}d_2(\sigma)]\right]}$$

APPENDIX: Charles Hermite's polynomials…

Let's define these polynomials before we study their properties. The core idea is to expand what we saw already with the normal distribution: it is grossly "invariant or close" through a Fourier transform operator, which gives us an elegant way to generalize the resolution of the Black and Scholes equation when we deal with "distributions" for the "rate" or the "trading costs" that are not

necessarily normal but can be well approached through their projection on the basis of the Hermite's functions that themselves derive from the Hermite's polynomials. (see right below)

$$H_k(x) = (-1)^k e^{x^2} \frac{d}{d^k}\left[e^{-x^2}\right]$$

We will consider a certain type of orthogonality using the following scalar product so that we establish the important projection properties of these polynomials.

$$< H_k|H_l >= \int_{-\infty}^{+\infty} H_k(x)H_l(x)e^{-x^2}dx$$

We can prove it with integration by parts first (we make a more interesting proof next…) k>l:

$$< H_k|H_l >= \int_{-\infty}^{+\infty} H_k(x)H_l(x)e^{-x^2}dx = \int_{-\infty}^{+\infty} (-1)^k e^{x^2} \frac{d}{d^k}\left[e^{-x^2}\right]H_l(x)e^{-x^2}dx$$

$$< H_k|H_l >= \int_{-\infty}^{+\infty} H_l(x)(-1)^k \frac{d}{d^k}\left[e^{-x^2}\right]dx$$

$$Let's\ assume\ that\ k > l, we\ integrate\ over\ the\ \mathrm{k}\ part:$$

$$< H_k|H_l >= \int_{-\infty}^{+\infty} H_l(x)(-1)^k \frac{d}{d^k}\left[e^{-x^2}\right]dx$$

$$= \left[(-1)^k \frac{d}{d^{k-1}}\left[e^{-x^2}\right]H_l(x)\right]_{-\infty}^{+\infty} - \int_{-\infty}^{+\infty} (-1)^k \frac{d}{d^{k-1}}\left[e^{-x^2}\right]\frac{dH_l}{dx}(x)\,dx$$

$$< H_k|H_l >= \int_{-\infty}^{+\infty} (-1)^{k+1} \frac{d}{d^{k-1}}\left[e^{-x^2}\right]\frac{d}{d^1}[H_l(x)]dx$$

We then just need to repeat the integration by parts "k" times overall to obtain:

$$< H_k|H_l >= \int_{-\infty}^{+\infty} (-1)^{2k} \frac{d}{d^k}[H_l(x)]e^{-x^2}dx$$

Now we set that k>l, and we are to show that $H_l(x)$ is a polynomial of degree "l" with the highest factors in x that is worth$(2)^l$. We just need to remind the very definition of the polynomials to see that:

$$H_k(x) = (-1)^k e^{x^2} \frac{d}{d^k}\left[e^{-x^2}\right]$$

We can think in a recursive manner. Indeed , each time we derive a factor (-2x) comes to multiply the highest existing power law of "x" and we apply a power law of (-1) that comes to annihilate the apparition of the "-" sign on (-2x). On top of that we need to include the factorial factor inherited from the consecutive derivations of the term of the highest degree: for H_k the highest degree if "k" and thus "k"derivations will generate a k! factor

We thus infer that the scalar product is 0 when k and l are distinct.

Thus we can provide a general result: $< \boldsymbol{H_k}|\boldsymbol{H_l} >_{e^{-x^2}} = \boldsymbol{\delta_{k,l}} \boldsymbol{2^k(k)!}\sqrt{\boldsymbol{\pi}}$ $\left(\boldsymbol{recall}\ldots \int_{-\infty}^{+\infty} \boldsymbol{e^{-\frac{1}{2}u^2}}\boldsymbol{du} = \sqrt{\boldsymbol{2\pi}}\right)$

Another proof of all this can be made more synthetically using the generating function….

Let's write then that, with a contour integral using a radius that goes to 0: $e^{-x^2} = \frac{1}{2i\pi}\oint \frac{e^{-z^2}}{z-x} dz_{r\to 0}$

To prove it is rather straightforward from the general finding that for a function "f" that is having no singularity inside a circle C of radius r (we make a change of coordinates:

$$\overline{f(x)_{C_r}} = \frac{1}{2i\pi}\oint \frac{f(z)}{z-x} dz_r = \frac{1}{2i\pi}\int_0^{2\pi} \frac{f(\theta)}{re^{ir}} ire^{ir\theta} d\theta = \frac{1}{2\pi}\int_0^{2\pi} f(\theta) d\theta$$

Since the function "f" is bounded on the finite contour the integral sum converges towards the average value of f and, when the radius tends towards 0, the convergence is done in "norm" towards "f(x)". Let's observe that this "average" theorem works actually on any closed contour because then we could represent "r" as "$r(\theta)$" and simply "measure" the size of the contour by its perimeter:

$$\overline{f(x)_C} = \frac{1}{2i\pi}\oint \frac{f(z)}{z-x} dz_C$$

Proof: the only singularity for $\frac{f(z)}{z-x}$ is in "z=x" since f itself is regular and when the radius is not equal to 0, we can decompose the contour into a circle around z-x=0 and side contours where there is no singularity for $\frac{f(z)}{z-x}$. Therefore the integral on these contours is nil all along. So, by running the contour through this circle and the side contours or directly through the initial contour, we have the same number of singularities, ie just one in z-x=0, and this amounts to conclude on the more general result.

We can directly infer the consecutive derivatives of this same function in terms of the contour integral of radius 0 as : : $\frac{d}{dx^k}\left[e^{-x^2}\right] = \frac{1}{2i\pi} k! \oint \frac{e^{-z^2}}{(z-x)^{k+1}} dz_{r\to 0}$….it is a "+" because the "-x" is balanced with the negative power law each time…

We then infer an expression for the Hermite's polynomials:

$$H_k(x) = (-1)^k e^{x^2} \frac{d}{d^k}\left[e^{-x^2}\right] = \frac{(-1)^k k!}{2i\pi} e^{+x^2} \oint \frac{e^{-z^2}}{(z-x)^{k+1}} dz_{r\to 0}$$

Let's now introduce the "generating function" associated to the Hermite's polynomials:

$$G_H(t,x) = \sum_{k=0}^{k\to+\infty} \frac{t^k}{k!} H_k(x) = \frac{e^{+x^2}}{2i\pi} \oint \sum_{k=0}^{k\to+\infty} \frac{(-t)^k}{k!} \frac{e^{-z^2}}{(z-x)^{k+1}} dz_{r\to 0}$$

$$G_H(t,x) = \sum_{k=0}^{k\to+\infty} \frac{t^k}{k!} H_k(x) = \frac{e^{+x^2}}{2i\pi} \oint \frac{e^{-z^2}}{(z-x)^1} \sum_{k=0}^{k\to+\infty} \frac{(-t)^k}{(z-x)^k} dz_{r\to 0}$$

If we consider a complex "t" such that $\left|\frac{t}{(z-x)^1}\right| < 1 \; for \; any \; z$(considering that anyway z has a module that is close to zero…this means that we just need to keep "t" between "-x" and "+x" in practice), we have a converging series:

$$\sum_{k=0}^{k\to+\infty} \frac{(-t)^k}{(z-x)^k} = \frac{1}{1+\frac{t}{z-x}} = \frac{z-x}{z-x+t}$$

$$G_H(t,x) = \sum_{k=0}^{k\to+\infty} \frac{t^k}{k!} H_k(x) = \frac{e^{+x^2}}{2i\pi} \oint \frac{e^{-z^2}}{(z-x)^1} \frac{z-x}{z-x+t} dz_{r\to 0} = \frac{e^{+x^2}}{2i\pi} \oint \frac{e^{-z^2}}{z-(x-t)} dz_{r\to 0}$$

$$using\ the\ Theorem\ of\ Cauchy\ where\ f(a) = \frac{1}{2i\pi} \oint \frac{f(z)}{z-a} dz_{r\to 0}\ \ for\ \mathrm{f}\ bein\ regular\ in\ "a"$$

$$G_H(t,x) = \sum_{k=0}^{k\to+\infty} \frac{t^k}{k!} H_k(x) = e^{-(x-t)^2} e^{+x^2} = e^{-t^2+2xt}$$

There is an even more direct way to set this expression via the Taylor-Young formula.

Indeed since $H_k(x) = (-1)^k e^{x^2} \frac{d}{d^k}\left[e^{-x^2}\right]$

$$G_H(t,x) = \sum_{k=0}^{k\to+\infty} \frac{t^k}{k!} H_k(x) = \sum_{k=0}^{k\to+\infty} \frac{t^k}{k!} (-1)^k e^{x^2} \frac{d}{d^k}\left[e^{-x^2}\right] = e^{x^2} \sum_{k=0}^{k\to+\infty} \frac{(-t)^k}{k!} \frac{d}{d^k}\left[e^{-x^2}\right]$$

The Taylor-Young formula is:

$$f(u) = \sum_{k=0}^{k\to+\infty} \frac{(u-x)^k}{k!} \frac{d}{d^k}[f](x)$$

Setting $f(x) = e^{-x^2}\ and\ u - x = -t\ ie\ u = x - t$ we find that:

$$e^{-(x-t)^2} = \sum_{k=0}^{k\to+\infty} \frac{(-t)^k}{k!} \frac{d}{d^k}\left[e^{-x^2}\right](x)$$

Therefore: $G_H(t,x) = e^{-(x-t)^2} e^{x^2} = e^{-t^2+2tx}$

Let's now consider the scalar products of 2 Hermite polynomials again:

$$< H_k | H_l >= \int_{-\infty}^{+\infty} H_k(x) H_l(x) e^{-x^2} dx$$

Astutely let's consider the scalar product of 2 generating function values, one in "t" and another one in "u":

$$< G_H(t,x) | G_H(u,x) >= \int_{-\infty}^{+\infty} G_H(t,x) G_H(u,x) e^{-x^2} dx$$

Ok we consider here the generalized integral over series, ie infinite sums of infinite sums. Fortunately our core function is smooth!

$$< G_H(t,x) | G_H(u,x) > = \int_{-\infty}^{+\infty} \sum_{kl=0}^{k,l\to+\infty} \frac{t^k}{k!} \frac{u^l}{l!} H_k(x)\, H_k(x)\, e^{-x^2} dx = \sum_{kl=0}^{k,l\to+\infty} \frac{t^k}{k!} \frac{u^l}{l!} < H_k | H_l >$$

The good news is that we have a pretty simple expression for the generating functions and they define analytic functions in "t" or "u" equally…

Thus $< G_H(t,x)|G_H(u,x) >= \sum_{kl=0}^{k,l\to+\infty} \frac{t^k}{k!}\frac{u^l}{l!} < H_k|H_l > = \int_{-\infty}^{+\infty} e^{-t^2-2xt} e^{-u^2-2xu} e^{-x^2} dx$

$$< G_H(t,x)|G_H(u,x) >= e^{2u} \int_{-\infty}^{+\infty} e^{-u^2-2xu-2ut-2xt \quad ^2-x^2} dx = e^{+2ut} \int_{-\infty}^{+\infty} e^{-(u+x+t)^2} dx$$

The generalized integral is well known given that the bounds are infinite (it does not depend upon any value of "t" or "u" actually): $\int_{-\infty}^{+\infty} e^{-(u+x+t)^2} dx = \sqrt{\pi}$.

Hence we can write that : $< G_H(t,x)|G_H(u,x) >= \sqrt{\pi} e^{+2ut} = \sum_{k=0}^{k\to+\infty} \sqrt{\pi} \frac{2^k}{k!} u^k t^k$

We formerly wrote that $< G_H(t,x)|G_H(u,x) >= \sum_{kl=0}^{k,l\to+\infty} \frac{t^k}{k!}\frac{u^l}{l!} < H_k|H_l >$

This leads to the same result since we can identify the derivative values in 0 for these functions that are smooth anyway: $< H_k|H_l >= \delta_{k,l}\sqrt{\pi} k!\, 2^k$

So now we would consider the Hermite's functions that derive from these polynomials:

$$h_n(x) = \frac{e^{-\frac{x^2}{2}} H_n(x)}{\sqrt{\sqrt{\pi} n!\, 2^n}} = \frac{e^{-\frac{x^2}{2}}}{\sqrt{\sqrt{\pi} n!\, 2^n}} (-1)^n e^{x^2} \frac{d}{d^n}\left[e^{-x^2}\right] = \frac{(-1)^k}{\sqrt{\sqrt{\pi} n!\, 2^n}} e^{\frac{1}{2}x^2} \frac{d}{d^n}\left[e^{-x^2}\right]$$

We would define a different scalar product but with the goal to generate the same results that matter to us:

$$< h_k|h_l >= \int_{-\infty}^{+\infty} h_k(x) h_l(x) dx =< H_k|H_l >= \frac{1}{\sqrt{\sqrt{\pi} k!\, 2^k}} \frac{1}{\sqrt{\sqrt{\pi} l!\, 2^l}} \int_{-\infty}^{+\infty} H_k(x) H_l(x) e^{-x^2} dx$$

Then $< h_k|h_l >= \delta_{k,l}$

We will now use these Hermite's functions to define a very interesting properties with regards to the Fourier transform… which is what could be very helpful to generalize the resolution of the Black and Schole equation….including then any kind of function…

Let's first set a very useful property of the Fourier transforms for these Hermite's functions….We start with the root function itself e^{-x^2}

$$F\left(e^{-x^2}\right)(u) = \int_{-\infty}^{+\infty} e^{-x^2} e^{-iux} dx = e^{-\frac{1}{4}u^2} \int_{-\infty}^{+\infty} e^{-(x+\frac{iu}{2})^2} dx = \sqrt{\pi} e^{-\frac{1}{4}u^2}$$

Note: quick reminder on : $\int_{-\infty}^{+\infty} e^{-x^2} dx = \sqrt{\pi}$

We integrate in 2 dimensions and use the theorem of Fubini on a disk of radius R and move R towards infinity next. Here is the quick proof because it is very useful in many situations…We start from the integration of what will be almost the square of the integral that we target posing $x = r\cos\theta$ *and also* $y = r\sin\theta$ *with a Jacobian matrix:* $J = \begin{bmatrix} \cos\theta & \sin\theta \\ -r\sin\theta & r\cos\theta \end{bmatrix} = r$

$$by\ application\ of\ Fubini's\ theorem : I(R) = \int_0^{+R} \int_0^{2\pi} e^{-x^2} e^{-y^2} dxdy = \int_0^{+R} \int_0^{2\pi} e^{-r^2} (r) dr\, d\theta$$

$$I(R) = \int_0^{+R} \int_0^{2\pi} r e^{-r^2} dr d\theta = \int_0^{2\pi} \left(\int_0^R r e^{-r^2} dr \right) d\theta = \int_0^{2\pi} \left[\frac{1}{2} e^{-r^2} \right]_0^R d\theta$$

$$I(R) = \frac{1}{2}\int_0^{2\pi} 1 - e^{-\frac{1}{2}R^2} d\theta = \pi\left(1 - e^{-\frac{1}{2}R^2}\right)$$

The function "I(R)" is well behaved and we can easily prove the convergence of :

$$\lim_{R\to\infty} I(R) = \pi$$

Finally , on a square of length "R", we would define:

$$J(R) = \int_0^{+R}\int_0^{R} e^{-x^2}\, e^{-y^2} dxdy$$

The Pythagoras theorem indicates that a square is containing the disk of radius R but the square itself is included in the disk of radius $\sqrt{2}R$. *The function that we integrate is always positive.Thus we can bracket J(R) as follows:*

$$I(R) < J(R) < I\left(\sqrt{2}R\right) and\ so\ \pi\left(1 - e^{-\frac{1}{2}R^2}\right) < J(R) < \pi\left(1 - e^{-R^2}\right)$$

And we deduce for sure that: $\lim_{R\to\infty} I(R) = \pi = \left(\int_{-\infty}^{+\infty} e^{-x^2} dx\right)^2$ *hence* $\int_{-\infty}^{+\infty} e^{-x^2} dx = \sqrt{\pi}$

We can likewise infer that $F\left(e^{-\frac{1}{2}x^2}\right)(u) = \int_{-\infty}^{+\infty} e^{-\frac{1}{2}x^2} e^{-iux} dx = \sqrt{\pi} e^{-\frac{1}{2}u^2}$

For that matter let's just observe that if we pose $z = \frac{z}{\sqrt{2}}\ and\ v = u\sqrt{2}$:

$$F\left(e^{-z^2}\right)(u) = \int_{-\infty}^{+\infty} e^{-z^2} e^{-ivx} dx = e^{-\frac{1}{4}v^2}\int_{-\infty}^{+\infty} e^{-(z+\frac{iv}{2})^2} dx = \sqrt{\pi} e^{-\frac{1}{4}v^2}$$

Let's now look at the following Fourier transforms:

$$F\left(e^{\frac{1}{2}x^2}\frac{d^k\left[e^{-x^2}\right]}{dx^k}\right)(u) = \int_{-\infty}^{+\infty} e^{\frac{1}{2}x^2}\frac{d^k\left[e^{-x^2}\right]}{dx^k} e^{-iux} dx$$

$$as\ we\ showed\ earlier\ F(e^{-x^2})(u) = \sqrt{\pi} e^{-\frac{1}{4}u^2}$$

We are going to use the generating function by considering altogether all the "k" functions

$$G_H(t,x) = \sum_{k=0}^{k\to+\infty} \frac{t^k}{k!} H_k(x) = e^{-(x+t)^2} e^{+x^2} = e^{-t^2+2xt}$$

Let's recall that could have obtained the result much faster than what we did before….

$$G_H(t,x) = \sum_{k=0}^{k\to+\infty} \frac{t^k}{k!} H_k(x) = \sum_{k=0}^{k\to+\infty} \frac{t^k}{k!}(-1)^k e^{x^2}\frac{d}{d^k}\left[e^{-x^2}\right] = \sum_{k=0}^{k\to+\infty} \frac{(-t)^k}{k!} e^{x^2}\frac{d}{d^k}\left[e^{-x^2}\right]$$

We recognize here the Taylor expansion of the function $e^{-(x-t)^2}$ times e^{x^2}. Therefore:

$$G_H(t,x) = \sum_{k=0}^{k\to+\infty} \frac{t^k}{k!} H_k(x) = e^{-(x-t)^2+x^2} = e^{-t^2+2xt}\ CQFD$$

But the relation that we proved in the first place runs even deeper, ie

$$H_k(x) = (-1)^k e^{x^2} \frac{d}{d^k}\left[e^{-x^2}\right] = \frac{k!}{2i\pi} e^{+x^2} \oint \frac{e^{-z^2}}{(z-x)^{k+1}} dz_{r\to 0}$$

Now, because the Fourier transform is a linear operator and because we apply a polynomial expression I, "t" or in "u" through the Generating function, we can compute astutely all the Fourier transforms in one go….But we actually only can compute Fourier transforms if we add some $e^{-b^{\ 2}}$ with b>0 term. Otherwise the generalized sum or integrals would NOT converge.

$$F_{e^{-b^{\ 2}}G_H(t,x)}(y) = \int_{-\infty\to x}^{x\to+\infty} \sum_{k=0}^{k\to+\infty} \frac{t^k}{k!} H_k(x) e^{-b^{\ 2}-ixy} dx = \sum_{k=0}^{k\to+\infty} \int_{-\infty\to x}^{x\to+\infty} \frac{t^k}{k!} H_k(x) e^{-bx^2-ixy} dx$$
$$= \sum_{k=0}^{k\to+\infty} \frac{t^k}{k!} F_{e^{-bx^2}H_{k(x)}}(y)$$

On the other hand: $F_{e^{-bx^2}G_H(t,x)}(y) = \int_{-\infty\to x}^{x\to+\infty} e^{-t^2+2xt-ixy-b^{\ 2}} = e^{-t^2} \int_{-\infty\to x}^{x\to+\infty} e^{-bx^2-x(iy-2t)} dx$

$$F_{e^{-b^{\ 2}}G_H(t,x)}(y) = e^{-t^2} \int_{-\infty\to x}^{x\to+\infty} e^{-b\left(x^2+\frac{2x\left(\frac{iy}{2}-t\right)}{b}+\frac{\left(\frac{iy}{2}-t\right)^2}{b^2}\right)} e^{+b\frac{\left(\frac{iy}{2}-t\right)^2}{b^2}} dx$$

$$F_{e^{-bx^2}G_H(t,x)}(y) = e^{-t^2+b\frac{\left(\frac{iy}{2}-t\right)^2}{b^2}} \int_{-\infty\to x}^{x\to+\infty} e^{-b\left(x+\frac{\left(\frac{iy}{2}-t\right)}{b}\right)^2} dx$$

Since $\int_{-\infty}^{+\infty} e^{-\frac{1}{2}z^2} dz = \sqrt{2\pi}$, we need to set $z = \sqrt{2b}\left(x + \frac{\left(\frac{iy}{2}-t\right)}{b}\right)$ so that $\frac{z^2}{2} = b\left(x + \frac{\left(\frac{iy}{2}-t\right)}{b}\right)^2$ modulo also that $dx = \frac{dz}{\sqrt{2b}}$ as a result :

$(reminder: \int_{-\infty}^{+\infty}\int_{-\infty}^{+\infty} e^{-\frac{1}{2}(x^2+y^2)} dxdy = \int_{R=0}^{R\to+\infty}\int_{u=0}^{u\to 2\pi} Re^{-\frac{1}{2}R^2} dRdu = 2\pi = \left(\int_{-\infty}^{+\infty} e^{-\frac{1}{2}z^2} dz\right)^2)$

So….

$$\int_{-\infty\to x}^{x\to+\infty} e^{-b\left(x+\frac{\left(\frac{iy}{2}+t\right)}{b}\right)^2} dx = \sqrt{\frac{2\pi}{2b}} = \sqrt{\frac{\pi}{b}}$$

And therefore: $F_{e^{-b^{\ 2}}G_H(t,x)}(y) = \sqrt{\frac{\pi}{b}} e^{-t^2+b\frac{\left(\frac{iy}{2}-t\right)^2}{b^2}} = \sum_{k=0}^{k\to+\infty} \frac{t^k}{k!} F_{e^{-bx^2}H_{k(x)}}(y)$

$$-t^2 + b\frac{\left(\frac{iy}{2}-t\right)^2}{b^2} = -t^2\left(1-\frac{1}{b}\right) - \frac{1}{4b}y^2 - \frac{1}{b}ity = -\frac{1}{4b}y^2 - t^2\frac{(b-1)}{b} - \frac{1}{b}ity$$

We observe that if 0<b<1 we may have a problem to retrieve a Fourier transform that resurrects the Hermite polynomial since, what we would like to have is something on the line of $-a(t^2 + 2t\frac{y}{d})$:

$$-t^2 + b\frac{\left(\frac{iy}{2}-t\right)^2}{b^2} = -\frac{1}{4b}y^2 - \left[t\left(\frac{(b-1)}{b}\right)^{\frac{1}{2}}\right]^2 - 2\left(\frac{(b-1)}{b}\right)^{-\frac{1}{2}}\frac{iy}{2b}t\left(\frac{(b-1)}{b}\right)^{\frac{1}{2}}$$

Let's focus on the term $\left(\frac{(b-1)}{b}\right)^{-\frac{1}{2}}\frac{iy}{2b}$….say for example that 0<b<1:$i^2\left(\frac{(1-b)}{b}\right)^{-\frac{1}{2}}=-\left(\frac{(1-b)}{b}\right)^{-\frac{1}{2}}$ with $\left(\frac{(1-b)}{b}\right)^{-\frac{1}{2}}$ being a real number. If now b>1, then we still have the same result but in this case $\left(\frac{(1-b)}{b}\right)^{-\frac{1}{2}}$ is going to be a pure imaginary number

We thus infer, from the fact we looked for some $-a(t^2-2(cy)t)$, that allows us to apply the more generic expression $G_H(t,x)=\sum_{k=0}^{k\to+\infty}\frac{t^k}{k!}H_k(x)=e^{-(x-t)^2+x^2}=e^{-t^2+2xt}$ transposing t-> $t\left(\frac{(b-1)}{b}\right)^{\frac{1}{2}}$ and

x->$\left(\frac{(1-b)}{b}\right)^{-\frac{1}{2}}\frac{iy}{2b}$:

$$F_{e^{-b\ ^2}G_H(t,x)}(y)=\sqrt{\frac{\pi}{b}}e^{-t^2+b\frac{\left(\frac{iy}{2}-t\right)^2}{b^2}}=\sqrt{\frac{\pi}{b}}e^{-\frac{1}{4b}y^2-\left[t\left(\frac{(b-1)}{b}\right)^{\frac{1}{2}}\right]^2-2\left[\left(\frac{(b-1)}{b}\right)^{-\frac{1}{2}}\frac{iy}{2b}\right]t\left(\frac{(b-1)}{b}\right)^{\frac{1}{2}}}=$$

$$\sum_{k=0}^{k\to+\infty}\frac{t^k}{k!}F_{e^{-bx^2}H_{k(x)}}(y)=\sqrt{\frac{\pi}{b}}\sum_{k=0}^{k\to+\infty}\left(\frac{(b-1)}{b}\right)^{\frac{k}{2}}\frac{t^k}{k!}e^{-\frac{1}{4b}y^2}H_k\left(\left(\frac{(1-b)}{b}\right)^{-\frac{1}{2}}\frac{iy}{2b}\right)$$

$$Let's\ notice\ that\ \left(\left(\frac{(1-b)}{b}\right)^{-\frac{1}{2}}\frac{y}{2b}\right)^2=\frac{\frac{1}{4b^2}y^2b}{1-b}=\frac{y^2}{4b(1-b)}$$

We also can infer that , in general, for any value of b actually, that

$$\boldsymbol{F_{e^{-bx^2}H_{k(x)}}(y)=\sqrt{\frac{\pi}{b}}e^{+\frac{b}{4(1-b)}y^2}\left(\frac{(1-b)}{b}\right)^{\frac{k}{2}}\frac{(i)^k}{k!}e^{-\left(\left(\frac{(1-b)}{b}\right)^{-\frac{1}{2}}\frac{y}{2b}\right)^2}H_k\left(\left(\frac{(1-b)}{b}\right)^{-\frac{1}{2}}\frac{y}{2b}\right)}$$

In the formula above we see that for a given adequate "b" value we obtain almost an eigen value ie a "b-Hermite function" in "y" has a Fourier transform that turns out to be the $\frac{1-b}{b}Hermite\ function\ in\frac{y}{2b}$ ". It is therefore easy to invert the process thereafter to "solve" an equation in Fourier transforms and next "infer" what the native solution is going to be.

We can note the nice symmetry around b=1/2…we retrieve a well known result

$$\boldsymbol{\underbrace{F_{e^{-\frac{1}{2}x^2}H_{k(x)}}}_{h_k(x)}(y)=\sqrt{2\pi}\frac{(i)^k}{k!}\underbrace{e^{-\frac{1}{2}(y)^2}H_k(y)}_{h_k(y)}}$$

That shows that Hermite's functions are eigen values for the space of Fourier transforms for "b=$\frac{1}{2}$"

Why should we care?!

In a stochastic equation like the one of Black&Scholes we have 2 functions that we could project on the basis of Hermite's function so that, after the move to Fourier transforms and the resolution in a polynomial form, we can easily project this solution again on the Hermite's functions basis and use the fact that the Hermite's functions themselves are the eigen value of the Fourier operator because we can notice as well that $F^{-1}\left(f_{(x)}\right)(y) = \frac{1}{\sqrt{2\pi}}\int f(x)e^{+ixy}dy = F\left(f_{(x)}\right)(-y) = F\left(f_{(-x)}\right)(y)$ and this implies that

$$F°F(f)(x) = F_{F_{f(x)}(y)}(x) = F^{-1}_{F_{f(x)}(y)}(-x) = F^{-1}_{F_{f(x)}(-y)}(x) = F^{-1}_{F_{f(-x)}(y)}(x) = f(-x)$$

$$F°F(f)(z) = \frac{1}{\sqrt{2\pi}}\int \frac{1}{\sqrt{2\pi}}\int f(x)e^{-ixy}\, e^{-iyz}dxdy = \frac{1}{\sqrt{2\pi}}\int \frac{1}{\sqrt{2\pi}}\int f(x)e^{-i(-x)(-y)}\, e^{+i(-y)z}dxdy$$

$$F°F(f)(z) = (-1)^2\int \frac{1}{\sqrt{2\pi}}\int f(-x')e^{-i(x')(y')}\, e^{+i(y')z}dx'dy' = F^{-1}_{F_{f(-x)}(y')}(z) = f(-z)$$

We infer also that $F°F°F°F = Id$

Technically we know that a convolution product will induce the product of the respective Fourier transforms. Here if we can think about reversing the Fourier transform of a convolution product involving initially 2 Hermite functions having the same "b" but not the same index, we end up with an integral that is not nil only when the index is "k" on both as we will see. Below this hopefully will permit to directly operate on the "coordinates" of the functions via convolution products.

Let's sketch it briefly . Say we have 2 functions "f" and "g" that we project on a family of functions of the kind of the Hermite functions, ie $A_k e^{-b(x)^2}H_k(x)$. Using the generating function we will be able to secure that they are orthogonal and we will find the appropriate values for the factor A_k so that they are of norm 1 as per the measure that secure the orthogonality.

How do we achieve that? Answer we first compute the product of 2 generating functions involving a "u" and a "v" as follows:

$$< G_{e^{-bx^2}H}(u,x)|G_{e^{-bx^2}H}(v,x) >= \sum_{kl=0}^{k,l\to+\infty}\frac{t^k}{k!}\frac{u^l}{l!} < e^{-bx^2}H_k|e^{-bx^2}H_l >$$

$$with\ < e^{-bx^2}H_k|e^{-bx^2}H_l > = \int_{-\infty\to x}^{x\to+\infty} e^{-2bx^2}H_k(x)H_m(x)e^{-dx^2}dx$$

We know that we achieve the orthogonality by simply picking "d" the "measure" as "d=1-2b".

In order to achieve the norm 1 effect, we will infer the factors A_k through ensuring that we can invert the sigma and the integral signs all along. Therefore :

$$< G_{e^{-bx^2}H}(u,x)|G_{e^{-bx^2}H}(v,x) >= \int_{-\infty\to x}^{x\to+\infty} e^{-x^2-u^2+2xu-\ ^2+2xv}dx$$

$$< G_{e^{-bx^2}H}(u,x)|G_{e^{-bx^2}H}(v,x) >= e^{+2uv}\int_{-\infty\to x}^{x\to+\infty} e^{-x^2-u^2+2xu-\ ^2+2xv-2uv}dx$$

$$< G_{e^{-bx^2}H}(u,x)|G_{e^{-bx^2}H}(v,x) >= e^{+2uv}\int_{-\infty\to x}^{x\to+\infty} e^{-(x-(u+v))^2}dx$$

$$By\ a\ similar\ change\ of\ variable: based\ on\ \int_{-\infty}^{+\infty} e^{-z^2}dz = \sqrt{\pi}$$

$$< G_{e^{-bx^2}H}(u,x)|G_{e^{-bx^2}H}(v,x) >= \sqrt{\pi}e^{+2uv} = \sum_{kl=0}^{k,l\to+\infty} \frac{v^k}{k!}\frac{u^l}{l!} < H_k|H_l > = \sum_{k=0}^{k\to+\infty} \sqrt{\pi}\frac{2^k}{k!}u^k v^k$$

We can see **that as long as we pick a "d", ie a well adapted measure (0≤d=1-2b with 0<b≤1/2) to the value that we picked with "b"** (the convenient one as mentioned is "b=1/2" but we can pick any value in]0;1/2] for b and that works.

So we infer that $A_k = \left(\sqrt{\pi}\frac{2^k}{k!}\right)^{-\frac{1}{2}}$ and we set $g_k(x) = A_k e^{-b(x)^2}H_k(x)$

The nice outcome is that only the "k x k " terms are not zero through this "scalar product" via the projection on this generalized basis of Hermite functions.

So let's consider the case of the Black&Scholes equation where we look at the trading costs for example as entering into some convolution product with the "price" function of some vanilla option that follows the stochastic equation "on average" made through this convolution product. Let's project the 2 functions on some "b-Hermite functions" basis. We know that the Fourier transform of their convolution product will be the product of the respective Fourier transforms. The latter, as we saw for any appropriate "b " value in]0,1[, will project onto a similar basis of Hermite functions ie not on "b , y" but on "$\frac{1-b}{b}$ and $\frac{y}{2b}$". Thus picking a good "d= $1-\frac{1-b}{2b} = \frac{3b-1}{2b} > 0$" we are certain then that an additional "averaging" modulo this measure will retain only the "diagonal" elements ie only the terms "k x k" as proved right before. Then it is straightforward to infer the terms of the of the "solution" in Fourier transform terms "on average based on the measure set by d". And we just need to reverse the mutation that we established right above to infer the factors of the native solution. So we would not have a closed form expression but a series instead projecting the solution onto the "b-Hermite functions basis" . But there is a more direct way…

What happens if we project our target functions like the option price "f", the "trading cost" or the "risk neutral rate r" and if we now look and Fourier transform of their convolution product?

We said that we might either " count " the empirical evidence or else build the convolution product around a convenient Gaussian kernel.

Let's just note what happens when we consider the Fourier transform of the convolution product for 2 of our core functions:

$$F_{\int e^{-b(u)^2}H_k(u)e^{-b(x-u)^2}H_k(x-u)du}(y) = \int_{-\infty\to x}^{x\to+\infty}\int \left(e^{-b(u)^2}H_k(u)e^{-b(x-u)^2}H_k(x-u)du\right)e^{-ixy}dx$$

$$F_{\int e^{-b(u)^2}H_k(u)e^{-b(x-u)^2}H_k(x-u)du}(y) = F_{e^{-b(u)^2}H_k(u)}(y)F_{e^{-b(u)^2}H_m(u)}(y)$$

And since we care about what $\int e^{-b(u)^2}H_k(u)e^{-b(x-u)^2}H_k(x-u)du$ is, we target

$$\int e^{-b(u)^2}H_k(u)e^{-b(x-u)^2}H_k(x-u)du = \frac{1}{2\pi}\int_{-\infty\to y}^{y\to+\infty} e^{+ixy}\int F_{\int e^{-b(u)^2}H_k(u)e^{-b(x-u)^2}H_k(x-u)du}(y)dy$$

So we really look at:

$$\int e^{-b(u)^2}H_k(u)e^{-b(x-u)^2}H_k(x-u)du = \frac{1}{2\pi}\int_{-\infty\to y}^{y\to+\infty} e^{+ixy}F_{e^{-b(u)^2}H_k(u)}(y)F_{e^{-b(u)^2}H_m(u)}(y)dy$$

However , as much we could set conveniently the Fourier transforms we end up having to compute the reverse Fourier transform of in some simplified way:

$$\frac{1}{2\pi}\int e^{-c(y)^2}H_k(ay)e^{-c(y)^2}H_m(ay)e^{+ixy}dy$$

Recall:

$$F_{e^{-bx^2}H_{k(x)}}(y) = \sqrt{\frac{\pi}{b}}e^{+\frac{b}{4(1-b)}y^2}\left(\frac{(1-b)}{b}\right)^{\frac{k}{2}}\frac{(i)^k}{k!}e^{-\left(\left(\frac{(1-b)}{b}\right)^{-\frac{1}{2}}\frac{y}{2b}\right)^2}H_k\left(\left(\frac{(1-b)}{b}\right)^{-\frac{1}{2}}\frac{y}{2b}\right)$$

$$F_{e^{-bx^2}H_{k(x)}}(y) = \sqrt{\frac{\pi}{b}}\frac{(i)^k}{k!}e^{+\left(\frac{y}{2\sqrt{\frac{1}{b}-1}}\right)^2}\left(\sqrt{\frac{1}{b}-1}\right)^k e^{-\left(\frac{y}{2\sqrt{b(1-b)}}\right)^2}H_k\left(\frac{y}{2\sqrt{b(1-b)}}\right)$$

Our problem is that there is NO easy way to handle the term e^{+ixy} in $\int e^{-b(y)^2}H_k(y)e^{-b(y)^2}H_m(y)e^{+ixy}dy$, because this term induces a "shift" in the exponentials but NOT in the Hermite's polynomials. What is this product worth?!

We consider a core function $f_k(x) = x^k e^{-ax^2}$ that is the "k-th leg" of a Kernel $K(x) = e^{-a(x-y)^2}$.

We infer rather easily that: $\widehat{f_k}(y) = i^{-k}(4a)^{-\frac{k}{2}}\sqrt{\frac{\pi}{a}}H_k\left(\frac{y}{\sqrt{4a}}\right)e^{-\frac{y^2}{4a}}$

Let's indeed remember 2 important properties with a peculiar definition of the Fourier transform:

1- $F_{xf}(y) = \int xf(x)e^{-ixy}dx = i\frac{dF_f}{dy}$ and thus $F_{x^k f}(y) = \int x^k f(x)e^{-ixy}dx = i^k\frac{d^k F_f}{dy^k}$

2- $F_{e^{-ax^2}}(y) = \int e^{-ax^2}e^{-ixy}dx = \sqrt{\frac{\pi}{a}}e^{-\frac{1}{4a}y^2}$ and therefore , once we combine the result 1- with this one here: $\frac{d^k F_f}{dy^k} = \sqrt{\frac{\pi}{a}}\frac{d^k\left(e^{-\frac{1}{4a}y^2}\right)}{dy^k} = i^k(-1)^k\sqrt{\frac{\pi}{a}}(4a)^{-\frac{k}{2}}H_k\left(\frac{y}{\sqrt{4a}}\right)e^{-\frac{1}{4a}y^2}\left(consider\ z = \frac{y}{\sqrt{4a}}\right)$

3- $which\ induces\ that\ F_{x^k e^{-ax^2}}(y) = (-i)^k\sqrt{\frac{\pi}{a}}(4a)^{-\frac{k}{2}}H_k(\frac{y}{\sqrt{4a}})e^{-\frac{1}{4a}y^2}$

This result is quite convenient for us because this gives a "hint" at the kind of "transform" we need so that we can easily "project" convolution products…

We just set that $F_{x^k e^{-a\ 2}}(y) = (-i)^k\sqrt{\frac{\pi}{a}}(4a)^{-\frac{k}{2}}H_k(\frac{y}{\sqrt{4a}})e^{-\frac{1}{4a}y^2}$ ….and actually we know that $F_{F_{x^k e^{-a\ 2}}(y)}(-x) = 2\pi x^k e^{-a\ 2}$ because $f(x) = \underbrace{\frac{1}{2\pi}\int_{-\infty\to y}^{y\to+\infty} e^{-i(-x)y}\underbrace{\left(\int_{-\infty\to u}^{u\to+\infty} f(u)\,e^{-iuy}du\right)}_{g(y)=F_{f(u)}(y)}dy}_{F_{g(y)}(-x)\to 2\pi f(x)}$

So…If, instead of a straight product of Hermite's functions we inherited of some convolution product of such Hermite's function with some varying factor "b" we then may operate a "measured" convolution product so that we obtain functions like $x^k e^{-ax^2}$ as Fourier transforms actually.

The relation above implies that $F_{F_{x^k e^{-ax^2}}(y)}(-x) = F_{(-i)^k\sqrt{\frac{\pi}{a}}(4a)^{-\frac{k}{2}}H_k(\frac{y}{\sqrt{4a}})e^{-\frac{1}{4a}y^2}}(-x) = 2\pi x^k e^{-ax^2}$

We can then make few adjustments: $F_{H_k(\frac{y}{\sqrt{4a}})e^{-\frac{1}{4a}y^2}}(x) = 2(i)^k\sqrt{a\pi}(4a)^{+\frac{k}{2}}(-x)^k e^{-a^2}$

We next "normalize y", and that is tricky and not really useful:

$$F_{H_k\left(\frac{y}{\sqrt{4a}}\right)e^{-\frac{1}{4a}y^2}}(x) = \int_{-\infty\to y}^{y\to+\infty} H_k\left(\frac{y}{\sqrt{4a}}\right)e^{-\frac{1}{4a}y^2}\, e^{-ixy}dy = \sqrt{4a}\int_{-\infty\to z}^{z\to+\infty} H_k(z)e^{-z^2}\, e^{-i\sqrt{4a}xz}dz$$

$$F_{H_k(\frac{y}{\sqrt{4a}})e^{-\frac{1}{4a}y^2}}(x) = \sqrt{4a}\,F_{H_k(z)e^{-z^2}}\left(\sqrt{4a}x\right) = 2(i)^k\sqrt{a\pi}(4a)^{+\frac{k}{2}}(-x)^k e^{-ax^2}$$

$$F_{H_k(z)e^{-z^2}}(z) = (-i)^k\sqrt{\pi}(z)^k e^{-\frac{1}{4}z^2}$$

This last result could be obtained straight from integration by part on f provided f is smooth enough

$F_{\frac{df}{dx}}(y) = \int \frac{df}{dx}(x)e^{-ixy}dx = iyF_f(y)$ given that :

$$H_k(x) = (-1)^k e^{x^2}\frac{d^k}{dx^k}\left[e^{-x^2}\right] = (-1)^k\frac{k!}{2i\pi}e^{+x^2}\oint\frac{e^{-z^2}}{(z-x)^{k+1}}dz_{r\to 0}$$

So we are back to the problem of determining what the convolution product of 2 Hermite's functions would be modulo a well chosen "measure". We saw that a straight usual convolution product does not work well in that the expression is not user friendly. We then had use of functions like $e^{-b(u)^2}H_k(u)$. The resulting Fourier transform of a usual convolution product like

$$\int e^{-b(u)^2}H_k(u)e^{-b(v-u)^2}H_k(v-u)du$$

Would induce a problematic formula that induced a "shift" within the Hermite's functions

$$\frac{1}{2\pi}\int e^{-c(y)^2}H_k(dy)e^{-c(y)^2}H_m(dy)e^{+ixy}dy$$

We would like to have a "normalized" integral that opens the way to $(-x)^k e^{-ax^2}$:

$$\int e^{-b(u)^2}H_k(u)e^{-b(v-u)^2}H_m(v-u)d\mu_v(u) = \int e^{-(u)^2}H_k(u)e^{-(v-u)^2}H_m(v-u)du$$

We infer from that $\boldsymbol{d\mu_v(u) = e^{-(1-b)((u)^2+(v-u)^2)}du}$ and observe that the "measure" becomes a function of "v". This is a special convolution product….let's not forget it!

Moving to the Fourier transform:

$$F_{\int e^{-(1)(u)^2}H_k(u)e^{-(1)(v-u)^2}H_m(v-u)d\mu_v(u)}(y) = \int\int e^{-(1)(u)^2}H_k(u)e^{-(1)(v-u)^2}H_m(v-u)e^{-i(u+v-u)y}dvdu$$

$$F_{\int e^{-(1)(u)^2}H_k(u)e^{-(1)(v-u)^2}H_m(v-u)d\mu_v(u)}(y) = F_{e^{-(1)(u)^2}H_k(u)}(y)F_{e^{-(1)(u)^2}H_m(u)}(y)$$

$$F_{\int e^{-(u)^2}H_k(u)e^{-(v-u)^2}H_m(v-u)d\mu_v(u)}(y) = \sqrt{\pi}(-i)^{k+m}\sqrt{\pi}(z)^{k+m}e^{-\frac{1}{2}z^2}$$

We then should consider the expression in a more convenient way:

$$F_{\int e^{-(u)^2}H_k(u)e^{-(v-u)^2}H_m(v-u)d\mu_v(u)}(y) = \sqrt{\pi}\left(\frac{1}{\sqrt{2}}\right)^{k+m}(-i)^{k+m}\sqrt{\pi}\left(\sqrt{2}z\right)^{k+m}e^{-\frac{1}{4}(\sqrt{2}z)^2}$$

We deduce straightaway the result of a convolution product given the special "measure" here:

$$\int e^{-(u)^2}H_k(u)e^{-(v-u)^2}H_m(v-u)d\mu_v(u) = \sqrt{2\pi}(\frac{1}{\sqrt{2}})^{k+m}H_{k+m}\left(\frac{v}{\sqrt{2}}\right)$$

This is quite interesting when we now consider the convolution product, with this quite specific measure, ie

$d\mu_v(u) = e^{-(1-b)((u)^2+(v-u)^2)}du$, that would differ from this "other measure" that we would use to set the orthogonality and projection criteria as $\boldsymbol{with} < \boldsymbol{e^{-bx^2}H_k|e^{-bx^2}H_l} > = \int_{-\infty\to x}^{x\to+\infty} \boldsymbol{e^{-2bx^2}H_k(x)H_m(x)e^{-dx^2}dx\ where\ clearly\ d = 1 - 2b}$.

So we both use a special scalar product and a special convolution product that depends on our central variable itself.

If we recall now our original vectors on a function "f" and a function "g" as :

$A_k = \left(\sqrt{\pi}\frac{2^k}{k!}\right)^{-\frac{1}{2}}$ and we set $g_k(x) = A_k e^{-b(x)^2}H_k(x)$:

$$f(x) = \sum_{k=0}^{k\to+\infty} a_k g_k(x) \ \ and\ g(x) = \sum_{k=0}^{k\to+\infty} b_k g_k(x)$$

We would set the convolution product with this special measure now:

$$h(v) = \int f(u)g(v-u)d\mu_v(u) = \int \sum_{k,m=0}^{k,m\to+\infty} a_k b_m g_k(u)g_m(x-u)d\mu_v(u)$$

$$h(v) = \sum_{k,m=0}^{k,m\to+\infty} a_k b_m \int g_k(u)g_m(x-u)d\mu_v(u)$$

$$= \sum_{k,m=0}^{k,m\to+\infty} a_k b_m A_k A_m \int e^{-(u)^2}H_k(u)e^{-(v-u)^2}H_m(v-u)d\mu_v(u)$$

$$h(v) = \sum_{k,m=0}^{k,m\to+\infty} a_k b_m \int g_k(u)g_m(x-u)d\mu_v(u) \ = \sum_{k,m=0}^{k,m\to+\infty} a_k b_m A_k A_m \sqrt{2\pi}(\frac{1}{\sqrt{2}})^{k+m}H_{k+m}\left(\frac{v}{\sqrt{2}}\right)$$

We then "see" that we would have to sum all the terms having the same "k+m" index actually!

$$h(v) = \sum_{k,m=0}^{k,m\to+\infty} a_k b_k \int g_k(u)g_m(x-u)d\mu_v(u)$$

$$= \sum_{i=k+m=0}^{i=k+m\to+\infty} \sqrt{2\pi}(\frac{1}{\sqrt{2}})^{i}H_i\left(\frac{v}{\sqrt{2}}\right)\sum_{k=0}^{k=i} a_k A_k b_{i-k}A_{i-k}$$

We thus “only” need to compute the convolution “product” of the series depicting each function at each order level. This is “nice” enough when one knows the profile of the Hermite’s functions that become more and more “oscillating’ as the index “k+m=i” grows.

This is going to be very convenient to aggregate the risk-neutral weightings or the trading costs weightings as compounding distributions versus the “price” distribution itself (over time for example). How will it fit with the derivatives that are involved. The idea here is to reduce all this to compounded normal distributions….

Now if we consider that the function f for example is a normal distribution that we will make converge towards a Dirac measure. This proxy function will look like $e^{-q(u)^2}$ with “q” going towards infinity….

The “profile” of this function is very simple to compute in the basis of Hermite’s functions:

$$with\ < e^{-bx^2} H_k | e^{-qx^2} > = \int_{-\infty \to x}^{x \to +\infty} e^{-x^2} H_k(x) e^{-(q-b)x^2} dx$$

Of course we face a fundamental issue since the “measure” applied to the convolution product is a functions of “v” ie the “price” or the “random variable” in our broader issue that is all about solving the differential equation. Yet, we might actually be still capable of setting the proper factors for the adjoint functions like the “risk neutral” impact and the “trading cost” since “v” is actually the “sum” of 2 random factors, one being the “path followed” by the random variable the other being the “systemic + idiosyncratic risk” that runs in general independently so, our “v” resulting from the congruence of these 2 streams. So the trick seems nonsense while, as soon as we consider the adjoint function as one joint distribution and the “measure” being another “independent” joint distribution we have no margin to maneuver.

In order to compute the factors for our “Dirac” proxy we will use again the generating function….But….We face a problem….the derivation will slide in some ‘x’ or some ‘x²” as a factor of Hermite’s functions…

Fortunately there is a convenient recurrence between the Hermite’s functions that we will detail here.

$$g_k(x) = A_k e^{-bx^2} H_k(x) = A_k e^{-bx^2} e^{+x^2} \frac{\mathrm{d^k}\left[e^{-x^2}\right]}{\mathrm{dx^k}} = \mathrm{A_k} e^{(1-b)x^2} \frac{\mathrm{d^k}\left[e^{-x^2}\right]}{\mathrm{dx^k}}$$

$$\frac{d[g_k(x)]}{dx} = \frac{d[A_k e^{-bx^2} H_k(x)]}{dx} = g_{k+1}(x) + 2x(1-b) g_k(x)$$

We need to get rid of the “x” factor here and replace it with another basis function. We will use the properties of the Hermite polynomials in their native format.

One more check!....

$$\sum_{k=0}^{k \to +\infty} \frac{(t)^k}{k!} H_k(x) = \sum_{k=0}^{k \to +\infty} \frac{(-t)^k}{k!} e^{+x^2} \frac{\mathrm{d^k}\left[e^{-x^2}\right]}{\mathrm{dx^k}} = e^{+x^2} e^{-(x-t)^2} = e^{-x^2+2xt}$$

Now one key relation concerns the Hermite polynomials themselves…

$$H_k(x) = (-1)^k e^{x^2} \frac{d}{d^k}\left[e^{-x^2}\right] = (-1)^k \frac{k!}{2i\pi} e^{+x^2} \oint \frac{e^{-z^2}}{(z-x)^{k+1}} dz_{r \to 0}$$

$$2xH_k(x) = (-1)^k \frac{k!}{2i\pi} e^{+x^2} \oint \frac{2xe^{-z^2}}{(z-x)^{k+1}} dz_{r\to 0}$$
$$= (-1)^k \frac{k!}{2i\pi} e^{+x^2} \oint \frac{-2(z-x)e^{-z^2} + 2ze^{-z^2}}{(z-x)^{k+1}} dz_{r\to 0}$$

$$-(-1)^k \frac{k!}{2i\pi} e^{+x^2} \oint \frac{e^{-z^2}}{(z-x)^k} dz_{r\to 0} = kH_{k-1}(x)$$

$$and \oint \frac{2ze^{-z^2}}{(z-x)^{k+1}} dz_{r\to 0} = \left[\frac{e^{-z^2}}{(z-x)^{k+1}}\right]_o + (-(k+1)) \oint \frac{e^{-z^2}}{(z-x)^{k+2}} dz_{r\to 0} = H_{k+1}(x)$$

$$Therefore\ 2xH_k(x) = kH_{k-1}(x) + H_{k+1}(x)$$

This means that we can always replace an “x” by neighboring functions modulo factors in “k” “k-1” and “k+1”.

So let’s compute the factors of a given proxy for the Dirac measure that we label for now as e^{-bx^2}…..

$$< A_k e^{-bx^2} H_k | e^{-qx^2} > = \int_{-\infty\to x}^{x\to+\infty} A_k H_k(x) e^{-(q+b)x^2} e^{-(1-2b)x^2} dx \ with\ A_k = \left(\sqrt{\pi}\frac{2^k}{k!}\right)^{-\frac{1}{2}}$$

We really care again about this integral :

$$I_k(q,b) = \int_{-\infty\to x}^{x\to+\infty} H_k(x) e^{-(q+b)x^2} e^{-(1-2b)x^2} dx = \int_{-\infty\to x}^{x\to+\infty} H_k(x) e^{-(1+q-b)x^2} dx$$

Using the generating function we can rather quickly infer the value…

$$\sum_{k=0}^{k\to+\infty} \frac{(t)^k}{k!} I_k(q,b) = \sum_{k=0}^{k\to+\infty} \int_{-\infty\to x}^{x\to+\infty} \frac{(t)^k}{k!} H_k(x) e^{-(1+q-b)x^2} dx$$
$$= \int_{-\infty\to x}^{x\to+\infty} \sum_{k=0}^{k\to+\infty} \frac{(t)^k}{k!} H_k(x) e^{-(1+q-b)x^2}\, dx$$

$$\int_{-\infty\to x}^{x\to+\infty} \sum_{k=0}^{k\to+\infty} \frac{(t)^k}{k!} H_k(x) e^{-(1+q-b)x^2}\, dx = \int_{-\infty\to x}^{x\to+\infty} e^{-(q-b)x^2} \underbrace{\sum_{k=0}^{k\to+\infty} \frac{(t)^k}{k!} H_k(x) e^{-x^2}}_{e^{-t^2+2xt}} dx$$

$$\int_{-\infty\to x}^{x\to+\infty} \sum_{k=0}^{k\to+\infty} \frac{(t)^k}{k!} H_k(x) e^{-(1+q-b)x^2}\, dx = \int_{-\infty\to x}^{x\to+\infty} e^{-(q-b)x^2} e^{-t^2+2xt} dx$$
$$= e^{-t^2} \int_{-\infty\to x}^{x\to+\infty} e^{-(q-b)\left(x^2 - \frac{2xt}{q-b} + \frac{t^2}{(q-b)^2}\right)} e^{+\frac{t^2}{q-b}} dx$$

$$\int_{-\infty\to x}^{x\to+\infty} \sum_{k=0}^{k\to+\infty} \frac{(t)^k}{k!} H_k(x) e^{-(1+q-b)x^2}\, dx = e^{-t^2\frac{(q-b-1)}{q-b}} \int_{-\infty\to x}^{x\to+\infty} e^{-(q-b)\left(x-\frac{t}{q-b}\right)^2} dx$$

This formula will work well whenever q>b and we obtain:

$$\int_{-\infty\to x}^{x\to+\infty} \sum_{k=0}^{k\to+\infty} \frac{(t)^k}{k!} H_k(x) e^{-(1+q-b)x^2}\, dx = \sqrt{\frac{\pi}{q-b}} e^{-t^2\frac{(q-b-1)}{q-b}}$$

We can deduce the terms for our future Dirac proxy function:

$$I_{2k}(q,b) = \int_{-\infty \to x}^{x \to +\infty} H_k(x) e^{-(q+b)x^2} e^{-(1-2b)x^2} dx = \sqrt{\frac{\pi}{q-b}} \left(\frac{(q-b-1)}{q-b} \right)^k$$

$$and\ the\ odd\ terms\ are\ all\ nil.$$

We will need now to "explain" how the convolution product operates here to include the derivatives at the limit…

One last key point relates to "what " is needed so that we build a family of orthogonal polynomials from some Gaussian distribution that also handless conveniently so the derivations: we actually need to secure that the derivative is like "ax+b"* "f". Thus any such "kernel" produces derivatives that can be expressed as some 'non polynomial' recurrence. It also enables us to provide a projection basis that morphs the convolution product into a convolution product of the factors of projection. The "price" to pay is that the determination of the factors is to be a bit tricky for the" parameter functions" like the trading costs and the risk neutral rate impacts. The convolution product shall have to mimic reality: the measure of the convolution product is going to bias the calculation. In what follows, let's remember that we might deploy the hermite's functions and measures through a change of variables that will be based ipon the constant values, ie as if we were on the simple Black and Scholes model with constant values that we described before. The derivatives would be based on this core normal distribution.

We can use this basis of polynomials to project any function or any target solution of some differential equation. When we would use the Fourier transform on the terms involved in some SDE like the Black and Scholes equation we could infer the Fourier transform of the solution and directly obtain the actual solution: numerically speaking this is just a change of support from say "u" (the Fourier variable) back to the original variable "x"….or almost!....We may need to run recursive convolution products…of hermite's functions actually…Let's see how it comes…

APPENDIX: the polynomial roots convergence on the unit circle when the factors are normally distributed

Proof of the polynomial having random factors theorem:

The idea is to start with a polynomial: $P_n(z) = a_n z^n + a_{n-1} z^{n-1} + a_{n-2} z^{n-2} + \cdots a_1 z + a_0$

We pose: $P_n(z) = \underbrace{a_n z^n - a_n}_{f(z)=f_n(z)} + \underbrace{a_{n-1} z^{n-1} + a_{n-2} z^{n-2} + a_{n-3} z^{n-3} + \cdots a_1 z + a_0 + a_n}_{g(z)=g_n(z)}$

The fundamental concept is that polynomials tend to have constant values when z is small and tend to diverge to infinity, positive or negative, when z becomes big. This applies also when we are in the set of complex numbers and reason with the module of z , Ie $z = x + iy\ abd\ |z| = \sqrt{x^2 + y^2}$

The idea is that below 1 or above 1 in modulus, the 2 functions have a lower and lower chance to be close to 0 as n grows. For 'f' the only possibility for that would be that $a_n = 0$, and for "g" we would

need either to have $a_{n-1} = a_{n-2} \ldots = 0$ for the area above module=1 or else $a_0 + a_n = 0$ for the "module below 1" case. The probability that "f" is close to 0 goes lower and lower as we pick a_n among an ever higher number following the normal distribution. In a similar fashion, for the case of "g", we have the same outcome as n grows be that for $a_{n-1}\, when\, |z| \gg 1$ or for $a_0 + a_n = 0\, for\, |z| < 1$. In other words, these possibilities tend towards 0 when n grows (see the comment below) . So, for "f" or for "g" likewise, when n grows, the extreme module values (either 0 or infinity) indicate a nil probability that either function is nil.

Therefore, with n growing and growing, the only case when we can have a decent probability for the module of "f+g" to be nil is when the module of z is 1 or close to 1. Then we will show that, again as n grows, that f dominates g more likely than not due to the much higher number of factors that combine to form a sort of Fourier series here that dives towards zero anyway.

Applying the theorem of Rouché we would conclude that the polynomial will have its roots tend towards the unit circle AND actually follow a uniform distribution along the cyclotomic roots actually. The lemma of Riemann-Lebesgue will help quantify the probability that the module of "g" is eventually superior to the module of "f" modulo the fact that this module of f is high outside the unit circle while the module of "g" instead will tend to be close to zero when n is high.

Side comment on the probability that one factor like $a_n = 0$ when "n" grows:

The idea is that the factor is one among many within one draw that is normally distributed. So for a degree "n", we have "n+1" values that are extracted from a normal distribution. We would set the "likely" range as the one that is defined by the expectation of the max. We then can use the value that is inferred from the probability law of the max itself that is

$$P(Max(a_0 \ldots a_n) \geq M) = P_{rob} N_{0,1}\, (X \geq M)^n$$

We would infer the density for the max by derivation and the expectation by proxying the average of the Max. That would deliver an interval $[-E_{Maximum}; +E_{maximum}]$ that would define for our draw of coefficients $(a_0, a_1, a_2 \ldots)$ the likely range of variations of our n+1 values. Here we want to know "how many times" we would have value that is 0 ie that sits in the middle interval.

Then we have a range that we would split into "n+1" or "n" sub-intervals and then compute the probability as per the normal distribution that X lies within the interval containing 0. When n is large the expectation of the max is $\sqrt{2Ln(n)}$. Therefore, the full interval will be $2\sqrt{2Ln(n)}$ and the sub-interval that contains 0 would have a width that would be $\frac{\sqrt{2Ln(n)}}{n} \to 0$ when n grows while the density of the normal distribution remains bounded anyway since the variance is 1 anyway. This shows that anyway, when n grows the possibility of having a 0 or something infinitely close to 0 converges towards 0 and therefore the probability of "f" being nil is becoming nil too when n grows.

If now we consider an annulus formed by 2 circles: one inner circle of radius 1-a and another circle of radius 1+a. We will show that for any "a" there is a degree level "n_a" such that the probability of the module of "g" to be greater than the module of "f" is a declining function of n_a and this index actually tends towards infinity when a tends towards 0. So, when a tends towards 0, n_a tends towards infinity, and the function that is an upper bound for the probability that the module of "g" is higher than "f" goes to zero, ie "f" dominates "g" almost surely so. We then will apply the theorem of Rouché on unit disk to state that the roots of f+g actually tends towards the roots of "f" that is the fully uniform cyclotomic distribution.

Beware here of the fact that we deal with a limit case…in other words, for a given degree "n" that is NOT infinity, we will have surely "f" having only cyclotomic roots and "g" more often than not be dominated by "f"….but not in full in the general case, which implies that some roots of "g" will differ from the ones of "f". Yet, in a full dominance of "f" the roots would be exactly identical….which suggests in reverse that, because "g" is not the nil polynomial in general, the dominance is never full before infinity too. Indeed if "g" is not nil, we are certain otherwise that the roots of f+g shall differ from the ones of f. Yet, when "n" grows, the discrepancy vanishes progressively. As "f" dominates "g" more and more surely from a statistical standpoint. We can only think in "probabilistic" terms because we assume that the coefficients are "normally distributed" which means strictly any real value is "possible" even though it is not "likely". In other words, there is always the possibility that "g" has a huge coefficient, which then would make it dominate "f". But that would be "exceptional" statistically speaking….and yet never "impossible"…Now, back briefly to our context of the transformer blocks piling in through a random "normal" process, we really cave about the "expectation", not the exception.

One remark is that we really will need that, through the CLT the factors land on the standard normal distribution. This is it. This what the ChatGPT model relies on: through the many random normalizations and geometrical distorsions that the attention, or the MLP, induce, we always revert to the unit circle. Although it is not intrinsically "normal" in distribution terms, it remains homogeneous and it remains a Markov process. Therefore the CLT applies and we know that the compound effect will follow a normal distribution.

Let's consider first the function $f_a(z) = a_n(z^n - 1)$ in the annulus delimited by the circles of respective radii "1+a" and "1-a". We care to prove that for any z in this annulus then the function f_a will have a certain value for its modulus and the other term , namely what we call "g"; will have most of the time a lower module value. So let's call $M_{f_{a_z}} = |f_a(z)|$. We now look for the probability that the module of the other function, namely $g_a(z) = \underbrace{a_{n-1}z^{n-1} + a_{n-2}z^{n-2} + a_{n-3}z^{n-3} + \cdots a_1 z + a_0 + a_n}$, has a higher module than $M_{f_{a_z}}$. Of course is all depends upon the factors a_k that are randomly distributed among themselves. From one set of factors to another the functions will change. But we can think in terms of what we expect to get on average, whatever the set of factors will be for a given degree "n". For any given "z", depending on the sample of factors that we would get, we would have each time a different situation. However, on average, we might get a regular pattern of behavior.

Back to the context of ChatGPT, remember that there is a generation of random coefficients like these for every single word. So, again, from one block to the next, from one word to the next, we only know that the coefficients across squared matrices of 1024 lines and 1024 columns, ie 1 048 576 coefficients that mix together in each block to generate in the end a polynomial that would be of degree say 16+16=32…well each word has its meaning, its positional embedding in the process and will be influenced more or less remotely by the other words in some sinusoidal fashion ie cyclical and rather aribitrarily centered around 0 before being scaled back, as a group, to a norm 1 value…So we really cannot control that: it is random as soon as we consider a model that "works" for every word in every possible position among 1023 other words alike.

All we want is to make sure is that in all these possible situations we remain consistent. For that we want to converge to the cyclotomic solutions even if there is still some possible motion around that shared convergence point. We saw above that we need to show, a bit paradoxically, that all these "other coefficients" will weigh little if nothing when it is about the ultimate "profile" of the output.

Thus we look for:

$$P\left(\left|g_{a_{sample}}(z)\right| \geq M_{f_{a_{z_{sample}}}}\right)$$

This probability matters because it means that if the probability is low on average, then the dominance has a great probability instead to be verified and we therefore prove the ultimate conclusion that we aim to prove here: at the end of the day, the meta words will land "rather close always" on the same kind of values that are here the cyclotomic solutions to $z^n - 1 = 0$ which we know are $e^{\frac{2i}{n}}$

Now we know also that: $\left|g_{a_{sample}}(z)\right| = |a_{n-1}z^{n-1} + a_{n-2}z^{n-2} + a_{n-3}z^{n-3} + \cdots a_1 z + a_0 + a_n|$

And:

$$|a_{n-1}z^{n-1} + a_{n-2}z^{n-2} + a_{n-3}z^{n-3} + \cdots a_1 z + a_0 + a_n| \leq |a_{n-1}z^{n-1}| + |a_{n-2}z^{n-2}| + \cdots |a_0| + |a_n|$$

Therefore , the probability of a higher module being above the same value is higher:

$$P\left(\left|g_{a_{sample}}(z)\right| \geq M_{f_{a_{z_{sample}}}}\right) \leq P\left(|a_{n-1}z^{n-1}| + |a_{n-2}z^{n-2}| + \cdots |a_0| + |a_n| \geq M_{f_{a_{z_{sample}}}}\right)$$

But we would not benefit from the proximity from 0 for the sum of the factors. We should then consider z as $z = re^{iu} = (1 + x)e^{iu}$ and express that function "g" as a function of "x" plus a constant complex term. We can notice that the function g is close to a Fourier series where "u" is the dual variable and the power law "k" is the native variable of the discrete series.

$$g\left(z = re^{iu}\right) = a_{n-1}r^{n-1}e^{(n-1)iu} + a_{n-2}r^{n-2}e^{(n-2)u} + \cdots + a_1 re^{iu} + a_0 \ + a_n$$

When we consider the case when r=1 and u=0 we know the sum of the factors converges fast towards 0.

What happens when r differs slightly from 1 and u ranges over [0,2π]?

In particular we care to compare the module of g for a given (long enough) set of factors a_k, against the module of $f_a(z)$.

We then realize that we deal with an inner function "h" that we know only from its Fourier series coefficients:

$$h_{F_k} = a_k r^k \ and \ h_{F_0} = a_0 + a_n$$

We will limit ourselves for the outer circle to the circle of radius 1 because inverting z leads to a quite symmetrical initial problem and therefore the same ultimate answer.

Let's check that quickly here….

Starting from the initial problem related to the polynomial:

$$P_n(z) = a_n z^n + a_{n-1}z^{n-1} + \cdots a_1 z + a_0$$

Let's say we factorize as follows:

$$P_n(z) = z^n\left(a_n + \frac{a_{n-1}}{z} + \frac{a_{n-2}}{z^2} + \cdots \frac{a_1}{z^{n-1}} + \frac{a_0}{z^n}\right)$$

Then, we realize that when we look at the zeros of the polynomial:

$$Q_n\left(\frac{1}{z}\right) = a_n + \frac{a_{n-1}}{z} + \frac{a_{n-2}}{z^2} + \cdots \frac{a_1}{z^{n-1}} + \frac{a_0}{z^n}$$

We look at the same problem since only the "index" of the coefficients has changed, not the constraints of the question asked here. Thus whatever value for z when z has a module that is higher than 1, it is exactly the same problem in $\frac{1}{z}$ ie for a complex the module of which is inferior to 1. The fact that for low modules the polynomial would barely move away from the constant term value, or else for high modules the polynomial would diverge as per the sign of the coefficient a_n $R_{eal}(z^n)$ is irrelevant since in both ends there are no zeros. We aim to prove that as n grows the roots of the polynomial P or the polynomial Q likewise will be found uniformly spread on the unit circle.

Now we should consider that this sum is the start of a Fourier factor in the context of the Rieman Lebesgue lemma that states:

on a bounded interval [a,b] that for any function f that is in $\boldsymbol{L_{1[a,b]}(\mathbb{R})}$ ie such that $\boldsymbol{\int |f(x)| dx < +\infty}$ that $\boldsymbol{\lim_{t\to+\infty} F_{f(x)}(t) = \lim_{|t|\to+\infty} \int f(x) e^{+itx} dx = 0}$

we have to make the distinction in a first step between "n" and "t" showing that a function h_n of the variable "u" finds its "limit" Fourier factor converging towards 0. We will show the convergence to 0 with "t" and next infer that "n" mapping "t" in discrete strides, the limit is the same ie "0".

We can see that the function "h" can be expressed as :

$$h_n(u) = \sum_{k=0}^{n-1} h_{F_k} e^{iku} = \sum_{k=0}^{n-1} b_k r^k e^{iku}$$

We can introduce the term e^{int} and, sliding an integral then we get:

$$h_n(u) = \sum_{k=0}^{n-1} b_k r^k e^{iku} = \int_a^b \sum_{k=0}^{n-1} F_{n_k}(x)\, e^{inx} dx$$

We would like now to find a function that is the sum of sub-functions $F_{n_k}(x)$ of "x" over a shared interval [a,b] so that $b_k r^k e^{iku} = \int_a^b F_{n_k}(x) e^{inx} dx \ then\ f_n(x) = \sum_{k=0}^{n-1} F_{n_k}(x)$

We do not need that the components of f or the many F_{n_k} to be smooth. We will only need to prove that the integral of f on the interval [a,b] is absolutely convergent. We then can simply set a=1 and b=0, ie an interval [0,+1] and the function $F_{n_k}(x) = e^{-inx} b_k r^k e^{iku}$ on this interval and 0 elsewhere.

As a result: $f_n(x) = \sum_{k=0}^{n-1} F_{n_k}(x) = \sum_{k=0}^{n-1} e^{-inx} b_k r^k e^{iku}$

In order to apply the lemma of Riemann-Lebesgue, we need to show that $\int |f_n(x)| dx$ exists. That is very clear for any finite value of "n". But the challenge is to prove that this remains true when "n" goes to infinity.

But first let's apply the lemma for any "n": since the function f is absolutely convergent in integral terms then its limit Fourier factor, ie $\boldsymbol{\lim_{t\to+\infty} F_{f_n(x)}(t) = \lim_{|t|\to+\infty} \int f_n(x) e^{+itx} dx = 0}$

To do that we will look at the intermediate series: $S_{f_n} = \sum_{k=0}^{k=n-1} |\int F_{n_k}(x)\,dx|$

Actually we can use the relation above since only all the Dirac measures below N are triggered: $S_{f_n} = \sum_{k=0}^{k=n-1} |e^{-inx} b_k r^k e^{iku}| = \sum_{k=0}^{k=n-1} |b_k| r^k$

We only need to prove that this series has a limit and this is achieved if we simply prove that it has an upper bound given that it is a monotonic growing sequence on "n". We can notice that the factors b_k play a critical role here.

If we just consider that $|a_k|$ appears exactly 2 times more often than a_k because the distribution is perfectly symmetrical we can easily compute the limit of the series that will converge absolutely. We simply consider that when N grows it is just as if we added N random variables each following the normal distribution.

Please note for now on that we really need , only, that the series is centered on 0 and that its variance is finite….

One the one hand we know that $\sum_{k=0}^{k=n-1} r^k$ will converge as long as r<1 and will surely diverge when r=1. On the other hand we know that the coefficients $b_k\ or\ a_k$ are normally distributed. So, on balance, they average around zero as per a variance that is "1". So this will more than compensate, as we will see later, the potential divergence of the series above for r=1. However, the coefficients $b_k\ or\ a_k$can take arbitrarily high values. Yet, if it occurs, it must be so rare that "on average" this is negligible on our matter. This is what we will prove now.

So, among the many a_k, the range of the normal distribution will be covered through some empirical range that will inclue all the coefficients as they are drawn. And for each "bucket" of possible values we would have a finite numbers of indices of type "k" that would find the coefficient $b_k or\ a_k$ to stand in this interval. Within this interval we would have only a fraction of the sum of the power laws of "r" that we mentioned with$\sum_{k=0}^{k=n-1} r^k$.

So, on the on hand, every interval that would cover the range of values taken in one draw of coefficients would at the most contain $\sum_{k=0}^{k=n-1} r^k$. this is an upper bound for all the sub intervals. On the other hand, a draw of factors a_k can always be the very same value…even though that would be a rare situation. Alternatively some coefficients might be arbitrarily high and offsetting each other. But, also, since the variance must remain 1 overall, there will be a much much larger number of other coefficients that wil be 0 or close.

Now we know that each value for each bucket follows the normal distribution and we also know that once some records are allocated to one bucket, they will not go to another bucket. Therefore, when N gets very high we will find many values associated to a power law of "r" between 0 and N. If we assume that r<1, the sum of the terms will always be bounded upwards by the sum of all the power laws that are possible and, even so, they will be present in a number that is a fraction of this total. As a consequence when we would "group" the factors by buckets of values, we will get a sum of power laws of "r" as follows:

$$B_{ucket_m} = A_{buck\ \ _m}\left(r^{k_{m_1}} + r^{k_{m_2}} + \cdots\right) \leq A_{bucket_m}\left(\sum_{k=0}^{k=N-1} r^k\right)\left(N_{norm}(0,1,A_{bucke\ _m})(1+\varepsilon)\right.$$

So we notice the term in "epsilon" that is here to show that we still run a approximation that, when N gets infinite induces that the "epsilon" converges to 0. With limited draws, ie n small, one coefficient may be very in module and the others may not get really close to 0, while the variance of

course would take a large value. But this is not our matter here: we instead look at cases where "n" is large, the probability to have some very large value is not nil but on balance this forces all the other values to ensure an average at 0 and a variance of 1 or close. This constraint is mandated by our core assumption: the coefficients follow a normale distribution and therefore the more they are (ie the higher n is), the closer their average is to 0 and the closer their variance is to 1.

In any event there is an upper bound "M" for this "epsilon" value: indeed, under the key assumption that "r" is <1 all along, we know that the CLT will secure a convergence of the distribution of our empirical limited sums in number towards the normal distribution. This secures that the "epsilon" converges and therefore, as a series of index "n", it is bounded. The dominance of the lower power laws or "r" versus the actual ones is sure. The key really is that, when we opt to just "count" the number of occurrence of factors within predefined buckets, we approach the normal distribution better and better as N grows. Hence the upper bound that we define here

More, as we increase N, exactly like when we actually run a Monte Carlo simulation, we would check on the coherence between the sampling and the target distribution by grouping the sample values in sub-intervals and refine these intervals width as we get more samples. When we would draw the empirical distribution in "bars", the latter would fit better and better the smooth line of the limit distribution as N grows.

Of course we speak in "module" terms that, on average will NOT converge to 0 like the native samples. But it is enough to notice the symmetry of the distribution or, more generally, that be that in the "positive values" direction or in the "negative values" direction the variance is limited and therefore the expection if bounded.

So we can define an upper bound that can hardly be better if we want to cover all the possible cases. For the normal distribution the integrals are easy to compute but, as we hinted at before, we only really need to have a bounded variance and expectation" In every possible directions that define a basis on the space of variation for the coefficients":

$$S_{f_N} \leq (1+M)\left(\sum_{k=0}^{k=N-1} r^k\right) 2\int_0^{+\infty} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}\, x dx$$

$$S_{f_N} \leq \frac{(1+M)(1-r^N)}{1-r} 2\int_0^{+\infty} \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}x^2}\, x dx \leq 2\frac{(1+M)}{1-r}$$

As long as |r|<1, we prove that the series is upper bounded and monotonic, therefore converging. Thus the function f_n is L_1 on the interval [0,1]

We therefore can apply the lemma of Riemann-Lebesgue and again the absolute convergence allows to state that: $\mathbf{lim}_{t\to+\infty} \boldsymbol{F_{Lim_{n\to+\infty} f_n(x)}(t)} = \boldsymbol{Lim_{n\to+\infty}}[\mathbf{lim}_{|t|\to+\infty} \int \boldsymbol{f_n(x) e^{+itx} dx}] = \mathbf{0}$

We therefore proved here that $\int_a^b \sum_{k=0}^{+\infty} F_{n_k}(x)\, e^{itx} dx \; tends\; to\; 0$ when t tends towards infinity because $\sum_{k=0}^{+\infty} F_{n_k}(x) = f_{+\infty}(u)$ is L1

Now our series is more like $\int_a^b \sum_{k=0}^{n} F_{n_k}(x)\, e^{inx} dx$

We have a sense that when "n" tends towards infinity the "sigma" converges towards the function$f_{+\infty}(u)$ and also the Fourier factor therefore goes to the limit of the Fouier factor of this same function at infinity. This is what happens indeed.

Now we have not concluded fully on the case where actually "t=n" while n goes to infinity……We have just proved for now that for any "t" the limit value of the Fourier factor of the limit sum goes to zero. We have also proved, thanks to the absolute convergence that eliminates a possible influence of n on the fact that the increments anyway add less and less, that this remains true if and when "n" goes to infinity. We notice here the crucial role played by r<1.

What can we say then if, for a given "n", we actually compute the Fourier term, and its module, at "t=n"? Well we know that this value in module will be a sequence of terms that goes to zero as n goes towards infinity since the function is L1. Indeed, for any boundary value "b" centered on 0, be itself being arbitrarily low, we will find a value for "n" , call it n1,where, beyond that value n1 the Fourier factor of any value "t=n" higher than this one will have its value below a distance b/2 from 0. This results simply from the application of the Riemann Lebesgue lemma. And we will find another lower bound value for "n" , noted n2 this time where, beyond any higher n value guarantees that the function f_n is close enough to $f_{+\infty}$, for a given "u" that is common to all these functions here, to its limit ie at a distance form of b/2at the most. Using the triangular inequality we then conclude that this Fourier factor is close to the limit at a distance b at the most (b/2+b/2). We therefore prove that the function "h" goes to zero at "b" residual distance when "n" is beyond the maximum between the 2 threshold values that we described. For any arbitrarily low value of "b" we will find a rank "n=max'n1,n2)" beyond which our series is at a distance "b" from 0 at the most. We conclude therefore that the limit of our sequence is also 0.

Thus, beyond a certain "index" "n", the module of our original function "g" is arbitrarily low. This proves the dominated nature of "g" by the very original function $f_a(z) = a_n(z^n - 1)$ for any complex "$z = re^{iu}$ with r<1".

Let's crucially notice here that we only need that the distribution is symmetrical (for the convenience of the proof), that only the module of the factors would matter in fact and a reasonable distribution of the diverse "signs" of the factors across the range of possible values. In other words we only really need a bounded expectation in every possible direction and the abounded variance in every possible direction.

Let's notice that that if the variance and expectations are finite, ie if the squared module of the factors in question is converging in distribution terms as per the distribution of these factors, the proof still works. In other words, the normal distribution is not mandatory: we need an average at 0, we need a reasonable symmetry around that 0 and we need a finite variance and expectation in module. More…The simple fact that if we divide all the factors by their variance does not change the location of the roots, while it will surely change their variance to "1", supports the view that we only need a finite variance here. The symmetry around 0 is more generally the need to have a central symmetry to easily compute the moment of the module of the coefficients. But really we only need to have module moment along every "direction " form a function of the direction that is L1 on $[0, +\infty[$. In other words, if the coefficients were complexes we only need every "beam" to end up being L1 in distribution terms.

So if we focus on the probability below in the annulus that is bounded by the unit circle and r<1 always (a>0) having set a z complex value accordingly:

$$P\left(\left|g_{a_{sample}}(z)\right| \geq M_{f_{a_{z_{sample}}}}\right)$$

We can observe then that if we grow "n" the module of $f_a(z) = a_n(z^n - 1)$ will grow (as long as r<1) and the module of $\left|g_{a_{sample}}(z)\right|$ will tend to 0. The probability above can therefore only go to 0 when n grows. This means that strictly inside the unit disk, when n grows, the probability to have "g" dominate $f_a(z) = a_n(z^n - 1)$ in module goes to zero. This implies that our native polynomial, being the sum of $f_a(z) = a_n(z^n - 1)\ and\ g_{a_{sample}}(z)$ finds its zeros on right where (Rouché's theorem proof consequence) $f_a(z) = a_n(z^n - 1)$ has its zeros.

But the latter has no zeros outside the unit circle.So we so far have proved that when n gets higher and higher, the function f_n will dominate almost surely the function g_n that contains "all the other factors that are not associated wint "n". We prove that for any complex number "z" that has a module r<1. And therefore, no matter how close we get to the unit circle, ie r=1, without "touching" it, the potentials "zeros" of the polynomial will be those of f_n. But the problem is: f_n has all its zeros uniformly distributed on the unit circle. Intuitively we sense that if we pick one of these zeros and arbitrarily "look" into some inner nerighborhood of this root of f_n we will find for all the z values that $f_n(z) + g_n(z)$ can only cross 0 where and when $f_n(z)$ crosses 0…when n is large enough…on average…Can we "push" this conclusion to the unit circle then and conclude once for all with no ambiguity?

We always revolve around the same question then, from the lemma of Riemann-Lebesgue: " is the series of the $|a_k|$ converging? We add up an infinity of the absolute value of values that are distributed normally. This sum is clearly not bounded, and there is always a probability that it goes beyond any value. We cannot run by the former method to define an upper bound and prove that the series of modules is bounded, therefore proving that the function $f_{+\infty}(u)$ Is L_1. The Riemann-Lebesqge lemma lemma does not apply.

However, in any neighborhood of r=1 we have this conclusion. We also know that , aside from z=1 where we have a problem for now because $f_a(z) = a_n(z^n - 1) = 0$(ie $f_n\ being\ nil$ it cannot possibly dominate $g_n(1)$ that is not nil in module in general, the module of the latter function is not 0. Let's therefore focus on the case where r=1 but z is not equal to 1 itself.

Then the function "g" below:

$$g\left(z = re^{iu}\right) = a_{n-1}r^{n-1}e^{(n-1)iu} + a_{n-2}r^{n-2}e^{(n-2)u} + \cdots + a_1re^{iu} + a_0 \ + a_n$$

It becomes, for r=1:

$$g\left(z = e^{iu}\right) = a_{n-1}e^{(n-1)iu} + a_{n-2}e^{(n-2)u} + \cdots + a_1e^{iu} + a_0 \ + a_n = g_1(u)$$

We should consider the Fourier series of $g_n(u)$ on the interval $[0,2\pi]$. What is the Fourier series of each term of rank 'k'?

$$X_{km} = \frac{1}{2\pi}\int_0^{2\pi} b_k e^{iku-i} \quad du = \begin{cases} 0\ if\ m \neq k \\ b_k\ if\ m = k \end{cases}$$

So this I function $g\left(z = e^{iu}\right)$can be seen as a Fourier series and we can see that it converges towards 0 when u tends towards 0 because the sum of the factor a_k tends towards 0.

Let's reformulate the function that we study here so that we better stick to the standard definition of Fourier series:

$$h_{2\pi}(v) = a_{n-1}e^{(n-1)i2\pi v} + a_{n-2}e^{(n-2)2\pi iv} + \cdots + a_1 e^{i2\pi v} + a_0 \ + a_n = g_1(u)$$

Let's consider a value that is not 0 for v but rather a rational number like $u = \frac{p}{q}$ with "p" and "q" integers being positive and u is one value in the interval $[0,1[$. For very large values of N, we know that after every loop over "q" consecutive values for "k", we land on the same argument in the complex exponential $e^{\frac{ikp}{q}2\pi}$ because any new q-increase only generates a value $2i\pi \ and \ e^{2i\pi} = 1$. Then for every such value of "k" we will land on one of the "p" possible classes of angles of the type $\frac{2\pi}{q} \ where \ k = m[q]$. Depending on the PGCD between p and q we may have classes that melt together we may have less distinct classes. But we can always define "q" different classes, and for each class, anyway, there we will see the extracted sum of factors a_k that will count as normally distributed elements of a sum, that necessarily converge to 0 for each of the q different classes. We can conclude that the target function converges towards 0 for all the "rational" multiples of 2π. Thus when "n" goes towards infinity we get an infinity of coefficients that "fall" into one class. For all the q classes, we will sum the coefficients of this angle and get the expectation…that is zero…provided n is high enough… This works for any value "u" on the unit circle that has an angle "v" modulo 2π where the "angle" is like $v = \frac{p}{q}$. These are rational numbers as we know and for each of them there is a level for "n" that will be high enough so that every one of the q different clases will have enough "samples" of the normal distribution to average close to 0.

The interval $[0, 1]$ being densely filled with such rational number, any real fraction can be approached through these rational numbers and the difference can be shrunk while the limit remains 0 because there will be offsetting differences when all the classes have a sum of zero anyway. That seems to work but we need to prove still that this number "n" exists for a real number that is only the "limit" of a sequence of rational numbers…. The proof lies again on the fact that if we consider our "real number v" and the function that expresses g_n on the unit circle, whether u look at it from the "u" standpoint, or from the "v" standpoint, u can say that this is always an holomorph function ie infinitely derivable, in other words very smooth. In an arbitrarily small neighborhood of this "real angle" number , no matter how small, you will find 2 rational numbers that will be so close to the real v angle that the function is arbitrarily close to the value taken by the same function having the 2 neighboring rational numbers. We are certain of that because the function has a nice smooth firs derivative everywhere.

$$h_{2\pi}(v) = a_{n-1}e^{(n-1)i2\pi v} + a_{n-2}e^{(n-2)2\pi iv} + \cdots + a_1 e^{i2\pi v} + a_0 \ + a_n = g_1(u)$$

Thus picking an arbitrary low positive value "b" again, we would look for rational numbers that beyond a certain level of "n" each, call it n+ and n-, will place $h_{2\pi}(v_-)$ and $h_{2\pi}(v_3)$ where $v_- \leq v_{real} \leq v_+$ at a distance to $h_{2\pi}(v_{real})$ that is inferior to d/2. How can we be sure of that? We can assert this because, the derivative of the function is bounded everywhere in this compact set irrespective of the 3 angles (the unit circle is a compact set)

$|h_{2\pi}(v_{real}) - h_{2\pi}(v_-)| \leq M_{deriv}, |v_{real} - v_-|$ and $[h_{2\pi}(v_{real}) - h_{2\pi}(v_+)[\leq M_{real}, |v_{real} - v_+|$

If we set "d", we infer another derived bound $\varepsilon = \frac{d}{2M_{deriv}} \ and \ thus \ picking$, $|v_{real} - v_-| < \varepsilon$ and

Likewise , $|v_{real} - v_+| < \varepsilon$

Then we infer for any value of d>0 that is arbitrarily close to 0 that:

$|h_{2\pi}(v_{real}) - h_{2\pi}(v_-)| \leq M_{deriv}, |v_{real} - v_-| < \frac{d}{2}$

and $[h_{2\pi}(v_{real}) - h_{2\pi}(v_+)[\leq M_{real}, |v_{real} - v_+| < \frac{d}{2}$

How can we sure of that? Simple: there is a sequence of rational numbers that converges towards v_{real} $and\ therefore\ there\ is\ a\ pair\ of\ integers\ so\ that \frac{p_-}{q_-} \leq v_{real} w < \frac{p_-+1}{q_-}$ We just need to choose an integer q_- that is high enough so that $\frac{1}{q} < \varepsilon$. We then verify te inequalities. In order to spot the right q_- value it is simple: we take $q_- = Ent\left(\frac{1}{\varepsilon}\right) + 1\ since\ then\ p_- = E(q_- v_{real})$. We can verify than that $p_- \leq q_- v_{real} < p_- + 1\ and\ therefore\ \frac{p_-}{q_-} \leq v_{real} w < \frac{p_-+1}{q_-} < \frac{p_-}{q_-} + \varepsilon$

So we have just proved that the function $g_n(u)$ is arbitrarily close to 2 values of the same function g_n but in slightly different "angles". And formerly we proved that for each of them, for this same arbitrarily low distance from 0, there is lower bound value for "n" above which the the function g_n is itself close enough to zero, the limit when n goes to infinity. So once all this is established we know that for each of the 2 rational numbers that bracket the real angle "v" we can find a lower bound for "n" above which each series will be itself at this distance "d/2" from 0. We just need to pick the highest of the 2 for each d. Since it always exists and we have proved by the triangular inequality again that the function g_n(u) converge to 0 when n is higher than a certain value that we can define rationally….while the function f_n is only nil on n roots on the unit circle. So we have proved that g_n is more and more dominated by f_n along the unit circle.

Again now, considering any neighborhood inside or at the unit circle we can define a certain "n" values above which all the z values converging towards a "z" being on the unit circle will ascertain the statistical dominance of $f_n\ over\ g_n$.And we also can define another certain "n" above which all the rational angles are also ascertaining the statistical dominance of $f_n\ over\ g_n$. The rational angles forming a dense subset of the angles we can therefore use again the fact that our functions have a nice firs derivative to finish the proof: the dominance is every where on and inside the unit circle. As explained above, looking at the case of z that have a module above 1 reverts back to the problem that we have dealt with here: only the zeros on the unit circle of "f_n' will matter when "n" grows to infinity. QED

Let's observe here that this step through the rational angles with complex factors works as long as we keep extracting a finite number of series from an infinite countable series that sums up to zero. The "counting" process is entirely defined by the choice of "z", irrespective of the "vectors" of the coefficients that may be involved in each random draw. Thus, for example, if we ran the same thing in parallel on any number of dimension, this would work as long as the "vectors" average to the nil vector when n goes infinite. And we could prove the same thing, as we saw above, provided the distribution has an L1 property across the many possible directions. The L2 property for the variance would naturally ensue on the unit disk anyway. In layman's terms we need the distribution of the coefficients to simply be centered on 0 and to be L1 on its domain of definition. This is very well fitted to the case of ChatGPT and other LLMs!

To close the matter, even the original part could , within the application of the Riemann-Lebesgue lemma AND the subsequent part for r=1, involve vectors of many dimensions, the "module" being replaced by a "norm". All we need is vectors or tensors centered on 0 and the distribution being L1 on its domain. Which again is a good fit with the goal of the LLMs.

Now a key point is that we cannot deploy a commutative ring based on a field beyond the set of complex number. The quaternions (Hamilton) were not commutative, ie we could not express geometric views in arithmetic format in 4 dimensions or more. Typically a product of matrices is not

commutative in general. But we can build a full arithmetic with a commutative product in 2 dimensions with complex numbers. Thus if we split every real coordinate into 2 through a complex number set, we then need a power law of 2 as a number of coordinates. This rise on power law of 2 may look like a luxury but on balance we then have a fully commutative calculation space. If indeed we want to apply this theorem to many dimensions using simply matrices would not work well all the time because the way the polynomial is built up implies a certain order of products of matrices. And to predict vectors" that may appear at different positions like words typically, a usual product of matrices may induce disturbing imbalances. We need a symmetry on the products of matrices due to that "position of the word that could change" thing and the only way to secure this is to remain in "complex coordinates" ie a power law of 2 when we increase the capacity of the model. This last point may not be mandatory though…. We look for symmetries but we also look for asymmetries….maybe "meaning" Is there: in the asymmetry…

This means that the Taylor series that we defined for "g" converges towards 0 when n becomes infinite which means that the dominance is here too on the unit circle: the conclusion is therefore that when n grows the polynomial has its roots that converges towards the regular uniformly displaced cyclotomic roots. And this is the case for many other distributions than the normal districbution provided they are centered on 0 and are L1.